

ESP32: практичний довідник

Польовий довідник з розробки, прошивки, діагностики й ремонту

ESP32: практичний довідник

Польовий довідник з розробки, прошивки, діагностики й ремонту

Ярослав Войтович

ESP32: практичний довідник

Польовий довідник з розробки, прошивки, діагностики й ремонту

Автор — Ярослав Войтович.

Ревізія 2026-08-26. Видання перше.

Ліцензія. Текст довідника — Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0). Приклади коду — MIT.

Довідник дозволено **вільно друкувати, копіювати і роздавати**, зокрема комерційним друком, за умови зазначення авторства; похідні матеріали поширюються на тих самих умовах. Повні тексти ліцензій — у репозиторії проекту.

Що це за книга. Не джерело істини першої інстанції, а **зібрання**: відомості з великої кількості відкритих джерел — документації Espressif, вихідних текстів ESP-IDF і esptool, специфікацій виробників мікросхем — зведені до купи, впорядковані й звірені з першоджерелами. Книга не заміняє документацію Espressif; вона дає швидку робочу базу тому, хто відкриває для себе вбудовану електроніку.

Книга публічна. Текст, приклади коду й **увесь реєстр фактчекінгу** — з дослівними цитатами, адресами джерел і станом перевірки кожного твердження — відкриті й доступні за адресою github.com/ivoitovych/esp32-handbook-ua. Перевірити будь-яке число з цих сторінок можна не питаючи автора.

Як зроблено. Книгу складено приблизно за 48 годин, і **справжнього рецензента вона не мала**: жодна людина не прочитала всі сторінки поспіль перед друком. Замість людської вичитки застосовано автоматизовану перевірку. Понад 7800 тверджень виділено в окремі одиниці з власними ідентифікаторами; кожна має запис у реєстрі й клас перевірки — від «першоджерело отримано, цитату наведено дослівно» до «ще не звірено». Три незалежні автоматичні звірки стежать, щоб твердження не лишалося без запису, щоб цитата підпирала саме те, що нею підпирають, і щоб кожна цитата дослівно стояла за названою адресою. Докладно — у вступному розділі.

Помилки тут є. Автоматика зменшує їхню кількість, але не доводить до нуля. Числа, від яких залежить ціле, звіряйте з першоджерелом.

Застереження. Матеріал описує стандартну інженерну роботу з мікроконтролерами: схемотехніку, код, протоколи, живлення, ремонт. Автор не несе відповідальності за наслідки застосування наведених відомостей. Робота з мережевим живленням, літєвими акумуляторами і радіопередавачами регулюється окремими нормами — дотримання їх лишається на відповідальності читача.

Технічні дані наведено за документацією Espressif Systems станом на дату ревізії. Кремний, документація і тулчейн змінюються — звіряйте критичні значення з першоджерелом.

Складено вільним програмним забезпеченням: Typst, pandoc. Гарнітури: Libertinus, DejaVu Sans Mono.

Зміст

Про цей довідник	2
Ярус 0. Картки	6
K1. Тріаж невідомої плати	7
K2. Збереження стану перед втручанням	9
K3. Підключення до ПК: кабель → драйвер → порт → права	11
K4. Download mode вручну (BOOT + EN)	13
K5. Прошивка готового образу	15
K6. Читання boot-логу	17
K7. Guru Meditation і backtrace за 60 секунд	19
K8. Топ-15 несправностей: симптом → причина → дія	21
K9. Піни: обмеження і призначення	23
K10. Шпаргалка команд	25
K11. Ніколи	27
K12. Польовий комплект і закупівля	29
K13. Живлення: пошук несправності	31
K14. Сумісність логічних рівнів	33
K15. Чек-лист серійної прошивки	35
Частина I. Платформа	38
01. Що таке ESP32 і коли його брати	39
02. Лінійка чипів і правило фокуса	44
03. Архітектура SoC	49
04. Периферія на кристалі — огляд	53
Частина II. Електроніка для програміста	58
05. Електроніка за один вечір	59
06. Живлення і струмоспоживання	65
07. Розпіновка й обмеження GPIO	71
Частина III. Плати й підключення	78
08. Модулі, плати, анатомія девборди	79
09. Підключення до комп'ютера	83
10. Комплект інструментів	88
Частина IV. Інструментарій	92
11. ESP-IDF + розширення VS Code	94
12. Arduino core для ESP32	100
13. PlatformIO і pioarduino	104
14. MicroPython, ESPHome, Wokwi	108
15. Вибір, поєднання, офлайн-комплект	112
Частина V. Прошивка	116
16. Як завантажується ESP32	117
17. esptool	122
18. Розділи флешу	129
19. OTA-оновлення	134
20. Бекап, відновлення, «цеглинка»	139

21. Серійна прошивка партії	143
Частина VI. Невідомий пристрій	147
22. Збереження стану перед втручанням	148
23. Тріаж невідомої плати	152
24. Чужа прошивка: форензика	156
Частина VII. Налагодження і діагностика	160
25. Serial monitor і логування	161
26. Читання збоїв	166
27. JTAG і покрокове налагодження	172
28. Апаратна діагностика сигналів	176
29. Симптомний troubleshooting	180
Частина VIII. Програмування	185
30. Застосунок і модель пам'яті	186
31. FreeRTOS і конкурентність	191
32. Надійність і обробка помилок	197
33. Периферія з коду: GPIO, таймери, PWM, ADC	202
Частина IX. Дротові інтерфейси	208
34. UART, RS-485, Modbus	209
35. I ² C	213
36. SPI	218
37. 1-Wire	223
38. CAN (TWAI)	226
Частина X. Бездротовий зв'язок	231
39. Wi-Fi	232
40. Мережеві протоколи	237
41. Bluetooth і BLE	242
42. ESP-NOW	246
43. LoRa та інші радіо	251
Частина XI. Периферія	256
44. Як підключити незнайомий модуль	257
45. Сенсори	261
46. Дисплеї і введення	265
47. Ключі, реле, узгодження рівнів	269
48. Двигуни й сервоприводи	274
49. Карти пам'яті, камери, звук	278
Частина XII. Безпека	282
50. Безпека пристрою	283
Частина XIII. Збирання і супровід	288
51. Паяння і монтаж	289
52. Від макетки до постійного монтажу	293
53. Автономне живлення	297
54. Корпус і захист середовища	302
55. Польова діагностика й ремонт	306
56. Паспорт виробу і передача	310

Частина XIV. Від задачі до пристрою	315
57. Постановка і архітектура	316
58. Прототип → тест → надійний вузол	320
Частина XV. Проекти	324
59. Моніторинг датчиків з веб-інтерфейсом	325
60. Автономний логер	334
61. Радіоканал між двома платами	342
62. Керування виконавчими механізмами	348
63. Протокольний міст	356
Додатки	361
Додаток А. Повні пінаут-таблиці	362
Додаток В. Симптом → причина → фікс	366
Додаток С. Повні шпаргалки команд	373
Додаток D. Voot-повідомлення, скидання, паніки	378
Додаток Е. Інтерфейс → пристрої → бібліотека	386
Додаток F. Офлайн-пак	390
Додаток G. Глосарій	393
Додаток H. Джерела й посилання	401
Вкладиші	404
Вкладиш: ринок України	405
Вкладиш: компонентний довідник	407
Вкладиш: радіочастотні норми	411

Моєму братові
Ігнату Великоівану

Про цей довідник

Ця книга написана для програміста, який уміє писати код, але ніколи не тримав у руках осцилограф. Завтра вам можуть дати коробку незнайомих плат, чужий пристрій без документації або задачу «зроби, щоб воно передавало телеметрію, термін — до ранку». Довідник для цього.

Це не курс. Курс читають з початку; довідник відкривають на потрібній сторінці. Лінійне читання — лише один зі способів користуватися цією книгою, і не найчастіший.

Що це за книга, і чим вона не є

Це не джерело істини першої інстанції. Це зібрання: інформація з великої кількості відкритих джерел — документації Espressif, вихідних текстів ESP-IDF і esp8266, специфікацій виробників мікросхем, галузевих стандартів — зведена до купи, впорядкована й перевірена.

Цінність тут не в авторській новизні. Вона в трьох речах: **зібрано в одному місці, зведено до спільної структури й звірено з першоджерелами.** Те, що інакше довелося б збирати тижнями по десятках документів різними мовами, лежить під однією обкладинкою в порядку, у якому це потрібно на столі.

Тому книга не заміняє документацію Espressif і не претендує на це. Вона дає **швидкий старт і робочу базу:** досвідчений програміст, який відкриває для себе вбудовану електроніку, отримує звідси достатньо, щоб за день почати рухатися самостійно — і далі йти вже до першоджерел, яких у природі незліченно.

Як зроблено цю книгу, і чому це варто знати

Книгу зроблено приблизно за 48 годин. Це сказано не як похвальба, а як застереження: **вона не мала справжнього рецензента.** Жодна людина не прочитала всі 413 сторінок від початку до кінця перед тим, як їх надруковано. Для книги такого обсягу це серйозна вада, і читач має про неї знати.

Замість людської вичитки застосовано **автоматизовану перевірку**, і зроблено її стільки, скільки вдалося встигнути.

Кожне твердження в книзі, яке взагалі можна звірити, виділено в окрему одиницю з власним ідентифікатором. Таких одиниць **понад 7800.** Кожна має запис у реєстрі — дослівний текст книги поруч із дослівною цитатою з джерела, адресою цього джерела й датою отримання.

Одиниці розкладено за класами перевірки:

Клас	Що означає
A	першоджерело отримано, цитату наведено дослівно
B	першоджерело отримано, твердження впливає з нього однозначно
C	джерело назване, але недосяжне; записано, що саме в ньому шукати
D	перевіряється арифметикою; зовнішнє джерело не потрібне
E	поза зовнішньою звіркою: редакційне рішення, порада, рамка викладу
F	ще не звірено

Три різні автоматичні перевірки працюють над цим щодня. Перша стежить, щоб жодне твердження не лишилося без запису. Друга — щоб цитата справді підпирала те, що нею підпирають. Третя, суто механічна, звіряє **кожну цитату з файлом за названою адресою**, символ за символом: вона ловить переказ замість цитати, зшиті до купи речення й посилання, що ведуть у нікуди.

Окремий реєстр зберігає **спростовані формулювання** — те, що в книзі колись стояло й виявилось хибним, разом із взірцем для пошуку. Якщо хибне формулювання колись повернеться в текст, збирання впаде.

Чого це все не дає

Помилки в цій книзі є. У книжках такого обсягу вони є завжди, а тут до звичайних причин додається стислий термін і брак людської вичитки.

Автоматика зменшує кількість помилок, але не доводить її до нуля — і кожен із трьох рівнів перевірки має власну сліпу пляму. Механічна звірка цитати не бачить, що дослівна цитата зі справжнього документа підпирає хибний висновок. Такі випадки в цій книзі знаходили, і саме вони найдорожчі.

Тому правило для читача просте: **числа, від яких залежить ціле — напруги, струми, номери пінів, адреси — звіряйте з документацією Espressif на момент читання.** Довідник каже, що дивитися й де останнє слово лишається за першоджерелом.

Що вважається відомим

Ви пишете код мовою C або C++ — принаймні читаете його без словника. Ви користуєтеся командним рядком і git. Ви розумієте, що таке компілятор, покажчик і стек.

Все «залізне» подається з нуля: закон Ома, логічні рівні, підтягування, чому дешевий USB-хаб призводить до перезавантажень. Якщо ви це знаєте — пропустайте частину II.

Межі змісту

Довідник охоплює стандартну embedded-інженерію: залізо, код, протоколи, периферію, живлення, збирання, ремонт. Матеріал побудований навколо ESP32 як допоміжного (companion) контролера — вузла, що стоїть збоку від основної системи і спілкується з нею через UART або CAN.

Позначки

Чотири типи блоків трапляються по всьому тексту. Вони набрані окремо не для краси: кожен позначає місце, де читач найчастіше втрачає плату, час або гроші.

ЖИВЛЕННЯ

Живлення. Струми, просадки, вимоги до джерела. Найчастіша причина «незрозумілих глюків», які насправді не глюки.

УВАГА

Увага. Граблі, на які наступають усі. Дешевше прочитати, ніж відтворити самостійно.

ЗАКУПІВЛЯ

Закупівля. Що саме брати, на що дивитися в оголошенні, які позиції не варті грошей.

НЕЗВОРОТНЕ

Незворотне. Операція, яку не можна скасувати: перепалений eFuse, стертий флеш без дампа, спалений пін. Читати до, а не після.

Правила, що стосуються не всіх чипів, позначені областю дії: **classic** — тільки ESP32 classic, **S3** — тільки S3, **C3** — тільки C3. Там, де правило стосується інших сімейств, позначка називає і їх: **S2**, **C6**, **H2**. Без позначки правило стосується всієї лінійки.

Маршрути читання

Довідник має кілька входів. Оберіть той, що описує вашу ситуацію.

Дали невідому плату або пристрій. Тріаж плати (K1) → збереження стану (розділ 22) → тріаж (розділ 23) → чужа прошивка (розділ 24).

Треба тільки прошити готовий образ. Підключення до ПК (K3) → прошивка (K5) → esptool (розділ 17). Через FreeRTOS ви не проходитье — це інша задача.

Прошити партію плат. Серійна прошивка (розділ 21) → чек-лист серійної прошивки (K15).

Лагодити чуже. Збереження стану (K2) → тріаж (розділ 23) → симптомний troubleshooting (розділ 29) → польова діагностика (розділ 55).

Писати прошивку з нуля. ESP-IDF (розділ 11) → структура застосунку (розділ 30) → FreeRTOS (розділ 31), далі частини IX–XI за потребою.

Паяти і збирати корпус. Електроніка за вечір (розділ 05) → паяння (розділ 51) → корпус (розділ 54).

Щось робоче за дві години. Швидкі шляхи (розділ 14) → проєкт «моніторинг датчиків» (розділ 59).

Мовна політика

Українська назва подається разом із канонічним англійським терміном у дужках при першій згадці в розділі: черги (queues), підтягування (pull-up), таблиця розділів (partition table). Далі в розділі вживається українська форма.

Усталені терміни, які в галузі не перекладають, лишаються як є: brownout, watchdog, backtrace, strapping, bootloader. Шукати їх у пошуку ви все одно будете англійською.

Назви команд, файлів, реєстрів і функцій ніколи не перекладаються і набрані моноширинним шрифтом: `idf.py`, `app_main`, `sdkconfig`.

Версії інструментів

Кожне значення в довіднику прив'язане до конкретних версій тулчейну. Таблиця версій друкується на початку частини IV і генерується під час збирання з файлу `toolchain-baseline.yaml` — не переписується вручну в тексті. Рядки, звирені з першоджерелом, позначені окремо від незвирених.

Дата ревізії стоїть на титульній сторінці. Кремній, документація і тулчейн змінюються; критичні числові значення звіряйте з документацією Espressif на момент читання.

Швидкопсувне

Ціни, наявність на ринку, конкретні позиції постачальників і радіочастотні норми не входять до основного тіла. Вони живуть в окремих датованих вкладах, які перевидаються без зміни нумерації книги. Якщо вкладиш у ваших руках старший за рік — вважайте його орієнтиром, а не довідкою.

З чого почати закупівлю

Мінімальний комплект, з яким можна робити все, що описано в перших семи частинах, — на картці K12. Загальна сума менша, ніж здається; половина позицій купується один раз на роки.

Ярус 0. Картки

*Пятнадцать сторінок, які мають бути в руках,
а не у файлі. Друкуються окремо на А4,
ламінуються, їдуть у поле. Решта книги
пояснює, чому там написано саме це.*

К1. Тріаж невідомої плати

Плата в руках, походження невідоме. Порядок дій — згори вниз. Не перестрибуйте: кожен крок відсікає варіанти для наступного.

1. Дивитися, не вмикати

Прочитати напис на металевій кришці модуля — це головне джерело істини:

Напис на модулі	Чип	Що це значить
ESP32-WROOM-32	ESP32 classic	двоядерний Xtensa, без PSRAM
ESP32-WROVER	ESP32 classic	1. PSRAM; вона займає частину пінів
ESP32-S3-WROOM-1	ESP32-S3	двоядерний, native USB
ESP32-C3-MINI-1	ESP32-C3	одноядерний RISC-V
ESP8266 / ESP-12	не ESP32	інша архітектура, інший тулчейн

Далі — очима, без живлення: чи цілий USB-роз'єм, чи не здутий стабілізатор, чи немає слідів припою між сусідніми пінами, чи не обвуглене щось.

2. Живлення — до першого підключення

НЕЗВОРОТНЕ

Логіка ESP32 — **3.3 В**. П'ять вольтів на будь-який GPIO вбивають пін або чип. На платі є пін 5V/VIN — він іде на стабілізатор, а не в чип; це єдиний вхід, куди 5 В подавати можна.

Мультиметром у режимі прозвонки, живлення вимкнене: короткого замикання не має бути ні між 3V3 і GND, ні між 5V і GND. Дзвенить — далі не вмикати, шукати замикання.

3. Підключити і подивитися лог

USB-кабель має бути **data**, а не тільки для заряджання. Термінал на 115200 бод, потім коротко натиснути EN (RST).

- З'явився осмислений текст → чип живий, є прошивка. Далі — картка [K6](#).
- Текст «кракозябрами» → не та швидкість. Читається на **74880** — це ESP8266, не ESP32 (у нього ROM говорить саме так).
- Один рядок, тиша, знову той самий рядок → boot loop. Далі — картка [K7](#).
- Порожньо → чип не стартує або немає порту. Далі — картка [K3](#).

4. Опитати чип

```
esptool --port /dev/ttyUSB0 flash-id
```

Шапка з'єднання називає сімейство і ревізію — звірити з написом на модулі. flash-id називає обсяг флешу: менший за очікуваний означає клон із перемаркованим модулем.

УВАГА
Якщо esptool не бачить чип — це ще не вирок платі. Спершу картка K4: вхід у download mode вручну кнопками BOOT + EN.

5. Зберегти стан — до будь-яких змін

Перш ніж прошивати, стирати чи міняти конфігурацію — зняти повний дамп флешу. Далі — картка K2. Це єдина операція, після якої плату завжди можна повернути в початковий стан.

Дерево рішень

Що маємо	Що з цим робити
Чип відповідає, лог осмислений	робочий пристрій → K2, потім K6
Чип відповідає, лог порожній	немає прошивки → K5
Чип відповідає, boot loop	K7, потім K8
Чип не відповідає, порт є	K4 (download mode вручну)
Чип не відповідає, порту немає	K3 (кабель, драйвер, права)
K3 по живленню	ремонт, не прошивка → K13

K2. Збереження стану перед втручанням

Усе, що нижче, робиться **до** першої зміни. Після erase-flash або перепрошивки повернути початковий стан можна лише з того, що ви встигли зняти. Двадцять хвилин зараз — або незворотна втрата.

1. Фото

Обидві сторони плати, з такої відстані, щоб читалися написи на чипах. Окремо — кожен роз'єм із під'єднаними дротами, **до** від'єднання. Окремо — положення всіх перемичок і мікроперемичаків.

Знімати при доброму світлі й без спалаху: спалах засвічує лазерне маркування на чипах, а саме воно потім знадобиться.

2. Схема з'єднань

Записати, який дріт куди йде. Не «синій до плати», а: синій → GPIO4, червоний → 3V3, чорний → GND. Кольори дротів у чужих виробках не означають нічого — покладатися можна лише на записане.

3. Вимірювання

До подачі живлення, мультиметром у режимі прозвонки: опір між 3V3 і GND, між 5V і GND. Записати обидва значення.

Під живленням: реальна напруга на 3V3 і на вході. Записати. Ці чотири числа — єдина база для порівняння, якщо потім щось зміниться.

4. Дамп флешу

Головне. Обсяг флешу спершу дізнатися, інакше дамп буде неповний:

```
esptool --port /dev/ttyUSB0 flash-id
esptool --port /dev/ttyUSB0 read-flash 0 ALL dump-2026-08-26.bin
```

ALL читає рівно стільки, скільки є на чипі. Якщо esptool цього не підтримує — підставити обсяг явно: 0x400000 для 4 МБ, 0x800000 для 8 МБ.

Перевірити результат одразу: розмір файлу має точно дорівнювати обсягу флешу. Файл, менший за очікуваний, — обірваний дамп, а не «стисненіший».

5. Куди це кладеться

Один каталог на пристрій, назва — дата й що це. В середині: фото, текстовий файл із записами, дамп. Дамп без записів через місяць нічого не означає.

НЕЗВОРОТНЕ

Дамп, що лежить лише на робочому ноутбуку, — це не резервна копія. Скопіювати в друге місце **до** того, як почнете втручатися.

К3. Підключення до ПК: кабель → драйвер → порт → права

Порт не з’явився. Причина майже завжди в одному з чотирьох місць, і шукати їх треба саме в цьому порядку — від найчастішого до найрідшого.

1. Кабель

Найчастіша причина. Дешеві кабелі з комплектів до павербанків мають лише дві жили живлення і **жодних даних**. Зовні не відрізняються.

Перевірка: під’єднати цим кабелем будь-який пристрій, що точно визначається (телефон, флешку). Не визначився — кабель для заряджання, взяти інший.

2. Живлення і сам чип

Індикатор живлення на платі горить? Ні — кабель, роз’єм або плата. Роз’єм micro-USB на дешевих платах відривається разом із площадками; поворушити його при увімкненому живленні: блимає — тріщина в пайці.

3. Драйвер USB-UART мосту

Що саме стоїть на платі — видно за маркуванням меншого чипа біля роз’єма:

Маркування	Міст	Драйвер
CP2102, CP2102N	Silicon Labs	Windows: з сайту SiLabs. Linux: у ядрі
CH340, CH341	WCH	Windows: з сайту WCH. Linux: у ядрі
CH9102, CH9102F	WCH	Windows: окремий, не той, що для CH340
немає окремого чипа	native USB S3 C3	драйвер не потрібен

Windows: Диспетчер пристроїв → жовтий знак оклику означає драйвер, а не залізо.
Linux: `dmesg | tail` одразу після під’єднання показує, чи ядро взагалі побачило пристрій.

4. Права на порт (Linux)

Порт `/dev/ttyUSB0` є, але програма пише «Permission denied» — це права, а не несправність:

```
sudo usermod -aG dialout $USER
```

Далі — **перезайти в систему**. Без цього група не застосується, і здаватиметься, що команда не спрацювала.

УВАГА

Не запускати прошивку через `sudo`, щоб «обійти права». Це працює один раз і залишає по собі файли проекту, що належать `root`, — далі збирання ламається у неочевидний спосіб.

Якщо порт є, а зв'язку немає

Порт зайнятий іншою програмою: відкритий монітор, Arduino IDE, screen. Одночасно порт тримає лише один процес. Закрити все зайве і повторити.

K4. Download mode вручну (BOOT + EN)

Плата не переходить у режим прошивки сама. Це нормальна ситуація для частини плат, а не ознака несправності.

Що take download mode

Щоб прийняти прошивку, чип має стартувати не в застосунок, а в ROM-бутло-адер. Вибір робиться **в момент скидання** за станом strapping-піна.

classic **S3** Вирішує GPIO0:

- GPIO0 вільний (підтягнутий вгору) → звичайний старт застосунку;
- GPIO0 притиснутий до землі → download mode.

C3 Вирішує пара пінів: до землі притискається GPIO9, а GPIO8 при цьому має лишатися високим. Кнопка BOOT на платах C3 діє саме на GPIO9, тому послідовність нижче не змінюється.

Кнопка BOOT (іноді I00, FLASH) саме притискає GPIO0 до землі. Кнопка EN (іноді RST, RESET) — скидання.

Послідовність

Порядок обов'язковий: стан GPIO0 читається один раз, у момент відпускання скидання.

1. Натиснути і **тримати** BOOT.
2. Не відпускаючи BOOT, коротко натиснути й відпустити EN.
3. Зачекати півсекунди.
4. Відпустити BOOT.

Ознака успіху — у моніторі порту рядок виду `waiting for download`. Якщо монітор закритий, просто запустити прошивку: вона має початися без «Failed to connect».

Чому авторесет не спрацював

На платі є схема, що смикає GPIO0 і EN сигналами DTR/RTS USB-мосту. Вона не універсальна і не працює, коли:

- на GPIO0 або GPIO2 навішано зовнішню обв'язку, яка їх утримує;
- плата без такої схеми взагалі (голі модулі, частина клонів);
- живлення просідає під час скидання — конденсатори не встигають;

- драйвер мосту не керує DTR/RTS (трапляється на CH9102 у Windows).

Плати без кнопок

Голий модуль або плата без кнопок: перемичкою або пінцетом замкнути GPIO0 на GND, потім коротко замкнути EN на GND і відпустити, далі прибрати перемичку з GPIO0. Послідовність та сама.

УВАГА

classic На платах ESP32-CAM кнопки BOOT немає взагалі: GPIO0 з'єднується з GND перемичкою на самій платі. Після прошивки перемичку обов'язково зняти, інакше плата так і лишиться в download mode.

Якщо не допомогло

Понизити швидкість: --baud 115200. Спробувати інший USB-порт напряду, без хаба. Зняти всі дроти зі strapping-пінів — під час старту вони мають бути вільні.

classic Це GPIO0, GPIO2, GPIO5, GPIO12, GPIO15; **S3** GPIO0, GPIO3, GPIO45, GPIO46; **C3** GPIO2, GPIO8, GPIO9 (картка K9).

K5. Прошивка готового образу

Прошити зібраний кимось образ, не збираючи проєкт.

Три образи і їхні адреси

Повна прошивка ESP-IDF — це три файли, кожен на своїй адресі:

Файл	Що це	classic, S2	S3, C3, C6, H2	P4, C5, H4
bootloader.bin	другий бутлоадер	0x1000	0x0	0x2000
partition-table.bin	таблиця розділів	0x8000	0x8000	0x8000
застосунок .bin	сама програма	0x10000	0x10000	0x10000

НЕЗВОРОТНЕ

Різниця в адресі бутлоадера — найчастіша причина «прошилося без помилок, але плата мовчить». Команда з інструкції для ESP32 classic кладе бутлоадер S3 на 0x1000, тобто не туди. Спершу визначити чип (картка [K1](#)), потім брати адресу.

Команда

```
esptool --port /dev/ttyUSB0 --baud 460800 write-flash -z \  
  0x1000 bootloader.bin \  
  0x8000 partition-table.bin \  
  0x10000 app.bin
```

 У команді вище стоїть 0x1000 — адреса **classic i S2**. Для решти чипів перший рядок інший: див. таблицю вище. Правила «що новіше, то ближче до нуля» немає — адресу задає ROM чипа.

-z — стиснення при передачі. Уже ввімкнене; писати треба лише разом із --no-stub, де воно типово вимкнене. Не з'єднується — знизити до --baud 115200.

Якщо образ **один** файл (зібраний через merge-bin), адреса завжди 0x0, незалежно від сімейства чипа: зсуви вже всередині файлу.

```
esptool --port /dev/ttyUSB0 write-flash 0x0 merged.bin
```

Синтаксис: v5 проти v4

Команди вище — для esptool v5. Якщо у вас v4 (іде з ESP-IDF 5.x), то замість дефісів підкреслення і потрібен суфікс .py:

```
esptool.py --port /dev/ttyUSB0 write_flash -z 0x1000 bootloader.bin
```

Перевірити свою версію: `esptool version` або `esptool.py version`.

Перевірка після прошивки

Не «прошилося без помилок», а:

1. `esptool --port /dev/ttyUSB0 verify-flash 0x10000 app.bin` — звіряє вміст флешу з файлом;
2. відкрити монітор на 115200 і скинути плату кнопкою EN;
3. прочитати boot-лог (картка K6).

Прошивка вважається успішною тільки після третього пункту.

УВАГА

Дамп флешу зняти **до** прошивки, не після (картка K2). Після `write-flash` початкового вмісту вже немає.

K6. Читання boot-логу

Монітор на 115200 бод, натиснути EN. Перші рядки друкує ROM, і саме вони кажуть, чому плата поводить себе так, як поводить себе.

Перший рядок — головний

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
```

rst: — **причина останнього скидання**. Найчастіші значення для classic (повна таблиця — додаток D):

Код	Назва	Що сталося
0x1	POWERON_RESET	подано живлення або натиснуто EN. Норм
0x3	SW_RESET	скидання з коду (esp_restart)
0x5	DEEPSLEEP_RESET	прокинувся з deep sleep. Норм
0x7	TG0WDT_SYS_RESET	спрацював watchdog таймера 0
0x8	TG1WDT_SYS_RESET	спрацював watchdog таймера 1
0x9	RTCWDT_SYS_RESET	спрацював RTC watchdog
0xc	SW_CPU_RESET	скидання ядра з коду; часто після паніки
0xf	RTCWDT_BROWN_OUT_RESET	просіло живлення
0x10	RTCWDT_RTC_RESET	RTC watchdog скинув усе, разом з RTC

boot: — куди пішов чип: SPI_FAST_FLASH_BOOT — звичайний старт із флешу; DOWNLOAD_BOOT — режим прошивки: classic S3 GPIO0 притиснутий до землі, C3 GPIO9 (картка K4).

classic Саме **число** після boot: — маска рівнів на strapping-пінах: 0x01=GPIO5, 0x02=GPIO15, 0x04=GPIO4, 0x08=GPIO2, 0x10=GPIO0, 0x20=GPIO12. Отже 0x13 — норма, а **виставлений 0x20 означає GPIO12 високий**: флеш отримав 1.8 В і плата мовчить (додаток D).

ЖИВЛЕННЯ

rst:0xf — це не програмна помилка. Це живлення. Шукати треба джерело, кабель або конденсатори, а не баг у коді. Найчастіше з’являється в момент увімкнення Wi-Fi, бо саме там піковий струм.

Далі — другий бутлоадер

```
I (29) boot: ESP-IDF v6.0.2 2nd stage bootloader
I (33) boot.esp32: SPI Flash Size : 4MB
I (52) boot: Partition Table:
I (56) boot: ## Label          Usage          Type ST Offset   Length
```

Три речі, які тут читаються безкоштовно: **версія IDF**, якою зібрано прошивку; **обсяг флешу**, який бачить бутлоадер; **уся таблиця розділів** із адресами. Це готова відповідь на «а що там усередині» — без розбирання дампа.

Число в дужках — мілісекунди від старту. Стрибок у цьому числі показує, де саме прошивка задумалася.

Що означає тиша

Видно	Що це
нечитний набір символів	не 115200. Читається на 74880 — це ESP8266
перші рядки, далі тиша	застосунок стартував і не логує. Це може бути норма
ті самі рядки по колу	boot loop → картка K7
invalid header: 0x...	застосунку немає або адреса не та → картка K5
зовсім порожньо	немає порту чи живлення → картка K3 . classic Або GPIO15 притиснутий до землі: він глушить лог ROM, плата при цьому справна

УВАГА
Рядки ROM завжди йдуть на **115200** — і незалежно від швидкості застосунку, і незалежно від кварцу на платі. Якщо ROM видно, а далі каша — швидкість застосунку інша, і це нормально.

Якщо ж на 115200 не читається нічого, а на **74880** з’являється осмислений текст — у вас **ESP8266**, а не ESP32: у його ROM швидкість виходить саме такою.

K7. Guru Meditation і backtrace за 60 секунд

```
Guru Meditation Error: Core 0 panic'ed (LoadProhibited). Exception was unhandled.
Core 0 register dump:
PC      : 0x400d1234  PS      : 0x00060730  A0      : 0x800d5678
...
Backtrace: 0x400d1234:0x3ffb1f30 0x400d5678:0x3ffb1f50
```

Це не «плата зламалася». Це звіт про те, де саме програма померла.

1. Прочитати причину

Причина	Що сталося	Куди дивитися
LoadProhibited	читання за недійсною адресою	покажчик NULL або звільнений
StoreProhibited	запис за недійсною адресою	те саме, але на запис
InstrFetchProhibited	перехід на недійсну адресу	зіпсований покажчик на функцію
IllegalInstruction	виконання не-коду	пошкоджений стек, переповнення
LoadStoreAlignment	невирівняний доступ	читання 32 біт з непарної адреси
Interrupt wdt timeout	ISR або critical section триває задовго	код у перериванні

Task watchdog got triggered — **не паніка**: це окреме повідомлення, і система при ньому лишається живою. Воно саме називає винуватця в рядку `Tasks currently running`. Причина майже завжди — цикл без `vTaskDelay`.

Найчастіші дві — `LoadProhibited` і `StoreProhibited`, і обидві майже завжди означають одне: **розмінування покажчика, який не той, що ви думаєте**. Найчастіше — результат `malloc`, який не перевірили.

2. Розшифрувати backtrace

Backtrace — це ланцюжок адрес. Сам по собі він нечитний; його треба перекласти в назви функцій і номери рядків. `idf.py monitor` робить це автоматично, якщо запущений з каталогу того самого проекту.

Вручну, коли лог знято з чужого пристрою і є `.elf`:

```
xtensa-esp32-elf-addr2line -pfiaC -e build/app.elf 0x400d1234 0x400d5678
```

Інструмент **свій для кожної архітектури**: **S3** — xtensa-esp32s3-elf-addr2line, **C3** та інші RISC-V — riscv32-esp-elf-addr2line.

Читати **знизу вгору**: нижні кадри — хто викликав, верхній — де впало.

3. Якщо .elf немає

Без .elf адреси перекласти нема в що: символів у прошивці немає. Лишається причина паніки і PC — цього досить, щоб відрізнити збій у власному коді від збою в стеку Wi-Fi, але не досить для рядка.

УВАГА

.elf того самого збирання, що й .bin, — єдине, що робить backtrace читним. Зберігати .elf разом із кожною прошивкою, яку віддали в поле. Перезібрати «такий самий» пізніше не вийде: адреси зсунуться.

4. Коли паніка не перша

Після паніки чип скидається — і в логу з'являється rst:0xc. Якщо причина паніки лишилася, це стає boot loop: паніка → скидання → паніка. Дивитися треба **найперший** дамп після подачі живлення, а не сотий.

Coredump у флеші (якщо ввімкнено в menuconfig) зберігає стан усіх задач, а не лише тієї, що впала: idf.py coredump-info.

К8. Топ-15 несправностей: симптом → причина → дія

Порядок — за частотою в житті, а не за складністю. Повний покажчик з посиланнями на розділи — додаток В.

#	Симптом	Найчастіша причина	Що робити
1	Порт не з'являється	кабель тільки для заряджання	інший кабель → К3
2	«Failed to connect»	плата не в download mode	BOOT + EN → К4
3	Прошилося, плата мовчить	адреса бутлоадера не та для цього чипа	звірити чип і адресу → К5
4	rst:0xf у логу	просідає живлення (brownout)	джерело, кабель, конденсатор → К13
5	Перезавантаження при вмиканні Wi-Fi	джерело не тягне піковий струм	1 A+, короткий кабель, 470 мкФ по живленню
6	Boot loop	паніка в застосунку	перший дамп після живлення → К7
7	Каша в моніторі	не та швидкість	ROM завжди 115200; 74880 = ESP8266 → К6
8	I ² C-пристрій не знаходиться	немає підтягування або земля не спільна	4.7 кОм на SDA і SCL, спільний GND
9	ADC читає дурницю	classic S2 S3 ADC2 не працює при Wi-Fi	перенести на ADC1
10	GPIO поводить дивно при старті	це strapping-пін	інший пін → К11
11	classic GPIO 6–11 нічого не роблять	вони зайняті флешем	не чіпати ці піни взагалі
12	Працює від USB, не працює від БЖ	немає спільної землі або просадка	з'єднати GND, зміряти під навантаженням
13	Сенсор гріється, значення плывуть	подано 5 В на 3.3-вольтовий вхід	звірити datasheet, конвертер рівнів

#	Симптом	Найчастіша причина	Що робити
14	Wi-Fi бачить мережу, не під'єднується	пароль, канал 12–13 або 5 ГГц	звірити SSID, канал; ESP32 не бачить 5 ГГц
15	Після erase-flash нічого немає	стерто разом із калібруванням і NVS	залити дамп → K2; надалі дампи до

Три правила, що економлять години

Спершу живлення, потім код. Більшість «плаваючих багів» — просадка напруги. Зміряти під навантаженням дешевше, ніж читати код.

Одна зміна за раз. Змінили дві речі — не знаєте, яка допомогла.

Мінімальний тест окремо. Незнайомий модуль перевіряється на голій платі з трьома дротами, а не всередині проекту з двадцятьма.

УВАГА

Симптом «іноді працює» майже ніколи не є програмним. Дивитися в такому порядку: живлення → пайка → земля → рівні → таймінги → і лише потім код.

К9. Піни: обмеження і призначення

Повні пінаут-таблиці плат — додаток А. Тут — те, через що плати не стартують і піни не працюють.

ESP32 classic (WROOM-32, DevKitC)

Піни	Що з ними
6, 7, 8, 9, 10, 11	зайняті флешем. Не використовувати ніколи
34, 35, 36, 37, 38, 39	тільки вхід. Немає виходу і немає підтягування
0, 2, 5, 12, 15	strapping: стан при старті визначає режим завантаження
1 (TX), 3 (RX)	консоль UART0. Зайняті логом і прошивкою
12 (MTDI)	вгору при старті → флеш живиться 1.8 В. На тривольтовому (майже всі модулі) плата не стартує
25, 26	єдині DAC-виходи
32–39	ADC1 — працює завжди
0, 2, 4, 12–15, 25–27	ADC2 — не працює при увімкненому Wi-Fi

Вільні без застережень: 4, 13, 14, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33.

Поширена домовленість (не апаратна прив'язка): I²C — SDA 21, SCL 22; SPI — MOSI 23, MISO 19, SCK 18, CS 5. Апаратні піни — додаток А.

ESP32-S3 (WROOM-1, DevKitC-1)

Піни	Що з ними
26–32	флеш і PSRAM. Не чіпати
33–37	додатково зайняті на модулях з Octal PSRAM (N16R8)
0, 3, 45, 46	strapping
19, 20	native USB (D–, D+)
1–10	ADC1
11–20	ADC2

УВАГА

S3 Вхід у бутлоадер: $\text{GPIO0} = 0$, а GPIO46 при цьому **низький або вільний**. Підтягнутий угору GPIO46 у download mode не пускає. Якщо на цих пінах щось висить — знімати.

Увага: на C3 правило **протилежне** — там другий пін має бути високим.

ESP32-C3 (MINI-1, SuperMini)

Піни	Що з ними
12–17	флеш. Не чіпати
2, 8, 9	strapping
18, 19	USB-Serial-JTAG. Перевизначили — втратили налагодження
0–4	ADC1
5	ADC2 — не використовувати , апаратна вада

Вхід у бутлоадер: GPIO9 притиснутий до землі при скиданні, GPIO8 при цьому має бути високим. Комбінація $\text{GPIO8} = 0$ і $\text{GPIO9} = 0$ недійсна.

Правило, що економить найбільше часу

Пінаут плати **важливіший за пінаут чипа**. Виробник плати міг вивести не всі піни, підписати їх по-своєму, повісити світлодіод або підтягувальний резистор. Те, що пін є на схемі чипа, не означає, що він вільний на цій платі.

НЕЗВОРОТНЕ

Пін із позначкою «тільки вхід» не стає виходом від налаштування в коді. Спроба керувати з нього навантаженням — це або нічого, або пошкоджений пін.

K10. Шпаргалка команд

Синтаксис esptool v5 (дефіси, без .py). Для v4 — підкреслення і суфікс .py: esptool.py write_flash. Перевірити своє: esptool version.

esptool

```
# що за чип і ревізія — у шапці з'єднання перед будь-якою командою
esptool --port /dev/ttyUSB0 flash-id          # обсяг і виробник флешу
esptool --port /dev/ttyUSB0 read-flash 0 ALL dump.bin    # повний дамп
esptool --port /dev/ttyUSB0 write-flash -z 0x10000 app.bin # залити
esptool --port /dev/ttyUSB0 verify-flash 0x10000 app.bin # звірити
esptool --port /dev/ttyUSB0 erase-flash          # стерти все (Δ див. K2)
esptool --port /dev/ttyUSB0 --baud 115200 ... # повільніше, надійніше
esptool --chip esp32 merge-bin -o all.bin --flash-mode dio \
0x1000 boot.bin 0x8000 pt.bin 0x10000 app.bin # --chip обов'язковий; 0x1000 →
classic/S2, інші чипи — див. таблицю
```

idf.py

```
idf.py create-project my-project    # новий проєкт (назва латиницею)
idf.py set-target esp32s3           # Δ стирає sdkconfig
idf.py menuconfig                   # налаштування
idf.py build                         # зібрати
idf.py -p /dev/ttyUSB0 flash        # залити
idf.py -p /dev/ttyUSB0 monitor      # монітор з розшифровкою backtrace
idf.py -p /dev/ttyUSB0 flash monitor # найчастіша команда
idf.py fullclean                    # коли збирання поводитьсь незрозуміло
idf.py size                         # скільки зайнято флешу і RAM
idf.py coredump-info                # розбір coredump із флешу
idf.py merge-bin -o all.bin          # один образ; адреси з конфігурації проєкту
```

Є проєкт — idf.py merge-bin (адрес набирати не треба). Є лише .bin-файли — esptool --chip ... merge-bin з адресами вище.

Монітор

idf.py monitor: вийти — Ctrl+J. Скинути плату — Ctrl+T, потім Ctrl+R.

```
minicom -D /dev/ttyUSB0 -b 115200    # вийти: Ctrl+A, потім X
screen /dev/ttyUSB0 115200            # вийти: Ctrl+A, потім K
picocom -b 115200 /dev/ttyUSB0        # вийти: Ctrl+A, потім Ctrl+X
```

Порт

```
ls /dev/ttyUSB* /dev/ttyACM*          # Linux: що є
dmesg | tail                           # що ядро побачило при під'єднанні
sudo usermod -aG dialout $USER         # права; далі ПЕРЕЗАЙТИ в систему
lsof /dev/ttyUSB0                      # хто тримає порт
```

/dev/ttyUSB* — зовнішній міст (CP2102, CH340). /dev/ttyACM* — native USB S3 C3 .

Адреси

Що	classic, S2	S3, C3, C6, H2	P4, C5, H4
bootloader	0x1000	0x0	0x2000
partition table	0x8000	0x8000	0x8000
застосунок	0x10000	0x10000	0x10000
зібраний merge-bin	0x0	0x0	0x0

K11. Ніколи

Кожен пункт нижче — незворотний або майже незворотний. Прочитати до, а не після. Решта помилок лікується.

НЕЗВОРОТНЕ

Не палити eFuse наосліп. esepfuse записує біти лише в один бік — з 0 у 1, назад ніколи. Помилковий біт може назавжди відібрати JTAG, download mode або можливість перепрошивки. Не запускати esepfuse burn-*, поки не зрозуміло дослівно, що робить кожен її аргумент.

Остання перепона — набрати слово BURN у відповідь на запит. Прапорець --do-not-confirm її знімає; у чужому скрипті він означає, що плата згорить без питання.

НЕЗВОРОТНЕ

Не вмикати Flash Encryption і Secure Boot «щоб подивитися». В release-режимі це односторонні двері: чип перестає приймати непідписані прошивки, а флеш стає нечитним поза цим конкретним чипом. Дамп, знятий до цього, ще можна залити; знятий після — ні. Пробувати тільки на платі, яку не шкода.

НЕЗВОРОТНЕ

Не стирати флеш без дампа. erase-flash знищує NVS разом із калібруванням радіо, збереженими креденцями і конфігурацією конкретного пристрою. Це не завжди відновлюється перезбиранням прошивки. Спершу картка [K2](#).

НЕЗВОРОТНЕ

Не подавати 5 В на GPIO. Логіка ESP32 — 3.3 В. П'ять вольтів вбивають пін, іноді весь порт, іноді чип. Найчастіші джерела: HC-SR04 (вихід ЕСНО), релейні модулі з VCC 5 В, «сумісні з Arduino» датчики. Дільник або конвертер рівнів — обов'язково.

Виняток єдиний: пін 5V/VIN — це вхід стабілізатора, туди 5 В можна.

НЕЗВОРОТНЕ

classic Не чіпати GPIO 6, 7, 8, 9, 10, 11. Вони з'єднані з мікросхемою флешу на самому модулі. Будь-яка спроба їх використати підвищує чип або псує вміст флешу. На пінауті вони часто виведені — це не означає, що вони вільні.

Не незворотне, але дороге

Не паяти під живленням. На жалі незаземленого паяльника може бути наведений потенціал відносно землі плати, і цього досить, щоб пробити вхід. Живлення від'єднується завжди.

Не під'єднувати земляний щуп осцилографа абиде. У приладах із мережевим живленням земля щупа — це земля розетки. Одне невдале дотикання коротить живлення схеми через прилад.

Не тримати strapping-піни навантаженими під час старту. classic GPIO0, GPIO2, GPIO5, GPIO12, GPIO15. Підтягнутий не в той бік GPIO12 перемикає флеш на 1.8 В, і тривольтовий флеш — а він майже скрізь — не стартує зовсім.

Не міняти дві речі одночасно. Не незворотно, але з'їдає більше часу, ніж усе перелічене вище разом.

К12. Польовий комплект і закупівля

Актуальні ціни, наявність і конкретні магазини — у датованому вкладишні ринку, не тут. Тут — що саме шукати і чому.

Мінімум, без якого не працює

Позиція	На що дивитися
Плата ESP32 DevKit	38 пінів, USB-UART CP2102 або CH9102
Кабель USB data	перевірити на телефоні перед виїздом
Мультиметр	прозвонка зі звуком, вимірювання постійної напруги
Паяльник із терморегулятором	60 Вт, жало «скіс» 2–3 мм
Припій і флюс	0.5–0.8 мм зі свинцем; флюс-гель у шприці
Макетка + Dupont	усі три види: тато-тато, тато-мама, мама-мама
USB-хаб із власним живленням	знімає половину «незрозумілих глюків»

Те, що окупається на другому пристрої

Позиція	Навіщо
Логічний аналізатор 8 каналів	бачити I ² C і SPI наживо замість гадання
Лабораторний БЖ з обмеженням струму	обмеження струму рятує плату при КЗ
USB-тестер напруги і струму	ловить просадки і рахує споживання
Термофен	зняти модуль, не відірвавши площадки
Пінцет, бокорізи, оплітка	зняти зайвий припій, а не розмазати
Набір резисторів і конденсаторів	4.7 кОм і 10 кОм, 100 нФ і 470 мкФ — найчастіші

Витратне, яке завжди закінчується не вчасно

Гребінки (male і female), термоусадка кількох діаметрів, дроти 22–26 AWG трьох кольорів, ізоляційна стрічка, спирт і безворсові серветки, запасні кабелі USB.

ЗАКУПІВЛЯ

Три позиції, на яких не варто економити. Мультиметр — дешевий бреше на межах вимірювання. Паяльник — без терморегулятора або не гріє, або палить площадки. Кабель — половина «плата не визначається» саме тут.

Три позиції, де дешеве нормальне. Макетка, Dupont-дроти, гребінки. Брати з запасом: вони губляться і псуються.

Що покласти в сумку, яка їде в поле

Плата з уже залитою робочою прошивкою · перевірений кабель · павербанк · мультиметр · викрутки · ніж · ізоляційна стрічка · термоусадка · запасні гребінки · роздруковані й заламіновані картки **K1–K15**.

Ноутбук із **офлайн-комплектom** (додаток F): встановлений тулчейн, завантажена документація, закешовані бібліотеки. У полі інтернету немає, і саме там він потрібен найбільше.

K13. Живлення: пошук несправності

Більшість «плаваючих багів» — це живлення. Симптоми виглядають програмними, тому люди читають код. Ця картка — перше, що робиться замість цього.

Ознаки, що це живлення

- `rst:0xf` у boot-лозі — **гарантовано живлення**, не код
- перезавантаження саме при вмиканні Wi-Fi, двигуна, реле
- «іноді працює», залежить від кабелю чи розетки
- працює від USB, не працює від зовнішнього джерела
- `MD5 of file does not match` при прошивці
- датчики віддають шум

Чотири вимірювання

Мультиметр, у цьому порядку.

#	Що міряти	Норма	Якщо ні
1	3V3–GND, прозвонка, живлення вимкнене	не дзвенить	K3 — не вмикати, шукати
2	Вхідна напруга на роз'ємі	5 В $\pm 5\%$ (від USB)	джерело, кабель, роз'єм
3	3V3 на холостому ходу	3.2–3.4 В	стабілізатор
4	3V3 під навантаженням, Wi-Fi увімкнений	не нижче 3.0 В	 причина знайдена

Четвертий рядок — головний. Три перші часто в нормі, а провал видно лише під навантаженням.

Порядок лікування, від дешевого

1. **Інший кабель.** Короткий, товстий, завідомо data. Розв'язує половину випадків.
2. **Порт напряму, без хаба.** Або хаб із власним живленням.
3. **Конденсатор 470 мкФ** між 3V3 і GND поруч із модулем. Копійки, ставиться за хвилину.
4. **Керамічний 100 нФ** біля кожної мікросхеми.
5. **Окреме джерело від 1 А.** Не 500 мА: платити треба за піки.

- 6. **Живлення 3.3 В напряду**, минаючи бортовий стабілізатор — коли він слабкий на клоні.
- 7. **Знизити пікове споживання**: потужність передавача, частота ядра.

Скільки треба струму

Джерело для ESP32 з Wi-Fi — **щонайменше 1 А**, навіть якщо середнє споживання сто міліампер. Передавач вмикається імпульсами, і платить джерело саме за них.

Двигун, серво, реле, підсвітка TFT — **окреме живлення**, спільна земля.

НЕЗВОРОТНЕ

Не вимикати детектор brownout у menuconfig, «щоб не заважав». Перезавантаження зникнуть, захист теж — і замість чесного скидання прийдуть пошкоджені NVS і збої, які неможливо пояснити.

Це діагностика, а не перешкода.

Автономне живлення

- **Дільник вимірювання напруги вмикати транзистором** — інакше він стає головним споживачем уві сні
- **Buck-boost, не LDO** — LDO відсікає половину ємності 18650
- **Плата розробки для сну не годиться**: USB-міст, стабілізатор і світлодіод не сплять. 20 mA замість 20 мкА — це вони
- **Не заряджати літій нижче 0 °C**

Швидка таблиця

Симптом	Найімовірніше
rst:0xf	кабель, хаб, немає конденсатора
Перезавантаження при Wi-Fi	джерело не тягне піків
Перезавантаження при двигуні	немає окремого живлення
Стабілізатор гарячий	перезавантаження або пробій
Працює від USB, не від БЖ	немає спільної землі
Сон 20 mA замість 20 мкА	плата розробки, а не чип

K14. Сумісність логічних рівнів

Логіка ESP32 — 3.3 В. Половина модулів «для Arduino» — 5 В. Зовні роз’єми однакові.

НЕЗВОРОТНЕ

5 В на GPIO вбивають пін, іноді чип. Єдиний виняток — пін 5V/VIN: це вхід стабілізатора, а не вхід у чип.

Живлення і рівні — різні питання

Модуль може живитися від 5 В і мати виводи на 3.3 В. Або живитися від 3.3 В і не сприймати 3.3 В як одиницю.

Перевіряти рівні на сигнальних виводах у datasheet, а не вгадувати за написом живлення.

Що на платі модуля	Живлення	Сигнали
Немає стабілізатора	3.3 В	3.3 В — прямо
Є стабілізатор 5→3.3	5 В	перевіряти
Є стабілізатор і конвертер	5 В	5 В — потрібен конвертер

Два напрямки

ESP32 → пристрій на 5 В. Часто працює прямо: більшість 5-вольтових входів бере 3.3 В за одиницю. Не завжди — звірятися.

Пристрій на 5 В → ESP32. Зниження обов’язкове.

Чим знижувати

Дільник напруги — два резистори. Дешево.

5 В — [10 кОм] — [20 кОм] — GND
 └─ до GPIO (≈3.3 В)

Годиться: повільні однонаправлені сигнали. **Не годиться:** I²C (двонаправлений), швидкий SPI (завалює фронти).

Модуль конвертера рівнів на польових транзисторах — двонаправлений, працює з I²C, коштує копійки. LV до 3.3 В, HV до 5 В, землі з’єднані.

Тримати пару в шухляді: потреба виникає раптово.

Часті винуватці 5 В

Модуль	Де 5 В
HC-SR04	вивід ЕСНО
Релейні модулі	вхід керування і живлення котушки
Дисплеї «для Arduino»	сигнальні лінії
MAX485, TJA1050, MCP2551	вивід Rx трансивера
Логіка 74НС при живленні 5 В	усі виходи

Спільна земля

НЕЗВОРОТНЕ

Усі з'єднані пристрої мусять мати спільну землю — крім тих, де стоїть гальванічна розв'язка (оптопара, ізолюваний трансивер).

Симптом відсутньої землі: живлення є, обміну немає, або обмін із випадковими помилками.

Зворотна помилка: з'єднати землі там, де розв'язка стоїть саме для того, щоб їх не з'єднувати. Тоді ізоляція перестає існувати, а схема виглядає робочою.

Перевірка за десять секунд

Мультиметром, до з'єднання:

1. Напруга на виводі живлення модуля — та, що очікуєте?
2. Напруга на сигнальному виводі в спокої — 3.3 чи 5?
3. Прозвонка землі між модулем і платою.

Три вимірювання дешевші за спалений пін.

K15. Чек-лист серійної прошивки

Прошити партію так, щоб через півроку можна було відповісти, що в кожному пристрої.

Підготовка (один раз на партію)

- ☐ Зібрано **один образ** через `merge-bin` — одна команда, одна адреса, нема чого переплутати (без проекту — K10)
- ☐ `--flash-mode`, `--flash-size`, `--flash-freq` збігаються з `sdkconfig` проекту
- ☐ Образ перевірено на одній платі повністю: прошивка → `boot-лог` → функціональний тест
- ☐ Збережено: `.bin`, `.elf` **того самого збирання**, `sdkconfig`, версія ESP-IDF, версії компонентів
- ☐ Заведено журнал партії
- ☐ Живлення для прошивки — окреме, з запасом, не від ноутбука

```
idf.py merge-bin -o vyrib-v1.4.bin      # адреси — з конфігурації проекту
```

На кожную плату

☐ Крок	Чим
☐ 1. Прошити	<code>write-flash -z 0x0 vyrib-v1.4.bin</code>
☐ 2. Звірити	<code>verify-flash 0x0 vyrib-v1.4.bin</code>
☐ 3. Записати унікальне	NVS: номер, ключі, калібрування
☐ 4. Зняти MAC	єдиний надійний ідентифікатор плати
☐ 5. Прочитати <code>boot-лог</code>	пристрій справді стартує
☐ 6. Функціональна перевірка	те, заради чого виріб існує
☐ 7. Маркувати	номер, версія, дата — на видний бік
☐ 8. Записати рядок у журнал	

Кроки 2, 5 і 6 — обов'язкові, не опції. «Прошилося без помилок» ловить не все: крок 6 виявляє справний образ на платі з непропаяним модулем.

Журнал партії

№	MAC	Версія	Дата	Контроль	Примітка
0041	A0:B7:...:14	v1.4	2026-08-26	OK	
0043	A0:B7:...:31	v1.4	2026-08-26	брак	не стартує

Спільне і унікальне

НЕЗВОРОТНЕ

Спільний образ ≠ спільна конфігурація.

Залити двадцятьом платам однаковий NVS означає двадцять пристроїв з однаковою особистістю: однаковий серійний номер, однакові ключі. Виявиться це пізно, у полі, і виглядатиме як містика — два пристрої «крадуть» з'єднання одне в одного.

Один ключ на всі також означає, що захоплення одного компрометує всі.

Якщо треба лише відрізнити пристрої — **беріть MAC**: він унікальний від заводу і не потребує нічого.

```
nvs_partition_gen.py generate config-0042.csv nvs-0042.bin 0x6000
esptool --port PORT write-flash 0x9000 nvs-0042.bin
```

Якщо плата не прошивається

Не зупиняти партію. Відкласти, позначити, продовжити з рештою.

Найчастіші причини саме в партії: непропаяний модуль, замикання при монтажі, плата іншої ревізії в тій самій коробці.

Прискорення і журнал партії

Кілька портів паралельно, оснастка з того pins, ведення журналу і відновлення після браку — розділ 21. Тут лише те, що робиться руками над кожною платою.

Дві дрібниці, що економлять найбільше: скрипт із `set -e` (інакше збій губиться в потоці виводу) і звертання до порту через `/dev/serial/by-id/`, а не `ttyUSB0` — номери плутаються після кожного перевикання.

Незворотне

НЕЗВОРОТНЕ

- `erase-flash` — тільки після дампа (картка K2)

- `espefuse burn-*` — не запускати, доки не зрозуміло дослівно, що робить кожен аргумент
- Flash Encryption і Secure Boot у `release` — незворотні. На партію — лише після випробування на платі, яку не шкода

Частина І. Платформа

Що це за кремній, чим сімейства різняться і коли ESP32 — не той вибір.

01. Що таке ESP32 і коли його брати

ESP32 — сімейство мікроконтролерів із вбудованим радіо, яке зробила китайська компанія Espressif. Головна причина його поширеності проста: це чи не єдиний спосіб отримати керований процесор **разом із Wi-Fi** за ціну, порівнянну з ціною самого мікроконтролера.

Для програміста, який приходить із великих систем, це виглядає дивно: щоб під'єднати щось до мережі, зазвичай треба щонайменше одноплатний комп'ютер. Тут — чип за кілька доларів, який стартує за мілісекунди, споживає мікроампери уві сні й не має операційної системи, яку треба адмініструвати.

Три рівні, які постійно плутають

Це перше джерело непорозуміння, і його варто рознести одразу.

Кристал (SoC). Власне чип: ESP32, ESP32-S3, ESP32-C3. Ядро, пам'ять, периферія, радіо. Окремо його ніхто не купує.

Модуль. Кристал плюс кварц, флеш-пам'ять, антена і обв'язка, залиті в екранований корпус із металевою кришкою: ESP32-WR00M-32, ESP32-S3-WR00M-1, ESP32-C3-MINI-1. Модуль уже сертифікований і готовий до монтажу на власну плату.

Плата розробки. Модуль плюс USB-роз'єм, міст USB-UART, стабілізатор, кнопки і гребінки: DevKitC, DevKit V1, SuperMini. Те, що лежить у вас на столі.

Практична цінність цього поділу: **напис на металевій кришці модуля — головне джерело істини** про те, що у вас у руках (розділ 23). Плата може називатися як завгодно і продаватися ким завгодно; модуль підписаний виробником кристала.

І другий наслідок: коли ви робите власний виріб, ви ставите на плату **модуль**, а не кристал. Це знімає з вас розводку радіочастотної частини, узгодження антени і сертифікацію — три речі, кожна з яких складніша за решту виробу.

Що дає ESP32

Радіо на кристалі. Wi-Fi і Bluetooth без зовнішніх мікросхем. Для більшості сімейств — обидва.

Достатньо потужне ядро. 160–240 МГц і сотні кілобайтів RAM — це на порядок більше, ніж у класичних 8-бітних мікроконтролерів. Тут вміщається стек TCP/IP, TLS, веб-сервер і ще лишається місце на власну логіку.

FreeRTOS із коробки. Операційна система реального часу вже вбудована у фреймворк: задачі, черги, семафори доступні одразу, без окремого портування (розділ 31).

Багата периферія. UART, I²C, SPI, I²S, ADC, DAC, PWM, CAN, апаратний таймер, апаратний генератор імпульсів. Плюс матриця GPIO, яка дозволяє вивести майже будь-який сигнал майже на будь-який пін (розділ 04).

Глибокий сон. Одиниці мікроампер у deep sleep — достатньо, щоб пристрій жив на батарейках місяцями (розділ 06).

Ціна і доступність. Плата розробки коштує менше за обід. Це змінює підхід: плату не шкода спалити, і можна тримати запас.

Де ESP32 недоречний

Чесний перелік — це те, що відрізняє довідник від реклами.

Жорсткий реальний час із гарантіями. Радіостек виконується на тому самому чипі й іноді забирає керування. Якщо вам потрібна гарантована реакція за фіксовані мікросекунди — беріть STM32 або окремий чип на критичну частину.

Функціональна безпека і сертифікація. Медичне, авіаційне, промислове обладнання з вимогами сертифікації. ESP32 туди не йде.

Аналогові вимірювання високої точності. Вбудований ADC нелінійний і шумний; поруч працює радіо. Потрібна точність — зовнішній ADC по SPI (розділ 33).

Дуже низьке споживання в активному режимі. У сні ESP32 хороший, у роботі з радіо — ні. Для пристрою, що має роками жити на одній батарейці й при цьому постійно щось робити, є ефективніші платформи.

Задачі, де радіо не потрібне. Якщо зв'язку не треба, ESP32 — це переплата за складність. Простіший мікроконтролер буде дешевшим, передбачуванішим і споживатиме менше.

Порівняння з тим, з чим його порівнюють

	ESP32	AVR (Arduino Uno)	STM32	RP2040	Raspberry Pi
Радіо на борту	так	ні	ні (крім WB/WL)	ні (крім W)	так
Частота	160–240 МГц	16 МГц	24 МГц – сотні МГц	133 МГц	понад 1 ГГц

	ESP32	AVR (Arduino Uno)	STM32	RP2040	Raspberry Pi
RAM	сотні КБ	2 КБ	десятки– сотні КБ	264 КБ	гігабайти
ОС	FreeRTOS	немає	немає або RTOS	немає або RTOS	Linux
Старт	мілісекун- ди	миттєво	миттєво	миттєво	десятки се- кунд
Сон	одиниці мкА	мкА	мкА	мА	не перед- бачено
Реальний час	посере- дньо	добре	дуже до- бре	добре	погано
Ціна плати	низька	низька	середня	низька	висока

Як це читати практично.

Проти AVR. ESP32 переважає в усьому, крім простоти й передбачуваності. Arduino Uno досі має сенс там, де задача тривіальна і цінується те, що нічого несподіваного не станеться.

Проти STM32. STM32 кращий у реальному часі, у різноманітті периферії і в промисловій доступності на роки вперед. ESP32 кращий у радіо і в швидкості початку роботи. Часто правильна відповідь — обидва: STM32 керує процесом, ESP32 стоїть збоку і забезпечує зв'язок (розділ 57).

Проти RP2040. RP2040 має PIO — програмовані блоки вводу-виводу, які роблять неможливе можливим у нестандартних протоколах. ESP32 має радіо. Вибір визначається тим, що з двох потрібніше.

Проти Raspberry Pi. Це різні класи. Pi — комп'ютер із Linux: файлова система, що псується від зникнення живлення, десятки секунд на завантаження, споживання у ватах. ESP32 — мікроконтролер: стартує миттєво, живе на батарейці, не має чого ламати при вимиканні.

УВАГА

Найчастіша помилка вибору — узяти Raspberry Pi туди, де потрібен мікроконтролер. Пристрій, який має стояти без обслуговування і переживати зникнення живлення, на Linux потребує уваги до карти пам'яті, файлової системи і коректного завершення роботи. Мікроконтролер цього класу проблем не має взагалі.

ESP32 як допоміжний контролер

Окрема і часто найправильніша роль: ESP32 не керує системою, а стоїть **збоку** від неї.

Основний контролер робить свою роботу — керує механізмом, читає датчики, дотримується таймінгів. ESP32 підключений до нього по UART або CAN і відповідає лише за зв'язок: віддає телеметрію, приймає команди, оновлює себе по повітрю.

Чому так часто краще:

- проблеми з радіо не впливають на основну логіку;
- основний контролер можна взяти той, що краще підходить до задачі;
- відповідальність розділена, і кожную частину можна тестувати окремо;
- ESP32 можна перепрошити або замінити, не чіпаючи основну систему.

Це схема, навколо якої побудований розділ [57](#), і вона згадується в книзі далі не раз.

Де жити: документація і спільнота

ESP-IDF Programming Guide — основна документація фреймворку. Має версії: дивіться саме ту, що відповідає вашій (розділ [11](#)). Найчастіша причина «в документації написано, а не працює» — відкрита сторінка іншої версії.

Datasheet сімейства — електричні параметри, межі, розводка. Те, що треба відкривати перед проектуванням плати.

Technical Reference Manual (TRM) — опис регістрів і периферії на низькому рівні. Товстий документ, який читають не підряд, а за покажчиком, коли треба зрозуміти, чому периферія поводить себе саме так.

Приклади в самому ESP-IDF — каталог `examples/`. Це найкорисніший і найменш використовуваний ресурс: там є робочий приклад майже на кожную периферію, і він точно відповідає вашій версії фреймворку.

Форум і GitHub Issues Espressif — там відповідають розробники, і багато відповідей ніде більше не існує.

ЗАКУПІВЛЯ

Офлайн-комплект варто зібрати заздалегідь: Programming Guide, datasheet, TRM, встановлений тулчейн і закешовані бібліотеки (додаток F). У полі інтернету немає, і саме там документація потрібна найбільше.

Що з цього треба запам'ятати

Кристал, модуль і плата — три різні речі; істина написана на модулі.

ESP32 беруть заради радіо. Якщо радіо не потрібне — це переплата за складність. Він поганий у жорсткому реальному часі й у точних аналогових вимірюваннях; це лікується поділом задачі між двома чипами.

Роль допоміжного контролера збоку від основної системи — часто найправильніша, а не запасна.

02. Лінійка чипів і правило фокуса

«ESP32» — не один чип, а сімейство, що росте вже років десять. Плутанина тут коштує реальних грошей: код, написаний під один чип, не завжди працює на іншому, а плата, куплена за назвою «ESP32», може виявитися чимось зовсім іншим.

Правило фокуса

Довідник не описує рівно всі сімейства однаково детально — це зробило б його вдвічі товщим і вдвічі менш корисним. Замість цього:

Детально: ESP32 classic і ESP32-S3. Перший — те, що найчастіше лежить у шухляді й продається на кожному кроці. Другий — те, що варто брати для нового проєкту.

Коротко: ESP32-C3 як «дешевий малий» — коли треба недорого, компактно і без надмірностей.

Оглядово: S2, C6, H2 — там, де вони дають щось, чого немає в інших.

Рядком у таблиці: C2, C5, C61, H4, P4 — щоб ви знали, що вони є, і не вважали незнайому назву помилкою.

Кожне правило, що стосується не всієї лінійки, позначене областю дії: classic

S3 C3 . Без позначки правило стосується всіх.

Зведена таблиця

	ESP32	S2	S3	C3	C6	H2
Ядро	Xtensa LX6	Xtensa LX7	Xtensa LX7	RISC-V	RISC-V	RISC-V
Ядер	2	1	2	1	1	1
Частота, МГц	240	240	240	160	160	96
SRAM, КБ	520	320	512	400	512	320
PSRAM	так	так	так	ні	ні	ні
Wi-Fi	так	так	так	так	Wi-Fi 6	ні
BT Classic	так	ні	ні	ні	ні	ні

	ESP32	S2	S3	C3	C6	H2
BLE	так	ні	так	так	так	так
802.15.4	ні	ні	ні	ні	так	так
USB	ні	OTG	OTG + JTAG	Serial- JTAG	Serial- JTAG	Serial- JTAG

Уся таблиця звірена з першоджерелами: ядра, радіо, PSRAM, Wi-Fi 6 і USB — із заголовками можливостей ESP-IDF (`soc_caps.h`); обсяги SRAM і частоти — з datasheet сімейств. Подробиці звірки — `docs/fakty.md` у репозиторії.

УВАГА

Обсяг SRAM у таблиці — це фізична пам'ять чипа, а не та, яку отримає ваш застосунок. Різниця більша, ніж здається.

На ESP32 classic із 520 КБ як звичайні дані (DRAM) адресуються близько 328 КБ — решта доступна лише як пам'ять інструкцій. На S3 із 512 КБ вікно даних — близько 480 КБ. Далі від цього ще відрізають своє бутлоадер, стек ROM, Wi-Fi і сам застосунок.

Тому число з таблиці годиться для порівняння чипів між собою і не годиться для планування буфера. Реальну відповідь дає лише сам чип: `heap_caps_get_free_size` (розділ 30).

Заразом видно, чому на C3, C6 і H2 різниця менша: у них вікна даних і інструкцій збігаються, і 400 чи 512 КБ адресуються цілком.

Два значення варто зазначити окремо, бо в мережі щодо них трапляється хибна інформація: **ESP32-S2 працює до 240 МГц** (не 120), а **ESP32-H2 — до 96 МГц**.

Три речі, які плутають найчастіше

УВАГА

Bluetooth Classic є лише в ESP32 classic. Усі пізніші сімейства — тільки BLE. Це не «поки не реалізовано», а відсутність апаратного блоку.

Практичний наслідок: профіль SPP — послідовний порт по Bluetooth, на якому тримається безліч старих проєктів і на який розраховують дешеві термінальні застосунки для телефона, — на S3 і C3 недоступний у принципі. Проєкт, що переїжджає з classic на S3, доведеться переписувати на BLE (розділ 41).

УВАГА

ESP32-S2 не має Bluetooth узагалі — ні classic, ні BLE. Це найдивніша позиція в лінійці й найчастіше джерело розчарувань: чип купують як «новіший ESP32», а він не вміє того, що вміє старий.

УВАГА

PSRAM підтримують лише classic, S2 і S3. У C3, C6 і H2 зовнішньої псевдостатичної пам'яті не буде за жодних умов — апаратної підтримки немає.

Це головне обмеження C3: 400 КБ SRAM — це стеля, і вона визначає, що на ньому можна робити. Кадровий буфер камери, великий веб-інтерфейс, TLS із кількома одночасними з'єднаннями — все це впирається саме сюди.

Xtensa проти RISC-V: чи має це значення

ESP32, S2 і S3 побудовані на ядрах Xtensa; C3, C6, H2 і решта нової лінійки — на RISC-V.

Для прикладного коду — практично ні. Ви пишете на C або C++, і компілятор ховає різницю. Той самий код збирається під обидві архітектури.

Де різниця вилазить:

- **готові двійкові бібліотеки** без вихідних текстів не переносяться;
- **асемблерні вставки** доведеться переписувати;
- **інструмент розшифровки backtrace** інший: xtensa-esp32-elf-addr2line проти riscv32-esp-elf-addr2line (розділ 26);
- **прошивка не переноситься** взагалі: образ для classic на C3 не запрацює.

Espressif послідовно переходить на RISC-V у нових чипах, тому в довгостроковій перспективі це напрямок лінійки.

Що переноситься між сімействами, а що ні

Переноситься майже завжди: код на ESP-IDF, написаний через штатні драйвери; логіка застосунку; робота з Wi-Fi і мережею; FreeRTOS.

Потребує уваги: номери пінів — вони інші скрізь (картка K9); Bluetooth — див. вище; обсяг пам'яті, якщо ви поклалися на PSRAM; периферія, якої в цільовому чипі просто немає.

Не переноситься: зібрана прошивка; двійкові бібліотеки; асемблер; код, що звертається до регістрів напряду.

Перенесення проєкту на інший чип в ESP-IDF починається з однієї команди:

```
idf.py set-target esp32s3
```

УВАГА

`set-target` **стирає** `sdkconfig`. Усі налаштування, зроблені через `menuconfig`, повертаються до типових. Це не помилка, а необхідність: багато параметрів специфічні для чипа. Але зроблене без збереження `sdkconfig.defaults` доведеться налаштовувати заново.

Решта лінійки, рядком

ESP32-C2 (він же ESP8684) — здешевлений: менше пам'яті, менше периферії. Для дуже простих і дуже масових виробів.

ESP32-C5 — перший у лінійці з Wi-Fi у двох діапазонах, 2,4 і 5 ГГц. Знімає обмеження, через яке ESP32 не бачить сучасні 5-гігагерцові мережі.

ESP32-C61 — здешевлений варіант у тому ж напрямку, що C6.

ESP32-H4 — розвиток лінії H2: без Wi-Fi, з наголосом на 802.15.4 і низьке споживання.

ESP32-P4 — окремий випадок: потужний застосунковий процесор **без радіо взагалі**. Розрахований на графіку, відео й інтерфейси; зв'язок додається окремим чипом.

ЗАКУПІВЛЯ

Наявність нових сімейств у продажу відстає від анонсів на місяці, а підтримка в ESP-IDF, Arduino core і сторонніх бібліотеках — ще більше. Чип, що вийшов пів року тому, часто означає: приклади є, а бібліотеки на потрібний вам датчик — ще ні.

Для виробу, який треба зробити зараз, це аргумент на користь classic або S3. Актуальна наявність — у датованому вкладиші ринку, не тут (P5).

Типова задача → чип

Задача	Чип	Чому
Перший проєкт, навчання	classic	найбільше прикладів і статей
Новий серйозний проєкт	S3	два ядра, USB-JTAG, PSRAM, актуальний
Дешево і масово	C3	ціна і розмір, якщо 400 КБ вистачає
Требa Bluetooth Classic / SPP	тільки classic	більше ніде немає
Камера, дисплей, буфери	S3 з PSRAM	пам'ять — вирішальна

Задача	Чип	Чому
Батарейка на роки, без Wi-Fi	H2	немає Wi-Fi, дуже низьке споживання
Zigbee, Thread, Matter	C6 або H2	тільки в них є 802.15.4
Wi-Fi 6 у щільній мережі	C6	єдиний з Wi-Fi 6 у цій таблиці
Мережа 5 ГГц	C5	решта лінійки 5 ГГц не бачить
Налагодження без адаптера	S3, C3	вбудований USB-JTAG (розділ 27)

Практична порада про парк плат

Якщо ви робите більше одного пристрою, варто звести парк до **двох** позицій: одна «робоча конячка» і одна дешева для простих вузлів. Наприклад, S3-DevKitC-1 і C3 SuperMini.

Причина не в економії, а в тому, що кожна нова плата — це свій пінаут, свої граблі, своя схема авторесету і свій драйвер мосту. Три однакові плати в шухляді корисніші за шість різних.

Що з цього треба запам'ятати

Bluetooth Classic — лише classic. S2 без Bluetooth узагалі. PSRAM — лише classic, S2, S3.

Xtensa проти RISC-V для прикладного коду не має значення; для двійкових бібліотек, асемблера і готових прошивок — має.

`set-target` стирає `sdkconfig`.

Для нового проєкту типова відповідь — S3; для навчання — classic; для дешевого вузла — C3, якщо 400 КБ вистачає.

03. Архітектура SoC

Цей розділ пояснює, чому ESP32 поводить себе так, як поводить себе. Він не потрібен, щоб змигнути світлодіодом, і стає необхідним рівно тоді, коли з'являються питання виду «чому програма падає лише при увімкненому Wi-Fi» або «чому цей масив не вміщається, хоча пам'яті вистачає».

Ядра і хто на них живе

classic **S3** ESP32 classic і S3 мають **два ядра**. Вони називаються PRO_CPU (ядро 0) і APP_CPU (ядро 1) — назви історичні й нічого не означають: ядра рівноправні.

Розподіл за замовчуванням, який варто знати:

Ядро 0 зайняте стеком Wi-Fi і Bluetooth. Радіо — це не магія в антені, а код, що виконується на тому самому процесорі, і виконується він переважно тут.

Ядро 1 здебільшого вільне, і саме там за замовчуванням запускається `app_main`.

Практичний наслідок: задача, прив'язана до ядра 0, конкурує за час із радіостеком. Якщо у вас щось із жорсткими таймінгами — його місце на ядрі 1 (розділ 31).

C3 C3, C6, H2 і S2 **одноядерні**. Там радіостек і ваш код ділять один процесор, і затримки від радіо помітніші. Це одна з реальних цін дешевизни C3.

УВАГА

Двоядерність не робить програму вдвічі швидшою автоматично. Вона дає можливість **розвести** конкуренцію: важку роботу на одне ядро, зв'язок на інше. Код, написаний як один потік, використовує одне ядро й лишить друге простоювати.

Пам'ять: чому її «не вистачає» при вільних кілобайтах

Це найчастіше джерело здивування. Пам'ять ESP32 не є одним однорідним масивом; вона поділена на області з різними властивостями, і `malloc` може повернути `NULL`, коли сумарно вільно чимало.

ROM. Зашитий на заводі код: ROM-бутлоадаер, частина бібліотечних функцій. Не змінюється, місця не займає у вашому бюджеті.

SRAM — основна швидка пам'ять на кристалі. Ділиться на дві ролі:

- **IRAM** (instruction RAM) — звідси виконується код. Сюди потрапляє те, що має працювати швидко або має бути доступним, коли флеш недоступний: обробники переривань, критичні функції.
- **DRAM** (data RAM) — тут живуть дані: змінні, стеки задач, купа.

RTC RAM — маленька область (одиниці кілобайтів), яка **переживає deep sleep**. Все інше при глибокому сні втрачається. Сюди кладуть лічильники і стан, який має пережити пробудження (розділ 06).

Зовнішня флеш — те, де лежить прошивка. Код виконується звідти через кеш, а не з RAM: у RAM він просто не помістився б.

PSRAM — зовнішня псевдостатична пам'ять (pseudo-static RAM), від 2 до 8 МБ. Є лише в classic, S2 і S3 (розділ 02). Повільніша за SRAM, бо ходить через ту саму шину, що й флеш, але дозволяє тримати великі буфери: кадри камери, зображення для дисплея, великі структури.

УВАГА

З цього випливає найчастіша пастка: **malloc повернув NULL, хоча «пам'ять є»**. Причини бувають три.

Фрагментація. Купа зайнята дрібними шматками, суцільного блоку потрібного розміру немає. Виникає від виділення й звільнення в циклі.

Не та область. Буфер для DMA має лежати в DRAM, доступний контролеру DMA. Звичайний malloc може віддати пам'ять, яка формально є, але для DMA не годиться — для цього є `heap_caps_malloc` із явним зазначенням властивостей.

PSRAM не увімкнено. `CONFIG_SPIRAM` типово вимкнено, і плата з розпаяною мікросхемою без цього її не бачить. А от коли ввімкнено, malloc уже сам виносить у PSRAM усе від 16 KB — вмикати це окремо не треба (розділ 30).

Подивитися реальну картину: `heap_caps_get_free_size` і `heap_caps_print_heap_info` (розділ 30).

Кеш і чому виконання з флешу іноді зупиняється

Код лежить у флеші, а виконується процесором через кеш. Поки потрібна ділянка в кеші — все швидко. Коли ні — процесор чекає читання з флешу.

Звідси наслідок, який породжує дуже загадкові збої: **у момент операції з флешем виконання коду з флешу неможливе**. Коли система пише в NVS або стирає сектор, кеш вимикається — і будь-яка функція, що в цей момент має виконатися з флешу, впаде.

Стосується це насамперед обробників переривань. Саме тому ISR, які можуть спрацювати під час роботи з флешем, позначають атрибутом `IRAM_ATTR` — щоб вони лежали в IRAM, а не у флеші:

```
static void IRAM_ATTR gpio_isr_handler(void *arg) {
    // цей код виконається навіть тоді, коли кеш вимкнено
}
```

Ціна — IRAM небагато, і кожна функція з IRAM_ATTR займає її назавжди (розділ 31).

Тактування

Частота ядра — 240 МГц для classic, S2, S3; 160 МГц для C3 і C6; 96 МГц для H2. Її можна знизити, і це прямий спосіб зменшити споживання.

Кілька джерел тактування з різними властивостями:

- **зовнішній кварц** (типово 40 МГц) — точний, основне джерело;
- **внутрішній RC-генератор** — не потребує деталей, але «пливе» від температури;
- **RTC-генератор** — повільний, працює під час сну.

УВАГА

Годинник реального часу, побудований на внутрішньому RC-генераторі, за добу набігає помітну похибку. Якщо пристрою потрібен точний час і немає мережі для SNTP — потрібна зовнішня мікросхема RTC із власним кварцом і батареєю (розділ 60).

Динамічне керування частотою (DFS) дозволяє системі самій знижувати частоту, коли роботи немає, і піднімати під навантаженням. Вмикається в menuconfig і дає відчутну економію в пристроях, що більшість часу чекають (розділ 06).

RTC-домен і ULP

Окрема частина кристала, що лишається живою, коли основні ядра сплять:

- **RTC RAM** — дані переживають deep sleep;
- **RTC GPIO** — частина пінів здатна будити чип;
- **RTC-таймер** — будильник;
- **ULP-співпроцесор** — крихітний процесор, що може працювати, поки основні ядра вимкнені.

ULP уміє небагато: прочитати ADC, перевірити пін, порівняти з порогом, розбудити основну систему, якщо щось сталося. Але споживає він мікроампери.

Типове застосування: пристрій спить, ULP раз на секунду міряє рівень і будить систему, лише коли значення вийшло за межу. Замість того щоб будити повноцінну систему сто разів даремно, її будять один раз по ділу (розділ 06).

Програмується ULP окремо — власним асемблером або, у пізніших сімействах, на C для RISC-V-варіанта ULP. Це помітно складніше за звичайний код, і братися за нього варто тоді, коли бюджет живлення справді не сходиться.

Memory map: навіщо це знати

Кожна область пам'яті має свій діапазон адрес. Знати таблицю напам'ять не треба, але вміти прочитати адресу — корисно: **за адресою в дампі паніки одразу видно, куди зверталися**.

Практичні орієнтири для `classic`:

- адреса виду `0x400d....` — код, що виконується з флешу;
- адреса виду `0x3ffb....` — дані в DRAM, зазвичай стек;
- адреса виду `0x3f4....` — PSRAM;
- адреса близько нуля (`0x0–0x40`) — розіменування NULL зі зсувом поля структури.

Останній випадок покриває більшість `LoadProhibited` на практиці (розділ 26).

Що з цього треба запам'ятати

Радіостек — це код на ядрі 0, а не окремий пристрій. На одноядерних чипах він конкурує з вашим кодом за той самий процесор.

Пам'ять неоднорідна: `malloc` може повернути NULL при формально вільних кілобайтах через фрагментацію, вимоги DMA або невикористану PSRAM.

Під час операції з флешем код із флешу не виконується — звідси `IRAM_ATTR` на обробниках переривань.

RTC RAM переживає `deep sleep`; усе інше — ні.

За адресою в дампі паніки видно, у яку область зверталися.

04. Периферія на кристалі — огляд

Периферія — це апаратні блоки всередині чипа, які роблять роботу без участі процесора: передають байти по UART, генерують імпульси PWM, міряють напругу. Процесор лише налаштовує їх і забирає результат.

Це оглядовий розділ: що взагалі є в чипі й навіщо. Робота з коду — розділ [33](#), конкретні інтерфейси — частини IX і XI.

GPIO matrix: чому піни майже взаємозамінні

Найважливіша архітектурна особливість ESP32, і вона суттєво відрізняє його від класичних мікроконтролерів.

У звичайному мікроконтролері периферія жорстко прив’язана до пінів: UART1 виходить на конкретні дві ніжки, і перенести його неможливо. Тут між периферією і пінами стоїть **комутаційна матриця**: майже будь-який сигнал майже будь-якого блоку можна вивести на майже будь-який пін — програмно, без перепаявання.

Практичні наслідки величезні:

- розводити плату значно легше: сигнал ведеться туди, куди зручно, а призначення задається в коді;
- конфлікт «дві бібліотеки хочуть той самий пін» розв’язується налаштуванням;
- чужа плата, де все розведено дивно, все одно працює.

Де межі матриці:

- **Швидкі сигнали** — SPI на високих частотах, I²S, native USB — мають «рідні» піни. Через матрицю вони теж працюють, але з обмеженням частоти: маршрут через матрицю додає затримку.
- **ADC, DAC і touch** прив’язані жорстко. Це аналогові блоки, і матриця на них не поширюється: ADC1 у classic — це саме GPIO 32–39, і жодне налаштування цього не змінить.
- **Піни флешу** classic GPIO 6–11 зайняті фізично.
- **Тільки-вхідні піни** classic GPIO 34–39 не мають вихідного драйвера. Матриця не додасть його.

УВАГА

Звідси головне правило пінів: **свобода стосується цифрових інтерфейсів, обмеження — аналогових і службових**. Перш ніж призначити пін, звіря-
тися з картою **K9**, а не з інтуїцією.

Що є в чипі

Цифрові інтерфейси зв'язку

Блок	Що це	Розділ
UART	послідовний порт; консоль і RS-485	34
I ² C	дві лінії, багато пристроїв, невисока швидкість	35
SPI	швидка шина, окремий вибір на кожен пристрій	36
I ² S	цифровий звук: мікрофони, підсилювачі	49
TWAI	CAN-шина, сумісна з автомобільною	38
SDMMC	картки пам'яті на повній швидкості	49
USB OTG	S3 повноцінний USB: пристрій або хост	09
USB-Serial-JTAG	S3 C3 консоль і JTAG одним кабелем	27

Керування сигналами

Блок	Що це	Розділ
LEDC	PWM: яскравість світлодіодів, сервоприводи	33
MCPWM	classic S3 моторний PWM: мертвий час, аварійне вимкне- ння	48
RMT	точні послідовності імпульсів: WS2812, ІЧ	33
PCNT	апаратний лічильник імпульсів: енкодери, витратоміри	33
Таймери	апаратні таймери загального призначення	33

Аналогові

Блок	Що це	Розділ
ADC	вимірювання напруги	33
DAC	classic S2 справжній аналоговий вихід	33
Touch	ємнісні сенсори дотику	33

Службові

Апаратні прискорювачі криптографії (AES, SHA, RSA), генератор випадкових чисел, апаратний watchdog, контролер DMA.

Кілька блоків, які варто знати заздалегідь

RMT — недооцінений блок. Задумувався для інфрачервоних пультів, а виявився ідеальним для будь-яких точних імпульсних послідовностей. Керування адресними світлодіодами WS2812, де таймінги вимірюються сотнями наносекунд, робиться саме через RMT — і робиться апаратно, без навантаження на процесор і без залежності від того, чим зайнята система. Він же вміє вимірювати тривалість вхідних імпульсів.

PCNT рахує імпульси сам. Енкодер, витратомір, лічильник обертів — усе це не потребує переривань на кожен імпульс: процесор просто читає накопичене значення, коли йому зручно.

MCSPWM classic S3 зроблений спеціально для силової електроніки: вміє мертвий час між плечима моста, апаратне аварійне вимкнення за зовнішнім сигналом, синхронізацію каналів. Керувати двигуном звичайним PWM теж можна, але саме ці функції рятують силовий каскад від наскрізних струмів.

TWAI — це CAN. Назва інша через ліцензування торгової марки, суть та сама. Дає обмін із «дорослими» контролерами й автомобільною електронікою (розділ 38).

Апаратна криптографія робить TLS придатним до використання. Без прискорювачів рукоустискання на мікроконтролері триває неприйнятно довго.

Що є в якому чипі

	classic	S2	S3	C3	C6	H2
UART	3	2	3	2	2 + 1 LP	2
I ² C	2	2	2	1	1 + 1 LP	2
SPI, усього	3	3	3	2	2	2
SPI, вільних	2	2	2	1	1	1
I ² S	2	1	2	1	1	1
TWAI (CAN)	1	1	1	1	2	1
DAC	2	2	ні	ні	ні	ні
Touch	10	14	14	ні	ні	ні

	classic	S2	S3	C3	C6	H2
USB	ні	OTG	OTG+JTAG	JTAG	JTAG	JTAG

Кількості звірені з заголовками можливостей самого ESP-IDF (`soc_caps.h`) — тим джерелом, за яким збирається фреймворк.

Рядок «SPI, вільних» — це те, що лишається після контролера, зайнятого флешем і PSRAM. Саме це число має значення при проєктуванні; у datasheet наведено більше.

C6 Запис «+ 1 LP» для C6 не описка: третій UART і другий контролер I²C там **низькоспоживчі**. Вони живуть у LP-домені й працюють тоді, коли основна система спить, — тобто датчик можна опитувати, не будячи чип. Для звичайного застосунку доступні два UART і один I²C, тобто рівно як у C3.

Важливо не порахувати їх як звичайні: `soc_caps.h` дає для C6 `SOC_UART_NUM = 3` і `SOC_I2C_NUM = 2`, але поруч стоять `SOC_UART_HP_NUM = 2`, `SOC_UART_LP_NUM = 1` і така сама пара для I²C. Саме друга пара й описує те, що доступне звичайному застосунку.

Рядок «Touch» показує **придатні до використання** канали, а не все, що рахує апаратура: на S2 і S3 нульовий канал зайнятий внутрішнім каналом придушення шуму, тож із п'ятнадцяти лишається чотирнадцять.

УВАГА

Два спостереження з цієї таблиці, що впливають на вибір чипа.

DAC є лише в classic і S2. Справжній аналоговий вихід більше ніде не з'явився. Там, де він потрібен, — або classic, або зовнішній ЦАП по I²C.

Touch немає в C3, C6, H2. Ємнісні кнопки на цих чипах доведеться робити зовнішньою мікросхемою.

Скільки периферії можна ввімкнути одночасно

Питання виникає, коли проєкт розростається. Обмежень три, і всі практичні:

Піни. Найжорсткіше обмеження. У 38-пінової плати вільних пінів насправді деє два з половиною десятки, і кожен інтерфейс їх з'їдає: I²C — два, SPI — чотири плюс по одному на кожен пристрій, UART — два.

Канали DMA. Швидкі блоки (SPI, I²S, ADC у режимі потоку) використовують DMA, і каналів обмежена кількість.

Переривання і час процесора. Периферія працює апаратно, але результати хтось має забирати. Десять джерел переривань із високою частотою з'їдять ядро незалежно від того, скільки блоків є в чипі.

Практичний вихід, коли пінів не вистачає: розширювачі портів по I²C (PCF8574, MCP23017), зсувні регістри по SPI, або перенесення частини задачі на другий мікроконтролер (розділ 57).

Як читати datasheet про периферію

Три речі, які варто перевіряти перед тим, як закладати блок у проєкт:

Кількість екземплярів — скільки саме UART чи SPI, і скільки з них вільні після флешу й консолі.

Обмеження частот — SPI через матрицю обмежений 40 МГц замість 80 на рідних пінах; нижче 40 МГц різниці немає (розділ 36).

Взаємні конфлікти — classic S2 S3 ADC2 не працює при Wi-Fi; USB-JTAG займає конкретні піни; DAC ділить піни з іншими блоками.

Останній пункт — найчастіше джерело сюрпризів на пізньому етапі, коли плата вже розведена.

Що з цього треба запам'ятати

Матриця GPIO робить цифрові інтерфейси майже вільними у виборі пінів; аналогові блоки й піни флешу вона не зачіпає.

RMT керує адресними світлодіодами апаратно; PCNT рахує імпульси без переривань; MCPWM дає мертвий час і аварійне вимкнення для силових каскадів.

DAC — тільки classic і S2. Touch — тільки classic, S2, S3.

Реальне обмеження проєкту — не кількість блоків, а кількість пінів.

Частина II. Електроніка для програміста

Рівно стільки заліза, скільки треба, щоб не спалити плату і зрозуміти, чому вона поводить ся дивно.

05. Електроніка за один вечір

Цей розділ — мінімум, після якого ви перестанете палити плати і почнете розуміти, чому вони поведуться дивно. Не курс електроніки: рівно те, що потрібно, щоб з'єднати цифрові пристрої й не зробити з них диму.

Якщо ви це знаєте — пропускайте до розділу 06.

Чотири величини

Напруга (U, вольти). Різниця потенціалів між двома точками. Завжди **між** — фраза «на цьому проводі 5 вольтів» неповна, поки не сказано, відносно чого. Відносно землі, майже завжди.

Струм (I, ампери). Скільки заряду проходить за секунду. Струм не «подають» — його **споживають**: джерело пропонує напругу, а скільки взяти струму, вирішує навантаження.

Опір (R, оми). Наскільки провідник заважає струму.

Потужність (P, вати). $P = U \times I$. Те, що перетворюється на тепло.

Закон Ома: $U = I \times R$. Практично він потрібен у трьох випадках: порахувати резистор для світлодіода, порахувати дільник напруги і зрозуміти, чому на довгому тонкому дроті напруга просіла.

УВАГА

Аналогія з водою працює краще, ніж здається. Напруга — тиск. Струм — витрата. Опір — вузькість труби. Джерело живлення тримає тиск; навантаження вирішує, скільки протече. Тонкий довгий дріт — вузька труба: на кінці тиск нижчий, ніж на початку. Саме це відбувається з дешевим метровим USB-кабелем, і саме тому плата від нього перезавантажується.

Резистор для світлодіода

Найчастіший розрахунок. Світлодіод не можна вмикати без резистора: він не обмежує струм сам і згорає.

Формула: $R = (U_{\text{живлення}} - U_{\text{світлодіода}}) / I_{\text{бажаний}}$

Для типового червоного світлодіода від 3.3 В: падіння на ньому близько 2 В, бажаний струм 5–10 мА.

$R = (3.3 - 2) / 0.007 \approx 185 \text{ Ом} \rightarrow$ беремо найближчий більший, 220 Ом.

Резистор 220–330 Ом підходить майже завжди. Синій і білий світлодіоди мають більше падіння (близько 3 В), тому від 3.3 В світять слабо — це нормально.

ЖИВЛЕННЯ

Пін ESP32 віддає обмежений струм, і найпоширеніше уявлення про це неточне в обидва боки.

40 мА — це типова віддача на максимальній силі драйвера, а не гранично допустиме значення. У datasheet воно стоїть у таблиці робочих характеристик (DC Characteristics), а не серед абсолютних меж.

Гранично допустиме нормується лише сумарно — 1200 мА на всі виводи разом. Це далека стеля, а не робочий орієнтир.

І 40 мА не є сталою. Що більше пінів одного домену віддають струм одночасно, то менше дістається кожному — приблизно до 29 мА. Тому розрахунок «межа 40, беру 35 із запасом» на кількох активних пінах виходить за реальне значення, точно дотримуючись арифметики.

Ще одна асиметрія: **на віддачу пін дає більше, ніж на прийом** — на максимальній силі драйвера 40 мА проти 28 мА. Різниця невелика, але в схемі, де світлодіод можна повісити і катодом на пін, і анодом, вибір напрямку — це вибір між двома різними межами.

Окремо варто знати про домен VDD_SDIO: там віддача вдвічі менша, 20 мА.

За замовчуванням драйвер стоїть на **середній** силі, тобто помітно менше за максимум. Конкретного числа в міліамперах на цей випадок не дає ані datasheet, ані ESP-IDF: в API це GPIO_DRIVE_CAP_DEFAULT, підписане просто як «medium».

Практично: яскравий світлодіод на 20 мА напряму працює на межі розумного. Десять таких — не «за сумарною межею» (це шоста частина від 1200 мА), а за межею того, що домен віддасть на пін: віддача просяде, і додасться нагрів. Усе, що споживає більше кількох міліампер, вмикається через транзистор (розділ 47).

Логічні рівні: головна причина спалених плат

Цифровий сигнал — це дві напруги. Для ESP32: близько 0 В — логічний нуль, близько 3.3 В — логічна одиниця.

Проблема в тому, що величезна кількість модулів «для Arduino» працює на **5 В**. Зовні роз'єми однакові, дроти однакові, і з'єднати їх дуже легко.

НЕЗВОРОТНЕ

П'ять вольтів на GPIO вбивають пін, а іноді чип. Вхід ESP32 розрахований на 3.3 В; захисні діоди всередині починають проводити струм при перевищенні й вигорають.

Єдиний виняток — пін 5V/VIN: це вхід стабілізатора, а не вхід у чип. Туди 5 В подавати можна і треба.

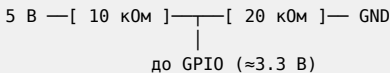
Найчастіші джерела 5 В на сигнальній лінії: ультразвуковий далекомір HC-SR04 (вихід ЕСНО), релейні модулі, дисплеї «для Arduino», логічні мікросхеми серії 74НС при живленні 5 В.

Що робити в різних напрямках:

ESP32 → пристрій на 5 В. Часто працює без нічого: більшість 5-вольтових входів сприймає 3.3 В як одиницю. Не завжди — звертатися з datasheet модуля.

Пристрій на 5 В → ESP32. Потрібне зниження **обов'язково**.

Дільник напруги — два резистори. Найдешевший спосіб для повільних сигналів:



Дільник підходить для однонаправлених повільних сигналів. Для швидких (SPI) і двонаправлених (I²C) він не годиться: перший завалює фронти, другий просто не працює.

Конвертер рівнів — маленька плата за копійки, вирішує обидва випадки. Для I²C потрібен саме двонаправлений, на польових транзисторах. Тримати пару в шухляді (розділ 47).

Підтягування: чому вхід «бовтається»

Цифровий вхід, до якого нічого не під'єднано, не читає нуль. Він читає **випадкове значення**: ловить наводки і перемикається сам.

Кнопка, з'єднана з землею, дає нуль при натисканні — і невідомо що при відпусканні. Щоб було визначено, потрібен **підтягувальний резистор**:

Pull-up — резистор від піна до 3.3 В. У спокої вхід читає одиницю, натиснута кнопка притискає до нуля. Найпоширеніша схема.

Pull-down — резистор до землі, дзеркально.

Хороша новина: у ESP32 підтягувальні резистори **вбудовані** і вмикаються з коду:

```

gpio_config_t cfg = {
    .pin_bit_mask = (1ULL << GPIO_NUM_4),
    .mode = GPIO_MODE_INPUT,
    .pull_up_en = GPIO_PULLUP_ENABLE,
};
gpio_config(&cfg);
  
```

УВАГА

classic Піни 34–39 **не мають вбудованого підтягування**. Взагалі. Це не налаштовується — апаратної схеми немає. Кнопка на GPIO34 без зовнішнього резистора працювати не буде, і це виглядає як несправність плати (картка K9).

Зовнішнє підтягування для I²C — окрема історія: там вбудованого замало, потрібні справжні резистори 4.7 кОм (розділ 35).

Open-drain: коли кілька пристроїв на одній лінії

Звичайний вихід активно тримає лінію в обох станах. Два таких виходи на одному дроті, що хочуть різного, — це коротке замикання.

Open-drain уміє лише притискати лінію до землі, а «відпускати» — пасивно. Лінія піднімається підтягувальним резистором. Тоді до неї можна чіпляти скільки завгодно пристроїв: хто завгодно може притиснути, а одиниця з'являється, коли всі відпустили.

На цьому побудовані I²C і 1-Wire. Знати це треба, щоб розуміти, чому в цих шинах підтягування обов'язкове, а без нього нічого не працює.

Земля: правило, яке порушують найчастіше

Усі з'єднані пристрої мусять мати спільну землю. Сигнал — це напруга відносно землі; без спільної точки відліку «3.3 В» не означає нічого.

Симптом відсутньої спільної землі: пристрої живляться, індикатори горять, обміну немає. Або обмін є, але з випадковими помилками.

Це найчастіша помилка при живленні від окремого джерела: плюс подали, а землю з'єднати забули (розділ 29).

Конденсатори: чому вони скрізь

Конденсатор накопичує заряд і віддає його швидко. У цифровій схемі це локальний запас енергії на випадок різкого споживання.

Коли ESP32 вмикає передавач, споживання стрибає за мікросекунди. Джерело і дроти так швидко не встигають — напруга просідає, і чип перезавантажується по brownout (розділ 06).

Два типи, обидва потрібні:

Керамічний 100 нФ біля кожної мікросхеми, якомога ближче до її ніжок живлення. Гасить високочастотні кидки. Дешевий, і його не буває забагато.

Електролітичний або танталовий 100–470 мкФ біля роз'єма живлення. Тримає напругу при повільніших просадках — саме те, що рятує від brownout при вмиканні радіо.

ЖИВЛЕННЯ

Конденсатор 470 мкФ між 3V3 і GND поруч із модулем — найдешевше розв'язання найчастішої проблеми в усій книзі. Коштує копійки, ставиться за хвилину, знімає цілий клас «незрозумілих перезавантажень».

Полярність електролітичного конденсатора обов'язкова: мінус позначений смугою. Переплутаний конденсатор здувається, іноді голосно.

MOSFET як ключ

Коли треба ввімкнути щось, що споживає більше, ніж віддає пін, — двигун, стрічку світлодіодів, реле, нагрівач — між ними ставлять транзистор.

Найзручніший варіант — **N-канальний MOSFET логічного рівня**. Ключове слово «логічного рівня» (logic level): такий відкривається від 3.3 В. Звичайний MOSFET від 3.3 В відкривається частково, гріється і згорає.

Схема low-side: навантаження між плюсом і стоком, витік на землю, затвор — на GPIO через резистор 100–220 Ом, і резистор 10 кОм із затвора на землю, щоб при старті ключ був закритий.

УВАГА

Резистор із затвора на землю важливіший, ніж здається. Під час завантаження GPIO ще не налаштований і «висить» — без цього резистора навантаження може ввімкнутися саме в момент старту. Для нагрівача або двигуна це не дрібниця.

Захисний діод на індуктивності

Реле, двигун, електромагнітний клапан — це котушка. При вимиканні струму котушка створює викид напруги зворотної полярності, і цей викид пробиває транзистор.

Лікується одним діодом (наприклад, 1N4007), під'єднаним **паралельно котушці, у зворотному напрямку**: катодом до плюса. У нормальній роботі він закритий; у момент вимикання приймає викид на себе.

Правило просте: **будь-яка котушка — діод обов'язково**. Модулі реле зазвичай мають його на платі; саморобна схема — ні (розділ 47).

ESD і як убити плату за десять секунд

Статична електрика — це кіловольти. Розряд із пальця легко пробиває вхід мікросхеми. Плата при цьому може не померти одразу, а почати працювати «дивно».

Топ-5 способів убити плату, у порядку популярності:

1. **5 В на GPIO.** Абсолютний лідер.
2. **Переплутані живлення і земля.** Особливо на Dupont-роз'ємах, які легко зсунути на один контакт.
3. **Паяння під живленням.** Наведений потенціал на жалі незаземленого паяльника пробиває вхід.
4. **Замикання сусідніх пінів** краплею припою або пінцетом.
5. **Статика** — сухе повітря, синтетичний одяг, плата за виводи.

Профілактика проста і безкоштовна: перед тим як узяти плату, торкнутися заземленого металу; тримати за краї, а не за виводи; від'єднувати живлення перед будь-яким паянням; двічі перевіряти живлення і землю перед подачею напруги.

Повний перелік незворотного — картка [K11](#).

Що з цього треба запам'ятати

Напруга завжди **між** двома точками; спільна земля обов'язкова.

3.3 В — межа для GPIO. 5 В можна тільки на VIN.

Вхід без підтягування читає випадкове значення. classic Піни 34–39 вбудованого підтягування не мають.

100 нФ біля кожної мікросхеми і 470 мкФ біля роз'єма живлення знімають більшість «незрозумілих» збоїв.

Будь-яке навантаження понад кілька міліампер — через транзистор. Будь-яка котушка — з діодом.

06. Живлення і струмоспоживання

Якщо в цій книзі є один розділ, який економить найбільше часу, — це він. Величезна частка «плаваючих багів», «незрозумілих перезавантажень» і «воно іноді працює» — це живлення, а не код.

Причина, чому це так важко впізнати: симптоми виглядають програмними. Пристрій перезавантажується посеред роботи з мережею; датчик видає сміття; Wi-Fi відвалюється. Людина йде читати код і не знаходить нічого, бо в коді нічого й немає.

Куди подавати живлення

На платі розробки зазвичай три способи, і плутати їх дорого.

USB-роз'єм. 5 В іде на бортовий стабілізатор, той робить 3.3 В. Найпростіший і найчастіший шлях.

Пін 5V або VIN. Те саме, мінаючи USB-роз'єм. Сюди можна подавати 5 В (часто й трохи більше — залежить від стабілізатора на платі).

Пін 3V3. Це **вихід** стабілізатора. Подавати сюди зовнішні 3.3 В можна, але тоді бортовий стабілізатор має лишатися без входу — інакше два джерела працюють одне проти одного.

НЕЗВОРОТНЕ

Подати 5 В на пін 3V3 означає подати їх напряму в чип, мінаючи стабілізатор. Це майже гарантована смерть плати.

Помилка трапляється частіше, ніж здається: на гребінці піни 5V і 3V3 часто стоять поруч, підписані дрібно, а Dupont-роз'єм легко зсунути на один контакт.

Скільки воно споживає

Порядки величин, які варто тримати в голові. Точні цифри для конкретного чипа — в його datasheet; наведене нижче взято з документації сімейств і **до першоджерела не звірено** (позначено в реєстрі фактів).

Режим	Порядок споживання
Deep sleep	10 мкА; з живленням ULP — близько 150 мкА
Light sleep	сотні мкА — одиниці мА
Активний, радіо вимкнене	десятки мА

Режим	Порядок споживання
Активний, Wi-Fi у роботі	близько сотні мА середнє
Піки при передачі Wi-Fi	сотні мА, короткими сплесками

Ключове тут — останній рядок. Споживання ESP32 **нерівномірне**. Середнє значення нічого не каже про те, що відбувається в мілісекундному масштабі: передавач вмикається імпульсами, і саме ці імпульси провалюють напругу.

ЖИВЛЕННЯ

Практичний висновок: **джерело для ESP32 з Wi-Fi має тягнути щонайменше 1 А**, навіть якщо середнє споживання — сто міліампер. Не тому, що пристрій стільки з'їсть, а тому, що джерело мусить тримати напругу під короткими піками.

Тут варто знати, з чим сперечаємося. **500 мА — це не чиясь наївність, а паспортний мінімум самої Espressif**: у datasheet стоїть IVDD, current delivered by external power supply, Min 0.5 A.

Тобто блок на 500 мА формально відповідає вимозі — і все одно найпоширеніша помилка в саморобних виробках. Причина в тому, що паспортний мінімум передбачає джерело, яке справді тримає свої 500 мА на сплеску; дешевий блок із написом «500 мА» просідає задовго до цього.

Тому 1 А — це не суперечність datasheet, а запас на якість джерела.

Brownout: як воно виглядає і що з ним робити

У чипі є вбудований детектор зниження напруги. Коли напруга падає нижче порогу, він примусово скидає чип, щоб той не працював у невизначеному стані й не пошкодив вміст флешу.

Як розпізнати. У boot-лозі:

```
rst:0xf (RTCWDT_BROWN_OUT_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
```

або пряме повідомлення:

```
Brownout detector was triggered
```

rst:0xf — це **живлення**. Не помилка в коді. Скільки б ви не читали код, причина не там (картка K6).

Причини за частотою:

1. Кабель USB — тонкий, довгий, дешевий. Абсолютний лідер.
2. USB-хаб без власного живлення.
3. Порт ноутбука, що обмежує струм.

4. Немає конденсатора по живленню.
5. Джерело, що не тягне пікового струму.
6. Розряджений акумулятор.
7. Слабкий стабілізатор на платі-клоні.

Що робити, у порядку дешевизни:

- **Короткий і товщий кабель.** Безкоштовно, розв'язує половину випадків.
- **Конденсатор 470 мкФ** між 3V3 і GND поруч із модулем. Копійки.
- **Хаб із власним живленням** або порт напрямку, без хаба.
- **Окреме джерело від 1 А.**
- Знизити пікове споживання: обмежити потужність передавача, знизити частоту ядра.

УВАГА

Спокуса вимкнути детектор brownout у menuconfig дуже велика: перезавантаження зникають, і здається, що проблема розв'язана.

Насправді проблема лишилася, а захист зник. Тепер чип працює при зниженій напрузі — і замість чесного перезавантаження ви отримуєте пошкоджений NVS, біті записи у флеш і збої, які взагалі неможливо пояснити.

Детектор brownout — це діагностика, а не перешкода. Вимикати його можна лише свідомо і в останню чергу.

Режими енергоспоживання

Active. Все увімкнено. Ядро працює, радіо доступне.

Modem sleep. Ядро працює, радіо вимикається між сеансами зв'язку. Wi-Fi-з'єднання при цьому зберігається: чип прокидається на маячки точки доступу. Прозорий для застосунку і дає відчутну економію.

Light sleep. Ядро зупинено, тактування знижено, **вміст RAM зберігається**. Прокидання швидко, і програма продовжує з того самого місця. Периферія переважно вимкнена.

УВАГА

classic Пін, який мусить тримати рівень крізь light sleep, треба спершу перевірити на приналежність до домену `VDD_SDIO`. На WROOM-32 до нього належать GPIO16 і GPIO17. Домен у сні вимикається, і пін, виставлений високим, тихо просідає — реле відпускає, транзистор закривається, EN зовнішньої мікросхеми падає.

Лікується одним рядком до засинання:

```
esp_sleep_pd_config(ESP_PD_DOMAIN_VDDSDIO, ESP_PD_OPTION_ON);
```

Ціна — домен лишається під живленням, тобто споживання в сні росте. Тому це рішення, а не типове налаштування: якщо рівень тримати не конче, дешевше взяти інший пін.

Deep sleep. Живиться лише RTC-домен. Вміст SRAM **втрачається** переживає сон тільки RTC RAM. Прокидання = перезавантаження: код починає з початку.

ULP. Основні ядра сплять, крихітний співпроцесор працює: міряє, порівнює, будить систему за умовою (розділ 03).

УВАГА

Головне непорозуміння з deep sleep: **це не пауза, а перезавантаження**. Після пробудження виконується `app_main` з початку, всі змінні ініціалізуються наново. Стан, який має пережити сон, кладеться в RTC RAM:

```
RTC_DATA_ATTR int lichyllyk_probudzen = 0;
```

Або в NVS — але NVS це запис у флеш, а флеш має обмежений ресурс запису. Для лічильника, що збільшується щохвилини, RTC RAM правильніша.

Приклад засинання на 60 секунд:

```
esp_sleep_enable_timer_wakeup(60ULL * 1000000); // мікросекунди
esp_deep_sleep_start(); // назад не повертається
```

Розбудити можуть таймер, RTC GPIO, ULP або touch — залежно від сімейства.

Бюджет енергії: як рахувати

Для автономного пристрою розрахунок робиться до складання, а не після.

Ємність джерела. Ходові елементи 18650 — від 2500 до 3500 мА·год (розділ 53). Із запасом на старіння і холод беріть 70 % від номіналу.

Споживання за цикл. Пристрій прокидається, міряє, передає, засинає. Порахувати треба **кожну фазу окремо**: скільки мілісекунд і скільки міліампер.

Приклад: прокидання і вимірювання — 200 мс при 40 мА; передача по Wi-Fi — 2 с при 150 мА середнього; сон — решта при 20 мкА.

Заряд за цикл: $0.2 \text{ с} \times 40 \text{ мА} + 2 \text{ с} \times 150 \text{ мА} \approx 308 \text{ мА} \cdot \text{с} \approx 0.086 \text{ мА} \cdot \text{год}$.

При одному циклі на годину: $0.086 + 0.02 \approx 0.106 \text{ мА} \cdot \text{год}$ на годину. З 2000 мА·год: близько 18 000 годин, тобто понад два роки.

ЖИВЛЕННЯ

Розрахунок майже завжди виявляється оптимістичним, і головний винуватець — **час під'єднання до Wi-Fi**. Дві секунди в прикладі вище — це добрий випадок. У реальності під'єднання може займати п'ять-десять секунд, а на межі покриття — не відбутися взагалі, і чип буде намагатися, доки не спрацює таймаут.

Саме тут ESP-NOW (розділ 42) виграє на порядок: передача без під'єднання займає мілісекунди замість секунд. Для датчика на батарейці це різниця між місяцем і роком.

Перевірка вимірюванням обов'язкова. Розрахунок дає порядок; реальність дає число. Без вимірювання бюджет — здогадка (розділ 58).

Як міряти споживання

USB-тестер. Вставляється між портом і платою, показує напругу, струм і накопичений заряд. Найдешевший інструмент, дає середнє споживання відразу. Піків не покаже — інерція вимірювання завелика.

Мультиметр у розрив. Точніший для середнього. Проблема та сама: піки не видно, і при перемиканні діапазону мультиметр може на мить розірвати коло і перезавантажити плату.

Шунт і осцилограф. Єдиний спосіб побачити реальну картину: резистор 0.1 Ом у розрив землі, осцилограф на ньому. Струм = напруга / опір. Показує піки, тривалість передачі, форму споживання — усе, що потрібно для чесного бюджету.

Спеціалізовані прилади вимірюють від мікроампер до сотень міліампер з високою роздільною здатністю. Дорого, але для серйозної роботи над автономністю — єдиний зручний варіант.

УВАГА

Міряючи deep sleep, пам'ятайте: **ви міряєте плату, а не чип**. На платі розробки живуть USB-міст, стабілізатор і світлодіод живлення, і всі вони споживають незалежно від того, що робить ESP32.

Плата, яка в deep sleep показує 20 мА замість 20 мкА, — це не несправний чип. Це світлодіод і міст. Для реальної автономності потрібен власний монтаж без цієї обв'язки, або плата, спроектована під низьке споживання (розділ 53).

Стабілізатори на платах

LDO (лінійний) — стоїть майже на всіх платах розробки. Простий, тихий, не створює завад. Різницю напруг перетворює на тепло: при вході 5 В і виході 3.3 В

третина енергії йде в нагрів. Для живлення від USB це прийнятно, для батарейки — марнотратно.

Buck (понижувальний імпульсний) — ефективність 85–95 %. Для автономних пристроїв правильний вибір. Створює високочастотні завади, тому для аналогових вимірювань після нього ставлять LDO (розділ 53).

ЗАКУПІВЛЯ

На дешевих клонах плат стабілізатор часто слабший за оригінальний і перегрівається при роботі радіо. Симптом: плата працює кілька хвилин, потім починає перезавантажуватися; корпус стабілізатора гарячий на дотик.

Лікується зовнішнім живленням 3.3 В на відповідний пін, минаючи бортовий стабілізатор.

Що з цього треба запам'ятати

`rst:0xf` означає живлення. Читати код при ньому марно.

Джерело має тягнути 1 А, навіть якщо середнє споживання — сто міліампер: платить воно за піки.

Конденсатор 470 мкФ біля модуля і короткий товстий кабель — найдешевше розв'язання найчастішої проблеми в книзі.

Вимикати детектор brownout — це прибрати діагностику, а не проблему.

Deep sleep — це перезавантаження; переживає його лише RTC RAM.

Бюджет енергії перевіряється вимірюванням, і найбільший ризик у ньому — час під'єднання до Wi-Fi.

07. Розпіновка й обмеження GPIO

Матриця GPIO робить більшість пінів взаємозамінними (розділ 04) — і саме через це легко забути, що взаємозамінні **не всі**. Піни-винятки не позначені на платі жодним чином, поведуться дивно і псують нерви довше, ніж будь-яка інша дрібниця.

Швидка довідка — картка K9. Тут — чому кожне обмеження існує, бо зрозуміле обмеження запам'ятовується, а список — ні.

Strapping-піни

При скиданні ROM-бутлоадер має вирішити, звідки завантажуватися. Джерелом рішення служать кілька звичайних GPIO, стан яких читається **один раз**, у момент відпускання скидання (розділ 16). Далі ці піни працюють як звичайні.

classic ESP32 classic: GPIO0, GPIO2, GPIO5, GPIO12, GPIO15.

Пін	Що задає	Наслідок помилки
GPIO0	звичайний старт або download mode	плата не стартує в застосунок
GPIO12	напругу живлення флешу: високий = 1.8 V	тривольтовий флеш не стартує
GPIO2	разом із GPIO0	заважає входу в download mode
GPIO15	чи друкує ROM boot-лог	лог зникає повністю
GPIO5	таймінги SDIO-веденого	рідко помітно поза SDIO

УВАГА

classic GPIO15, притиснутий до землі, вимикає boot-лог ROM.

У джерела формулювання пряме: MTD0, поданий низьким, глушить повідомлення, які друкує ROM-бутлоадер. Пін має внутрішнє підтягування вгору, тому не під'єднаний = високий = звичайний вивід.

Наслідок для діагностики важливий і неочевидний. «Плата мовчить на 115200» звично означає порт, живлення чи швидкість (картка K6) — але на classic це може означати просто резистор або світлодіод на GPIO15. Плата при цьому працює нормально, вона лише мовчить.

Перевірка коштує нуль: зняти обв'язку з GPIO15 і скинути.

НЕЗВОРОТНЕ

classic GPIO12 (MTDI) — найзліший пін у всій лінійці. Він задає напругу внутрішнього стабілізатора VDDSDIO, від якого живиться мікросхема флешу: високий рівень при старті означає **1.8 В**, низький — 3.3 В.

Уся зловісність — у тому, що на переважній більшості модулів (WROOM-32 і подібних) флеш **тривольтовий**. Він отримує 1.8 В, не запускається, і плата не подає ознак життя: ні логу, ні реакції, ні повідомлення про помилку.

Звідси й друга половина пастки: на модулях, де флеш справді на 1.8 В, той самий високий рівень — правильне налаштування. Тобто поведінка залежить від модуля, а не лише від піна, і «у сусіда працює» тут нічого не доводить.

Втішна половина: **сам пін має внутрішнє підтягування вниз**, тож ні до чого не під'єднаний GPIO12 = низький = 3.3 В = правильно. Тобто чип безпечний за замовчуванням, і високим його робить **тільки те, що причепили ззовні**.

Саме тому діагностика проста, а плати все одно викидають, вважаючи їх мертвими. Достатньо зняти те, що тримає GPIO12 високим.

Найчастіші винуватці: підтягувальний резистор, поставлений «про всяк випадок»; світлодіод із резистором на живлення; JTAG-адаптер (розділ 27); довгий вільний дріт, що ловить наводку.

S3 S3: GPIO0, GPIO3, GPIO45, GPIO46. Вхід у бутлоадер — GPIO0 притиснутий до землі, а GPIO46 при цьому **низький або вільний**. Підтягнутий угору GPIO46 не дає ввійти в download mode.

C3 C3: GPIO2, GPIO8, GPIO9. Вхід у бутлоадер — GPIO9 притиснутий до землі, GPIO8 при цьому високий. Комбінація GPIO8 = 0 і GPIO9 = 0 недійсна.

УВАГА

Другий strapping-пін працює на S3 і C3 у протилежні боки. Це джерело помилок при перенесенні плати з одного чипа на інший.

	Головний пін	Другий пін для входу в бутлоадер
classic	GPIO0 = 0	GPIO2 низький або вільний
S3	GPIO0 = 0	GPIO46 низький або вільний
C3	GPIO9 = 0	GPIO8 високий

І поняття «недійсна комбінація» існує лише в правому стовпці для C3 (і решти RISC-V): там GPIO8 = 0 разом із GPIO9 = 0 дає непередбачувану поведін-

ку. На classic і S3 такої комбінації немає — там неправильний рівень другого піна просто не пускає в download mode.

У звичайному режимі (головний пін високий) другий пін ігнорується взагалі — на всіх сімействах.

Практичне правило: strapping-піни можна використовувати, але як **виходи**, і бажано ті, що ні до чого не під'єднані під час старту. Вхід, кнопка чи датчик на strapping-піні — джерело збоїв, які проявляються раз на десять увімкнень.

УВАГА

Внутрішнє підтягування тут — 45 кОм, і цього мало для кнопки.

Кожен strapping-пін має внутрішній резистор саме такого номіналу. Він задає стан за замовчуванням, коли до піна нічого не під'єднано, — і саме тому вільний пін поводить себе передбачувано.

Але 45 кОм — це слабо. Кнопка boot на власній платі, зроблена «просто перемикачем на землю», конкурує з внутрішнім підтягуванням і на довгих доріжках або при наводці може не пересилити його надійно.

Правильно: **сильна підтяжка вниз, 10 кОм на землю**. Це рекомендація самої документації esptool, а не запас «про всяк випадок».

Піни флешу: не чіпати

classic GPIO6, GPIO7, GPIO8, GPIO9, GPIO10, GPIO11 з'єднані з мікросхемою флешу всередині модуля.

Вони **виведені на гребінку** більшості плат, підписані як звичайні GPIO — і це чиста пастка. Спроба їх використати підвищує чип або псує вміст флешу.

Правило категоричне: **classic** шість пінів 6–11 не існують. Ніколи, за жодних умов, у жодному проєкті.

УВАГА

classic На модулях із PSRAM до цього переліку додаються GPIO16 і GPIO17. Документація ESP-IDF називає їх в одному рядку з 6–11: «GPIO 6-11 and GPIO16-17 are usually connected to the SPI flash and PSRAM integrated on the module».

Різниця між шісткою й цією парою — у слові «usually». Піни 6–11 зайняті завжди; 16 і 17 — лише там, де на модулі є PSRAM, тобто на WROVER і подібних. На голому WROOM-32 вони вільні.

Практично це означає, що правило «шість пінів» безпечне лише доти, доки ви знаєте, який модуль тримаєте. Взяли WROVER за схемою, накресленою для WROOM, — і GPIO16 уже нічий.

S3 На S3 те саме стосується GPIO26–GPIO32, а на модулях з Octal PSRAM (N16R8 і подібні) — додатково GPIO33–GPIO37.

C3 На C3 — GPIO12–GPIO17, і окремо GPIO11: його майданчик у матриці називається VDD_SPI, тобто це вивід живлення самої флеш-пам'яті, а не звичайний пін. Перевести його в GPIO можна лише пропаленням eFuse VDD_SPI_A5_GPIO — незворотним (картка K11), і на модулі з внутрішнім флешем це забирає в флешу живлення. Рахуйте GPIO11 серед зайнятих.

УВАГА

S3 Це найпоширеніша причина «купив S3 із 16 МБ і 8 МБ PSRAM, а пінів менше, ніж на classic». Октальна PSRAM з'їдає п'ять додаткових пінів, і на платі вони або не виведені, або виведені й непридатні.

Перед проєктуванням плати на S3 варто точно знати, який модуль стоїть: N8 і N16R8 мають різну кількість доступних пінів.

Тільки-вхідні піни

classic GPIO34–GPIO39 мають лише вхідний буфер. У них немає:

- вихідного драйвера — керувати з них нічим не можна;
- **вбудованого підтягування** — ні pull-up, ні pull-down.

Друге важливіше, бо менш очевидне. Кнопка на GPIO34 без **зовнішнього** резистора не працює: вхід бовтається і читає випадкове (розділ 05). Виглядає як несправний пін або несправна кнопка.

Налаштуванням у коді це не змінюється: апаратної схеми немає.

У пізніших сімействах (S3, C3) тільки-вхідних пінів немає — усі повнофункціональні.

Аналогові обмеження

ADC1 і ADC2. У чипі два аналого-цифрові перетворювачі, і вони не рівноправні.

УВАГА

classic **S2** **S3** **ADC2 ділиться з Wi-Fi, і радіо має пріоритет.** Драйвер це передбачає: `adc_oneshot_read` розводиться з драйвером Wi-Fi і при зайнятості ADC2 повертає **помилку**, а не зіпсоване число.

Тобто зіпсованих даних чекати не варто — варто чекати читання, яке перестало вдаватися. Що гірше для того, хто не перевіряє код повернення: датчик просто «замовкає» на старому значенні.

Симптом: датчик читається правильно, доки не викликано `esp_wifi_start`, після чого читання перестає вдаватися. Людина шукає помилку в коді вимірювання, а справа в тому, який саме пін обрано.

Лікування одне: перенести вимірювання на **ADC1** — **classic** це GPIO32–GPIO39.

DAC. Справжній аналоговий вихід є лише у двох сімействах, і пін у них **різні**:

	Канал 1	Канал 2
classic	GPIO25	GPIO26
S2	GPIO17	GPIO18

Більше ніде в лінійці DAC немає (розділи 04 і 33).

Плутати їх дорого вдвічі: на S2 GPIO25 не просто не має DAC — його там не існує взагалі, маска дійсних пінів вирізає 22–25.

Touch. Ємнісні сенсори прив'язані до конкретних пінів і є лише в classic, S2 і S3.

Для всіх трьох матриця GPIO не діє: це аналогові блоки, фізично з'єднані з конкретними ніжками.

Службові пін

UART0 **classic** GPIO1 (TX) і GPIO3 (RX) — це консоль. Через них іде boot-лог і прошивка. Використати їх під щось інше можна, але тоді ви втрачаєте і лог, і зручну прошивку — тобто саме те, чим діагностують проблеми.

Правило: чіпати UART0 тільки тоді, коли пінів справді не лишилося.

USB-JTAG **S3** GPIO19, GPIO20; **C3** GPIO18, GPIO19. Переналаштувати їх можна, але це вимикає покрокове налагодження (розділ 27).

Скільки пінів насправді

Практичний підрахунок для **classic** 38-пінової плати. З 34 GPIO відкидаємо:

- 6 пінів флешу (6–11) — не існують;
- 2 пін консолі (1, 3) — краще не чіпати;
- 6 тільки-вхідних (34–39) — придатні лише під датчики;

Лишається близько **20 повноцінних** пінів, з яких п'ять — strapping і потребують уваги.

Типовий проєкт витрачає їх швидко: I²C — 2, SPI — 4 плюс по одному на пристрій, UART до другого контролера — 2, кнопка — 1, світлодіод — 1, реле — 2. Двадцяти вистачає не завжди.

ЗАКУПІВЛЯ

Коли пінів не вистачає, варіанти в порядку зростання складності:

Розширювач портів по I²C — PCF8574 (8 пінів) або MCP23017 (16) за дві лінії. Найдешевше і найпростіше; підходить для кнопок, світлодіодів, реле — усього, де не потрібна швидкість.

Зсувний регістр 74HC595 на виходи, 74HC165 на входи — по SPI, каскадуються скільки завгодно.

Аналоговий мультиплексор CD4051 — вісім аналогових входів на один пін ADC.

Чип із більшою кількістю пінів — S3 має їх більше за classic.

Другий мікроконтролер — коли задача й так ділиться на дві частини (розділ 57).

Як читати pinout

Головне правило: **pinout плати важливіший за pinout чипа**.

Виробник плати міг вивести не всі піни; підписати їх власними іменами (D2, A0, SDA) замість номерів GPIO; повісити на пін світлодіод, резистор або кнопку; використати частину пінів під власні потреби.

Тому: спершу шукайте схему **конкретної плати**, і лише як довідку — datasheet чипа.

УВАГА

Плати з однаковою назвою бувають різними. «ESP32 DevKit V1» продається у 30- і 38-піновому варіантах, і це **різні плати** з різним розташуванням пінів. Пінаут, знайдений за назвою, може не відповідати тому, що у вас у руках.

Надійна перевірка: порахувати піни й звірити з картинкою. Займає десять секунд, рятує від дуже неприємних помилок (розділ 08).

Що з цього треба запам'ятати

classic GPIO 6–11 не існують. Це не рекомендація.

classic GPIO12 високий при старті = флеш отримує 1.8 В замість 3.3 В. На три-вольтовому флеші — а він майже на всіх модулях — плата мовчить, без жодного повідомлення.

classic GPIO 34–39 — тільки вхід і **без вбудованого підтягування**.

classic **S2** **S3** ADC2 не працює при Wi-Fi; вимірювання переносяться на ADC1.

Strapping-піни краще використовувати як виходи й лишати вільними під час старту.

Pinout конкретної плати, а не чипа; порахувати піни перед тим, як довіритися картинці.

Частина III. Плати й підключення

*Від кристала до того, що лежить на столі:
модулі, плати, клони, кабелі, інструмент.*

08. Модулі, плати, анатомія девборди

Між кристалом і тим, що лежить на столі, — два шари, і кожен додає своїх особливостей (розділ 01). Цей розділ про те, як розпізнати, що саме у вас у руках, і чого від нього чекати.

Модулі

Модуль — кристал плюс кварц, флеш, антена й обв’язка в екранованому корпусі. Це те, що ставлять на власну плату у виробі.

Модуль	Чип	Особливості
ESP32-WROOM-32	classic	базовий, без PSRAM. Найпоширеніший
ESP32-WROOM-32D, -32E	classic	пізніші ревізії того самого
ESP32-WROVER, -B, -E	classic	з PSRAM частина пінів зайнята нею
ESP32-S3-WROOM-1	S3	сучасний вибір; варіанти за флешем і PSRAM
ESP32-C3-MINI-1	C3	малий і дешевий

Суфікс кодує пам’ять: N8 — 8 МБ флешу; N16R8 — 16 МБ флешу і 8 МБ PSRAM. Літера U в кінці (WROOM-1U) означає роз’єм під зовнішню антену замість друкованої.

УВАГА

classic WROVER має PSRAM, і вона займає піни. На модулях із PSRAM частина GPIO недоступна, хоча на пінауті чипа вони є. Тому проєкт, розведений під WROOM, не завжди переноситься на WROVER без змін — попри те, що модулі однакові за розміром і виводами.

Анатомія плати розробки

Крім модуля, на платі є п’ять речей, і кожна вміє зіпсувати життя.

Стабілізатор (LDO). Робить 3.3 В із 5 В. На дешевих клонах слабший за оригінальний і перегрівається при роботі радіо (розділ 06).

Міст USB-UART. Створює порт у системі. CP2102, CH340, CH9102 — кожен зі своїм драйвером (розділ 09). **S3** **C3** На платах із native USB може бути відсутній.

Схема авторесету. Два транзистори, керовані DTR і RTS. Дозволяє прошивати без натискання кнопок. Не універсальна: коли не спрацьовує — картка K4.

Кнопки BOOT і EN. Перша притискає GPIO0 до землі, друга — скидання.

Світлодіоди. Живлення і, часто, один на GPIO. Той, що на GPIO, корисно знати за номером: він займає пін і дає безкоштовний індикатор.

Конкретні плати

ESP32-DevKitC V4 — офіційна плата Espressif на 38 пінів. Еталон: якщо в документації сказано «плата розробки», мається на увазі вона. Передбачувана, добре розведена, дорожка за клони.

DevKit V1 / DOIT — найпоширеніший клон.

УВАГА

«ESP32 DevKit V1» продається у **30- і 38-пінному** варіантах. Це різні плати з різним розташуванням виводів. Назва в оголошенні однакова, пінауті — ні.

Перевірка: порахувати піни й звірити з картинкою, перш ніж довіритися знайденому в мережі пінауту. Десять секунд, що рятують від дуже неприємних помилок.

ESP32-S3-DevKitC-1 — сучасний вибір для нового проекту. Варіанти за пам'яттю (N8, N16R8) визначають, скільки пінів лишиться (розділ 07).

S3 Має **два USB-роз'єми**: один на native USB чипа, другий на міст USB-UART. Вони дають різні порти і роблять різне. Половина питань «чому на S3 нічого не працює» знімається переставленням кабелю.

ESP32-C3 SuperMini — дуже мала й дуже дешева. Пінів мало, PSRAM немає. Для простого вузла — оптимально; для чогось більшого впирається в 400 КБ.

D1 Mini ESP32 — у форматі старих плат ESP8266, під ті самі плати розширення.

ESP32-CAM — модуль із камерою OV2640 і роз'ємом microSD. Найдешевший спосіб отримати мережеву камеру, і найнезручніша плата в цьому переліку: **немає USB-роз'єму взагалі**, потрібен окремий програматор або USB-UART-перехідник, а GPIO0 замикається на землю перемичкою вручну. Камера з'їдає більшість пінів (розділ 49).

LilyGO, M5Stack, Heltec — плати з дисплеєм, корпусом, акумулятором, LoRa. Зручні як готовий пристрій; ціна — власна розводка пінів, під яку доводиться шукати їхні бібліотеки.

Як розпізнати підробку

Ринок клонів великий, і не всі клони погані. Погані ті, що видають себе за інше.

Перемаркований чип. Напис на модулі каже одне, шапка esptool — інше (розділ 23). Звіряти обов’язково.

Менша флеш. flash-id показує 2 МБ там, де за написом має бути 4. Прошивка, зібрана під 4 МБ, не запрацює, і причина буде неочевидною.

Фейковий USB-міст. Клони CP2102 і FT232 працюють нестабільно або перестають визначатися після оновлення драйвера (розділ 09).

Слабкий стабілізатор. Плата працює кілька хвилин, потім починає перезавантажуватися; корпус стабілізатора гарячий (розділ 06).

Погана пайка модуля. Видно під лупою: холодні з’єднання, зсунутий модуль, залишки флюсу.

Мінімальна перевірка нової партії плат — три команди:

```
esptool --port /dev/ttyUSB0 flash-id
esptool --port /dev/ttyUSB0 flash-id
```

плюс подивитися boot-лог. Займає хвилину на плату і ловить майже все.

ЗАКУПІВЛЯ

Практична стратегія: **купувати з запасом і перевіряти одразу**. Плати дешеві, і партія з десяти майже завжди містить одну проблемну. Знайти її на столі при отриманні дешевше, ніж у полі всередині виробу.

Конкретні магазини, ціни й наявність — у датованому вкладиші ринку, не тут (P5).

Уніфікація парку

Порада, що економить більше часу, ніж будь-яка інша в цьому розділі: звести парк до **двох** позицій — одна робоча, одна дешева (розділ 02).

Кожна нова модель плати означає власний пінаут, власну схему авторесету, власний драйвер мосту, власні граблі. Три однакові плати корисніші за шість різних: усе, що ви вивчили про одну, переноситься на решту, а запасна завжди сумісна.

Для виробу, який робиться в кількох екземплярах, це не порада, а вимога: партія має бути з однієї моделі й бажано з однієї закупівлі.

Своя плата замість готової

Коли пристрій виходить із прототипу, готова плата розробки стає недоліком: зайвий розмір, зайве споживання (USB-міст і світлодіод не сплять), ненадійні гребінки.

Три шляхи, у порядку зростання зусиль:

Плата розробки на перфборді. Найшвидше. Годиться для одиничних виробів (розділ 52).

Модуль на власній платі. Ви ставите WROOM і мінімальну обв'язку: стабілізатор, конденсатори, роз'єм для прошивки. Радіочастина й сертифікація вже всередині модуля — це головна причина брати модуль, а не кристал.

Кристал на власній платі. Потребує розводки радіочастини, узгодження антени й окремої сертифікації. Має сенс від дуже великих серій.

УВАГА

На власній платі не забудьте вивести **піни для прошивки**: TX, RX, пін входу в бутлоадер (**classic** **S2** **S3** GPIO0, **C3** **C6** **H2** GPIO9), EN, 3V3, GND. Шість контактних площадок, які коштують нічого на етапі розводки і рятують виріб, коли треба перепрошити його через рік. Плата без доступу до прошивки — це виріб, який можна тільки викинути (розділ 56).

Що з цього треба запам'ятати

Модуль — джерело істини про те, що всередині; звир'ятати напис із шапкою і flash-id.

«DevKit V1» буває 30- і 38-піновий — це різні плати.

S3 У DevKitC-1 два USB-роз'єми, і вони роблять різне.

ESP32-CAM не має USB-роз'єму й потребує окремого програматора.

Парк плат — дві моделі, не шість.

На власній платі завжди виводити шість контактів для прошивки.

09. Підключення до комп'ютера

Найперший бар'єр між людиною і ESP32 — не код і не схемотехніка, а питання «чому не з'явився порт». На нього припадає більше згаяного часу початківців, ніж на будь-що інше в цій книзі, і майже завжди причина виявляється не в платі.

Порядок пошуку — картка К3. Тут — як воно влаштоване і чому ламається саме так.

Що взагалі відбувається

ESP32 розмовляє з комп'ютером по UART — послідовному інтерфейсу з двома лініями даних. У комп'ютера UART-порту немає вже років двадцять. Тому між ними ставлять **міст USB-UART**: окремий чип, який з боку комп'ютера виглядає як віртуальний послідовний порт, а з боку плати говорить по UART.

На типовій платі розробки цей міст уже стоїть — це той менший чип поруч із USB-роз'ємом. Він робить дві речі: передає дані і, через сигнали DTR/RTS, керує входом у режим прошивки (розділ 16).

Звідси випливає головне: **порт у системі створює міст, а не ESP32**. Порт з'явився — це ще не означає, що чип живий. Порт не з'явився — це майже ніколи не означає, що чип мертвий.

Мости і їхні драйвери

Маркування чипа	Виробник	Windows	Linux
CP2102, CP2102N	Silicon Labs	драйвер із сайту SiLabs	у ядрі
CH340, CH340G, CH341	WCH	драйвер із сайту WCH	у ядрі
CH9102, CH9102F	WCH	окремий драйвер, не той, що CH340	у ядрі
FT232RL	FTDI	драйвер FTDI	у ядрі

Чотири практичні наслідки.

CH9102 — окрема пастка, і подвійна. Він зовні схожий на CH340, часто стоїть на платах, що продаються як «з CH340», і потребує іншого драйвера. Людина ставить драйвер CH340, він не працює, і виникає висновок «плата бракована».

Друга половина пастки — в іменах портів. **CH9102 з'являється як /dev/ttyACM0, а не ttyUSB0**: у ядрі його обслуговує не ch341, а клас CDC-ACM. А правило «ttyACM означає native USB» (нижче в цьому ж розділі) саме тому й не абсолютне: ttyACM каже про **клас пристрою**, а не про те, чи є всередині окремий міст.

Практично: побачивши `ttyACM` на платі, яку вважали `classic`, не робіть висновку, що це S3. Дивіться на сам чип біля роз'єму.

FT232RL масово підробляють. Клони працюють, поки офіційний драйвер їх не розпізнає. Історично драйвери FTDI вміли робити підроблені чипи непрацездатними. Сьогодні це рідше, але сам факт: якщо плата з FT232 раптом перестала визначатися після оновлення драйвера — справа може бути в цьому.

Linux у більшості випадків не потребує нічого. Драйвери всіх поширених мостів у ядрі. Якщо порт не з'явився — дивіться `dmesg`, а не шукайте драйвер.

Різні плати — різні імена портів. Один і той самий комп'ютер може дати `/dev/ttyUSB0` одній платі й `/dev/ttyUSB1` іншій, і після перевикання номери міняються. Прив'язуватися до номера в скриптах — джерело сюрпризів.

Native USB: S3, C3 і новіші

S3 **C3** мають USB-контролер на самому кристалі. Мосту не потрібно взагалі: чип під'єднується до комп'ютера напряму.

Що це змінює:

Порт має інше ім'я. `/dev/ttyACM0` замість `/dev/ttyUSB0` у Linux; у Windows — теж COM-порт, але іншого класу. Це перше, що збиває з пантелику при переході з `classic` на S3.

Драйвер не потрібен. Пристрій відповідає стандарту USB CDC, який підтримується всіма системами з коробки.

JTAG іде тим самим кабелем. Повноцінне покрокове налагодження без жодного додаткового заліза (розділ 27).

Порт може зникати і з'являтися. Оскільки USB реалізує сам чип, при перезавантаженні або паніці порт відвалюється від системи і повертається за секунду. На мостовій платі такого не буває: міст живе окремо. Це нормальна поведінка `native USB`, а не несправність — але монітор порту при цьому доводиться пере-відкривати.

УВАГА

S3 Плати S3-DevKitC-1 мають **два** USB-роз'єми: один іде на `native USB` чипа, другий — на звичайний міст `USB-UART`. Вони роблять різні речі і дають різні порти. Половина питань «чому на S3 нічого не працює» знімається переставлянням кабелю в сусідній роз'єм.

Кабель

Найчастіша причина всього розділу, і вона банальна: **кабель без жил даних**. Кабелі з комплектів до павербанків, ліхтариків і зарядок часто мають лише живлення. Зовні вони не відрізняються ніяк.

Перевірка займає п'ять секунд: під'єднати цим кабелем телефон або флешку. Не визначилося — кабель для заряджання.

Друга кабельна причина — довжина і якість. Метрові тонкі кабелі дають просадку напруги, і прошивка обривається на `MD5 of file does not match data in flash!`. Короткий товстий кабель розв'язує більше проблем, ніж здається.

ЗАКУПІВЛЯ

Два-три завідомо справних кабелі, підписані маркером, — найдешевша інвестиція в цій книзі. Тримати окремо від решти. Кабель, який лежить у спільній коробці, рано чи пізно виявиться зарядним саме тоді, коли треба терміново.

Права на порт у Linux

Порт `/dev/ttyUSB0` є, програма пише `Permission denied`. Це права доступу, несправність:

```
sudo usermod -aG dialout $USER
```

Далі — **перезайти в систему**. Без цього нова група не застосується до поточної сесії, і виглядатиме, ніби команда не подіяла.

У деяких дистрибутивах група називається `uucp` замість `dialout` — подивитися можна так:

```
ls -l /dev/ttyUSB0
```

УВАГА

Спокуса запустити прошивку через `sudo` дуже велика, і вона працює — один раз. Далі виявляється, що каталог `build/` тепер належить `root`, і звичайне збирання ламається в неочевидний спосіб. Це витрачає більше часу, ніж перезайти в систему.

Порт зайнятий

```
Device or resource busy
```

Порт відкриває **лише один процес одночасно**. Найчастіші винуватці: забутий відкритий монітор, Arduino IDE у фоні, `screen` у сусідньому терміналі, `PlatformIO`.

Хто саме тримає порт:

```
lsuf /dev/ttyUSB0
```

Це найчастіша причина «esptool раптом перестав бачити плату» після того, як хвилину тому все працювало.

Діагностика в Linux: dmesg

Найшвидший спосіб зрозуміти, що сталося при під'єднанні:

```
dmesg | tail -20
```

Оразу після втикання кабелю. Три можливі результати:

Нічого не з'явилося — комп'ютер не побачив пристрій узагалі. Кабель, роз'єм або живлення плати.

Пристрій визначився, порт створено — все добре з боку системи, далі дивитися права і зайнятість.

Пристрій визначився з помилками (device descriptor read error, постійні перепід'єднання) — проблема живлення або кабелю. Класично на довгому кабелі через хаб.

Windows: диспетчер пристроїв

Жовтий знак оклику біля пристрою означає **драйвер**, а не залізо. Плата жива, система просто не знає, як з нею говорити.

Пристрій із назвою на кшталт USB2.0-Serial без COM-порту — той самий випадок.

Пристрій зникає і з'являється сам — живлення або кабель.

USB-хаб: коли він рятує, а коли шкодить

Рятує хаб із власним живленням. Він знімає навантаження з порту ноутбука, дає стабільну напругу і рятує сам порт при замиканні на платі. Для роботи з незнайомим залізом це правильна практика: дешевше замінити хаб, ніж материнську плату.

Шкодить дешевий хаб без живлення. Він додає просадку і затримки; на ньому не спрацьовує авторесет, обривається прошивка і плавають незрозумілі збої.

Правило просте: хаб із блоком живлення — так, хаб-«свисток» на чотири порти — ні.

Кілька плат одночасно

Коли в системі кілька плат, номери портів плутаються після кожного перевтикання. У Linux надійний спосіб — звертатися до пристрою за серійним номером мосту:

```
ls -l /dev/serial/by-id/
```

Шлях звідти не змінюється при перевиканні і не залежить від порядку під'єднання. Для скриптів серійної прошивки (розділ 21) це те, що варто виконувати замість `/dev/ttyUSB0`.

Що з цього треба запам'ятати

Порт створює міст, а не ESP32: поява порту не доводить, що чип живий, а відсутність — що мертвий.

Кабель перевіряється телефоном за п'ять секунд і є найчастішою причиною.

CH9102 потребує іншого драйвера, ніж CH340, попри схожість.

На S3 і C3 порт називається `ttuasm`, драйвер не потрібен, а на DevKitC-1 роз'ємів два, і вони роблять різне.

`sudo` для прошивки — не розв'язання проблеми прав, а створення нової.

Хаб із власним живленням допомагає; хаб без живлення шкодить.

10. Комплект інструментів

Перелік того, що справді використовується, з поясненням чому. Конкретні позиції, ціни й наявність — у датованому вкладиші ринку (P5); картка K12 — та сама інформація на одну сторінку в поле.

Головна ідея розділу: на трьох речах економити не варто, на решті — можна.

Мінімум, без якого не працює

Мультиметр. Найважливіший прилад у списку. Потрібні три функції: прозвонка **зі звуком** (щоб не відривати очей від щупів), вимірювання постійної напруги, вимірювання опору. Струм — рідше, але треба.

ЗАКУПІВЛЯ

Найдешевші мультиметри брешуть на межах діапазонів і мають повільну прозвонку, яка не встигає піскнути при швидкому дотику. Це саме той прилад, де перехід від найдешевшого до просто дешевого дає найбільшу різницю. Він живе роками і використовується щодня.

Паяльник із терморегулятором. Ключове — терморегулятор. Паяльник без нього або не гріє достатньо (холодні з'єднання), або перегрівається й відриває контактні площадки разом із доріжкою.

Потужність 60 Вт, жало «скіс» 2–3 мм як основне. Тонке конічне жало, всупереч інтуїції, гірше: воно гірше передає тепло, і паяти ним довше, а отже гарячіше для плати (розділ 51).

Припій і флюс. Припій зі свинцем (Sn63Pb37 або Sn60Pb40) 0.5–0.8 мм плавиться за нижчої температури й прощає більше помилок, ніж безсвинцевий. Флюс-гель у шприці — не «для професіоналів», а те, що перетворює паяння з боротьби на процедуру.

USB-кабелі data. Два-три завідомо справні, підписані маркером, короткі. Це найдешевша інвестиція в книзі й найчастіша причина втрачених годин (розділ 09).

Макетна плата і Dupont-дроти. Обов'язково всі три види: мама-мама, тато-тато, тато-мама. Брати з запасом — вони псуються й губляться.

USB-хаб із власним живленням. Знімає навантаження з порту ноутбука і рятує сам порт при замиканні на платі. Дешевше замінити хаб, ніж материнську плату (розділ 06).

Те, що окупається на другому пристрої

Логічний аналізатор на 8 каналів. Найкраще співвідношення користі до ціни в усьому переліку. Разом із PulseView перетворює «датчик не працює» на конкретний факт: чи почався обмін, чи відповів ведений, які байти пішли (розділ 28).

Дешеві моделі мають межу дискретизації близько 24 МГц — для I²C, UART і повільного SPI цього досить із запасом.

Лабораторний блок живлення з обмеженням струму. Обмеження струму — головна функція, а не регульована напруга. Виставили 200 мА — і замикання в новій схемі закінчується тим, що прилад просто не дасть більше, замість випаленої доріжки.

Для роботи з незнайомими платами це прилад, що окупається одним врятованим виробом.

USB-тестер напруги і струму. Вставляється між портом і платою. Ловить просадки, показує реальне споживання, рахує накопичений заряд. Коштує копійки, а відповідає на питання «чи вистачає живлення», яке інакше лишається здогадкою.

Термофен. Єдиний спосіб зняти модуль або мікросхему, не відірвавши площадки. Потрібен, коли доходить до ремонту (розділ 55).

Пінцет, бокорізи, оплітка для випаювання. Оплітка знімає зайвий припій, а не розмазує його. Без неї виправлення перемички між пінами перетворюється на псування плати.

Лупа або мікроскоп. Холодну пайку й перемичку між сусідніми ніжками незброєним оком не видно. Дешева USB-камера з підсвіткою закриває питання.

JTAG-адаптер. classic Потрібен лише для classic; на S3 і C3 вбудований (розділ 27). Купувати варто тоді, коли з'явилася конкретна задача, а не про запас.

Витратне, яке закінчується не вчасно

Гребінки male і female кількох висот; термоусадка трьох-чотирьох діаметрів; дроти 22–26 AWG у трьох кольорах (обов'язково червоний, чорний і будь-який третій); ізоляційна стрічка; ізопропіловий спирт і безворсові серветки для змивання флюсу; стяжки.

Окремо: **резистори і конденсатори розсипом.** Найчастіші номінали — 220 Ом (світлодіоди), 4,7 кОм (I²C), 10 кОм (підтягування), 100 нФ і 470 мкФ (живлення). Набір із сотні номіналів коштує дешево і рятує десятки разів.

Три позиції, де дешеве нормальне

Макетна плата. Різниці майже немає. Купувати кілька.

Dupont-дроти. Витратний матеріал.

Гребінки. Те саме.

Загальне правило: **вимірювальні прилади й те, що торкається плати гарячим, — не економити. Те, що з'єднує і тримається, — можна.**

Робоче місце

Кілька дрібниць, що впливають більше, ніж здається:

Світло. Направлене, яскраве. Половина дефектів пайки — це дефекти видимості.

Килимок. Термостійкий силіконовий: не плавиться від жала, дрібні деталі не розкочуються.

Антистатика. Браслет — ідеально; торкнутися заземленого металу перед роботою — безкоштовно і майже так само дієво (розділ 05).

Коробка з відділеннями. Плати без коробки перетворюються на купу однакових зелених прямокутників, у якій незрозуміло, що вже перевірене, а що ні.

Маркер по пластику. Підписувати плати одразу: номер, версія прошивки, дата. Плата без позначки через місяць не означає нічого (розділ 21).

Що покласти в сумку, яка їде в поле

Плата з уже залитою робочою прошивкою; перевірений кабель; павербанк; мультиметр; викрутки; ніж; ізоляційна стрічка; термоусадка; запасні гребінки; стяжки.

І окремо — **роздруковані й заламіновані картки К1–К15**. Вони для цього й зроблені: одна сторінка А4 на тему, читається при поганому світлі, не потребує заряду.

Ноутбук із **офлайн-комплектom** (додаток F): встановлений тулчейн, завантажена документація, закешовані бібліотеки, збережені образи прошивок. У полі інтернету немає, і саме там він потрібен найбільше (розділ 15).

УВАГА

Найчастіша помилка виїзного комплекту — узяти інструмент і забути **витратні матеріали**. Паяльник без припою, термоусадка не того діаметра, жодного запасного дроту. Комплект варто зібрати один раз, скласти перелік і зв'язати по ньому перед виїздом, а не згадувати щоразу наново.

Що з цього треба запам'ятати

Мультиметр, паяльник із терморегулятором і кабель — три позиції, на яких не варто економити.

Логічний аналізатор — найкраще співвідношення користі до ціни серед усього іншого.

Обмеження струму на блоці живлення рятує плати, а не регульована напруга.

Макетка, Dupont і гребінки — беріть дешеві й з запасом.

У виїзний комплект кладуть витратні матеріали, а не лише інструмент.

Частина IV. Інструментарій

ESP-IDF як нормативне ядро, Arduino як швидкий шлях, і чесна межа між ними.

Версії тулчейну

ревізія 2026-08-26

Що	Версія	Стан
ESP-IDF	6.0.2	звірено 2026-08-26
ESP-IDF, запасна гілка	5.5.x	звірено 2026-08-26
esptool	5.3.1	звірено 2026-08-26
Arduino core для ESP32	3.3.11	звірено 2026-08-26
Розширення ESP-IDF для VS Code	2.2.0	звірено 2026-08-26
pioarduino (форк platform-espressif32)	55.03.311	звірено 2026-08-26
build_pandoc	3.1.3	звірено 2026-08-26
build_typst	0.15.0	звірено 2026-08-26
build_fonts	Libertinus Serif (вбудований у Typst) + Liberation Sans + DejaVu Sans Mono	звірено 2026-08-26

ESP-IDF. поточний stable; підтримка гілки v6.0 до 2028-09-20

ESP-IDF, запасна гілка. лише якщо цього вимагає стороння бібліотека. Окремого статусу LTS не існує — 30 місяців підтримки має кожен major і кожен minor однаково

esptool. v5 перейшов на дефіси замість підкреслень (write-flash, chip-id) і прибрав суфікс .py. Інструкції з мережі під v4 дослівно не працюють

Arduino core для ESP32. опубліковано 2026-07-22

Розширення ESP-IDF для VS Code. офіційне розширення ESP-IDF від Espressif; ідентифікатор espressif.esp-idf-extension

pioarduino (форк platform-espressif32). community-форк platform-espressif32; офіційна платформа PlatformIO відстала і не має підтримки Arduino core 3.x. Версія знята з гілки main; у проєкті пінується конкретний тег релізу

build_pandoc. markdown → typst. Різниця між 3.1 і 3.10 змінює розбиття абзаців у typst-фрагменти й дає кілька сторінок на всю книгу

build_typst. typst → PDF, ставиться з rip. Між мінорними версіями змінюються виключка й розбиття рядків

build_fonts. Libertinus Sans у системі **немає** ні в M1, ні в M2 — заголовки падають на Liberation Sans в обох, тож шрифт не є причиною розбіжності сторінок. У шаблоні він лишається першим у переліку свідомо: якщо колись з'явиться, верстка стане кращою, а BUILD.txt покаже зміну виготовлювача

Ці значення не переписуються в тексті розділів (P4): вони живуть у файлі toolchain-baseline.yaml і потрапляють у книгу лише звідси. Рядок із позначкою **НЕ ЗВІРЕНО** означає, що значення вжите, але з першоджерелом не звірене — довіряти йому як решті не можна.

11. ESP-IDF + розширення VS Code

ESP-IDF (Espressif IoT Development Framework) — офіційний фреймворк Espressif. Це нормативне ядро довідника (P3): **усі приклади зобов’язані працювати тут**. Решта тулчейнів — надбудови над ним або альтернативи з власними обмеженнями.

Причина такого вибору не в ідеології. ESP-IDF — єдиний шлях, на якому доступна вся периферія, вся діагностика й уся документація, і єдиний, де відповідь на питання «чому воно так» існує в первинному вигляді.

Що це таке

ESP-IDF — не IDE і не редактор. Це набір із трьох частин:

Тулчейн — компілятор і бінарні утиліти під архітектуру чипа (різні для Xtensa і RISC-V, розділ 02).

Фреймворк — FreeRTOS, драйвери периферії, стеки Wi-Fi, Bluetooth, TCP/IP, TLS, файлові системи, система збирання.

Інструменти — `idf.py`, `esptool`, монітор, менеджер компонентів.

Редактор ви обираєте самі. Офіційне розширення для VS Code — зручна обгортка, але робота йде через ті самі команди, які можна набрати руками.

Встановлення

Windows. Офіційний інстальатор ESP-IDF Tools Installer ставить усе разом: тулчейн, Python, git, сам фреймворк. Це найпростіший шлях і рекомендований для Windows.

Linux і macOS. Клонування репозиторію і скрипт установлення:

```
IDF=v6.0.2 # версію брати з таблиці на початку частини
mkdir -p ~/esp && cd ~/esp
git clone -b $IDF --recursive https://github.com/espressif/esp-idf.git esp-idf-$IDF
cd esp-idf-$IDF && ./install.sh esp32,esp32s3,esp32c3
```

Гілка вказується явно — конкретна версія, а не master. Перелік цілей обмежує обсяг завантаження: тулчейни ставляться під кожную архітектуру окремо, і ставити всі немає сенсу.

Каталог названо за версією. Це не педантизм: щойно на машині з’явиться друга версія — а вона з’явиться, щойно ви візьмете чужий проєкт, — каталог `esp-idf` без номера стане джерелом плутанини, у якій неможливо сказати, що саме зараз активне.

Перед роботою в кожному новому терміналі:

```
./~/esp/esp-idf-v6.0.2/export.sh
```

Ця команда додає інструменти в PATH і ставить змінні середовища. Забути її — найчастіша причина «команду `idf.py` не знайдено».

УВАГА

Спокуса прописати `export.sh` у `.bashrc` є в усіх, і вона має ціну: якщо на машині кілька версій ESP-IDF (а це трапляється, щойно ви берете чужий проєкт), автоматична активація однієї з них створює дуже заплутані помилки збирання.

Надійніше — короткий псевдонім, який активує потрібну версію свідомо:

```
alias idf6='./~/esp/esp-idf-v6.0.2/export.sh'
alias idf5='./~/esp/esp-idf-v5.5/export.sh'
```

Версія: яку брати

Правило R4: версії живуть у `toolchain-baseline.yaml`, а не в тексті. Таблиця версій цієї ревізії надрукована на початку цієї частини — саме там дивитися, який номер підставляти в команди нижче.

Принцип вибору простий:

Беріть поточний `stable`. Кожен `major` і `minor` реліз підтримується 30 місяців від дати виходу. Окремого статусу LTS в ESP-IDF **не існує** — це поширена помилка форумів: у всіх релізів однаковий термін.

Ці 30 місяців діляться навпіл не порівну, і саме цей поділ, а не сам термін, відповідає на питання «яку брати»:

Період	Тривалість	Для нового проєкту
Service	перші 12 місяців	так
Maintenance	наступні 18 місяців	ні

У Service-періоді виправлення виходять регулярно; у Maintenance бекпортують лише серйозні й безпекові. Тобто версія, якій два роки, формально ще підтримується — але новий проєкт на ній починати не варто, хоч вона й не EOL.

Не беріть `master`. Це гілка розробки; вона ламається, і ваша проблема може виявитися чужим незавершеним комітом.

Старішу версію беріть лише тоді, коли цього вимагає конкретна стороння бібліотека, — і фіксуйте це в документації проєкту з причиною.

НЕЗВОРОТНЕ

Версія ESP-IDF фіксується на початку проєкту й записується. Не «остання», а конкретний тег.

Причина практична: перезібрати «такий самий» образ через рік на іншій версії майже ніколи не виходить. Адреси зсуваються, і збережений .elf перестає відповідати прошивці в полі — тобто backtrace із поля стає нерозшифровним (розділи 21, 26).

idf.py: команди, якими користуються

```
idf.py create-project my-project      # новий проєкт
cd my-project
idf.py set-target esp32s3             # цільовий чип
idf.py menuconfig                    # налаштування
idf.py build                          # зібрати
idf.py -p /dev/ttyUSB0 flash monitor
```

Остання — найчастіша команда в щоденній роботі: зібрати, залити, одразу відкрити монітор.

Решта, що трапляється:

Команда	Навіщо
idf.py fullclean	коли збирання поводитьься незрозуміло
idf.py size	скільки зайнято флешу і RAM
idf.py size-components	хто саме займає місце
idf.py coredump-info	розбір coredump із флешу (розділ 26)
idf.py openocd gdb	покрокове налагодження (розділ 27)
idf.py erase-flash	стерти (⚠ спершу дамп, картка K2)

УВАГА

idf.py set-target **стирає sdkconfig**. Усі налаштування з menuconfig повертаю-ться до типових.

Захист від цього — файл sdkconfig.defaults у корені проєкту: те, що в ньому, застосовується при кожному створенні конфігурації наново. Саме він має лежати в git, а не sdkconfig.

sdkconfig у репозиторії — поширена помилка: файл великий, змінюється від кожної дрібниці й породжує конфлікти при злитті.

Структура проєкту

```

my-project/
  CMakeLists.txt      ← корінь проєкту
  sdkconfig.defaults  ← налаштування, що йдуть у git
  sdkconfig           ← згенероване, у git не кладеться
  main/
    CMakeLists.txt
    main.c
  components/         ← власні компоненти
    my_sensor/
      CMakeLists.txt
      my_sensor.c
      include/my_sensor.h

```

Компонент — одиниця повторного використання: каталог із власним CMakeLists.txt, вихідними текстами й заголовками. main — теж компонент, просто особливий.

Мінімальний CMakeLists.txt компонента:

```

idf_component_register(
    SRCS "my_sensor.c"
    INCLUDE_DIRS "include"
    REQUIRES driver esp_timer
)

```

REQUIRES перелічує компоненти, чиї заголовки ви підключаєте. Помилка виду «файл не знайдено», коли файл є, — майже завжди відсутній REQUIRES.

menuconfig і sdkconfig

idf.py menuconfig — текстове меню налаштувань фреймворку. Розділів сотні; знати всі не треба, але кілька місць варто відвідати свідомо:

Що	Де саме
Розмір флешу і розбивка	Serial flasher config, Partition Table
Рівень логування	Component config → Log → Log Level
Частота ядра	Component config → ESP System Settings → CPU frequency
Watchdog і його таймаути	Component config → ESP System Settings
Coredump	Component config → Core dump
Підтримка PSRAM	Component config → ESP PSRAM
Оптимізація за розміром	Compiler options → Optimization Level → Optimize for size

Три перші пункти меню — Serial flasher config, Partition Table і Bootloader config — лежать у корені, решта всередині Component config. Це не косметика: у корені живе те, що стосується самої збірки й прошивки, а не окремого компонента.

Пошук усередині menuconfig — клавіша /. Це найкорисніша клавіша в усьому інтерфейсі: назви параметрів здебільшого відомі з документації, а шукати їх по деревах довго.

Менеджер компонентів

Сторонні бібліотеки ставляться з реєстру компонентів Espressif:

```
idf.py add-dependency "espressif/led_strip^3.0.3"
```

Це створює файл idf_component.yml, який фіксує залежності проєкту. Файл кладеться в git — саме він робить проєкт відтворюваним.

Номер версії в команді — з реєстру **на момент роботи**, а не з книги (P4): компоненти оновлюються значно частіше за ревізії довідника. Значок ^ означає «ця мажор-версія, будь-яка новіша minor», тож ^3.0.3 не візьме 4.x — і це саме те, чого хочеться від залежності у виробі.

Реєстр невеликий порівняно з екосистемою Arduino, і це реальне обмеження ESP-IDF: бібліотеки на конкретний датчик там може не бути (розділ 12).

Розширення для VS Code

Офіційне розширення від Espressif робить те саме, що idf.py, тільки кнопками, і додає інтеграцію з відлагоджувачем.

Ключове після встановлення — **вибрати гілку ESP-IDF**: розширення вміє встановити фреймворк саме або використати вже встановлений. Другий варіант надійніший, якщо ви вже працюєте з командного рядка: інакше на машині з'являються дві копії й починається плутанина, яка з них активна.

Що дає розширення понад командний рядок:

- **IntelliSense**, налаштований на конкретний чип: автодоповнення знає, які функції доступні саме тут;
- **налагодження в один клік** (розділ 27);
- **перегляд регістрів периферії** з розшифровкою бітових полів;
- **вбудований монітор** із розшифровкою backtrace.

УВАГА

Найчастіша проблема з розширенням — IntelliSense показує помилки там, де збирання проходить успішно. Причина зазвичай у тому, що конфігурація розширення вказує на іншу версію ESP-IDF, ніж та, якою збирається проєкт.

Червоні підкреслення при успішному `idf.py build` — це проблема редактора, а не коду.

Чому саме ESP-IDF, а не щось простіше

Чесна відповідь: ESP-IDF складніший за Arduino і вимагає більше часу на старті. Він виграє в тому, що стає видно пізніше:

Уся периферія доступна. Блоки на кшталт MCPWM, PCNT, TWAI в Arduino доступні частково або через сторонні обгортки (розділ 04).

Діагностика. CoreDump, розшифровка backtrace, JTAG, докладний лог підсистем — усе штатне (розділи 26, 27).

Керування пам'яттю й таймінгами. Розміри стеків, прив'язка до ядер, пріоритети задач, IRAM_ATTR — усе під контролем (розділи 30, 31).

Відтворюваність. Зафіксована версія фреймворку, зафіксовані компоненти, збережений `sdkconfig`. Виріб, який треба супроводжувати роками, будується так.

Arduino core лишається правильним інструментом для швидкого прототипування — і в довіднику він саме в цій ролі (розділ 12).

Що з цього треба запам'ятати

`export.sh` у кожному новому терміналі; свідомий псевдонім замість запису в `.bashrc`.

Версія фіксується на початку проєкту й записується. LTS в ESP-IDF не існує — у всіх релізів 30 місяців, із яких перші 12 — Service, і саме вони придатні для нового проєкту.

`set-target` стирає `sdkconfig`; у `git` кладеться `sdkconfig.defaults`.

`REQUIRES` у `CMakeLists.txt` — причина більшості «файл не знайдено».

Клавіша / у `menuconfig`.

12. Arduino core для ESP32

Arduino core — це шар сумісності поверх ESP-IDF. Не альтернативна платформа, не окремий фреймворк: усередині працює той самий ESP-IDF, той самий FreeRTOS, ті самі драйвери.

Розуміння цього факту знімає більшість запитань про межі Arduino: він не може менше, ніж ESP-IDF, — він просто показує менше.

Що під капотом

Коли ви пишете:

```
void setup() { Serial.begin(115200); }  
void loop() { Serial.println("привіт"); delay(1000); }
```

насправді відбувається таке: ESP-IDF стартує звичайним чином, створює задачу FreeRTOS (loopTask), у ній один раз викликає setup, а потім нескінченно викликає loop. Serial — обгортка над драйвером UART. delay — це vTaskDelay, тобто нормальне засинання задачі, а не холостий цикл.

Практичні наслідки, які варто знати одразу:

loop — звичайна задача FreeRTOS. Вона має свій стек і свій пріоритет, і поруч можуть працювати інші задачі.

Функції ESP-IDF доступні прямо з коду Arduino. Нічого не заважає підключити esp_sleep.h і викликати esp_deep_sleep_start() у скетчі. Це не хак, а нормальне використання.

delay не блокує систему. На відміну від AVR, тут під час затримки працює планувальник, і решта задач виконується.

УВАГА

Звідси випливає найкорисніше практичне знання про Arduino на ESP32: **коли впираєтеся в межу Arduino API — не переписуйте проєкт, а викличте функцію ESP-IDF безпосередньо.** Змішувати їх можна і нормально.

Саме так робиться доступ до тих блоків периферії, яких немає в Arduino API: MCPWM, PCNT, TWAI, тонке керування живленням.

Де вигреш

Бібліотеки. Головна і часто вирішальна причина. Екосистема Arduino має бібліотеку майже на кожен датчик, дисплей і модуль, який продається. Реєстр компонентів ESP-IDF значно менший (розділ 11).

Коли вам треба сьогодні прочитати незнайомий датчик, а бібліотека на нього існує тільки для Arduino — вибір очевидний.

Швидкість старту. Робочий скетч пишеться за хвилини, без CMake, без компонентів, без menuconfig.

Знайомий API. Якщо ви вже писали для Arduino, переносити знання майже не треба.

Прототипування. Перевірити ідею, знайти робочі таймінги датчика, показати замовнику щось живе — усе це швидше на Arduino.

Де стеля

Керування пам'яттю. Розміри стеків задач, вибір області для великих буферів, IRAM_ATTR — усе це або недоступне, або робиться в обхід Arduino API.

Діагностика. Coredump, докладне логування підсистем, розшифровка backtrace на місці — усе це доступно, але налаштовується не з Arduino IDE (розділ 26).

Розмір прошивки. Arduino core тягне за собою більше, ніж мінімально потрібно. На платі з 4 МБ це рідко проблема; при двох ОТА-слотах уже буває тісно (розділ 18).

Відтворюваність. Версії бібліотек із менеджера Arduino IDE фіксуються гірше, ніж залежності в idf_component.yml. Для виробу, який треба зібрати заново через три роки, це реальний ризик.

Якість сторонніх бібліотек. Дуже нерівна. Багато написано під AVR і перенесено формально: блокувальні затримки в мілісекунди, робота з пам'яттю в розрахунку на 2 КБ RAM, ігнорування помилок шини.

УВАГА

Типовий сценарій, через який проект впирається в стелю: бібліотека датчика всередині робить `delay(750)` під час вимірювання. На AVR це нормально; тут це означає, що задача стоїть три чверті секунди, і якщо від неї залежить щось із таймінгами — воно ламається.

Лікується або заміною бібліотеки, або переписуванням цієї частини на штатний драйвер ESP-IDF. Друге частіше.

Дві гілки: 2.x і 3.x

Arduino core версії 3.x — велике оновлення: він переїхав на ESP-IDF 5.x і разом із тим на нове покоління драйверів периферії.

Що зламалося при переході, за частотою:

API периферії. Функції керування PWM і аналоговим введенням змінили сигнатури. Код, знайдений у статті трирічної давності, не збереться.

Бібліотеки. Частина сторонніх бібліотек із 2.x на 3.x не працює; частина оновлена, частина покинута назавжди.

Поведінка за замовчуванням. Дрібні відмінності в ініціалізації периферії, які вилазять уже в роботі.

УВАГА

Практична порада: **не змішувати статті різних епох**. Знайшовши в мережі приклад, що не збирається, спершу подивіться на його дату й на те, під яку версію core він написаний. «Не працює» тут частіше означає «інша версія», ніж «помилка».

І окремо: гілка 4.x на ESP-IDF 6.x існує в статусі альфи. Для роботи вона не призначена.

Три способи використати Arduino core

Arduino IDE. Найпростіший вхід. Ставиться через менеджер плат за URL. Годиться для навчання і швидких перевірок; для проекту з кількох файлів незручний.

PlatformIO або pioarduino. Той самий core, але з нормальним керуванням залежностями, версіями і кількома цільовими середовищами в одному проекті (розділ 13).

Arduino як компонент ESP-IDF. Найцікавіший варіант: проект будується як звичайний проект ESP-IDF, а Arduino підключається компонентом. Тоді доступні `setup/loop` і бібліотеки Arduino — і водночас `menuconfig`, компоненти, розбивка флешу й уся діагностика.

Це правильний шлях для проекту, який почався як прототип на Arduino, а має стати виробом: не переписувати все одразу, а перенести під ESP-IDF і далі поступово замінювати частини.

Коли брати Arduino, а коли ESP-IDF

Ситуація	Що брати
Перевірити ідею за вечір	Arduino
Потрібна бібліотека, якої немає в IDF	Arduino або гібрид
Незнайомий датчик, треба сьогодні	Arduino
Виріб, який супроводжуватимуть роками	ESP-IDF

Ситуація	Що брати
Потрібні MCPWM, PCNT, TWAI	ESP-IDF або виклики IDF зі скетча
Жорсткі вимоги до пам'яті чи таймінгів	ESP-IDF
Серійне виробництво, OTA, відтворюваність	ESP-IDF
Прототип уже є, треба довести до виробу	Arduino компонентом IDF

Що з цього треба запам'ятати

Arduino core — це шар над ESP-IDF, а не окрема платформа; функції IDF доступні прямо зі скетча.

`delay` тут не блокує систему, а `loop` — звичайна задача FreeRTOS.

Головний виграш Arduino — бібліотеки; головна стеля — керування пам'яттю, відтворюваність і якість цих же бібліотек.

Перехід 2.x → 3.x зламав API периферії: приклад із мережі може не збиратися саме через це.

Прототип на Arduino доводиться до виробу підключенням Arduino компонентом ESP-IDF, а не переписуванням з нуля.

13. PlatformIO і pioarduino

PlatformIO — розширення для VS Code, що керує вбудованими проектами: тулчейни, платформи, бібліотеки, кілька цільових середовищ в одному проекті. Для ESP32 воно дає те, чого не дає Arduino IDE: фіксовані версії, нормальний менеджер залежностей і конфігурацію в одному файлі.

Ситуація навколо нього нетипова, і про неї треба знати до того, як почнете.

Статус: чому все складно

Підтримка ESP32 у PlatformIO забезпечується платформою `platform-espressif32`. Офіційна платформа від PlatformIO **відстала** від Arduino core: підтримка Arduino 3.x у ній офіційно не з'явилася, і через відсутність подальшого розвитку спільнота створила форк.

Форк називається **pioarduino** і супроводжується спільнотою. Він підтримує актуальні версії Arduino core і нові сімейства чипів.

Практичний висновок на сьогодні:

- працюєте з **Arduino 2.x** і старим кодом → офіційна платформа працює;
- потрібен **Arduino 3.x**, S3, C3, C6 або новіші → **pioarduino**.

УВАГА

Це та частина довідника, що застаріває найшвидше. Стан проектів може змінитися: офіційна платформа може наздогнати, форк — злитися назад або зупинитися.

Тому: перед початком нового проекту перевірте поточний стан обох, а не покладайтесь на цей текст. Версія, звірена на дату цієї ревізії, — у таблиці на початку частини; наскільки вона застаріла, залежить від того, коли ви це читаете.

Чому це взагалі варте уваги

Версії фіксуються в проекті. Це головне. `platformio.ini` лежить у git і повністю описує, чим збирається проект. Через рік на іншій машині збереться те саме.

Кілька середовищ в одному проекті. Один код, кілька цілей: `classic` і `S3`, налагоджувальна й робоча збірка, дві різні плати.

Менеджер бібліотек. Залежності записуються у файл із версіями, а не ставляться руками в спільний каталог, як в Arduino IDE.

Підтримка обох фреймворків. І Arduino, і ESP-IDF — через один інтерфейс.

platformio.ini

Весь проєкт описується одним файлом:

```
[env:esp32dev]
platform = https://github.com/pioarduino/platform-espressif32/releases/download/
stable/platform-espressif32.zip
board = esp32dev
framework = arduino
monitor_speed = 115200
upload_speed = 460800
build_flags =
    -DCORE_DEBUG_LEVEL=3
    -DMY_SENSOR_ADDR=0x76
lib_deps =
    adafruit/Adafruit BME280 Library @ ^2.2.2
```

УВАГА

Чому в рядку platform посилання, а не звичне espressif32 @ 6.5.0.

Офіційна платформа PlatformIO лишилася на Arduino 2.x. Запис espressif32 @ 6.5.0 збереться — і дасть Arduino 2.x, тобто не те, про що написано в розділі 12, і без підтримки нових чипів.

pioarduino розповсюджується не через реєстр PlatformIO, а архівом релізу, тому й рядок такий довгий. Мітка stable завжди вказує на поточний стабільний реліз форка — на момент цієї ревізії це Arduino 3.3.11 і ESP-IDF 5.5.5 (таблиця версій у частині IV).

Старий запис лишається доречним рівно в одному випадку: проєкт свідомо живе на Arduino 2.x і переносити його нікуди.

Розбір рядків, що мають значення.

platform. Тут — джерело платформи, а не лише її версія.

НЕЗВОРОТНЕ

Версію платформи треба пінити завжди. Запис без версії означає «бери найсвіжішу», і одного дня проєкт, що збирався роками, перестане збиратися — на іншій машині, в іншого розробника, або у вас після оновлення.

Це та помилка, що виявляється в найгірший момент: коли треба терміново перезібрати прошивку для виробу в полі.

Для pioarduino пінування — це заміна мітки stable на **тег релізу**:

```
platform = https://github.com/pioarduino/platform-espressif32/releases/
download/55.03.311/platform-espressif32.zip
```

Номер тегу — з таблиці версій у частині IV; вона єдине місце в книзі, де версії живуть, і оновлюється окремо від тексту (Р4).

board — ідентифікатор плати з переліку PlatformIO. Він задає чип, обсяг флешу і розбивку за замовчуванням. Плату, якої немає в переліку, описують власним файлом або беруть найближчу й правлять параметри.

framework — arduino або espidf.

build_flags — параметри компіляції й макроси. Зручний спосіб тримати конфігурацію в одному місці замість правок у коді.

lib_deps — залежності з версіями. ^2.2.2 означає «сумісні оновлення»; для виробничого проєкту точна версія надійніша.

Кілька середовищ

Типовий випадок — один проєкт на дві плати:

```
[env]
framework = arduino
monitor_speed = 115200
lib_deps = adafruit/Adafruit BME280 Library @ 2.2.2

[env]
platform = https://github.com/pioarduino/platform-espressif32/releases/download/
stable/platform-espressif32.zip

[env:classic]
board = esp32dev

[env:s3]
board = esp32-s3-devkitc-1
build_flags = -DHAS_PSRAM
```

Рядок platform винесено в спільну секцію навмисно: дві плати мають збиратися **однією** платформою, інакше різниця між середовищами перестане бути різницею плат. Для S3 це не косметика — офіційна платформа його новіші ревізії просто не знає.

Секція [env] — спільне для всіх. Збирання конкретного середовища — вибором у панелі PlatformIO або `pio run -e s3`.

Для виробу це зручно й іншим чином: окремі середовища для налагоджувальної збірки (докладний лог) і робочої (мінімум логу, оптимізація за розміром).

ESP-IDF усередині PlatformIO

framework = espidf дозволяє збирати проєкт ESP-IDF через PlatformIO. Працює, але з застереженням: версія ESP-IDF при цьому визначається версією платформи, а не вашим вибором, і зазвичай відстає від поточного stable.

Для серйозної роботи з ESP-IDF надійніше брати сам ESP-IDF з офіційним розширенням (розділ 11): там версія під вашим контролем, а вся штатна діагностика працює без обхідних шляхів.

PlatformIO виграє там, де головне — Arduino і бібліотеки.

Типові проблеми

Проект збирався, перестав. Майже завжди — незапінена версія платформи або бібліотеки. Перше, що перевіряти.

Бібліотека не знаходиться. Ім'я в `lib_deps` має точно відповідати реєстру, разом з іменем автора: `adafruit/Adafruit BME280 Library`, а не просто назва.

Плати немає в переліку. Взяти найближчу за чипом і обсягом флешу й поправити `board_build.*` параметрами.

Конфлікт із встановленим ESP-IDF. PlatformIO тримає власні тулчейни окремо. Змінні середовища від `export.sh`, активовані в тому ж терміналі, можуть заплутати збирання. Не змішувати в одному вікні.

Монітор не відкривається. Порт зайнятий іншим монітором — та сама причина, що скрізь (розділ 09).

Що обрати

Ситуація	Що брати
Arduino + багато бібліотек, проект із кількох файлів	PlatformIO / pioarduino
Одна плата, один файл, швидка перевірка	Arduino IDE
Кілька цільових плат з одного коду	PlatformIO / pioarduino
Виріб, ОТА, серійність, довгий супровід	ESP-IDF (розділ 11)
Потрібні найновіші чипи або Arduino 3.x	pioarduino

Що з цього треба запам'ятати

Офіційна платформа PlatformIO відстала від Arduino core; спільнотний форк `pioarduino` підтримує актуальні версії. Стан варто перевіряти перед початком проєкту — саме ця частина застаріває найшвидше.

Версію платформи і версії бібліотек пінити завжди. Незапінена версія ламає збирання в найгірший момент.

`platformio.ini` у `git` повністю описує проєкт — у цьому головна цінність.

Для ESP-IDF надійніше брати сам ESP-IDF, а не його через PlatformIO.

14. MicroPython, ESPHome, Wokwi

Три інструменти, які не є ESP-IDF і не претендують на це. Кожен у своїй ситуації дає робочий результат за час, за який на C ви ще не закінчите налаштування.

Розділ оглядовий: коли кожен із них — найкоротший шлях, і де межа, за якою доведеться переходити на щось інше.

MicroPython

Інтерпретатор Python, що виконується на самому чипі. Ви прошиваєте його один раз, далі працюєте з платою як з інтерактивною консоллю: набираєте рядок — він виконується негайно.

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)
while True:
    led.value(not led.value())
    time.sleep(0.5)
```

Де виграш. Цикл «змінив — побачив» вимірюється секундами, без компіляції й прошивки. Інтерактивна консоль (REPL) дозволяє смикнути пін або опитати датчик прямо руками — це найшвидший спосіб з'ясувати, як поводить себе незнайомий модуль. Для навчання і для людей, що вже знають Python, поріг входу мінімальний.

Де межа. Швидкість — десятки разів повільніше за C, і це відчутно на будь-чому інтенсивному. Пам'ять з'їдається інтерпретатором: на C3 з 400 КБ місця лишається небагато. Точні таймінги недосяжні: збирач сміття вмикається, коли вважає за потрібне. Частина периферії доступна частково.

Коли це правильний вибір. Розвідка незнайомого датчика; навчання; пристрій, де важлива швидкість написання, а не швидкість роботи; одноразова задача.

УВАГА

Навіть якщо проект буде на C, MicroPython вартий місця на одній платі в шухляді. Перевірити, чи взагалі відповідає незнайомий модуль, і за якою адресою, — три рядки в консолі проти написання, збирання й прошивання тестового скетча (розділ 44).

ESPHome

Радикально інший підхід: ви не пишете код узагалі. Ви описуєте пристрій у YAML, а ESPHome генерує з нього прошивку.

```

esphome:
  name: datchyk-teplytsi

esp32:
  board: esp32dev

wifi:
  ssid: !secret wifi_ssid
  password: !secret wifi_password

sensor:
  - platform: bme280_i2c
    temperature:
      name: "Температура"
    humidity:
      name: "Вологість"
    address: 0x76
    update_interval: 60s

```

Це повний опис робочого пристрою: під'єднання до Wi-Fi, читання датчика, віддача значень, веб-інтерфейс, **ОТА-оновлення з коробки**.

Де виграш. Датчик із телеметрією за десять хвилин без рядка коду. ОТА, веб-інтерфейс, відновлення зв'язку, інтеграція з системами домашньої автоматизації — усе вже є. Величезна бібліотека готових компонентів на конкретні датчики й дисплеї.

Де межа. Своя логіка описується обмежено: складніші сценарії доводиться вписувати шматками C++ у YAML, і це швидко стає незручним. Ви залежите від наявності компонента на ваше залізо. Прошивка виходить більшою, ніж написана вручну.

Коли це правильний вибір. Датчик або виконавчий пристрій, що передає дані й приймає команди. Кілька однотипних вузлів, які треба зробити швидко й однаково. Ситуація, коли пристрій має інтегруватися з готовою системою автоматизації.

ЗАКУПІВЛЯ

Практичний прийом: ESPHome добре працює як **проміжний етап**. Спершу YAML-описом перевіряється, що залізо зібране правильно й датчик реально читається. Далі, коли залізо доведене, пишеться власна прошивка — уже без сумнівів у монтажі.

Це економить найдорожче: час, витрачений на пошук помилки в коді, коли насправді проблема в пайці.

Wokwi

Симулятор ESP32 у браузері. Плата, датчики, дисплеї, дрони — усе віртуальне; код виконується по-справжньому, включно з Wi-Fi.

Де виграш. Заліза не потрібно взагалі. Можна перевірити логіку, поки плата ще їде поштою. Проектом можна поділитися посиланням — це найкращий спосіб показати комусь код разом зі схемою. Для навчання незамінний: не шкода нічого спалити.

Де межа. Симуляція не є реальністю. Немає просадок живлення, немає шуму, немає завалених фронтів на довгому дроті, немає дребезгу контактів — тобто немає рівно тих речей, що складають більшу частину цього довідника. Набір симульованих компонентів обмежений. Таймінги наближені.

Коли це правильний вибір. Перевірити алгоритм; навчатися; показати приклад іншій людині; підготуватися, поки заліза немає.

УВАГА

Головна пастка симулятора: код, що бездоганно працює у Wokwi, може не працювати на столі — і причина буде саме в тому, чого симулятор не моделює. Живлення, рівні, довжина дротів, підтягування.

Симулятор перевіряє **логіку**, а не пристрій.

Порівняння

	MicroPython	ESPHome	Wokwi
Треба залізо	так	так	ні
Треба писати код	так, Python	ні , YAML	так
Час до першого результату	хвилини	хвилини	секунди
ОТА з коробки	ні	так	—
Своя складна логіка	так	обмежено	так
Точні таймінги	ні	ні	наближено
Підходить для виробу	обмежено	так, у своїй ніші	ні

Як це поєднується з основним маршрутом

Ці три інструменти не конкурують з ESP-IDF — вони закривають різні етапи.

Розвідка заліза — MicroPython у консолі: чи відповідає модуль, за якою адресою, що повертає.

Перевірка ідеї — Wokwi, поки плати немає, або ESPHome, якщо задача типова.

Перевірка монтажу — ESPHome: якщо датчик читається під ним, залізо зібране правильно.

Виріб — ESP-IDF (розділ 11), коли потрібні надійність, керування пам'яттю, діагностика і супровід.

Помилка — брати інструмент не з того етапу: писати виріб на MicroPython через звичку або мучитися з ESP-IDF там, де потрібно швидко прочитати незнайомий датчик.

Що з цього треба запам'ятати

MicroPython — найшвидший спосіб з'ясувати, як поводить себе незнайомий модуль, навіть якщо проєкт буде на C.

ESPHome дає працюючий датчик із OTA за десять хвилин без коду і добре працює як спосіб довести, що залізо зібране правильно.

Wokwi перевіряє логіку, а не пристрій: у ньому немає живлення, рівнів і довжини дротів — тобто того, через що ламається реальне залізо.

15. Вибір, поєднання, офлайн-комплект

Підсумок частини: як обрати тулчейн під задачу, як переходити між ними без переписування з нуля, і що зробити заздалегідь, щоб працювати без інтернету.

Матриця вибору

Задача	Тулчейн	Чому
Перевірити ідею за вечір	Arduino	найкоротший шлях до результату
Розвідка незнайомого датчика	MicroPython	консоль, без збирання
Типовий датчик із телеметрією	ESPHome	ОТА і веб з коробки
Плати ще немає	Wokwi	залізо не потрібне
Проект із кількох файлів на Arduino	PlatformIO / pioarduino	версії і залежності
Виріб на роки, ОТА, серійність	ESP-IDF	відтворюваність і діагностика
Потрібні MCPWM, PCNT, TWAI	ESP-IDF	повний доступ до периферії
Жорсткі вимоги до пам'яті чи таймінгів	ESP-IDF	контроль над стеками і ядрами
Прошити партію готових плат	жоден	потрібен лише esptool (розділ 21)

Останній рядок варто помітити окремо: якщо задача — прошити готові плати, тулчейн не потрібен зовсім. Це інший маршрут (розділ 00).

Сценарій: прототип на Arduino → виріб на IDF

Найпоширеніший шлях реального проекту, і він **не вимагає переписування з нуля**.

Крок 1. Прототип на Arduino. Мета — з'ясувати, що задача взагалі розв'язується: датчик читається, зв'язок працює, логіка правильна. Швидкість тут важливіша за якість.

Крок 2. Зафіксувати те, що вже знаєте. Робочі таймінги, адреси на шині, реальні межі значень, послідовності ініціалізації. Це найцінніше, що дав прототип, і воно переноситься незалежно від мови.

Крок 3. Перенести під ESP-IDF, лишивши Arduino компонентом. Проєкт стає проєктом ESP-IDF: з'являються `menuconfig`, компоненти, розбивка флешу, повна діагностика. Код при цьому лишається тим самим — `setup/loop` і бібліотеки Arduino продовжують працювати (розділ 12).

Крок 4. Замінювати частини поступово. Спершу те, що впирається в межу: бібліотека з блокувальною затримкою, робота з пам'яттю, критичні таймінги. Решта може лишатися на Arduino API як завгодно довго.

Крок 5. Зміцнити. Обробка помилок, watchdog, OTA, безпечна поведінка при відмові (розділи 32, 19, 58).

Головна перевага цього шляху: **на кожному кроці є робочий пристрій**. Ви не опиняєтеся посеред переписування з непрацюючим кодом і невідомим терміном.

УВАГА

Помилка, якої варто уникнути: почати одразу з ESP-IDF задачу, у якій незрозуміло, чи взагалі працює залізо. Тоді доводиться одночасно розбиратися з фреймворком і з датчиком, і незрозуміло, що саме не працює.

Спершу довести, що залізо живе, — будь-яким найшвидшим способом. Потім робити з цього виріб.

Один проєкт трьома тулчейнами

Корисна вправа, яка займає вечір і після якої вибір перестає бути питанням віри. Візьміть найпростішу задачу — прочитати датчик і віддати значення по Wi-Fi — і зробіть її тричі.

Що стане видно на власному досвіді:

ESPHome — двадцять рядків YAML, працює за десять хвилин, разом з OTA і веб-інтерфейсом. І одразу видно межу: щойно потрібна власна логіка складніша за поріг, починається боротьба.

Arduino — сорок рядків, працює за півгодини. Бібліотека датчика знайшлася миттєво. Стелю не видно, поки проєкт малий.

ESP-IDF — більше коду, більше налаштувань, кілька годин. І одразу доступні речі, яких немає в перших двох: розмір стека задач, вибір ядра, рівень логу по підсистемах, `coredump`, розбивка флешу.

Висновок, який робиться після цього самостійно: жоден із трьох не є «кращим». Вони відповідають на різні питання.

Офлайн-комплект

У полі інтернету немає. Це не гіпотеза, а звичайна умова роботи, і саме там документація потрібна найбільше.

Зібрати заздалегідь, поки мережа є:

Тулчейн, встановлений і перевірений. Не «завантажений інсталятор», а встановлений і випробуваний збиранням хоча б одного проєкту. Половина проблем встановлення вилазить саме при першому збиранні, і робити це в полі — погана ідея.

Документація. ESP-IDF Programming Guide (доступний для завантаження цілком), datasheet усіх сімейств, з якими працюєте, TRM на основне.

Приклади. Каталог examples/ уже лежить усередині ESP-IDF — це найнедооціненіший офлайн-ресурс, який у вас і так є.

Бібліотеки й компоненти. Закешовані локально. Проєкт, що тягне залежності при збиранні, у полі не збереться.

Драйвери USB-мостів для Windows: CP2102, CH340, CH9102. Окремими файлами, не посиланнями.

Образи прошивок, які можуть знадобитися, разом із .elf того самого збирання (розділи 21, 26).

Схеми й пінауті плат, з якими працюєте — картинками, не закладками браузера.

Роздруковані картки K1–K15, заламіновані. Вони для цього й зроблені.

Повний перелік із поясненням, що саме завантажувати, — додаток F.

ЗАКУПІВЛЯ

Офлайн-комплект варто тримати не лише на робочому ноутбуку, а й на флешці. Ноутбук ламається, лишається чужий; комплект на флешці розгортається за годину, а зібраний наново — за день, і то якщо є мережа.

Що з цього треба запам'ятати

Вибір тулчейну — це вибір етапу, а не віри. Розвідка, прототип і виріб робляться різними інструментами.

Прототип на Arduino доводиться до виробу підключенням Arduino компонентом ESP-IDF, поступово, з робочим пристроєм на кожному кроці.

Не починати з ESP-IDF задачу, у якій ще невідомо, чи працює залізо.

Офлайн-комплект збирається заздалегідь і перевіряється збиранням, а не завантаженням.

Частина V. Прошивка

*Як воно завантажується, як туди потрапляє код,
і що робити, коли потрапило не те.*

16. Як завантажується ESP32

Між подачею живлення і першим рядком вашого коду проходить три етапи, і на кожному чип може зупинитися. Розуміння цього ланцюжка — різниця між «плата чомусь не працює» і «плата зупинилася на другому етапі, бо не знайшла таблицю розділів за адресою 0x8000».

Це найкорисніші двадцять хвилин, які можна витратити на теорію: майже вся діагностика прошивки (розділ 29) зводиться до питання «на якому етапі воно стало».

Три етапи

Етап 1 — ROM bootloader. Зашитий у кремній на заводі, змінити його неможливо. Він стартує завжди, незалежно від того, що у флеші. Саме тому плата з повністю стертим флешем все одно подає ознаки життя: ROM живий.

Завдання ROM-бутлоадера: подивитися на strapping-піни і вирішити, звідки брати наступний код — з флешу чи з UART.

Етап 2 — другий бутлоадер (bootloader.bin). Уже ваш, лежить у флеші, збирається разом із проектом. Він налаштовує тактування і флеш, читає таблицю розділів, обирає активний розділ застосунку, перевіряє його і передає керування.

Етап 3 — застосунок. Ініціалізація FreeRTOS, потім `app_main` (в ESP-IDF) або `setup/loop` (в Arduino core).

Головне практичне: **етапи 2 і 3 живуть у флеші за фіксованими адресами**, і якщо хоч одна адреса не та, ланцюжок рветься мовчки — без повідомлення про помилку, просто без результату.

Куди дивиться ROM: strapping-піни

Strapping-пін — це звичайний GPIO, стан якого читається **один раз**, у момент відпускання скидання, і потім більше не має значення для завантаження. Далі пін працює як звичайний.

Ключовий — GPIO0 classic S3 :

GPIO0 при скиданні	Куди піде ROM
високий (підтягнутий вгору)	звичайний старт із флешу
низький (притиснутий до землі)	download mode: чекає прошивку по UART

Для **C3** роль іншу грає пара пінів: GPIO9 притиснутий до землі вмикає download mode, і GPIO8 при цьому має бути високим.

Повний перелік strapping-пінів по сімействах — картка K9, вхід у download mode вручну — картка K4.

УВАГА

Це пояснює цілий клас загадкових несправностей: **зовнішня обв’язка на strapping-піні**. Світлодіод із резистором на GPIO0, датчик, що тримає лінію низькою, довгий дріт, який ловить наводку — і плата стартує не туди або не стартує взагалі. Причому в коді все правильно, і в 99 % часу пін поводить себе нормально: значення має лише мілісекунда після скидання.

classic Найзліший випадок — GPIO12 (MTDI). Він задає напругу живлення флешу. Підтягнутий вгору при старті — і плата не стартує взагалі, без жодного повідомлення.

Що робить другий бутлоадер

Другий бутлоадер лежить за адресою, яка **залежить від сімейства чипа**:

Сімейство	Адреса <code>bootloader.bin</code>
ESP32 classic, ESP32-S2	0x1000
ESP32-S3, C3, C6, H2	0x0
ESP32-P4, C5, H4	0x2000

Причина в кожному рядку своя, і в першому вона не та, про яку зазвичай думають. **classic** На classic перший сектор (0x0–0x1000) відведено під **IV і дайджест Secure Boot v1** коли secure boot не ввімкнено — а це звичайний випадок — сектор просто не використовується. **S2** На S2 він не використовується взагалі ніколи. У наступному поколінні зайвий сектор прибрати, і бутлоадер став на 0x0. У найновішому — перші два сектори (8 КБ) віддані менеджерів ключів апаратного шифрування флешу (AES-XTS), і бутлоадер зсунувся вдруте.

УВАГА

Правила «що новіше, то ближче до нуля» не існує — і саме таке припущення робить помилку. Значення задається ROM конкретного чипа й у ESP-IDF не налаштовується взагалі: це CONFIG_BOOTLOADER_OFFSET_IN_FLASH, і в довідці до нього сказано прямо, що воно визначене ROM.

Практичний висновок: адресу не пригадують, а дивляться. `idf.py flash` знає її сам; коли заливаєте `esptool` вручну — беріть адресу з таблиці вище для **свого** чипа, а не з чужої інструкції.

НЕЗВОРОТНЕ

Це найчастіша причина «прошилося без жодної помилки, а плата мовчить». Інструкція з інтернету, написана для ESP32 classic, кладе бутлоадер S3 на `0x1000` — тобто в порожнє місце. `esptool` при цьому не має підстав скаржитися: він робить рівно те, що просили. Спершу визначити чип (картка K1), потім брати адресу.

Далі бутлоадер читає **таблицю розділів** за адресою `0x8000` — ця адреса однакова на всіх сімействах. Таблиця займає цілий сектор флешу (`0x1000`, тобто 4 КБ), тому наступний розділ не може починатися раніше ніж `0x9000`. Детально про розділи — розділ 18.

Тут варта уваги асиметрія: адреса бутлоадера задана ROM і не налаштовується, а адреса таблиці розділів — звичайний параметр (`CONFIG_PARTITION_TABLE_OFFSET`), який **можна** зсунути. Це не дрібниця, бо саме цим зсувом лікують наступну проблему.

УВАГА

Простір бутлоадера — це проміжок до таблиці розділів, і він скінченний. `classic` На classic це `0x8000 - 0x1000 = 0x7000` (28 КБ); на S3 і C3, де бутлоадер починається з нуля, — цілі `0x8000` (32 КБ).

Звичайному бутлоадеру цього з великим запасом. Але шифрування флешу, Secure Boot і піднятий рівень логу бутлоадера додають відчутно, і збірка падає з `Bootloader binary size [...] is too large for partition table offset`.

Ліки в порядку дешевизни: повернути оптимізацію бутлоадера на «Size», знизити його рівень логу, і лише потім — відсунути таблицю розділів на адресу більшу за `0x8000`. Останнє тягне за собою перерахунок явних зсувів у CSV: жоден розділ не може починатися раніше ніж нова адреса таблиці плюс `0x1000`.

З таблиці бутлоадер дізнається, де лежить застосунок. Якщо розділів `ota_0/ota_1` кілька, вибір робиться за вмістом службового розділу `otadata` (розділ 19). Якщо є лише `factory` — беруть його.

Що видно в консолі

Монітор на **115200 бод** — це швидкість ROM, і вона не залежить від налаштувань вашого застосунку. Скинути плату кнопкою EN.

Звичайний старт:

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
config: 0, SPIWP:0xee
mode:DI0, clock div:2
load:0x3fff0030,len:1344
entry 0x400805e4
I (29) boot: ESP-IDF v6.0.2 2nd stage bootloader
I (33) boot.esp32: SPI Flash Size : 4MB
I (52) boot: Partition Table:
I (56) boot: ## Label           Usage            Type ST Offset   Length
I (63) boot:  0 nvs             WiFi data        01 02 00009000   00006000
...
I (xxx) cpu_start: Pro cpu up.
```

Читається тут більше, ніж здається. `rst`: — причина скидання (повна таблиця кодів на картці [K6](#)). Версія IDF, якою зібрано прошивку. Обсяг флешу, який **бачить бутлоадер** — а це не завжди те, що написано на модулі. І вся таблиця розділів з адресами: готова відповідь на «а що там усередині», без розбирання дампа.

Download mode:

```
rst:0x1 (POWERON_RESET),boot:0x3 (DOWNLOAD_BOOT(UART0/UART1/SDIO_REI_RE0_V2))
waiting for download
```

`waiting for download` — рівно те, чого треба досягти перед прошивкою.

Немає застосунку:

```
E (xxx) esp_image: image at 0x10000 has invalid magic byte (nothing flashed here?)
E (xxx) boot: Factory app partition is not bootable
```

Другий бутлоадер живий, розділи знайшов, але за адресою застосунку не те, чого він очікує. Або застосунок не заливали, або залили не туди.

Не знайдено таблицю розділів:

```
E (xxx) flash_parts: partition 0 invalid magic number 0xffff
E (xxx) boot: Failed to verify partition table
```

За адресою `0x8000` порожньо — типово після `erase-flash` без наступної повної прошивки.

Boot loop. Ті самі рядки повторюються по колу. Дивитися треба **найперший** дамп після подачі живлення, а не сотий: причина в першому, решта — наслідок. Розбір — картка [K7](#) і розділ [26](#).

Авторесет: чому він іноді не працює

Прошивати вручну кнопками незручно, тому на платах ставлять схему авторесету: два транзистори, керовані сигналами `DTR` і `RTS` USB-мосту. `esptool` перед

прошивкою смикає ці лінії в потрібній послідовності, чип сам заходить у download mode, і людині нічого натискати не треба.

Схема не універсальна. Вона не спрацює, коли:

- на GPI00 навішана зовнішня обв'язка, яка утримує лінію;
- плати без цієї схеми взагалі — голі модулі, частина клонів;
- живлення просідає під час скидання;
- драйвер мосту не керує DTR/RTS як треба (трапляється на CH9102 у Windows);
- USB-хаб додає затримок, і імпульси не потрапляють у вікно.

У всіх цих випадках лікування те саме — увійти в download mode руками, картка K4. Це не ознака несправної плати.

Скільки часу займає старт

Від подачі живлення до `app_main` — типowo десятки мілісекунд. Число в дужках у логу (`I (29) boot:`) — це мілісекунди від старту, і воно безкоштовно показує, **де саме прошивка задумалася**: стрибок з (52) на (1250) між двома рядками означає секунду очікування, і це майже завжди щось, що чекає таймауту.

ЖИВЛЕННЯ

Найпідступніша поведінка при старті — коли живлення просідає саме в момент увімкнення радіо, вже після успішного завантаження. Виглядає як збій прошивки; насправді це `rst:0xf (brownout)` у наступному рядку логу. Дивитися код тут марно, поки не зміряно напругу під навантаженням — розділ 06.

Що з цього треба запам'ятати

Три етапи: ROM → другий бутлоадер → застосунок. ROM не ламається ніколи; все, що ламається, лежить у флеші.

Адреса бутлоадера залежить від сімейства і задана ROM; адреса таблиці розділів на всіх сімействах однакова — `0x8000`, якщо її свідомо не пересунули.

Стан strapping-пінів має значення рівно одну мілісекунду після скидання — і саме тому помилки тут виглядають як містика.

Перший рядок логу називає причину попереднього скидання. Це найдешевша діагностична інформація в усій системі, і читати її треба першою.

17. esptool

esptool — програма, що розмовляє з ROM-бутлоадером чипа. Вона не знає нічого про ваш проєкт, не потребує встановленого ESP-IDF і працює з будь-якою платою на ESP32, включно з тією, прошивку якої зібрав хтось інший десять років тому.

Це головний інструмент для всього, що стосується чужого заліза: визначити чип, зняти дамп, залити готовий образ, стерти. Якщо з плати можна взяти хоч щось — це буде через esptool.

Дві версії з несумісним синтаксисом

Перше, що треба з'ясувати, — яка версія у вас:

```
esptool version
```

Команда друкує номер версії — це і є відповідь. Якщо esptool не знайдено взагалі, спробувати esptool.py version: у v4 інакшого імені немає. Зворотне не працює: у v5 обидва імені є, і те, що команда запустилася під іменем esptool.py, ще нічого не означає — дивитися треба на надрукований номер.

esptool v5 змінив командний рядок у двох місцях, і обидві зміни ламають копіювання команд з інтернету:

	v4 і раніше	v5
виклик	esptool.py	esptool
команди	write_flash, chip_id	write-flash, chip-id

Переважає більшість інструкцій, статей і відповідей на форумах написана під v4, і напрямки несиметричні.

Стара команда на новій версії поки що працює. У v5 старі імена позначені як застарілі: виконуються з попередженням і будуть прибрані в наступному мажор-релізі. Тобто write_flash на v5 спрацює — і тим неприємніше буде, коли одного дня перестане.

Нова команда на старій версії не працює. write-flash на v4 дає «невідома команда», і це збиває з пантелику: команда відома, просто пишеться інакше.

Перейменування торкнулося не лише команд, а й **опцій**: --flash_size, --flash_mode, --flash_freq, а також значень --before і --after.

У цьому довіднику команди подаються в синтаксисі v5. Щоб отримати варіант для v4: замінити дефіси на підкреслення і додати .py.

УВАГА

Версія esptool залежить не від вашого вибору, а від того, звідки вона взялася. Разом з ESP-IDF 5.x іде v4, разом з ESP-IDF 6.x — v5. Встановлена окремо через пір — та, що була свіжою на момент встановлення. Якщо у вас на машині є і те, і те, esptool і esptool.py можуть бути **різних версій** — це джерело дуже дивних годин.

Мінімум, з якого починається все

```
esptool --port /dev/ttyUSB0 flash-id
```

Перед виконанням **будь-якої** команди esptool встановлює зв'язок і друкує шапку з тим, що знайшов:

```
Chip type:      ESP32-D0WD-V3 (revision v3.1)
Features:      WiFi, BT, Dual Core, 240MHz, ...
Crystal frequency: 40MHz
MAC:          24:6f:28:xx:xx:xx
```

Сімейство, ревізія кремнію і MAC — саме тут. Це не результат команди, а **преамбула з'єднання**, спільна для всіх команд. Тому будь-яка команда, що взагалі під'єдналася, вже сказала, з чим має справу.

flash-id як перша команда зручна тим, що після шапки додає ще й те, що однаково знадобиться далі: виробник і **обсяг** флешу.

УВАГА

Чому не chip-id. Підкоманда з такою назвою існує, але вона успадкована від ESP8266, у якого справді був окремий Chip ID в efuse. У жодного чипа сімейства ESP32 його немає, і esptool на ньому відповідає попередженням:

```
ESP32 has no chip ID. Reading MAC address instead.
```

після чого друкує MAC. Тобто команда не помилкова — вона просто нічого не додає до шапки, а попередження лякає на рівному місці. Сімейство і ревізію вона не «називає»: їх назвала преамбула ще до неї.

Обсяг важливий двічі: він потрібен для дампа і він викриває підробки. Модуль з написом ESP32-WR00M-32 і флешем 2 МБ замість 4 МБ — перемаркований клон.

УВАГА

flash-id показує те, що каже сама мікросхема флешу. Бутлоадер у логу (розділ 16) показує те, що йому **skonфігуровано**. Розбіжність між цими двома числами означає неправильно зібрану прошивку: частина флешу просто не використовується, або, гірше, прошивка розраховує на пам'ять, якої немає.

Прочитати: read-flash

Найважливіша команда в усьому розділі, бо єдина незворотна операція — це та, перед якою не зробили дамп.

```
esptool --port /dev/ttyUSB0 read-flash 0 ALL dump-2026-08-26.bin
```

ALL читає рівно стільки, скільки є на чипі. Якщо ваша версія цього не розуміє — підставити обсяг явно: 0x400000 (4 МБ), 0x800000 (8 МБ), 0x1000000 (16 МБ).

Перевірка результату — одразу, не потім: **розмір файлу має точно дорівнювати обсягу флешу**. Файл, менший за очікуваний, — це обірваний дамп, а не «стиснений». Читання на високій швидкості через довгий кабель обривається частіше, ніж хотілося б; при найменшому сумніві повторити з --baud 115200 і порівняти розміри.

Можна читати й окремий шматок — наприклад, лише розділ NVS:

```
esptool --port /dev/ttyUSB0 read-flash 0x9000 0x6000 nvs.bin
```

Записати: write-flash

classic Адреси в цьому прикладі — для ESP32 classic:

```
esptool --port /dev/ttyUSB0 --baud 460800 write-flash -z \
  0x1000 bootloader.bin \
  0x8000 partition-table.bin \
  0x10000 app.bin
```

На **S3** **C3**, C6 і H2 бутлоадер іде не на 0x1000, а на 0x0; на P4, C5 і H4 — на 0x2000; решта адрес та сама (таблиця в розділі 16). Скопіювати цей приклад дослівно на інший чип означає покласти бутлоадер у порожнє місце — і esptool не поскаржиться.

Аргументи йдуть парами: адреса, файл. Скільки завгодно пар за один виклик.

--z вмикає стиснення при передачі. Воно **вже ввімкнене** за замовчуванням, тож у звичайній команді прапорець нічого не змінює. Сенс він має лише разом із --no-stub, де стиснення типово вимкнене — а це саме той випадок із клонами, який розібрано нижче. Користь від стиснення там подвійна: менше байтів пройшло довгим кабелем — менше нагод зіпсуватися.

--baud 460800 — розумний максимум для більшості мостів. Не з'єднується або обривається — знижувати: 230400, далі 115200. Швидкість тут не той параметр, на якому варто економити хвилини.

Адреси залежать від сімейства чипа — таблиця в розділі 16 і на картці K5. Це те місце, де помиляються найчастіше.

Стерти: erase-flash

```
esptool --port /dev/ttyUSB0 erase-flash
```

НЕЗВОРОТНЕ

erase-flash знищує **весь** флеш, включно з розділом NVS. У NVS лежать не лише ваші налаштування, а й калібрувальні дані радіо, збережені креденшили Wi-Fi і конфігурація конкретного пристрою. Перезбирання прошивки цього не повертає.

Дамп (read-flash) робиться **до**, а не після. Після — нема з чого.

Коли erase-flash справді потрібен: залишки старої розбивки заважають новій таблиці розділів; підозра на пошкоджений NVS, через який застосунок падає при старті; підготовка плати до продажу чи передачі.

Коли не потрібен: «щоб напевно». write-flash і так перезаписує те, що записує; стирати все заради оновлення застосунку — зайвий ризик.

Стерти лише частину — точніше і безпечніше:

```
esptool --port /dev/ttyUSB0 erase-region 0x9000 0x6000
```

Звірити: verify-flash

```
esptool --port /dev/ttyUSB0 verify-flash 0x10000 app.bin
```

Порівнює вміст флешу з файлом. «Прошилося без помилок» і «у флеші лежить те, що треба» — різні твердження, і verify-flash перетворює перше на друге. На серійній прошивці (розділ 21) це обов'язковий крок, не опція.

Зібрати один файл: merge-bin

Три файли на трьох адресах — незручно передавати іншій людині й легко переплутати. merge-bin склеює їх в один образ, у якому зсуви вже всередині:

```
esptool --chip esp32 merge-bin -o vyrib-v1.bin --flash-mode dio \
  0x1000 bootloader.bin \
  0x8000 partition-table.bin \
  0x10000 app.bin
```

--chip тут **обов'язковий**, і це єдина команда розділу, де він не опція. Решта команд працює через порт і визначає чип сама; merge-bin порту не має — вона складає файл офлайн. Без --chip esptool не вгадує, а зупиняється: Specify the --chip argument.

classic Адреса 0x1000 тут — знову classic, і тепер вона узгоджена з --chip esp32 у тому ж рядку; для інших чипів див. таблицю в розділі 16. Адреси всередині merge-bin мають бути ті самі, якими прошивали б окремо.

Отриманий файл заливається завжди на адресу 0x0, незалежно від сімейства чипа:

```
esptool --port /dev/ttyUSB0 write-flash 0x0 vyrib-v1.bin
```

Це формат, у якому варто віддавати прошивку тому, хто не читав цієї книги: одна команда, одна адреса, нема чого переплутати. Основа серійної прошивки — розділ 21.

Якщо проєкт під рукою — не набирайте адрес узагалі

Усе вище потрібне тоді, коли у вас на руках лише три .bin-файли. Якщо ж є сам проєкт ESP-IDF, є коротший і надійніший шлях:

```
idf.py merge-bin -o vyrib-v1.bin
```

Ця команда бере бутлоадер, таблицю розділів, застосунок і решту розділів **за конфігурацією проєкту**: адреси, чип, режим і частота флешу підставляються самі. Без параметрів результат лягає у build/merged-binary.bin.

УВАГА

Різниця не косметична. У ручному варіанті адреса бутлоадера набирається з голови, і саме там роблять помилку: 0x1000 на S3 дає образ, який прошивається без жодної скарги і не стартує (розділ 16).

idf.py merge-bin цю можливість прибирає повністю — воно читає адресу з конфігурації того самого проєкту, який ви щойно зібрали.

Правило просте: є проєкт — idf.py merge-bin; є тільки файли — esptool --chip ... merge-bin з адресами з таблиці.

Типові помилки і що вони означають

A fatal error occurred: Failed to connect to ESP32: No serial data received.

Найчастіша. Чип не в download mode. Спробувати вручну (картка K4), знизити швидкість, перевірити, що порт не зайнятий відкритим монітором.

Друга половина рядка залежить від версії. No serial data received. — esptool v4 і v5; Timed out waiting for packet header — старіші v3, які ще трапляються в готових складаннях і в чужих інструкціях. Причина в обох випадках та сама, і шукати варто за початком рядка — Failed to connect.

Serial port ... could not be opened: Permission denied

Права, не залізо. Linux: група dialout, потім **перезайти в систему** (картка K3). Не лікувати це запуском через sudo.

Device or resource busy

Порт тримає інша програма: монітор, Arduino IDE, screen. Одночасно порт відкриває лише один процес.

A fatal error occurred: MD5 of file does not match data in flash!

Записалося не те, що передавали. Причини за частотою: погане живлення, довгий чи неякісний кабель, завелика швидкість, зношений флеш. Знизити --baud, повторити. Повторюється стабільно на тій самій адресі — підозра на пошкоджену комірку флешу.

Invalid head of packet (0x00)

Зв'язок є, але на лінії сміття. Класично — плата стартує в застосунок, який сам щось пише в UART, поки esptool намагається говорити. Увійти в download mode вручну.

This chip is ESP32-S3, not ESP32. Wrong chip argument?

esptool визначив чип сам і побачив розбіжність із тим, що йому сказали. Прибрати --chip, дати визначити автоматично.

Плата лишається в download mode після прошивки S3 C3

Прошивка пройшла, а застосунок не стартував: чип так і сидить у завантажувачі. На платах із native USB (картка K3) причина в тому, що скидання по лінії RTS через USB-Serial/JTAG не завжди спрацьовує — фізичної лінії там немає.

Обхід — скидання внутрішнім watchdog замість RTS:

```
esptool --port /dev/ttyACM0 --after watchdog-reset write-flash 0x0 vyrib.bin
```

Побічний ефект, до якого треба бути готовим: порт перелічується заново, і /dev/ttyACM0 може стати /dev/ttyACM1. Монітор доведеться відкрити на новому імені.

На ESP32 classic, C6 і H2 цього режиму немає — там працює звичайне hard-reset по RTS.

Failed to start stub flasher. There was no response.

esptool вантажить у RAM невелику допоміжну програму («stub») для пришивлення. На частині клонів це не працює. Обійти: --no-stub, буде повільніше, але працюватиме.

Сусіднє попередження Stub flasher has been disabled for compatibility, set --no-stub to suppress this warning. — не помилка: esptool сам вимкнув stub і працює далі.

У v4 ті самі рядки коротші — Failed to start stub. Шукати варто за словом stub, а не за повним реченням.

Windows: Flash Download Tool

Espressif дає графічну програму (Flash Download Tool) — той самий функціонал у вікні. Вона зручна там, де прошивку заливає людина без командного рядка: оператор на складанні, замовник, колега.

Практично: підготувати merge-bin-образ, налаштувати один раз, зберегти конфігурацію і передати разом з інструкцією на одну сторінку (розділ 56). Все, що складніше, робиться з командного рядка — його простіше відтворити і покласти в скрипт.

Що з цього треба запам'ятати

Перевірити версію esptool **перш ніж** копіювати команду звідкись.

Сімейство, ревізію і MAC друкує **преамбула з'єднання**, а не окрема команда. flash-id — перша команда для будь-якої незнайомої плати: шапка плюс обсяг флешу.

read-flash робиться до першої зміни. Розмір файлу звіряється з обсягом флешу одразу.

verify-flash перетворює «прошилося» на «у флеші лежить те, що треба».

merge-bin — формат передачі прошивки людині, яка не мусить знати адрес.

18. Розділи флешу

Флеш ESP32 — не один суцільний шматок пам'яті, а набір областей із різним призначенням: бутлоадер, таблиця розділів, застосунок, сховище налаштувань, файлова система. Хто де лежить, описано в **таблиці розділів** (partition table) за адресою 0x8000.

Це та частина системи, яку більшість не чіпає роками — рівно до дня, коли застосунок перестає вміщатися у відведений розділ або треба додати ОТА. Тоді виявляється, що змінити розбивку неважко, а зламати нею робочий пристрій — легко.

Що таке таблиця розділів

Це список записів, кожен з яких каже: назва, тип, підтип, адреса початку, розмір. Бутлоадер читає цей список при кожному старті (розділ 16) і за ним знаходить застосунок.

Сама таблиця крихітна: 0xc00 байтів, тобто **не більше 95 записів**, плюс контрольна сума MD5 після них. Займає вона при цьому цілий сектор флешу — 4 КБ, — бо стерти флеш можна тільки секторами.

Типова розбивка для пристрою без ОТА виглядає так:

Назва	Тип	Підтип	Зсув	Розмір
nvs	data	nvs	0x9000	0x6000
phy_init	data	phy	0xF000	0x1000
factory	app	factory	0x10000	0x100000

Три речі, які варто прочитати з цієї таблиці одразу.

Застосунок починається з 0x10000 — це не випадкове число, а перша адреса після таблиці розділів і службових областей. Саме тому у всіх командах прошивки застосунок іде на 0x10000.

nvs лежить перед застосунком. Це сховище пар «ключ — значення»: налаштування, збережені креденшели Wi-Fi, лічильники. Воно переживає оновлення прошивки — і саме тому erase-flash такий болючий.

phy_init зберігає калібрувальні дані радіо. Маленький розділ, про який ніхто не думає, поки не зітре.

Як подивитися розбивку живого пристрою

Найдешевший спосіб — прочитати boot-лог: другий бутлоадаер друкує всю таблицю з адресами при кожному старті (розділ 16). Нічого розбирати не треба.

Якщо логу немає, таблицю можна зняти з флешу і розібрати:

```
esptool --port /dev/ttyUSB0 read-flash 0x8000 0x1000 pt.bin
python $IDF_PATH/components/partition_table/gen_esp32part.py pt.bin
```

Другий рядок друкує таблицю в тому самому CSV-форматі, у якому її пишуть. Це один із перших кроків форензики чужої прошивки — розділ 24.

CSV: як це описується в проєкті

У проєкті ESP-IDF розбивка задається текстовим файлом:

#	Name,	Type,	SubType,	Offset,	Size,	Flags
nvs,	data,	nvs,	0x9000,	0x6000,		
phy_init,	data,	phy,	0xF000,	0x1000,		
factory,	app,	factory,	0x10000,	1M,		
storage,	data,	spiffs,	,	1M,		

Порожній offset означає «одразу після попереднього» — так і треба робити: явні адреси в кожному рядку легко розсинхронізувати при першій же зміні розміру.

Розмір записується числом (0x100000), або з суфіксом (1M, 512K).

УВАГА

Розділи типу app мають бути вирівняні на 0x10000 (64 КБ). Це вимога апаратного відображення пам'яті, а не примха. Розділи типу data вирівнюються на 0x1000 (4 КБ). Якщо збирання скаржитися на вирівнювання — справа в цьому.

Вибір готової розбивки замість власної — `idf.py menuconfig`, розділ Partition Table. Там є типові варіанти: одна factory, дві OTA-області, варіанти під різні обсяги флешу. Для більшості задач власний CSV не потрібен.

NVS: де живуть налаштування

NVS (Non-Volatile Storage) — сховище пар «ключ — значення», розкладене по просторах імен. Саме тут має лежати все, що конкретне для **цього екземпляра** пристрою: серійний номер, калібрувальні коефіцієнти, адреса сервера, креденшили.

Чому саме тут, а не в коді: значення в NVS переживає оновлення прошивки. Один образ можна залити на сто плат, а різне для кожної записати в NVS окремо — основа серійного виробництва (розділ 21).

NVS стійкий до зникнення живлення: запис влаштований так, що обірвана операція не псує вже записане. Це не означає, що він незнищений — перепов-

нений або пошкоджений NVS призводить до помилок при старті, і в логу це видно явно (`nvs_flash_init failed`).

Стандартна реакція на пошкоджений NVS — стерти і переініціалізувати. Це робиться з коду, і це нормальна практика:

```
esp_err_t err = nvs_flash_init();
if (err == ESP_ERR_NVS_NO_FREE_PAGES ||
    err == ESP_ERR_NVS_NEW_VERSION_FOUND) {
    ESP_ERROR_CHECK(nvs_flash_erase());
    err = nvs_flash_init();
}
ESP_ERROR_CHECK(err);
```

НЕЗВОРОТНЕ

`nvs_flash_erase()` знищує всі налаштування пристрою. У прошивці, що йде в поле, цей код спрацює саме тоді, коли NVS переповнився — тобто несподівано, у роботі. Якщо серед налаштувань є те, чого не відновити (серійний номер, калібрування), його місце не в звичайному NVS, а в окремому розділі `data` тільки для читання, або в `eFuse`.

Файлові системи: LittleFS, SPIFFS, FAT

Коли треба зберігати файли — веб-сторінки, конфігурацію, логи, — потрібна файлова система в окремому розділі `data`.

	LittleFS	SPIFFS	FAT
Стійкість до знищення живлення	так, за задумом	ні	ні
Каталоги	так	ні, плоский простір	так
Знос флешу	вирівнюється власним механізмом	вирівнюється власним механізмом	лише через окремий шар <code>wear_levelling</code>
Швидкість при заповненні	не деградує	різко падає	рівна
Сумісність із ПК	ні	ні	так
У складі ESP-IDF	ні , окремий компонент	так	так

Практичний висновок простий: беріть LittleFS. SPIFFS вважається застарілим, не має каталогів і починає гальмувати, коли розділ заповнюється більш

ніж наполовину. Його розумно лишати тільки в чужому проєкті, який уже на ньому працює.

УВАГА

Останній рядок таблиці — це те, на чому спотикаються, шукаючи LittleFS у `menuconfig`. Його там немає: на відміну від SPIFFS і FAT, LittleFS **не входить до ESP-IDF** і ставиться з реєстру компонентів:

```
idf.py add-dependency "joltwallet/littlefs^1.22.3"
```

Після цього розділ у меню з'являється, а тип розділу в CSV пишеться як `littlefs`. Номер версії — з реєстру на момент роботи, він змінюється частіше за саму книгу.

FAT має сенс в одному випадку: коли той самий носій (найчастіше картку microSD) читатиме звичайний комп'ютер. На вбудованому флеші FAT сама по собі не вирівнює знос — за це відповідає окремий шар `wear_leveling`, який ESP-IDF підставляє під неї (`esp_vfs_fat_spiflash_mount_rw_wl`). Працює це чесно, але шар не безкоштовний ні за місцем, ні за швидкістю, а стійкості до зникнення живлення не додає. На картці — інша розмова (розділ 49).

ЖИВЛЕННЯ

Жодна файлова система на вбудованому флеші не переживе зникнення живлення **посеред запису** так, щоб гарантовано зберегти останній записаний файл. LittleFS гарантує, що ФС лишиться цілою і попередні дані доступні, — це не те саме, що «нічого не втрачено». Для логера, який пише постійно, це проєктне обмеження, а не деталь (розділ 60).

Як змінити розбивку і не зламати пристрій

Найчастіша причина міняти розбивку — застосунок переріс розділ. Помилка при збиранні виглядає прямо:

```
Error: app partition is too small for binary app.bin size 0x123456
```

Варіанти дій, у порядку від найдешевшого:

1. **Прибрати зайве з прошивки.** Рівень логування, невикористані компоненти, оптимізація за розміром (`-Os`) у `menuconfig`. `idf.py size` показує, хто саме займає місце.
2. **Взяти готову розбивку з більшим розділом застосунку** для вашого обсягу флешу.
3. **Написати власний CSV.**

НЕЗВОРОТНЕ

Зміна розбивки несумісна з OTA-оновленням. Пристрій у полі отримує через OTA лише новий образ застосунку — таблиця розділів при цьому не оновлюється. Якщо нова прошивка розрахована на іншу розбивку, вона або не влізе, або запишеться поверх чужих даних.

Практично це означає: **розбивку треба обирати з запасом на самому початку**, до того, як перший пристрій поїхав до замовника. Змінити її потім можна лише з фізичним доступом і повною перепрошивкою — а це поїздка до кожного пристрою.

Другий наслідок того самого: якщо ви змінили розбивку, а на платі лишився старий NVS зі старої розбивки за іншою адресою — застосунок побачить сміття. Після зміни розбивки повна прошивка робиться з попереднім erase-flash (і, звісно, з дампом до нього — картка [K2](#)).

Скільки чого відводити

Орієнтири для типового пристрою на 4 МБ флешу:

- nvs — 0x6000 (24 КБ) вистачає, поки ви не складаєте туди масиви.
- Застосунок з Wi-Fi і TLS — від 1 МБ. З BLE — більше.
- Дві OTA-області означають **подвоєння** місця під застосунок: два по 1.5 МБ — це 3 МБ із приблизно 3.9 МБ, доступних після службових областей. На все інше лишається менш ніж мегабайт.
- Файлова система — те, що лишилося.

ЗАКУПІВЛЯ

Плати з 4 МБ флешу — стандарт і найдешевші, і для більшості задач їх вистачає. Але як тільки в задачі з'являється OTA **разом із** веб-інтерфейсом або великими ресурсами, 4 МБ стають тісними. Модулі на 8 і 16 МБ коштують відчутно дорожче за різницю в ціні флешу — а от переробляти виріб під інший модуль на пізньому етапі дорожче в рази. Це та економія, яку варто рахувати на початку.

Що з цього треба запам'ятати

Таблиця розділів лежить на 0x8000 і друкується в boot-лозі при кожному старті — найдешевший спосіб дізнатися, що всередині чужого пристрою.

NVS переживає оновлення прошивки, і саме тому там живе все, що конкретне для екземпляра. І саме тому erase-flash без дампа незворотний.

LittleFS замість SPIFFS у будь-якому новому проекті.

Розбивку обирають один раз, на початку, з запасом: OTA її не оновлює, а зміна потім вимагає фізичного доступу до кожного пристрою.

19. ОТА-оновлення

ОТА (over-the-air) — оновлення прошивки без фізичного доступу до плати. Пристрій сам завантажує новий образ, кладе його в резервну область флешу і після перезавантаження стартує з неї.

Це та функція, яка перетворює виріб із «зробив і забув» на щось, що можна супроводжувати. І одночасно та, що найлегше перетворює двадцять робочих пристроїв на двадцять цеглинок, якщо зроблена наївно.

Механіка: два слоти і показчик

Розбивка з ОТА має два розділи для застосунку замість одного:

Назва	Тип	Що це
otadata	data	покажчик: з якого слоту стартувати
ota_0	app	слот А
ota_1	app	слот В

Працює це так. Пристрій виконується зі слоту ota_0. Приходить оновлення — воно записується в ota_1, при цьому робоча прошивка **не чіпається** і продовжує працювати. Коли образ записаний повністю і перевірений, змінюється otadata, пристрій перезавантажується і стартує зі слоту ota_1. Наступне оновлення піде у слот ota_0.

Ключове в цій схемі: **у будь-який момент до перемикання покажчика в пристрої є справна прошивка**. Обрив зв'язку посеред завантаження, зникнення живлення, зіпсований файл — усе це втрачає лише недописаний образ у неактивному слоті. Пристрій перезавантажиться в те, що працювало.

УВАГА

Розділ factory при цьому не обов'язковий. Класична схема «factory + ota_0 + ota_1» дає незмінний аварійний образ, з якого завжди можна стартувати, але з'їдає ще одну копію застосунку. На флеші 4 МБ це часто неможливо. Схema з двох слотів без factory — робоча і найпоширеніша.

Ціна: місце у флеші

Головне проєктне обмеження ОТА — **застосунок займає місце двічі**.

На флеші 4 МБ це виглядає так: службові області і NVS з'їдають близько 64 КБ, лишається приблизно 3.9 МБ. Два слоти по 1.5 МБ — це 3 МБ, і на файлову систему лишається менш ніж мегабайт. Прошивка з Wi-Fi, TLS і веб-інтерфейсом

у 1.5 МБ ще вміщається; додайте BLE або великі ресурси — уже ні, і тоді слот доводиться збільшувати за рахунок того самого мегабайта.

НЕЗВОРОТНЕ

Розбивку під OTA треба закласти **до того, як перший пристрій поїхав**. OTA оновлює лише образ застосунку — таблиця розділів через OTA не оновлюється ніколи. Пристрій, залитий з однією factory без слотів OTA, неможливо перевести на OTA дистанційно: для цього потрібна повна перепрошивка з фізичним доступом.

Практично: якщо є хоч найменша ймовірність, що виріб доведеться оновлювати в полі, — OTA-розбивка ставиться одразу, навіть якщо сама функція поки не написана. Місце коштує дешевше, ніж поїздка до кожного пристрою (розділ 18).

Rollback: захист від справної прошивки, яка не працює

Схема з двох слотів рятує від **зіпсованого завантаження**. Вона не рятує від іншого, значно гіршого випадку: образ завантажився ідеально, контрольна сума збіглася, пристрій стартував з нього — і саме ця нова прошивка не може під'єднатися до мережі. Пристрій живий, працює, і недосяжний. Наступне OTA до нього вже не дійде.

Від цього є механізм відкату (rollback). Вмикається в menuconfig: Bootloader config → Application Rollback → Enable app rollback support.

Логіка така. Після оновлення новий образ позначається як «випробувальний» (ESP_OTA_IMG_PENDING_VERIFY). Прошивка мусить сама, у процесі роботи, підтвердити, що вона справна:

```
esp_ota_mark_app_valid_cancel_rollback();
```

Якщо підтвердження не сталося і пристрій перезавантажився — бутлоадер бачить непідтверджений образ і повертається на попередній слот автоматично.

УВАГА

Уся цінність відкату — у тому, **де саме** стоїть виклик підтвердження. Поставити його першим рядком app_main означає підтвердити, що прошивка запустилася, — а це вона щойно й зробила, і від невдалого оновлення це не захищає ніяк.

Підтвердження має стояти після того, як виконано **умову, заради якої пристрій існує**: з'явився зв'язок із сервером, прочитався датчик, пройшла самоперевірка. Тобто після доказу, що новою прошивкою можна керувати далі. Найпростіший робочий критерій — успішне під'єднання до мережі і один вдалилий обмін із сервером.

Друга половина того самого механізму — watchdog. Прошивка, що зависла до підтвердження, має бути перезавантажена, інакше відкат не спрацює просто тому, що перезавантаження не станеться (розділ 32).

Як це виглядає в коді: esp_https_ota

Штатний спосіб в ESP-IDF — компонент esp_https_ota. Мінімальний робочий викилик:

```
esp_http_client_config_t http_cfg = {
    .url = "https://onovlennya.example/vyrib-v2.bin",
    .cert_pem = server_cert_pem_start,
    .timeout_ms = 10000,
};
esp_https_ota_config_t ota_cfg = { .http_config = &http_cfg };

esp_err_t err = esp_https_ota(&ota_cfg);
if (err == ESP_OK) {
    esp_restart();
} else {
    ESP_LOGE(TAG, "OTA не вдалося: %s", esp_err_to_name(err));
}
```

Компонент сам знаходить неактивний слот, пише в нього потоково (образ не треба вміщати в RAM), перевіряє заголовок і контрольну суму, перемикає otadata.

Для довгих оновлень і показу прогресу є покроковий варіант тих самих операцій: esp_https_ota_begin, esp_https_ota_perform у циклі, esp_https_ota_finish. Він потрібен тоді, коли під час завантаження треба годувати watchdog, малювати смужку прогресу або мати можливість скасувати.

ЖИВЛЕННЯ

ОТА — це кілька хвилин безперервної роботи радіо на прийом плюс запис у флеш. Споживання в цей час вище за звичайне робоче. Пристрій, що живиться від виснаженого акумулятора або від межового джерела, має шанс отримати brownout саме посеред оновлення. Схема з двох слотів це переживе, але оновлення не відбудеться — і так по колу. Для автономних пристроїв розумно перевіряти рівень живлення **перед** початком оновлення і не починати, якщо запасу мало (розділ 53).

TLS: мінімум, який не варто пропускати

Оновлення по звичайному HTTP означає, що будь-хто між пристроєм і сервером може підсунути свій образ. Для пристрою, що стоїть у полі, це не теоретична загроза.

Мінімум, який реально працює: HTTPS із перевіркою сертифіката сервера, захищеного в прошивку. Сертифікат кладеться в образ як вбудований ресурс, і пристрій відмовляється оновлюватися, якщо сервер не той.

Тут є пастка терміну дії: захищений сертифікат колись протермінується, і всі пристрої одночасно втратять здатність оновлюватися. Тому захищають не сертифікат сервера, а сертифікат **центру сертифікації**, яким його підписано, — він живе значно довше. Детальніше — розділ [50](#).

Наступний рівень — перевірка підпису самого образу (Secure Boot), щоб навіть скомпрометований сервер не міг залити чуже. Це вже незворотна операція над чипом, і читати про неї треба до, а не після — розділ [50](#) і картка [K11](#).

Інші способи

ArduinoOTA. Оновлення з середовища Arduino по локальній мережі: плата з'являється як мережевий порт. Зручно для розробки, коли плата вже змонтована в корпусі і діставати її незручно. Для виробу в полі не годиться: розрахована на довірену локальну мережу.

Оновлення з веб-сторінки. Пристрій піднімає власний веб-сервер із формою завантаження файлу; людина відкриває сторінку в браузері і віддає .bin. Найзручніший варіант там, де пристрій обслуговує людина на місці, а сервера оновлень немає взагалі. Обов'язково з паролем: форма без автентифікації — це запрошення залити в пристрій будь-що.

Оновлення в полі без ПК і без інтернету. Пристрій піднімає власну точку доступу, оператор під'єднується телефоном і віддає файл через ту саму веб-форму. Це найнадійніший спосіб для виїзного обслуговування: працює там, де немає ні мережі, ні ноутбука.

Що робити, коли оновлення не вдалося

Симптоми і дії, у порядку частоти:

ESP_ERR_OTA_VALIDATE_FAILED. Завантажене не є коректним образом застосунку. Найчастіше на сервері лежить не той файл: merge-bin-образ або повний дамп замість app.bin. Для OTA потрібен саме образ застосунку, без бутлоадера і таблиці розділів.

ESP_ERR_OTA_PARTITION_CONFLICT або «**немає вільного слоту**». Розбивка без OTA-розділів. Дистанційно не лікується.

Образ не влізав. Новий застосунок більший за слот. Слот змінити дистанційно неможливо — див. попередження вище.

Завантаження обривається щоразу. Перевірити таймаути HTTP-клієнта, стабільність зв'язку і рівень сигналу (RSSI). На межі покриття OTA не працює навіть

тоді, коли звичайний обмін даними ще проходить: тут значно більший обсяг і довша безперервна сесія.

Оновилося, і пристрій зник. Той випадок, від якого рятує лише rollback. Якщо він не був увімкнений — лишається фізичний доступ, картка K5.

Що з цього треба запам'ятати

Два слоти означають, що робоча прошивка не чіпається, поки нова не записана повністю. Обрив зв'язку не страшний.

Rollback — окремий механізм, і він працює лише тоді, коли підтвердження справності стоїть після виконання реальної умови, а не на початку `app_main`.

ОТА не оновлює таблицю розділів. Розбивку закладають з запасом до відправлення першого пристрою.

Оновлення по HTTP без перевірки сервера — це чужа прошивка у вашому пристрої, питання лише часу.

20. Бекап, відновлення, «цеглинка»

Питання, яке ставлять частіше за інші: **чи можна це вбити остаточно?**

Коротка відповідь: майже ні. Все, що лежить у флеші, лікується перепрошивкою. ROM-бутлоадер зашитий у кремній і не псується. Практично незворотний лише один клас операцій — запис eFuse.

Довга відповідь — цей розділ.

Що лікується завжди

Стертий або зіпсований флеш. Будь-який вміст флешу перезаписується. Пристрій без прошивки виглядає мертвим, але ROM живий і чекає на UART.

Boot loop будь-якої природи. Прошивка, що падає при старті, не заважає увійти в download mode: цей вибір робиться до того, як застосунок узагалі запуститься.

Зіпсований NVS. Стирається і переініціалізується (розділ 18).

Неправильна таблиця розділів. Перезаписується разом із рештою.

«Не бачить порт» після невдалої прошивки. Це майже завжди кабель, драйвер, права або зайнятий порт — картка K3, а не смерть чипа.

Спільне в усьому переліку: доки чип відповідає esptool шапкою з'єднання, він живий.

Що вбиває незворотно

НЕЗВОРОТНЕ

eFuse. Це набір однорозрядних запобіжників у кремнії. Запис пропалює біт **фізично** повернути його в нуль неможливо жодною командою, жодним програматором, ніколи.

Що можна втратити помилковим записом: доступ по JTAG, можливість увійти в download mode, можливість прошивати чип узагалі, здатність читати флеш поза цим конкретним чипом.

espfuse burn-* — єдина сімейка команд у всьому довіднику, яку не можна запускати «щоб подивитися, що буде». Дивитися можна лише читанням: espfuse summary.

НЕЗВОРОТНЕ

Flash Encryption і Secure Boot у release-режимі. Це односторонні двері, реалізовані через ті самі eFuse. Після ввімкнення чип перестає приймати не-підписані прошивки, а вміст флешу стає нечитним поза цим екземпляром чипа.

Дамп, знятий до ввімкнення, ще можна залити назад. Знятий після — марний: він зашифрований ключем, який не покидає чип. Пробувати це можна лише на платі, яку не шкода. Розділ 50.

Поза цими двома — фізичні пошкодження: спалений пін (5 В на GPIO), вигорілий стабілізатор, відірваний роз'єм, пробитий ESD-розрядом вхід. Це ремонт заліза (розділ 55), а не прошивки.

Дамп: єдина операція, що робить усе оборотним

Повний дамп флешу — страховка від усього переліку «лікується завжди». Без нього перепрошивка повертає пристрій до працездатності, але не до **того стану, в якому він був**: втрачаються налаштування, калібрування, креденшели, дані.

Порядок дій — картка K2. Коротко:

```
esptool --port /dev/ttyUSB0 flash-id
esptool --port /dev/ttyUSB0 read-flash 0 ALL dump-2026-08-26.bin
```

Перевірка одразу: **розмір файлу дорівнює обсягу флешу**. Менший файл — обірваний дамп. Повторити на --baud 115200.

Заливка назад:

```
esptool --port /dev/ttyUSB0 write-flash 0x0 dump-2026-08-26.bin
```

Адреса 0x0 — тому що дамп починається з нульового зсуву і містить усе, включно з бутлоадером на його правильному місці для цього чипа.

Дамп на іншу плату: що переноситься, а що ні

Спокуса очевидна: зняти дамп із робочого пристрою і залити на десять однакових плат. Це працює, але з двома застереженнями.

Перенесеться все, включно з тим, що мало бути унікальним. Серійний номер, ключі, ім'я пристрою — усе, що лежало в NVS, стане однаковим на всіх копіях. Для пристроїв, що спілкуються між собою або з сервером, це джерело дуже заплутаних збоїв. Правильний шлях — окрема конфігурація на кожен екземпляр (розділ 21).

Не перенесеться MAC-адреса. Вона зашита в eFuse кожного чипа, а не у флеші, і дампом не копіюється. Це добре: інакше два пристрої в одній мережі конфліктували б.

Калібрувальні дані радіо (`phy_init`) специфічні для екземпляра чипа. На практиці перенесення зазвичай минає без наслідків, але при межовій дальності зв'язку це один із підозрюваних.

УВАГА

Дамп із чипа одного сімейства не заливається на інше сімейство взагалі. Дамп з ESP32 classic на ESP32-S3 дасть плату, що мовчить: інші адреси, інший код. Перевірити чип перед заливкою — картка [K1](#).

Boot loop: покроковий алгоритм

Пристрій нескінченно перезавантажується. Розбір іде за логом, а не за здогадками.

Крок 1. Зняти найперший дамп після подачі живлення. Не сотий цикл, а перший: у першому — причина, в решті — наслідки. Відкрити монітор, **потім** подати живлення.

Крок 2. Прочитати `rst`: у першому рядку. Повна таблиця кодів на картці [K6](#). Три випадки, що покривають майже все:

- `rst:0xf` (`brownout`) → це **живлення**, не код. Розділ 06. Читати код далі безглуздо, поки не зміряно напругу під навантаженням.
- `rst:0xc` (`SW_CPU_RESET`) → програмне скидання, типово після паніки. Шукає Guru Meditation вище в лозі → картка [K7](#), розділ [26](#).
- `rst:0x7`, `rst:0x8`, `rst:0x9` (`watchdog`) → щось не віддає керування. Розділ 32.

Крок 3. Якщо паніки немає і живлення нормальне — дивитися, на якому рядку логів обривається цикл. Обрив на етапі бутлоадера означає проблему з розділами або образом; обрив після `cpu_start` — проблему в застосунку.

Крок 4. Відокремити прошивку від заліза. Залити свідомо справний мінімальний образ (наприклад, приклад `hello_world`). Boot loop зник — справа в прошивці. Лишився — справа в залізі або живленні.

Це найкорисніший крок у всьому алгоритмі, і його часто пропускають.

Коли `erase-flash` рятує, а коли його не можна

Рятує у трьох випадках: залишки старої розбивки заважають новій таблиці розділів; NVS пошкоджений так, що застосунок падає при старті; плата готується до передачі іншій людині.

НЕЗВОРОТНЕ

Не можна — доки не знято дампи. Це не порада, а єдина умова. `erase-flash` знищує NVS разом із калібруванням, креденшенлами і всім, що конкретне для цього пристрою. Перезбирання прошивки цього не повертає.

Особливо це стосується чужого пристрою, який ви бачите вперше: там у NVS може лежати єдина копія конфігурації, якої більше немає ніде.

Точковий варіант замість повного стирання — стерти лише те, що заважає:

```
esptool --port /dev/ttyUSB0 erase-region 0x9000 0x6000
```

Відновлення пристрою, який «нічого не бере»

Порядок від найдешевшого:

1. **Download mode вручну** — картка K4. Половина випадків закінчується тут.
2. **Знизити швидкість**: `--baud 115200`, далі `--no-stub`.
3. **Зняти навантаження зі strapping-пінів**. Від'єднати всю зовнішню обв'язку. **classic** GPIO0, GPIO2, GPIO5, GPIO12, GPIO15 під час старту мають бути вільні (повний перелік по сімействах — картка K9).
4. **Живлення від окремого джерела**, а не від USB-порту ноутбука через каб. Просадка під час прошивки виглядає як «MD5 of file does not match».
5. **Інший кабель, інший порт, прямо в комп'ютер без хаба**.
6. **espefuse summary** — подивитися, чи не ввімкнено Secure Boot або Flash Encryption попереднім власником. Читання нічого не пропалоє.

Якщо після всього чип не відповідає взагалі — це вже залізо: живлення на чипі, пайка модуля, кварц, USB-міст. Розділ 55.

Що з цього треба запам'ятати

Флеш лікується завжди. eFuse — ніколи. Ці дві фрази пояснюють, чому `espefuse burn-*` вимагає іншого ставлення, ніж усе інше в довіднику.

Дамп — єдина операція, що робить решту оборотною. Робиться до першої зміни, розмір звіряється одразу.

Boot loop розбирається за **першим** дампом логу після подачі живлення, і починається з `rst:.`

Заливка справного мінімального образу відокремлює прошивку від заліза за дві хвилини.

21. Серійна прошивка партії

Прошити одну плату і прошити двадцять — різні задачі. Різниця не в масштабі, а в тому, що при двадцяти платах з’являються питання, яких на одній не буває: чи всі прошилися, чи всі прошилися **однаково**, яка версія куди поїхала, і що робити з тим, що в кожній платі має бути своє.

Цей розділ — про процес. Той, хто прийшов сюди прошити партію готових плат, може не читати ні про FreeRTOS, ні про периферію: маршрут іде сюди і на картку K15.

Перше рішення: один файл замість трьох

Три файли на трьох адресах — нормально для розробки і погано для виробництва: три нагоди помилитися, помножені на кількість плат.

Складіть один образ (розділ 17). Якщо проєкт ESP-IDF під рукою — цим і обмежтеся, бо адреси підставить сама збірка:

```
idf.py merge-bin -o vyrib-v1.4.bin
```

Для виробництва це кращий варіант за ручний: жодного числа, набраного з голови, а отже й жодної нагоди зсунути бутлоадер.

Коли ж на руках лише готові .bin-файли — збирати доводиться вручну; **classic** адреси нижче для classic і S2, для решти чипів адреса бутлоадера інша (таблиця в розділі 16):

```
esptool --chip esp32 merge-bin -o vyrib-v1.4.bin --flash-mode dio \
0x1000 bootloader.bin \
0x8000 partition-table.bin \
0x10000 app.bin
```

Далі кожна плата прошивається однією командою з однією адресою:

```
esptool --port /dev/ttyUSB0 write-flash 0x0 vyrib-v1.4.bin
```

Тепер операцію можна віддати людині, яка нічого не знає про адреси розділів. Це і є мета.

УВАГА

--chip у merge-bin обов’язковий, бо порту немає й визначати чип нема звідки. Він же задає, з якою адресою бутлоадера образ має сенс.

--flash-mode, --flash-size і --flash-freq у merge-bin мають збігатися з тим, під що зібрано прошивку. Розбіжність дає плату, яка прошилася без помилок і не стартує. Найпростіше — узяти значення з sdkconfig проєкту, а не вгадувати.

Скрипт: те, що робить процес повторюваним

Ручна команда в терміналі не масштабується: помилку в ній видно не одразу і не завжди. Мінімальний скрипт, який робить прошивку і **перевірку**:

```
#!/bin/sh
set -e
PORT="${1:?вказіть порт: ./flash.sh /dev/ttyUSB0}"
IMAGE=vyrib-v1.4.bin

esptool --port "$PORT" --baud 460800 write-flash -z 0x0 "$IMAGE"
esptool --port "$PORT" verify-flash 0x0 "$IMAGE"
esptool --port "$PORT" read-mac | grep -i "MAC:"
echo "OK: $PORT"
```

Три речі, які тут важливі. `set -e` зупиняє скрипт на першій помилці — інакше збій прошивки лишиться непоміченим у потоці виводу. `verify-flash` перетворює «прошилося» на «у флеші лежить те, що треба». Виведений MAC іде в журнал: це єдиний надійний ідентифікатор плати.

Контроль після прошивки

«Прошилося без помилок» — не критерій приймання. Мінімальний контроль, який ловить майже все:

1. **verify-flash** — вміст флешу відповідає образу.
2. **Скидання і читання boot-логу** — пристрій справді стартує (розділ [16](#)).
3. **Одна функціональна перевірка** — те, заради чого виріб існує: світлодіод блимає, датчик читається, точка доступу піднялася.

Третій пункт ловить те, чого не спіймають перші два: справний образ, залитий на плату з непропаяним модулем або мертвим датчиком.

Живлення

Партія — це саме те місце, де вилазить межове живлення. Двадцять плат прошиваються по черзі, USB-хаб гріється, напруга просідає, і десь на чотирнадцятій починаються MD5 of file does not match. Живлення для серійної прошивки — окреме, з запасом, і бажано не від ноутбука.

Per-device: те, що в кожній платі своє

Найчастіша помилка серійного виробництва — залити двадцятьом платам однаковий образ разом із однаковим серійним номером, однаковим ключем і однаковим ім'ям пристрою. Наслідки виявляються пізно, у полі, і виглядають як містика: два пристрої «крадуть» одне в одного з'єднання.

Правильна схема розділяє два шари:

Спільне — образ прошивки. Однаковий для всієї партії, з merge-bin.

Унікальне — конфігурація в NVS. Своя для кожної плати: серійний номер, ключі, калібрувальні коефіцієнти, ім'я.

ESP-IDF дає інструмент, який робить із CSV готовий образ розділу NVS:

```
nvs_partition_gen.py generate config-0042.csv nvs-0042.bin 0x6000
esptool --port /dev/ttyUSB0 write-flash 0x9000 nvs-0042.bin
```

Адреса 0x9000 — початок розділу nvs у стандартній розбивці; звірити зі своєю таблицею розділів (розділ 18).

Альтернатива для невеликих партій: заливати спільний образ, а унікальне записувати через консоль після першого старту — прошивка при першому запуску просить серійний номер і зберігає його в NVS. Менше інструментів, але додає ручну операцію до кожної плати.

УВАГА

MAC-адреса кожного чипа унікальна від заводу і лежить в eFuse. Якщо все, що вам потрібно, — відрізнити пристрої один від одного, окремий серійний номер не потрібен: беріть MAC. Це прибирає цілий шар роботи разом із нагодою помилитися.

Маркування плат

Плата без позначки — плата, про яку через місяць невідомо нічого. На кожну наклеюється або пишеться маркером мінімум:

- порядковий номер у партії;
- версія прошивки;
- дата.

Наліпка клеїться на бік, який видно в зібраному виробі. Наліпка, що опинилася всередині корпусу, не існує.

Журнал партії

Один файл на партію, рядок на плату:

№	MAC	Версія	Дата	Контроль	Примітка
0041	A0:B7:...:14	v1.4	2026-08-26	OK	
0042	A0:B7:...:2C	v1.4	2026-08-26	OK	корпус подряпаний
0043	A0:B7:...:31	v1.4	2026-08-26	брак	не стартує, модуль

Цей файл відповідає на питання, які виникають через півроку і не мають іншого джерела відповіді: скільки зроблено, яка версія у пристрою з таким MAC, чи був цей екземпляр у браку.

Разом із журналом зберігаються **самі файли прошивки** тієї версії, що поїхала, і, обов'язково, `.elf` того самого збирання: без нього `backtrace` з поля не розшифрувати (картка [K7](#)).

НЕЗВОРОТНЕ

Версія прошивки, яка поїхала до замовника, має існувати в архіві як файли, а не як «гілка в `git`, з якої це збиралося». Перезібрати «такий самий» образ пізніше майже ніколи не виходить точно: змінилася версія тулчейну, змінилася бібліотека, змінився шлях збирання. Адреси зсунуться, і `.elf` перестане відповідати тому, що в полі.

Кілька плат одночасно

Коли партія вимірюється сотнями, послідовна прошивка стає вузьким місцем. Варіанти, у порядку складності:

Кілька портів паралельно. Скрипт запускається на `/dev/ttyUSB0`, `ttyUSB1`, `ttyUSB2` одночасно. Найдешевший спосіб отримати кратне прискорення. Обмеження — живлення хаба: чотири плати, що одночасно пишуть у флеш, дають помітний струм.

Flash Download Tool (Windows) вміє кілька плат в одному вікні — зручно там, де прошиває оператор без командного рядка (розділ [17](#)).

Прошивка до монтажу, на спеціальному ложементі з підпружиненими контактами (`pogo pins`). Це вже оснастка, і має сенс від сотень плат.

Якщо плата з партії не прошивається

Не зупиняти партію. Відкласти плату, позначити, продовжити з рештою, розібратися потім — розділ [55](#) і картка [K8](#). Найчастіші причини саме в партії: непропаяний модуль, замикання під час монтажу, плата іншої ревізії, що приїхала в тій самій коробці.

Що з цього треба запам'ятати

`merge-bin` — один файл, одна адреса, нема чого переплутати.

`verify-flash` і функціональна перевірка — обов'язкові кроки, а не опції.

Спільний образ і унікальний `NVS` — це два різні шари. Змішати їх означає отримати двадцять пристроїв з однаковою особистістю.

Журнал партії і збережені файли прошивки разом із `.elf` — те, без чого через півроку неможливо відповісти на просте питання «що в цьому пристрої».

Частина VI. Невідомий пристрій

*Плата чи виріб без документації: зберегти стан,
визначити, зрозуміти чужу прошивку.*

22. Збереження стану перед втручанням

Перед вами чужий пристрій. Ви ще нічого не зробили. Це найцінніший момент за весь час роботи з ним — і єдиний, коли можна зафіксувати, **яким він був**.

Далі буде інакше. Дроти від'єднуються і не згадається, який куди йшов. Перемички зміняться. Флеш перезапишеться. І тоді питання «а як воно працювало до нас» перестане мати відповідь.

Двадцять хвилин зараз — або незворотна втрата. Порядок дій на картці K2 тут — чому саме так і що з цим робити далі.

Правило, з якого все починається

Спершу фіксація, потім втручання. Не «подивимось, що воно робить, а потім задокументуємо» — бо перше ж вмикання може змінити стан: прошивка запише щось у NVS, спрацює watchdog, розрядиться акумулятор, який тримав RTC.

Це правило здається надмірним рівно до першого разу, коли воно знадобилося.

Фотографія як технічний документ

Фото тут — не «на всяк випадок», а джерело даних, до якого ви повертатиметесь.

Обидві сторони плати, з відстані, на якій читаються написи на чипах. Маркування модуля — головне джерело істини про те, що це взагалі таке (розділ 23), і воно легко стирається пальцями і спиртом.

Кожен роз'єм із під'єднаними дротами — до від'єднання. Це та фотографія, заради якої все робиться.

Положення перемичок і мікроперемикачів. DIP-перемикач має два стани і жодного напису про те, який був.

Загальний вигляд у корпусі — як воно було встановлено, куди йшли кабелі, де були стяжки.

Знімати при доброму розсіяному світлі й **без спалаху**: спалах засвічує лазерне маркування на чипах саме тоді, коли воно найпотрібніше.

УВАГА

Фото робиться телефоном, і на цьому етапі здається, що воно нікуди не дінеться. За тиждень у галереї буде дві тисячі знімків. Перекласти фото в каталог пристрою треба одразу, а не «потім».

Схема з'єднань словами

Фото показує, як воно виглядало. Текстовий запис показує, як воно було з'єднане, — і саме він потрібен, коли пристрій треба зібрати назад.

Записувати треба **сигнал, а не колір**:

```
GPI04 → синій → датчик DS18B20, лінія DATA
GPI021 → зелений → дисплей SSD1306, SDA
GPI022 → жовтий → дисплей SSD1306, SCL
3V3 → червоний → обидва пристрої
GND → чорний → обидва пристрої
GPI00 → білий → кнопка на землю (Δ strapping!)
```

Кольори дротів у чужих виробках не означають нічого: червоний може бути землею, чорний — сигналом. Покладатися можна лише на записане.

Останній рядок у прикладі — те, заради чого варто робити цей запис уважно. Кнопка на GPI00 пояснює, чому пристрій іноді не стартує (розділ 16), і без запису це б довелося шукати годинами.

Вимірювання: чотири числа

Мультиметром, **до подачі живлення**, у режимі прозвонки:

- опір між 3V3 і GND;
- опір між 5V/VIN і GND.

Дзвенить накоротко — далі не вмикати взагалі, шукати замикання (розділ 55). Це та перевірка, що рятує від «увімкнув і додав до несправностей ще одну».

Під живленням, мультиметром на постійній напрузі:

- реальна напруга на 3V3;
- реальна напруга на вході.

Ці чотири числа записуються. Вони — **база для порівняння**: коли через годину щось зміниться, буде з чим звірити. Без них «здається, просіло» лишається здогадкою.

Дамп флешу

Головне. Все попереднє можна відновити уважністю; це — ні.

Спершу дізнатися обсяг, інакше дамп буде неповний:

```
esptool --port /dev/ttyUSB0 flash-id
esptool --port /dev/ttyUSB0 read-flash 0 ALL dump-2026-08-26.bin
```

Перевірити розмір файлу одразу. Він має точно дорівнювати обсягу флешу. Менший — обірваний дамп, а не «стисненіший»; повторити на --baud 115200.

НЕЗВОРОТНЕ

Дамп, знятий після erase-flash або після перепрошивки, не має сенсу: він фіксує вже змінений стан. Порядок «спробуємо прошити, а якщо не вийде — знімемо дампи» не працює в принципі.

І ще: дампи, що лежать лише на робочому ноутбуку, — не резервна копія. Скопіювати в друге місце **до** початку втручання.

Що з дампа можна дістати, не змінюючи пристрій, — розділ 24.

Версія прошивки і те, що видно безкоштовно

Перед втручанням варто зняти boot-лог: відкрити монітор на 115200, подати живлення, зберегти вивід у файл.

Другий бутлоадер безкоштовно друкує версію ESP-IDF, обсяг флешу, який він бачить, і **всю таблицю розділів з адресами** (розділ 16). Це готова відповідь на «що всередині», без розбирання дампа.

Лог зберігається у файл, а не читається з екрана. Через годину екран прокрутиться.

Куди це все складається

Один каталог на пристрій. Назва — дата і що це, а не нова_папка_2.

```
2026-08-26-datchyk-teplytsi/
foto/
zyednannya.txt      ← схема словами
vymiryuvannya.txt   ← чотири числа
boot-log.txt
dump-2026-08-26.bin
notatky.txt          ← що робили і що вийшло
```

notatky.txt дописується по ходу роботи. Дамп без записів через місяць не означає нічого: невідомо, з якого він пристрою, до чи після втручання знятий і що з ним уже пробували.

Коли цей розділ можна пропустити

Чесно: коли пристрій ваш власний, зібраний вчора, і його вміст точно відтворюється з репозиторію. Тоді дампи справді зайвий.

У всіх інших випадках — від чужого виробу до власного пристрою, що пропрацював рік у полі і накопичив у NVS налаштування, яких немає в жодному файлі, — етап обов'язковий.

Що з цього треба запам'ятати

Фіксація йде **перед** втручанням, а не паралельно з ним.

Записується сигнал, а не колір дроту.

Чотири виміряні числа — база для порівняння, без якої «просіло» лишається здогадкою.

Дамп перевіряється за розміром одразу і копіюється в друге місце до початку роботи.

Boot-лог віддає версію IDF, обсяг флешу і таблицю розділів безкоштовно — за нього не треба платити нічим.

23. Тріаж невідомої плати

Тріаж — визначення, з чим ви маєте справу і що з цим можна робити. Порядок кроків не довільний: кожен відсікає варіанти для наступного і кожен дешевший за попередній у сенсі ризику.

Стисла версія — картка K1. Тут — те саме з поясненням, чому саме так, і що робити, коли крок не дав відповіді.

Жоден із шести кроків нижче нічого в пристрої не змінює, тому тріаж проходять до збереження стану (розділ 22). Збереження обов’язкове перед **першою зміною** — прошивкою, стиранням, правкою конфігурації; тріаж до неї не доходить.

Крок 1. Маркування: що це взагалі

Головне джерело істини — напис на металевій кришці модуля. Не на платі, не в оголошенні продавця, а на самому модулі.

Напис	Чип	Що це значить практично
ESP32-WR00M-32	ESP32 classic	двоядерний Xtensa, без PSRAM
ESP32-WR00M-32D, -32E	ESP32 classic	пізніші ревізії, той самий код
ESP32-WROVER, -B, -E	ESP32 classic	те саме + PSRAM
ESP32-S3-WR00M-1	ESP32-S3	двоядерний, native USB
ESP32-C3-MINI-1	ESP32-C3	одноядерний RISC-V
ESP8266, ESP-12, ESP-01	не ESP32	інша архітектура, інший тулчейн

Останній рядок трапляється частіше, ніж хотілося б: ESP8266 зовні схожий, продається поруч і в тих самих корпусах.

Суфікс після назви модуля кодує обсяг флешу і PSRAM: N8 — 8 МБ флешу, N16R8 — 16 МБ флешу і 8 МБ PSRAM. На **S3** це важливо особливо: модулі з Octal PSRAM додатково займають GPIO 33–37 (картка K9).

УВАГА

S3 Octal PSRAM треба зазначити в конфігурації — типово стоїть **Quad**. У ESP-IDF SPIRAM_MODE за замовчуванням SPIRAM_MODE_QUAD на всіх сімействах, тож модуль N16R8 із восьмилінійною PSRAM без правки менюконфігу поводить як плата без PSRAM узагалі.

Симптом характерний: `heap_caps_get_free_size(MALLOC_CAP_SPIRAM)` повертає нуль на платі, за яку заплачено саме через ті 8 МБ. Шукати при цьому починають несправність, а не налаштування.

Component config → ESP PSRAM → Mode (QUAD/OCT) of SPI RAM chip in use (розділ 11).

Маркування стерте або нечитне — не тупик. Переходьте до кроку 4: `esptool` назве сімейство сам, щойно під'єднається. Модуль при цьому лишиться невідомим, але чипа для більшості рішень достатньо.

Крок 2. Огляд без живлення

Очима, поки нічого не увімкнено:

- цілий USB-роз'єм, не хитається, площадки не відірвані;
- стабілізатор не здутий, без темних плям і запаху;
- немає перемичок припою між сусідніми пінами;
- немає слідів води, окислення, білого нальоту від флюсу;
- нічого не обвуглене.

Знайшли щось із переліку — далі йде ремонт (розділ 55), а не прошивка. Подавати живлення на плату з видимим дефектом означає з великою ймовірністю додати до однієї несправності другу.

Крок 3. Живлення: перевірка перед першим вмиканням

Мультиметр у режимі прозвонки, живлення від'єднане.

Між `V33` і `GND` не має бути короткого замикання. Між `5V/VIN` і `GND` — теж. Дзвенить — шукати замикання, не вмикати.

НЕЗВОРОТНЕ

Логіка ESP32 — **3.3 В**. Єдиний пін, куди можна подати 5 В, це `5V/VIN`: він іде на вхід стабілізатора, а не в чип.

Для незнайомої плати це питання не риторичне: буває, що плату живили нештатно, і на місці стабілізатора стоїть перемичка. Перед подачею живлення варто подивитися, чи стабілізатор узагалі є і чи він під'єднаний.

Перше вмикання — від джерела з **обмеженням струму**, якщо воно є (лабораторний БЖ, розділ 10). Обмеження на рівні 200–300 мА зупинить струм при замиканні замість того, щоб випалювати доріжку.

Немає такого джерела — вмикати від USB-порту з хабом, а не напряму від ноутбука: хаб дешевше замінити.

Після вмикання: реальна напруга на `V33`. Має бути близько 3.3 В. Помітно менше — просадка, і далі спершу розділ 06, а не прошивка.

Крок 4. Зв'язок і опитування чипа

Під'єднати USB, дочекатися появи порту (картка К3).

```
esptool --port /dev/ttyUSB0 flash-id
```

Шапка з'єднання, яку esptool друкує перед будь-якою командою, називає сімейство, ревізію кремнію і MAC (розділ 17). **Звірити з написом на модулі.** Розбіжність означає перемаркований модуль — це не робить плату непридатною, але тепер ви знаєте, що написам на ній вірити не можна.

flash-id називає обсяг флешу. Менший за очікуваний для цього модуля — клон із меншою пам'яттю. Практичний наслідок: прошивка, зібрана під 4 МБ, на 2 МБ не запрацює, і причина буде неочевидною.

Чип не відповідає — це ще не вирок. За порядком:

1. Download mode вручну — картка К4. Половина випадків закінчується тут.
2. Знизити швидкість: --baud 115200, далі --no-stub.
3. Зняти всю зовнішню обв'язку зі strapping-пінів.
4. Інший кабель, інший порт без хаба.

Не допомогло нічого — розділ 55: живлення на самому чипі, пайка модуля, кварц, USB-міст.

Крок 5. Boot-лог

Монітор на 115200, натиснути ЕМ. Розшифровка кожного рядка — картка К6 і розділ 16.

За один цикл скидання звідси дістається: причина попереднього скидання, версія ESP-IDF, обсяг флешу очима бутлоадера і **вся таблиця розділів з адресами**. Останнє особливо цінне: воно каже, чи є на пристрої OTA, чи є файлова система, скільки місця під застосунок.

Чого лог не скаже: що саме робить прошивка. Це наступний розділ.

Крок 6. Перевірка GPIO

Робиться в останню чергу і тільки за потреби — коли плату треба використати, а не просто визначити.

Небезпека цього кроку в тому, що не всі піни рівні. Перед подачею будь-чого на пін треба знати: чи це strapping-пін, чи він тільки на вхід, чи він не зайнятий флешем. Картка К9.

classic GPIO 6–11 не чіпати ніколи — вони з'єднані з мікросхемою флешу на модулі. Те, що вони виведені на гребінку, не означає, що вони вільні.

Безпечний спосіб перевірити пін — сконфігурувати його як вхід із підтягуванням і замикати на землю дротом. Ніякого струму, ніякого ризику.

Дерево рішень

Куди йти за результатом:

Що маємо	Висновок	Далі
Чип відповідає, лог осмислений	робочий пристрій з прошивкою	розділ 24
Чип відповідає, лог порожній	немає застосунку	картка K5
Чип відповідає, boot loop	падає при старті	розділ 20, потім 26
Чип відповідає, invalid header	застосунок не там або биті дані	розділ 18
Чип не відповідає, порт є	не в download mode	картка K4
Чип не відповідає, порту немає	кабель, драйвер, права	картка K3
K3 по живленню	несправність заліза	розділ 55
Напис ESP8266	інша платформа	ця книга не про це

Чого тріаж не дає

Тріаж відповідає на «що це і чи воно живе». Він **не** відповідає на «що воно робить» і «звідки взявся код». Для цього треба розібрати прошивку — розділ 24.

І окремо: тріаж не робить пристрій вашим. Якщо це чужий виріб, який доведеться повернути в робочому стані, все, що ви дізналися на цих шести кроках, отримано **без жодної зміни** в пристрої. Це властивість цього порядку дій, і її варто зберігати якнайдовше.

Що з цього треба запам'ятати

Порядок кроків важливіший за самі кроки: огляд перед живленням, живлення перед зв'язком, зв'язок перед прошивкою.

Напис на модулі звіряється з шапкою esptool. Розбіжність — це інформація, а не збій.

Чип, що не відповідає, у половині випадків просто не в download mode.

Усі шість кроків проходяться без жодної зміни в пристрої.

24. Чужа прошивка: форензика

Пристрій визначено, він живий, у флеші щось є (розділ 23). Питання змінюється: **що саме там лежить і що з ним можна зробити.**

Це розділ про роботу з двійковим образом без вихідних текстів. Мета — не «зламати», а зрозуміти рівно стільки, щоб ухвалити рішення: лагодити, переписати, лишити як є або відмовитися.

Передумова: дамп знято і перевірено (розділ 22). Усе, що нижче, робиться з **файлом дампа**, а не з пристроєм. Пристрій при цьому лишається недоторканим — і це головна перевага такого порядку.

Крок 1. Карта: таблиця розділів

Перше і найдешевше. Або з boot-логу, який друкує таблицю при кожному старті (розділ 16), або з дампа:

```
dd if=dump.bin of=pt.bin bs=1 skip=$((0x8000)) count=$((0x1000))  
python $IDF_PATH/components/partition_table/gen_esp32part.py pt.bin
```

Таблиця відповідає на кілька питань одразу.

Чи є ОТА. Два розділи ota_0/ota_1 плюс otadata означають, що пристрій розрахований на дистанційні оновлення (розділ 19). Один factory — не розрахований, і додати це дистанційно неможливо.

Чи є файлова система. Розділ типу spiffs, littlefs або fat означає, що на пристрої лежать файли: веб-інтерфейс, конфігурація, логи.

Скільки місця займає застосунок. Це орієнтир складності: 400 КБ — щось просте, 1.5 МБ — Wi-Fi зі стеком, TLS і веб-сервером.

Крок 2. Рядки: найшвидша розвідка

У кожній прошивці ESP-IDF лишається багато читабельного тексту. Це не недбалість розробника, а наслідок того, як влаштоване логування.

```
strings -n 6 dump.bin | less  
strings -n 6 dump.bin | grep -iE "v[0-9]+\.[0-9]+|20[0-9]{2}-"
```

Що там трапляється майже завжди:

Версія ESP-IDF і дата збирання. Рядки виду v5.1.2 і мітка часу компіляції. Це одразу каже, наскільки старий виріб і під який тулчейн його збирали.

Назви компонентів і тегів логування. wifi, esp_https_ota, nvs, а поруч — теги самого застосунку: SENSOR, MODBUS, PUMP_CTRL. Ці теги описують архітектуру виробу краще, ніж будь-яка документація, якої немає.

Тексти повідомлень. Failed to connect to broker, Calibration required — прямо називають, що пристрій робить і на що скаржитися.

Адреси й імена. URL сервера, ім'я точки доступу, топіки MQTT, шляхи до файлів.

УВАГА

Тут же трапляється те, чого не мало б бути: паролі й ключі, зашиті в код відкритим текстом. Це поширена і небезпечна практика (розділ 50).

Якщо ви працюєте з чужим виробом — знайдений пароль не є вашою здобиччю. Він є знахідкою, про яку варто повідомити власника, і причиною ставитися до дампа як до документа з чутливими даними: не класти в загальнодоступне місце, не надсилати в чат «щоб подивилися».

Крок 3. Що в NVS

NVS зберігає конфігурацію конкретного екземпляра. Часто саме там лежить відповідь на «чому цей пристрій поводить себе не так, як сусідній однаковий».

Витягти розділ (адреса і розмір — з таблиці розділів кроку 1):

```
esptool --port /dev/ttyUSB0 read-flash 0x9000 0x6000 nvs.bin
```

Формат NVS — сторінковий, з простором імен, ключем і типізованим значенням. Для розвідки зазвичай досить strings: імена ключів і текстові значення видно неозброєним оком. Для повного розбору є сторонні розбирачі формату NVS.

Що там типово лежить: SSID і пароль Wi-Fi, адреса сервера, серійний номер, калібрувальні коефіцієнти, лічильники напруцювання, стан автомата.

Крок 4. Файлова система

Якщо в таблиці є розділ spiffs або littlefs, з нього дістаються готові файли — часто це веб-інтерфейс цілком, разом із HTML і скриптами.

Витягти розділ так само, як NVS. Далі розбирати відповідним інструментом: mklittlefs, mkspiffs — обидва вмюють не лише пакувати, а й розпакувати.

Веб-інтерфейс, витягнутий звідси, — найшвидший спосіб зрозуміти функціонал виробу: у ньому видно всі поля налаштувань, усі команди і часто — прихований службовий розділ.

Крок 5. Чи є вихідні тексти

Перш ніж розбирати двійковий код, варто перевірити очевидне.

Рядки з кроку 2 часто містять назву проєкту, ім'я розробника, назву компанії або пряме посилання на репозиторій.

Пристрій може бути на відомій платформі. Теги логування виду `esphome` або `tasmota` означають, що це не унікальна розробка, а відкрита прошивка з конфігурацією. У такому випадку вихідні тексти є, і завдання зводиться до пошуку конфігурації, а не до реверсу.

Ліцензійні тексти в дампі. GPL і подібні вимагають надання вихідних текстів — і сам факт наявності такого тексту в прошивці є підставою запитати їх у виробника.

Крок 6. Що можна змінити без вихідних текстів

Реалістична межа проходить тут, і корисно знати її заздалегідь.

Можна змінити:

- вміст NVS — налаштування, адреси, креденшели, калібрування;
- вміст файлової системи — веб-сторінки, конфігураційні файли;
- залити зовсім іншу прошивку, свою, зберігши дамп старої.

Практично не можна:

- змінити логіку в скомпільованому коді. Теоретично можливо, практично — на порядки дорожче, ніж написати свою прошивку з нуля;
- перезібрати той самий образ із правкою: без вихідних текстів немає з чого збирати.

УВАГА

Звідси випливає найчастіше правильне рішення для чужого пристрою, який треба змінити: **не правити чуже, а написати своє**. Тріаж дав пінаут і схему з'єднань (розділ 22), рядки і веб-інтерфейс дали функціонал. Цього достатньо, щоб написати власну прошивку, яка робить те саме, — і вона буде супроводжуваною, на відміну від пропатченого двійкового файлу.

Старий дамп при цьому лишається: у будь-який момент можна повернути пристрій у початковий стан.

Відтворюване збирання

Питання, що виникає з іншого боку: як зробити так, щоб **ваш** виріб через п'ять років можна було зібрати заново і отримати той самий образ.

Умови, за яких це працює:

- зафіксована версія ESP-IDF (не «остання», а конкретний тег);
- зафіксовані версії всіх компонентів і бібліотек;
- збережений `sdkconfig` проєкту;
- збережений сам образ і `.elf` того збирання.

Останній пункт — найважливіший і найдешевший. Перезібрати «такий самий» образ пізніше майже ніколи не виходить точно; збережений файл не вимагає нічого, крім місця на диску (розділ 21).

Ризики втручання

НЕЗВОРОТНЕ

Перед будь-якою зміною в чужому пристрої — два питання.

Чи є дамп і чи він перевірений? Розмір файлу дорівнює обсягу флешу. Якщо ні — повернути пристрій у початковий стан буде нічим.

Чи не ввімкнено Flash Encryption або Secure Boot? `espefuse summary` (лише читання, нічого не пропалює). Якщо ввімкнено — дамп зашифрований ключем, що не покидає чип, і залити його назад на цей самий чип ще можна, а зрозуміти вміст — ні. Розділ 50.

Третій ризик — не технічний. Чужий пристрій може бути частиною системи, про яку ви не знаєте: зміна конфігурації одного вузла ламає роботу іншого. Перш ніж міняти — з'ясувати, з чим воно спілкується. Рядки з кроку 2 і топіки MQTT з кроку 3 зазвичай це показують.

Що з цього треба запам'ятати

Уся розвідка робиться з файлу дампа, а не з пристрою: пристрій лишається недоторканим до моменту рішення.

`strings` над дампом за п'ять хвилин дає версію IDF, архітектуру за тегами логування, адреси серверів і тексти повідомлень.

Змінити налаштування і файли можна; змінити логіку в скомпільованому коді — практично ні.

Найчастіше правильне рішення — не патчити чуже, а написати своє, зберігши дамп для повернення назад.

Частина VII. Налаштування і діагностика

Від логіки до логічного аналізатора: як довідатися, що саме сталося, замість того, щоб здогадуватися.

25. Serial monitor і логування

Лог — основний і часто єдиний інструмент налагодження вбудованої системи. Немає екрана, немає відлагоджувача під рукою, пристрій стоїть у корпусі на висоті чотирьох метрів — але UART працює завжди.

Цей розділ про дві різні речі, які легко плутають: **як прочитати лог** з пристроєм, який уже щось пише, і **як писати лог** так, щоб він допомагав через півроку.

Монітори портів

Усі роблять те саме: відкривають порт і показують байти. Різниця — у зручностях.

Монітор	Коли він	Вихід
idf.py monitor	працює зі своїм проектом ESP-IDF	Ctrl+J
Arduino IDE	працює в Arduino	кнопка
PlatformIO	працює в PlatformIO	Ctrl+C
minicom	чужий пристрій, Linux	Ctrl+A, потім X
screen	чужий пристрій, є під рукою скрізь	Ctrl+A, потім K
picocom	чужий пристрій, найпростіший	Ctrl+A, потім Ctrl+X
PuTTY	Windows	вікно

idf.py monitor вартий окремої згадки, бо робить дві речі, яких не роблять інші: **розшифровує backtrace** у назви функцій і номери рядків на льоту (розділ 26) і дозволяє скинути плату не торкаючись її (Ctrl+I, потім Ctrl+R).

Ціна цього — він працює лише з каталогу того проекту, з якого зібрано прошивку. Для чужого пристрою беріть будь-що інше.

Зняти лог з невідомого пристрою

Порядок на картці K6. Ключове тут — швидкість.

115200 бод — швидкість, на якій говорить ROM-бутлоадер ESP32. Вона не залежить ні від налаштувань застосунку, ні від частоти кварцу на платі. Тому: якщо перші рядки після скидання читаються, а далі йде каша — це нормально, просто застосунок перейшов на іншу швидкість.

Що пробувати для **застосунку**: 9600, 57600, 230400, 921600.

УВАГА

Порада «спробуйте 74880», що трапляється в статтях, стосується **ESP8266**, а не ESP32. У ESP8266 швидкість ROM пропорційна кварцу, і на типовому 26-мегагерцовому виходить $115200 \times 26/40 = 74880$.

Наслідок протилежний до очікуваного: якщо на платі, яку ви вважаєте ESP32, лог читається саме на 74880 — це не «інша швидкість ESP32», це **інший чип**. Звірити маркування модуля (розділ 23).

УВАГА

Лог зберігається у **файл**, а не читається з екрана. Через десять хвилин роботи буфер терміналу прокрутиться, і найцікавіше — перші рядки після подачі живлення — зникне назавжди.

```
picocom -b 115200 /dev/ttyUSB0 | tee log-2026-08-26.txt
```

Для idf.py monitor є **власний ключ**, і він кращий за перенаправлення:

```
idf.py monitor --save-log
```

Ім'я файлу монітор вибирає сам — `log.<ім'я-elf>.<дата-час>.txt` у поточному каталозі — і каже його вголос рядком `Logging is enabled into file ...`. Свого імені він **не приймає**: це прапорець, а не параметр зі значенням.

Різниця не косметична. Монітор — інтерактивна програма з кольорами й керуванням із клавіатури; через `| tee` у файл потрапляють ще й керівні послідовності ANSI, а сама інтерактивність псується.

`tee` лишається правильним інструментом для `picocom`, `screen` і решти моніторів без такого ключа.

Це та звичка, що відрізняє годину роботи від дня.

І окремо: монітор **тримає порт**. Прошивка при відкритому моніторі не почне-ться — `Device or resource busy`. Це найчастіша причина «esptool раптом перестав бачити плату».

ESP_LOGx: як писати лог

ESP-IDF дає п'ять рівнів і поняття тега.

```
static const char *TAG = "PUMP";

ESP_LOGE(TAG, "аварія: тиск %d поза межами", p); // Error
ESP_LOGW(TAG, "тиск близько до межі: %d", p);    // Warning
ESP_LOGI(TAG, "насос увімкнено");                 // Info
ESP_LOGD(TAG, "цикл %d, стан %d", i, state);      // Debug
ESP_LOGV(TAG, "сипі дані: %02x", raw);            // Verbose
```

Тег — це ім'я підсистеми. Він друкується в кожному рядку і дозволяє відфільтрувати потрібне з потоку. Один тег на файл або на логічний модуль; `static const char *TAG` на початку файлу — усталений спосіб.

Рівень визначає, чи потрапить рядок у прошивку взагалі. Глобальна межа задається в `menuconfig`: `Component config` → `Log` → `Log Level` → `Default log verbosity`. Все, що нижче межі, **вирізається компілятором** — не сповільнює роботу і не займає місця у флеші.

Це важлива властивість: `ESP_LOGD` у гарячому циклі не коштує нічого, поки рівень збирання `Info`.

Керування в `runtime` — коли треба підняти детальність окремої підсистеми без перезбирання:

```
esp_log_level_set("", ESP_LOG_INFO);
esp_log_level_set("PUMP", ESP_LOG_DEBUG);
esp_log_level_set("wifi", ESP_LOG_WARN);
```

Працює лише для тих рівнів, що не вирізані при компіляції.

УВАГА

Найчастіша пастка з логуванням: `esp_log_level_set` мовчки не робить нічого.

Виглядає так: у коді `esp_log_level_set("PUMP", ESP_LOG_DEBUG)`, у `ESP_LOGD` нічого не з'являється, помилки теж немає. Причина не в тегу і не в порядку викликів — рядка просто немає у прошивці.

Наївне лікування — зібрати все з `Debug` — псує головне: пристрій починає сипати налагоджувальним логом постійно, і в полі це і повільно, і незручно.

Правильний спосіб — сусідній пункт того самого меню, **Maximum log verbosity**. Він задає стелю того, що **компілюється**, окремо від того, що друкується за замовчуванням:

Параметр	Що робить
Default log verbosity	рівень, з яким прошивка стартує
Maximum log verbosity	до чого можна підняти в <code>runtime</code>

Отже `Default = Info`, `Maximum = Debug` дає рівно те, чого хочеться: тихий пристрій, який на команду піднімає детальність потрібної підсистеми без перезбирання. Ціна — трохи більший образ, бо рядки `ESP_LOGD` лишаються у флеші.

Типове значення `Maximum` — `Same as default`; тобто за замовчуванням запас не передбачено, і його треба ввімкнути свідомо.

Сам `esp_log_level_set` при цьому має працювати: за це відповідає `Enable dynamic log level changes at runtime`, і воно ввімкнене типово.

Як писати лог, щоб він допоміг через півроку

Різниця між логом, який рятує, і логом, який займає місце, — у кількох звичках.

Логувати переходи станів, а не факт виконання рядка. `ESP_LOGI(TAG, "тут")` не каже нічого. `ESP_LOGI(TAG, "стан: ОЧІКУВАННЯ → РОБОТА, причина: тиск %d", p)` каже все.

Логувати значення, а не висновки. «датчик несправний» — це вже інтерпретація, і вона може бути хибною. «датчик повернув -127, очікували [-55..125]» дає і висновок, і підставу для нього.

Помилки логувати з кодом і назвою.

```
esp_err_t err = i2c_master_probe(bus, ADDR, 100);
if (err != ESP_OK) {
    ESP_LOGE(TAG, "датчик 0x%02x не відповідає: %s", ADDR,
              esp_err_to_name(err));
}
```

`esp_err_to_name` перетворює число на читабельне ім'я. Без нього в лозі буде `0x105`, і його доведеться шукати по заголовках.

Не логувати в гарячому циклі на рівні Info. Кожен рядок — це байти через UART на 115200 бод, тобто приблизно 11 КБ/с. Рясний лог не просто засмічує вивід — він **уповільнює систему** і може сам стати причиною збою, який ви шукаєте.

УВАГА

Класична пастка: помилка зникає, коли ввімкнути детальний лог. Це майже завжди означає, що проблема в таймінгах — лог додав затримок і замаскував гонку. Шукати треба не в тому місці, де перестало відтворюватися, а в синхронізації (розділ 31).

Куди писати лог, крім UART

UART зникає, щойно пристрій поїхав у поле. Варіанти, що лишаються.

Кільцевий буфер у RAM. Останні `N` рядків тримаються в пам'яті і віддаються на запит — по HTTP, по команді з UART, у складі телеметрії. Найдешевший спосіб мати «що сталося перед збоєм» без зовнішньої пам'яті. Пережити перезавантаження може, якщо буфер лежить у RTC-пам'яті.

Файл на флеші або SD. Дає історію за дні й тижні. Два обмеження: знос флешу при частому записі (розділ 18) і те, що запис може обірватися на зникненні живлення. Для SD — розділ 49.

По мережі. Відправка логів на сервер — MQTT, HTTP, syslog. Найзручніше в експлуатації і найгірше саме тоді, коли треба найбільше: проблеми зі зв'язком не залогуються, бо логуються по зв'язку.

Coredump у флеші. Не лог, а знімок стану в момент паніки — розділ 26. Для рідкісних збоїв у полі це часто корисніше за будь-який лог.

Практичне поєднання для пристрою в полі: кільцевий буфер у RAM для поточного, coredump для паніки, відправка по мережі для того, що цікавить постійно.

Лог у продукційній прошивці

Ставити рівень `Verbose` у виробі, що поїхав, — погана ідея з трьох причин: гальмує, зношує флеш при записі у файл і **розкриває внутрішню кухню** тому, хто зніме дамپ (розділ 24).

Розумний компроміс: збирати з рівнем `Info`, тримати `Debug` доступним через `runtime-перемикач`, і не логувати креденшели ніколи — навіть на рівні `Verbose`, навіть тимчасово. Тимчасове має властивість доїжджати до замовника.

Що з цього треба запам'ятати

115200 — швидкість ROM, вона не залежить від застосунку.

Лог пишеться у файл (`idf.py monitor --save-log`, для решти моніторів `tee`), а не читається з екрана.

Монітор тримає порт: закрити перед прошивкою.

Логувати переходи станів і сирі значення, а не факт виконання рядка і не готові висновки.

Помилка, що зникає від детального логу, — це проблема таймінгів, а не розв'язана проблема.

26. Читання збоїв

Прошивка впала. У порту з’явився дамп регістрів, слово `Guru Meditation` і рядок незрозумілих чисел. Це не поламка плати — це докладний звіт про те, де саме програма померла, і читати його треба як звіт, а не як неприємність.

Стисла версія на 60 секунд — картка [K7](#). Тут — повний розбір і те, що з ним робити далі.

Анатомія паніки

```
Guru Meditation Error: Core 0 panic'ed (LoadProhibited). Exception was unhandled.

Core 0 register dump:
PC      : 0x400d2f1a  PS      : 0x00060730  A0      : 0x800d3045  A1      :
0x3ffb1f20
A2      : 0x00000000  A3      : 0x3ffb2010  A4      : 0x00000064  A5      :
0x00000001
...
EXCVADDR: 0x00000008  LBEG    : 0x400014fd  LEND    : 0x4000150d  LCOUNT  :
0xffffffff

Backtrace: 0x400d2f1a:0x3ffb1f20 0x400d3042:0x3ffb1f40 0x400d5a1c:0x3ffb1f70
```

Чотири поля, з яких читається майже все.

Причина в дужках — `LoadProhibited`. Що саме заборонено зробити.

PC — адреса інструкції, на якій упало. Це «де».

EXCVADDR — адреса, за якою намагалися звернутися. Це «куди». У прикладі `0x00000008` — тобто зверталися до поля структури зі зсувом 8 за покажчиком `NULL`. Найпоширеніший випадок у практиці.

Backtrace — ланцюжок викликів. Пари адреса:стек.

Причини паніки і що вони означають

Причина	Що заборонено	Що шукати
<code>LoadProhibited</code>	читання з недійсної адреси	<code>NULL</code> або звільнений покажчик
<code>StoreProhibited</code>	запис за недійсною адресою	те саме, на запис
<code>InstrFetchProhibited</code>	перехід на недійсну адресу	зіпсований покажчик на функцію
<code>IllegalInstruction</code>	виконання не-коду	переповнення стека, пошкоджена пам'ять

Причина	Що заборонено	Що шукати
LoadStoreAlignment	невирівняний доступ	32-бітове читання з непарної адреси
IntegerDivideByZero	ділення на нуль	дільник із датчика без перевірки

Практично: EXCVADDR близька до нуля ($0x0-0x40$) — це розіменування NULL зі зсувом поля. EXCVADDR виглядає як осмислена адреса, але доступ заборонено — покажчик на вже звільнену пам'ять.

УВАГА

Найчастіше джерело обох — malloc, результат якого не перевірили. На ESP32 пам'ять закінчується значно раніше, ніж на комп'ютері, і malloc повертає NULL не в теорії, а в четверо третій, коли під'єднався третій клієнт.

Друге за частотою — покажчик на локальний масив, повернений із функції. Компілятор попередить, якщо ввімкнені попередження; вони варті того, щоб бути ввімкненими.

Розшифровка backtrace

Самі по собі адреси нечитливі. Їх треба перекласти в назви функцій і номери рядків, і для цього потрібен .elf **того самого збирання**.

Автоматично. idf.py monitor, запущений з каталогу проекту, робить це на льоту: під дампом одразу з'являються імена функцій і рядки.

Вручну. Коли лог знято з чужого пристрою або збережений у файл:

```
xtensa-esp32-elf-addr2line -pfiaC -e build/app.elf \
0x400d2f1a 0x400d3042 0x400d5alc
```

Для **S3** — xtensa-esp32s3-elf-addr2line, для **C3** та інших RISC-V — riscv32-esp-elf-addr2line.

НЕЗВОРОТНЕ

C3 **C6** **H2** На RISC-V рядка backtrace: у дампі немає взагалі. Ядро друкує лише регістри; ланцюжок викликів **будує сам монітор** зі знімка стека. Це різні механізми: на Xtensa монітор розшифровує адреси, які надрукував чип, на RISC-V він відновлює послідовність, якої чип не друкував.

Наслідок практичний і неприємний: лог з C3, знятий через screen або picocom, взагалі не містить ланцюжка викликів — і його нізвідки взяти потім. Розшифровувати немає чого.

Тому на RISC-V лог знімають `idf.py monitor`, не чимось іншим. Якщо це неможливо (пристрій у полі, чужий термінал), ланцюжок можна попросити в самого чипа: `CONFIG_ESP_SYSTEM_USE_EH_FRAME` у `menuconfig`, меню `Backtracing method`. Ціна названа в документації прямо: розмір образу росте на 20–100 %, і в серійні збирання це вмикати не радять.

Прапорці: `-f` імена функцій, `-i` розкриття `inline`-викликів (важливо: без нього частина кадрів зникає), `-c` демангл C++ імен, `-r` читабельний формат, `-a` показувати адресу.

Читати знизу вгору. Нижній кадр — де почалося, верхній — де впало. Часто корисніший саме нижній: він каже, з якої задачі це прийшло.

НЕЗВОРОТНЕ

Без `.elf` того самого збирання `backtrace` нерозшифровний. Перезібраний «такий самий» проєкт не підходить: адреси зсуваються від будь-якої зміни — версії тулчейну, порядку файлів, прапорців.

`.elf` зберігається разом із кожним образом, що поїхав у поле (розділ 21). Це кілька мегабайтів, які вирішують, чи буде збій з поля розібраний за десять хвилин чи не буде розібраний узагалі.

Watchdog: два різні

Плутанина тут коштує часу, бо повідомлення схожі, а причини різні.

Task Watchdog Timer (TWDT).

```
E (5234) task_wdt: Task watchdog got triggered. The following tasks/users
did not reset the watchdog in time:
E (5234) task_wdt: - IDLE0 (CPU 0)
E (5234) task_wdt: Tasks currently running:
E (5234) task_wdt: CPU 0: my_task
```

Означає: задача не віддавала керування занадто довго. За замовчуванням стежать за IDLE-задачами — якщо IDLE не отримала часу, значить, хтось зайняв ядро повністю.

УВАГА

Два переліки в цьому дампі — різні, і плутати їх дорого.

Після першого рядка йдуть ті, хто **не встиг погодувати** watchdog. У типовому випадку це IDLE0 — тобто потерпілий, а не винуватець.

Tasks currently running: — те, що виконувалося в момент спрацювання. Ось тут і стоїть винуватець: `my_task` зайняв ядро й не дав IDLE запуснитися.

Шукати треба ім'я з **другого** переліку. Перший лише каже, на якому ядрі стало погано.

Типова причина — цикл без затримки:

```
while (1) {
    do_work();
    // немає vTaskDelay — IDLE ніколи не запуститься
}
```

Лікування — віддати керування: `vTaskDelay(pdMS_TO_TICKS(10))`. Якщо робота справді довга і переривати її не можна, задачу можна явно підписати на watchdog і годувати його: `esp_task_wdt_add, esp_task_wdt_reset`.

TWDT не вбиває систему миттєво — він друкує попередження і називає винуватця. Це діагностика, і вона дуже корисна: рядок `Tasks currently running` прямо каже, хто винен.

Interrupt Watchdog.

```
Guru Meditation Error: Core 0 panic'ed (Interrupt wdt timeout on CPU0)
```

Значно серйозніший. Означає, що переривання були заблоковані занадто довго: або обробник переривання виконується довго, або хтось надовго зайшов у критичну секцію.

Причини за частотою: важкий код в ISR (розділ 31), `portENTER_CRITICAL` навколо довгої операції, виклик у ISR чогось, що не можна викликати з ISR (`printf`, `malloc`, блокувальні функції).

Правило: **ISR має бути коротким**. Прочитати значення, покласти в чергу, вийти. Все інше — у задачі.

Причини скидання

Після паніки чип скидається, і наступний старт показує `rst:0xc (SW_CPU_RESET)`. Повна таблиця кодів — картка K6.

Три, що трапляються постійно:

`rst:0xf` — **brownout**, просіло живлення. Це не програмна помилка. Скільки б ви не читали код, причина в джерелі, кабелі або конденсаторах (розділ 06). З'являється найчастіше в момент увімкнення радіо, бо саме там піковий струм.

`rst:0xc` — програмне скидання ядра, типово після паніки. Шукати `Guru Meditation` вище в лозі.

`rst:0x7`, `rst:0x8`, `rst:0x9` — watchdog. Див. вище.

Прочитати причину з коду:

```
#include "esp_system.h"
ESP_LOGI(TAG, "причина скидання: %d", esp_reset_reason());
```

Корисно логувати це першим рядком у `app_main`: пристрій сам розповідає про свою попередню смерть.

Boot loop: коли паніка не перша

Паніка → скидання → та сама паніка → скидання. У порту тече нескінченний потік однакових дамтів.

Дивитися треба **найперший** дамт після подачі живлення. Порядок: відкрити монітор, **потім** подати живлення. Не навпаки.

Причина — в першому дамті. Решта — наслідки того, що причина не зникла.

Якщо перший дамт спіймати не вдається (пристрій у корпусі, живлення вмикається не вами), тут допомагає `coredump`.

Coredump: знімок стану у флеші

Вмикається в `menuconfig`: Core dump → призначення Flash. Потребує розділу типу `coredump` у таблиці розділів (розділ 18).

При паніці ESP-IDF записує у флеш стан **усіх задач**, а не лише тієї, що впала: їхні стеки, регістри, стан планувальника. Це переживає перезавантаження і зчитується потім:

```
idf.py coredump-info
idf.py coredump-debug
```

Другий відкриває GDB на збереженому стані: можна ходити по кадрах, дивитися змінні, перемикаючись між задачами — як при живому налагодженні, але постфактум.

Для рідкісних збоїв у полі — «падає раз на три дні» — це найкращий доступний інструмент. Лог такого не зловить: до моменту падіння цікаве вже прокрутилося.

ЖИВЛЕННЯ

Запис `coredump` — це запис у флеш у момент, коли система вже нестабільна. Якщо причина паніки — просадка живлення, `coredump` може не записатися або записатися частково. Тому `brownout` діагностується за `rst:`, а не за `coredump`.

Порядок дій при збої

1. `rst:` у першому рядку. Це живлення, `watchdog` чи паніка? Три різні шляхи.

2. **Причина паніки і EXCVADDR.** Найчастіше відповідь уже тут: LoadProhibited з EXCVADDR близько нуля — розіменування NULL.
3. **Backtrace через .elf.** Читати знизу вгору.
4. **Відтворити.** Збій, який не відтворюється, не полагоджений — він просто зараз не видно.
5. **Не відтворюється** — coredump і логуювання переходів станів (розділ 25).

Що з цього треба запам'ятати

EXCVADDR — найшвидша підказка: близько нуля означає NULL зі зсувом поля.

.elf того самого збирання — єдине, що робить backtrace читним. Зберігається разом із кожним образом, що поїхав.

Task WDT називає винуватця сам, у рядку Tasks currently running.

Interrupt WDT — це майже завжди довгий ISR або довга критична секція.

rst:0xf — живлення. Читати код при ньому марно.

Найперший дамп після подачі живлення, а не сотий.

27. JTAG і покрокове налагодження

Лог показує те, що ви здогадалися залогувати. Відлагоджувач показує все: поточне значення будь-якої змінної, вміст пам'яті, стек кожної задачі, стан регістрів периферії — і дозволяє зупинити програму в потрібній точці й піти далі по одній інструкції.

Це не заміна логу, а інший інструмент. Лог відповідає на «що відбувалося протягом години»; відлагоджувач — на «що зараз усередині цієї змінної».

Головна новина цього розділу: **на S3 і C3 для цього не потрібно жодного додаткового заліза.**

Вбудований USB-JTAG: S3, C3 і новіші

S3 **C3** мають на кристалі міст USB-Serial-JTAG. Той самий USB-кабель, яким ви прошиваєте плату, дає одночасно консоль і повноцінний JTAG. Ніякого зовнішнього адаптера, ніяких додаткових дротів.

Практично це означає, що бар'єр входу зник. Раніше покрокове налагодження було чимось, до чого треба готуватися: купити адаптер, розібратися з розводкою, підпаяти. Тепер це одна команда.

```
idf.py openocd
```

в одному терміналі, і в іншому:

```
idf.py gdb
```

Або все разом, в одній команді:

```
idf.py openocd gdb
```

УВАГА

C3 **S3** USB-JTAG займає конкретні піни: GPI018 і GPI019 на C3, GPI019 і GPI020 на S3. Якщо в проєкті ці піни переналаштовані під щось інше — USB-JTAG перестає працювати, і виглядає це як «відлагоджувач раптом не під'єднується».

Це та ситуація, коли варто спершу подивитися на розводку пінів, а не на налаштування OpenOCD.

VS Code: налагодження у вікні

Офіційне розширення ESP-IDF для VS Code налаштовує це саме. Ставиться точка зупинки клацанням на полі біля номера рядка, натискається запуск — далі звичайний інтерфейс відлагоджувача: змінні, стек викликів, покроковий прохід.

Що бачите під час зупинки:

- **Variables** — локальні змінні поточного кадру і глобальні;
- **Call Stack** — з якої функції прийшли, і **всі задачі FreeRTOS** окремими гілками, з можливістю перемкнутися в кожную;
- **Watch** — вирази, що обчислюються на кожній зупинці;
- **Peripherals** — регістри периферії з розшифровкою бітових полів.

Останнє варте окремої згадки: побачити, що саме лежить у регістрі конфігурації I²C, — часто швидший шлях до відповіді, ніж читати документацію про те, що там мало б лежати.

classic ESP32 classic: потрібен зовнішній адаптер

У classic вбудованого USB-JTAG немає. Потрібен апаратний адаптер: ESP-Prog від Espressif або будь-яка плата на FT2232H.

Підключення — чотири сигнали плюс земля, і всі чотири займають піни, які інакше були б вільні:

Сигнал	classic пін
TMS	GPIO14
TDI	GPIO12
TCK	GPIO13
TDO	GPIO15

УВАГА

classic GPIO12 і GPIO15 — це водночас strapping-піни (розділ 16). Адаптер, під'єднаний до GPIO12, може утримувати його високим під час скидання. Тоді флеш отримує 1.8 В замість 3.3 В і на тривольтовому модулі не запускається — плата мовчить, без жодного повідомлення. Це класична пастка першого підключення JTAG до classic.

Симптом: підключили відлагоджувач — плата перестала стартувати. Причини не в JTAG, а в тому, що GPIO12 задає напругу живлення флешу.

Чи воно того варте на classic. Чесна відповідь: у більшості випадків — ні. Чотири зайняті піни, зовнішня коробка, дроти, конфлікт зі strapping. Лог і coredump (розділ 26) покривають переважну більшість задач дешевше.

Коли справді варте: складна помилка з пошкодженням пам'яті, яку не видно логом; збій у чужому коді без вихідних текстів на рівні асемблера; робота з периферією, де треба дивитися регістри наживо.

Якщо є вибір платформи для проєкту, де очікується складне налагодження — це аргумент на користь S3.

Обмеження, про які варто знати заздалегідь

Зупинка ламає реальний час. Поки ви стоїте на точці зупинки, світ не чекає: спрацює watchdog, розірветься з'єднання Wi-Fi, переповниться буфер UART, партнер по шині вирішить, що ви мертві.

Практично: покрокове налагодження добре працює для логіки і погано — для всього, що має таймінги. Помилку в обміні по I²C зручніше дивитися логічним аналізатором (розділ 28), ніж покроково.

Watchdog доведеться вимкнути. Інакше кожна зупинка довше секунди закінчується перезавантаженням. У menuconfig на час налагодження вимикаються Task WDT та Interrupt WDT — і, обов'язково, вмикаються назад перед тим, як прошивка поїде кудись.

Оптимізація заважає. Зі стандартним -Og частина змінних «оптимізована геть» і не показується, а покроковий прохід стрибає по рядках не по порядку. Для важкого налагодження варто зібрати з -O0: menuconfig → Compiler options → Optimization Level → **Debug without optimization (-O0)**. Пункт Debug (-Og) у цьому ж переліку — це і є те, що стоїть за замовчуванням, тобто саме те, від чого ви тут тікаєте. Ціна -O0 — більша і повільніша прошивка.

Перше під'єднання майже ніколи не працює з першого разу. Драйвери USB у Windows, права на пристрій у Linux (правила udev), конфлікт із відкритим монітором порту. Це нормальний етап, а не ознака того, що щось зламано.

Що робити, коли відлагоджувач не під'єднується

За порядком, від найчастішого:

1. **Закрити монітор порту.** Він тримає пристрій.
2. **Права (Linux).** Потрібне правило udev для USB-JTAG; в ESP-IDF воно є в комплекті і ставиться один раз.
3. **Драйвер (Windows).** Для USB-JTAG треба призначити правильний драйвер утилітою на кшталт Zadig; для FT232 — драйвер FTDI.
4. **Піни JTAG зайняті проєктом** — див. попередження вище.

5. **classic** **GPIO12 заважає старту** — від'єднати адаптер, перевірити, що плата стартує без нього.
6. **eFuse вимкнув JTAG**. `espefuse summary` (лише читання). Якщо попередній власник спалив `JTAG_DISABLE` або ввимкнув Secure Boot — JTAG недоступний назавжди (розділ 20, картка K11).

Коли відлагоджувач не потрібен зовсім

Варто сказати прямо, бо це економить дні: більшість помилок у практиці вбудованої розробки не потребують JTAG.

Помилка живлення діагностується мультиметром (розділ 06). Помилка на шині — логічним аналізатором (розділ 28). Паніка — за `backtrace` і `coredump` (розділ 26). Логіка станів — логом переходів (розділ 25).

JTAG потрібен там, де всі чотири нічого не дали і треба подивитися всередину пам'яті. Це реальна, але не щоденна ситуація.

Що з цього треба запам'ятати

На S3 і C3 повноцінний JTAG уже є в чипі й доступний тим самим кабелем — `idf.py openocd gdb`.

На classic потрібен зовнішній адаптер, він займає чотири піни, два з яких — `strapping`, і GPIO12 уміє не дати платі стартувати.

Зупинка на точці ламає реальний час: `watchdog` доведеться вимкнути, а таймінгові помилки шукати іншим інструментом.

Більшість збоїв розбирається без JTAG узагалі.

28. Апаратна діагностика сигналів

Настає момент, коли код виглядає правильним, лог нічого не показує, а датчик мовчить. Питання «чи взагалі щось іде по проводу» неможливо розв’язати з боку прошивки: вона бачить лише те, що їй повернув контролер шини.

Тут потрібен інструмент, який дивиться на дріт.

Що чим міряють

Прилад	На що відповідає	Ціна питання
Мультиметр	яка напруга, чи є контакт, чи є КЗ	дешево
Логічний аналізатор	які байти йдуть по шині	недорого
Осцилограф	яка форма сигналу	дорого

Порядок у таблиці — це і порядок застосування. Мультиметром починають завжди: більшість несправностей знаходять саме на цьому кроці.

Мультиметр: що і де міряти

Живлення під навантаженням. Не на холостому ходу, а коли пристрій працює і радіо ввімкнене. Звз відносно GND: має бути близько 3.3 В. Просідання нижче 3.0 В — причина ваших «незрозумілих глюків» (розділ 06).

Спільна земля. Найчастіша помилка початківця. Між GND ESP32 і GND зовнішнього пристрою має бути нуль опору. Немає спільної землі — немає обміну, хоч сигнальні дрони й на місці.

Логічний рівень у спокої. На лініях I²C у спокої має бути 3.3 В (це роблять підтягувальні резистори). Нуль означає, що підтягування немає або лінія кимось притиснута — і шина не працюватиме ніколи.

Прозвонка. Обрив у дроті, тріщина в пайці, замикання між сусідніми пінами. Робиться при **вимкненому** живленні.

УВАГА

Мультиметр показує **середнє** значення. Сигнал, що перемикається сотні тисяч разів на секунду, покаже щось посередині між 0 і 3.3 В — і це нормально, а не несправність. Мультиметром перевіряють статику: живлення, контакт, рівень у спокої. Динаміку — іншими приладами.

Логічний аналізатор: головний інструмент розділу

Восьмиканальний USB-аналізатор коштує як пара плат і закриває майже всі питання по цифрових шинах. Це найкраще співвідношення користі до ціни в усьому переліку інструментів (розділ 10).

Він підключається паралельно до вже працюючої шини — нічого розривати не треба. Один дріт на кожен сигнал плюс обов'язково земля.

PulseView (з пакета sigrok) — вільна програма, що працює з більшістю дешевих аналізаторів. Вона не просто малює прямокутники: у ній є декодери протоколів, які перетворюють рівні на осмислені байти.

I²C наживо

Підключити два канали: SDA і SCL. Увімкнути декодер I²C. Запустити захоплення, викликати обмін із коду.

Що видно і що це означає:

Що на екрані	Висновок
нічого не рухається	контролер не почав обмін — справа в коді або пинах
SCL йде, SDA мовчить	ведений не відповідає: не та адреса або він мертвий
START, адреса, NACK	пристрою з такою адресою на шині немає
START, адреса, ACK, дані	шина працює, справа в інтерпретації даних
лінії не піднімаються до 3.3 В	немає підтягування або воно завелике
SCL розтягнутий веденим	clock stretching — ведений не встигає

Найцінніший рядок — четвертий: він знімає з шини всі підозри й переводить пошук у код. Це те, чого ніяк не отримати з боку прошивки.

UART і SPI

Так само. Для UART — один канал на лінію, декодер сам покаже байти, і одразу видно **невідповідність швидкості**: замість осмислених символів іде сміття зі стабільною структурою.

Для SPI потрібні чотири канали (SCK, MOSI, MISO, CS). Тут аналізатор особливо корисний, бо помилка в режимі SPI (розділ 36) з боку коду виглядає як «пристрій повертає нулі», а на екрані одразу видно, що дані зчитуються не по тому фронту.

УВАГА

Дешеві аналізатори мають межу частоти дискретизації — типово 24 МГц. Правило: частота вибірки має бути щонайменше вчетверо вищою за частоту сигналу. Для I²C (100–400 кГц) і UART запасу вистачає з головою. Для SPI на 40 МГц — ні, там уже потрібен осцилограф або дорожчий аналізатор.

Осцилограф: коли потрібен саме він

Аналізатор відповідає «які байти». Осцилограф відповідає «якої форми сигнал» — і це різні питання.

Осцилограф потрібен, коли:

- **сигнал аналоговий** — вихід датчика, пульсації живлення, шум на лінії ADC;
- **підозра на форму, а не на дані** — завалені фронти через довгий дріт або велику ємність шини, дзвін, викиди;
- **треба побачити просадку живлення** в момент увімкнення радіо: це мілісекунди, мультиметр їх не покаже;
- **шина швидша**, ніж бере аналізатор.

НЕЗВОРОТНЕ

Земляний щуп осцилографа з мережевим живленням з'єднаний із заземленням розетки. Причепити його до точки, що не є землею схеми, — означає закоротити цю точку на землю через прилад. У приладах, що живляться від мережі, це закінчується вигорілою доріжкою або гіршим.

Правило: земляний щуп іде тільки на землю схеми, і перед цим варто переконатися, що земля схеми не «висить» під потенціалом.

Практичний порядок пошуку

Коли периферія не працює, порядок такий — від дешевого до дорогого:

1. **Живлення пристрою.** Мультиметр: чи є на ньому напруга і яка.
2. **Спільна земля.** Прозвонка між землями.
3. **Рівень у спокої.** Чи піднято лінії до 3.3 В.
4. **Аналізатор: чи є взагалі активність** на шині.
5. **Аналізатор: чи відповідає ведений** — АСК чи НАСК.
6. **Дані.** Якщо байти йдуть — проблема в інтерпретації, а не в шині.

Дев'ять із десяти випадків закінчуються на кроках 1–3. Це варто пам'ятати, перш ніж перерахувати код удруге.

Дешевий заміник, коли приладів немає

Не завжди аналізатор під рукою. Кілька прийомів, що працюють на самому ESP32:

Сканер І²С (розділ 35) — програма, що перебирає всі адреси й друкує ті, що відповіли. Замінює аналізатор для питання «чи є пристрій на шині й за якою адресою».

Другий ESP32 як приймач. Підключити вільну плату до тієї самої лінії UART і слухати. Дає точний вміст обміну, хоч і без таймінгів.

Світлодіод на пін. Примітивно, але надійно: побачити, чи доходить керування до потрібної гілки коду, коли лог недоступний.

Осцилограф зі звукової карти. Для дуже повільних аналогових сигналів (одинарні кілогерців) — робочий варіант. Обов'язково через розв'язку: вхід звукової карти не терпить постійної складової і легко псується.

Що з цього треба запам'ятати

Мультиметр — статика: живлення під навантаженням, спільна земля, рівень у спокої. Дев'ять із десяти несправностей тут.

Логічний аналізатор із PulseView перетворює «датчик не працює» на конкретний факт: чи почався обмін, чи відповів ведений, які байти пішли. Найкраще співвідношення користі до ціни.

Осцилограф потрібен для аналогових сигналів, форми фронтів і просядок живлення тривалістю в мілісекунди.

Земляний щуп осцилографа — тільки на землю схеми.

29. Симптомний troubleshooting

Розділ побудований від симптому, а не від теми. Ви бачите конкретну поведінку — знаходьте її нижче і йдіть за посиланням.

Стисла версія на одну сторінку — картка [K8](#). Повний покажчик із посиланнями на всі розділи — додаток [B](#).

Три правила, що передують усьому

Спершу живлення, потім код. Більшість «плаваючих багів» — це просадка напруги. Зміряти дешевше, ніж перерхитати код (розділ [06](#)).

Одна зміна за раз. Змінили дві речі — не знаєте, яка допомогла, і не знаєте, чи друга не додала нової проблеми.

Мінімальний тест окремо. Незнайомий модуль перевіряється на голій платі з трьома дротами, а не всередині проєкту з двадцятьма (розділ [44](#)).

Не бачить плату, немає порту

Порт не з'являється в системі взагалі.

Кабель. Найчастіша причина, з відривом. Дешеві кабелі з комплектів до павер-банків не мають жил даних. Перевірка: під'єднати ним телефон.

USB-міст і драйвер. Подивитися маркування меншого чипа біля роз'єма і поставити правильний драйвер. Для CH9102 потрібен **не той самий** драйвер, що для CH340 — це окрема пастка (розділ [09](#), картка [K3](#)).

Роз'єм. micro-USB на дешевих платах відривається з площадками. Поворухити при увімкненому живленні: блимає індикатор — тріщина в пайці (розділ [55](#)).

Права (Linux). Порт є, доступу немає: група dialout, потім **перезайти в систему**.

Порт є, але «Failed to connect»

A fatal error occurred: Failed to connect to ESP32: No serial data received.

Чип не в download mode.

Дії за порядком: увійти вручну кнопками BOOT + EN (картка [K4](#)) → закрити монітор, який тримає порт → знизити --baud 115200 → додати --no-stub → зняти обв'язку зі strapping-пінів → під'єднати без хаба.

Прошилося, але плата мовчить

Найковарніша ситуація: помилки не було, результату немає.

Адреса бутлоадера не та для цього чипа. Причина номер один. 0x1000 для classic і S2, 0x0 для S3, C3, C6 і H2, 0x2000 для P4, C5 і H4 — повна таблиця в розділі 16. Команда, скопійована з чужої інструкції, кладе бутлоадер у порожнє місце, і esptool не має підстав скаржитися.

Залито не той файл. Для ОТА потрібен app.bin, а не merge-bin-образ; для повної прошивки — навпаки.

Застосунок не той під чип. Прошивка, зібрана під classic, на S3 не працює: інша архітектура.

Перевірка: verify-flash, потім boot-лог. Лог порожній — не стартує бутлоадер; лог e, invalid header — не знайдено застосунок.

Boot loop

Пристрій нескінченно перезавантажується.

Дивитися **найперший** дамп після подачі живлення: відкрити монітор, потім подати живлення. У першому — причина, в решті — наслідки.

Далі за rst: (картка K6): 0xf — живлення (розділ 06); 0xc — паніка (розділ 26); 0x7/0x8/0x9 — watchdog (розділ 32).

Повний алгоритм — розділ 20.

rst:0xf — brownout

Просіло живлення. **Це не помилка в коді**, скільки б ви його не читали.

Джерела за частотою: слабкий USB-порт або хаб без власного живлення; довгий тонкий кабель; відсутність конденсатора по живленню; джерело, що не тягне піковий струм радіо; розряджений акумулятор.

Що робити: зміряти напругу **під навантаженням**, з увімкненим Wi-Fi (розділ 28); поставити 470 мкФ по живленню поруч із модулем; коротший і товщий кабель; окреме джерело від 1 А (розділ 06).

Wi-Fi мовчить

Бачить мережу, не під'єднується. Звірити SSID і пароль посимвольно. ESP32 **не бачить мереж 5 ГГц** — це найчастіша причина «мережі немає в списку». Канали 12–13 доступні не за всіх налаштувань регіону.

Під'єднується і відвалюється. Живлення (див. вище) або слабкий сигнал. Подивитися RSSI: гірше за -80 дБм — робота на межі.

Працює поруч із роутером, не працює на місці. Антена, метал корпусу, розміщення (розділ 39).

I²C-пристрій не знаходиться

Немає підтягувальних резисторів. Шина I²C без них не працює: 4.7 кОм від SDA до 3.3 В і від SCL до 3.3 В. Частина модулів має свої на платі, частина — ні; два модулі зі своїми на одній шині дають замалий сумарний опір.

Немає спільної землі. Прозвонити.

Не та адреса. Запустити сканер (розділ 35): він друкує всі адреси, що відповіли. Багато модулів мають дві можливі адреси, обрані перемичкою.

Занадто довгі дроти. Ємність шини росте, фронти завалюються. Понад 30–50 см на макетці — вже ризик; лікується зниженням частоти до 100 кГц.

Порядок перевірки — розділ 28: рівень у спокої, потім активність на шині, потім АСК.

SPI не відповідає

Не той режим (CPOL/CPHA). Пристрій повертає нулі або сміття (розділ 36). Видно на аналізаторі одразу.

CS не той або не керується. Кожен ведений має власний CS.

Переплутані MOSI і MISO. Класика, особливо коли модуль підписаний SD1/SD0 замість звичних імен.

ADC читає дурницю

classic **S2** **S3** Використано ADC2 при увімкненому Wi-Fi. ADC2 тут недоступний, поки працює радіо. Перенести на ADC1 (GPIO 32–39) — картка K9.

Не враховано нелінійність. ADC у ESP32 нелінійний; без калібрування похибка помітна (розділ 33).

Шум по живленню. Аналогові вимірювання вимагають тихого живлення (розділ 53).

GPIO поводить дивно

Це strapping-пін. Стан при старті визначає режим завантаження. Перенести функцію на інший пін (розділ 16, картка K9).

classic **Це GPIO 6–11.** Зайняті флешем на модулі. Не використовуються ніколи, попри те, що виведені на гребінку.

Це вхід-тільки пін. **classic** GPIO 34–39 не мають виходу і не мають підтягування. Налаштуванням у коді це не змінюється.

УВАГА

Перш ніж гадати, що з піном не так, **спитайте в чипа**. ESP-IDF має функцію, яка друкує реальну конфігурацію піна — не ту, яку ви задавали, а ту, що зараз у регістрах:

```
gpio_dump_io_configuration(stdout, (1ULL << 4) | (1ULL << 18));
```

У виводі видно підтягування, напрямок, open-drain, силу драйвера і — що найцінніше — **хто саме тримає пін**: FuncSel показує, чи пін працює як GPIO, чи його забрала периферія через IOMUX, а рядки GPIO Matrix SigIn/SigOut ID називають сигнал, на який його перепризначили.

Це закриває цілий клас питань «я ж налаштував його на вихід» за секунду: частина драйверів переналаштовує піни під себе, і побачити це інакше можна лише читанням регістрів руками. Піни, зайняті системою, у виводі позначені як ****RESERVED****.

Працює від USB, не працює від зовнішнього живлення

Немає спільної землі між джерелом і рештою схеми. Прозвонити.

Просадка під навантаженням. Джерело тримає напругу на холостому ході і провалюється, коли вмикається радіо. Міряти під навантаженням.

Подано не на той пін. 5V/VIN — вхід стабілізатора; 3V3 — вихід. Подати 5 В на 3V3 означає подати їх напрямку в чип.

Сенсор гріється, значення пливуть

Подано 5 В на 3.3-вольтовий вхід. Модуль може працювати «майже нормально» і при цьому повільно вмирати. Звірити з datasheet (розділ 44).

Немає конвертера рівнів там, де він потрібен (розділ 47).

Самонагрів. Датчик температури, що стоїть біля стабілізатора або всередині щільного корпусу, міряє власне тепло, а не середовище.

Працює на столі, не працює в корпусі

Перегрів. Замкнений корпус без вентиляції; чип тротлить або перезавантажується (розділ 54).

Антенна біля металу. Металевий корпус — екран. Виносити антену (розділ 39).

Механіка. Вібрація і натяг кабелю виривають Dupont-роз'єми і ламають пайку. Постійний монтаж і strain relief (розділи 51, 54).

Конденсат. Перепад температур усередині герметичного боксу (розділ 54).

Симптом «іноді працює»

Найважчий клас. Майже ніколи не програмний.

Порядок підозр, суворо в цьому порядку: живлення → пайка → земля → логічні рівні → таймінги → і лише потім код.

Що допомагає: логувати переходи станів із часовими мітками (розділ 25); логувати `esp_reset_reason()` першим рядком у `app_main`; увімкнути `coredump` (розділ 26); поставити пристрій під навантаження і чекати з увімкненим логом у файл.

УВАГА

Збій, який зник після того, як ви щось змінили, але не відтворювався до цього стабільно, — **не полагоджений**. Він просто зараз не видно. Перш ніж закривати питання, треба вміти відтворити збій за бажанням. Інакше він повернеться в полі.

Коли нічого не допомагає

Три кроки, що розривають глухий кут:

Мінімальний приклад. Залити `hello_world` або `blink`. Працює — справа в прошивці, не в залізі. Дві хвилини, а відсікає половину простору пошуку (розділ 20).

Інша плата. Той самий код на іншому екземплярі. Працює — справа в конкретній платі.

Поділ навпіл. Прибрати половину периферії. Запрацювало — проблема в прибраному. Повторити.

Що з цього треба запам'ятати

Живлення перевіряється першим і **під навантаженням**.

Найперший дамп логу після подачі живлення, а не сотий.

`rst:0xf` означає, що читати код немає сенсу.

Мінімальний приклад за дві хвилини відокремлює прошивку від заліза.

Збій, який не відтворюється за бажанням, не полагоджений.

Частина VIII. Програмування

Модель пам'яті, FreeRTOS, надійність і периферія з коду.

30. Застосунок і модель пам'яті

Програма для ESP32 виглядає як звичайна програма на C, але виконується в середовищі, де кілька звичок із великих систем перестають працювати. Найголовніша з них — ставлення до пам'яті.

Що відбувається до входу в main

До того, як виконається перший рядок вашого коду, система вже зробила чимало (розділ 16): ROM запустив бутлоадер, той знайшов і перевірів застосунок, ініціалізувалися тактування, купа, периферія за замовчуванням, і **запустився планувальник FreeRTOS**.

`app_main` викликається як звичайна задача FreeRTOS. Не як точка входу програми — як одна із задач.

Практичні наслідки:

`app_main` може завершитися. І це нормально: система продовжує працювати, інші задачі виконуються далі. Задача `app_main` просто зникає, звільняючи свій стек.

У `app_main` уже можна створювати задачі, черги й таймери — планувальник працює.

`app_main` має обмежений стек, заданий у `menuconfig` (типово близько 3.5 КБ). Великий локальний масив тут падає так само, як будь-де.

В Arduino core те саме, тільки прикрите: `setup` і `loop` виконуються в задачі `loopTask` (розділ 12).

```
void app_main(void) {
    ESP_LOGI(TAG, "причина скидання: %d", esp_reset_reason());
    xTaskCreate(sensor_task, "sensor", 4096, NULL, 5, NULL);
    // app_main може тут завершитися — sensor_task працюватиме далі
}
```

Логувати причину скидання першим рядком — дешева звичка, що окупається: пристрій сам розповідає про свою попередню смерть (розділ 26).

Три види пам'яті і де вони закінчуються

Статична. Глобальні змінні й `static`. Виділяється при збиранні, розмір відомий наперед, ніколи не фрагментується. Найпередбачуваніша.

Стек. Локальні змінні. **У кожній задачі свій**, фіксованого розміру, заданого при створенні. Тут ховається найпідступніша помилка платформи.

Купа. `malloc` і `new`. Спільна на всі задачі. Тут ховається друга найпідступніша.

Стек: чому програма падає без причини

Розмір стека задається при створенні задачі — числом, яке легко недооцінити:

```
xTaskCreate(sensor_task, "sensor", 4096, NULL, 5, NULL);
//                ^^^^^ байтів стека
```

НЕЗВОРОТНЕ

Переповнення стека на мікроконтролері не дає ні винятку, ні повідомлення. Задача просто пише за межі свого стека — в чужу пам'ять. Наслідок може проявитися **пізніше й в іншому місці**: зіпсовані дані, `IllegalInstruction`, паніка в коді, який не має до цього стосунку.

Це найважчий для пошуку клас помилок на платформі саме тому, що причина й симптом розділені в часі.

Що з'їдає стек несподівано багато:

- **локальні масиви й буфери** — `char buf[2048]` це половина типового стека;
- **printf і форматування** — сотні байтів на виклик;
- **робота з плаваючою комою** на чипах без FPU;
- **глибока вкладеність викликів**, особливо в бібліотеках;
- **TLS** — рукоистискання потребує кількох кілобайтів.

Як з цим жити:

Великі буфери — не на стек. `static` або з купи.

Перевіряти залишок:

```
UBaseType_t zapas = uxTaskGetStackHighWaterMark(NULL);
ESP_LOGI(TAG, "найменший запас стека: %u байт", zapas);
```

Функція повертає мінімум, що лишився за весь час життя задачі. Значення менше кількох сотень байтів — привід збільшити.

Знати, що перевірка переповнення вже ввімкнена. У `menuconfig` це `Component config` → `FreeRTOS` → `Kernel` → `configCHECK_FOR_STACK_OVERFLOW`, і за замовчуванням там стоїть `Check using canary bytes (Method 2)`: у кінець стека кладуться контрольні байти, які перевіряються при кожному перемиканні контексту.

Отже, повідомлення ви отримаєте — але **не в момент помилки**, а при наступному перемиканні, і воно назве задачу, а не рядок. Це вже багато, і водночас саме тому причина й симптом розділені в часі. Вимикати цю перевірку (`No checking`) заради швидкості на етапі, коли прошивка ще пишеться, — погана угода.

Купа: чому malloc повертає NULL

Купа на ESP32 не однорідна (розділ 03), і malloc може повернути NULL при формально вільних десятках кілобайтів.

Фрагментація. Виділили й звільнили сотню разів різного розміру — купа стала «дірявою». Суцільного блоку потрібного розміру немає, хоча сума вільного велика.

УВАГА

Звідси головне правило пам'яті на цій платформі: **не виділяти й не звільняти в циклі**.

Буфери, потрібні постійно, виділяються один раз при старті й живуть до кінця. Це не оптимізація, а умова довгої безперервної роботи: пристрій, що фрагментує купу, падає не одразу, а через три дні — і зв'язок між причиною й наслідком знайти майже неможливо.

Не та область. Буфер для DMA має бути доступним контролеру DMA; звичайний malloc може віддати непридатну пам'ять. Для таких випадків є явний запит властивостей:

```
uint8_t *dma_buf = heap_caps_malloc(1024, MALLOC_CAP_DMA);
uint8_t *big     = heap_caps_malloc(65536, MALLOC_CAP_SPIRAM);
```

PSRAM є, але поводитьься не так, як гадають. Тут два поширені непорозуміння, і вони протилежні.

Перше: підтримку PSRAM треба **ввімкнути** — CONFIG_SPIRAM типово вимкнена, і плата з розпаяною мікросхемою без цього просто її не бачить.

Друге, і саме воно частіше: коли PSRAM увімкнено, malloc **уже** вміє віддавати з неї. Автоматичне винесення не треба вмикати — воно за замовчуванням, і має поріг:

Розмір виділення	Куди піде
менше 16 КБ	внутрішня SRAM
16 КБ і більше	PSRAM

Поріг — CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL, типово 16384, від 0 до

131072. Перевага м'яка: якщо в бажаній області місця немає, береться

інша, тож malloc не почне раптово повертати NULL при вільній пам'яті поруч.

Практичний наслідок: буфер на 64 КБ опиниться в PSRAM **без** жодного MALLOC_CAP_SPIRAM, а разом із тим і без гарантій щодо DMA та зі швидкістю

зовнішньої шини. `heap_caps_malloc` потрібен не щоб потрапити в PSRAM, а щоб керувати цим свідомо — і в обидва боки:

```
uint8_t *big = heap_caps_malloc(65536, MALLOC_CAP_SPIRAM); // напевно в PSRAM
uint8_t *fast = heap_caps_malloc(65536, MALLOC_CAP_INTERNAL); // напевно в SRAM
```

Дивитися, що відбувається:

```
ESP_LOGI(TAG, "вільно: %u, найбільший блок: %u",
          heap_caps_get_free_size(MALLOC_CAP_8BIT),
          heap_caps_get_largest_free_block(MALLOC_CAP_8BIT));
```

Друге число важливіше за перше. Коли вільно 40 КБ, а найбільший блок — 2 КБ, це і є фрагментація, і `malloc(8192)` поверне `NULL`.

І завжди перевіряти результат. `malloc`, результат якого не перевірили, — найчастіше джерело `LoadProhibited` на практиці (розділ 26):

```
uint8_t *buf = malloc(size);
if (buf == NULL) {
    ESP_LOGE(TAG, "не вистачило %u байт", size);
    return ESP_ERR_NO_MEM;
}
```

IRAM, DRAM і чому це вилазить

Код виконується з флешу через кеш. Під час операції з флешем (запис у NVS, стирання сектора) кеш вимикається, і виконання коду з флешу неможливе (розділ 03).

Функція, яка може спрацювати в цей момент — насамперед обробник переривання, — має лежати в IRAM:

```
static void IRAM_ATTR gpio_isr_handler(void *arg) { ... }
```

Дані, до яких така функція звертається, теж мають бути доступні: `DRAM_ATTR`.

IRAM небагато, і кожна така функція займає її назавжди. Помилка `section .iram0.text will not fit` при збиранні означає, що `IRAM_ATTR` роздали надто щедро.

volatile, атомарність, регістри

volatile каже компілятору не оптимізувати доступ до змінної. Потрібен для змінних, які змінює обробник переривання, і для доступу до регістрів.

volatile не робить операцію атомарною. Це найпоширеніше непорозуміння:

```
volatile int lychlynyk = 0;
// в ISR:
lychlynyk++; // читання + додавання + запис — три дії, не одна
```

На двоядерному чипі така операція може перерватися посередині. Для лічильників, що змінюються з переривання й читаються із задачі, правильні інструменти — атомарні операції або примітиви FreeRTOS (розділ 31).

32-бітне читання й запис вирівняного слова атомарні апаратно. Тому проста передача одного значення (прапорець, ціле число) із ISR у задачу через `volatile` працює. Складніші структури — ні.

Скільки в мене лишилося

```
idf.py size           # загальна картка: флеш і RAM
idf.py size-components # хто саме займає
```

`size-components` — найкорисніша команда, коли прошивка перестала вміщатися: одразу видно, що Wi-Fi і TLS з'їли більшість, а власний код — десяту частину (розділ 18).

Під час роботи:

```
ESP_LOGI(TAG, "вільно RAM: %u", esp_get_free_heap_size());
ESP_LOGI(TAG, "мінімум за весь час: %u", esp_get_minimum_free_heap_size());
```

Друге значення варто логувати періодично: воно виявляє повільний витік пам'яті, який інакше помітний лише через тижні.

Що з цього треба запам'ятати

`app_main` — звичайна задача FreeRTOS; вона може завершитися, і в неї обмежений стек.

Переповнення стека не дає повідомлення й проявляється пізніше та в іншому місці. Великі буфери — не на стек.

Не виділяти й не звільняти пам'ять у циклі: фрагментація вбиває пристрій через дні, а не одразу.

Найбільший вільний блок важливіший за суму вільного.

Результат `malloc` перевіряти завжди.

`volatile` не робить операцію атомарною.

31. FreeRTOS і конкурентність

FreeRTOS уже працює, коли викликається ваш перший рядок (розділ 30). Це не бібліотека, яку треба підключати, — це середовище, у якому виконується все.

Розділ для програміста, який знає потоки з великих систем: тут вони називаються задачами, поведінка схожа, але ціна помилки інша — немає захисту пам'яті, і зіпсована синхронізація призводить не до винятку, а до пошкоджених даних і паніки в іншому місці.

Задачі

Задача — функція, що виконується паралельно з іншими, з власним стеком.

```
static void sensor_task(void *arg) {
    while (1) {
        float t = read_sensor();
        ESP_LOGI(TAG, "температура %.1f", t);
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

xTaskCreate(sensor_task, // функція
            "sensor",     // ім'я для логів і діагностики
            4096,         // стек у байтах
            NULL,         // параметр
            5,           // пріоритет
            NULL);        // сюди можна отримати дескриптор
```

Два правила, які варто засвоїти одразу.

Задача не завершується. Це нескінченний цикл. Функція, що дійшла до кінця й вийшла, викликає паніку — задачу треба або зациклити, або явно видалити через `vTaskDelete(NULL)`.

Задача мусить віддавати керування. Цикл без затримки з'їдає ядро повністю, і рано чи пізно спрацює watchdog (розділ 26):

```
while (1) {
    do_work();
    // без vTaskDelay – Task WDT спрацює
}
```

`vTaskDelay` — не пауза процесора, а перехід задачі в сон: інші задачі працюють, споживання падає.

УВАГА

Різниця, яку варто зрозуміти один раз: `vTaskDelay(pdMS_TO_TICKS(10))` віддає керування, `esp_rom_delay_us(10000)` — ні. Друге крутить процесор вхолосту.

Короткі затримки на мікросекунди (таймінги протоколів) роблять другим способом; усе, що вимірюється мілісекундами, — першим.

Пріоритети

Число від 0 (найнижчий) до `configMAX_PRIORITIES - 1`. В ESP-IDF `configMAX_PRIORITIES` дорівнює 25, тобто найвищий доступний пріоритет — 24. Планувальник завжди виконує **найпріоритетнішу готову** задачу.

Практичні орієнтири:

Пріоритет	Для чого
1–4	фонові роботи, логуювання, необов'язкове
5	типовий рівень прикладних задач
10+	реакція на події, обробка з черг ISR
18–24	системні задачі; сюди краще не лізти

НЕЗВОРОТНЕ

Задача з високим пріоритетом, яка не блокується, **не дасть виконатися нічому нижчому**. Це не помилка планувальника, а його правило.

Симптом: додали «важливу» задачу з пріоритетом 20 і циклом без затримки — пристрій перестав відповідати повністю, включно з Wi-Fi. Причина не в радіо.

Високий пріоритет означає «швидко відреагувати й заснути», а не «важлива задача».

Два ядра і прив'язка

`classic` `S3` Ядро 0 переважно зайняте радіостеком, `app_main` за замовчуванням іде на ядро 1 (розділ 03).

Прив'язати задачу до ядра явно:

```
xTaskCreatePinnedToCore(motor_task, "motor", 4096, NULL, 10, NULL, 1);
// ^ ядро
```

Коли це має сенс: щось із жорсткими таймінгами — на ядро 1, подалі від радіо. Щось важке й тривале — теж на ядро 1, щоб не заважати мережі.

УВАГА

Двоядерність робить помилки синхронізації **реальними, а не теоретичними**. На одному ядрі дві задачі не виконуються одночасно фізично, і багато

некоректного коду роками працює випадково. На двох ядрах воно ламається одразу.

Класика: дві задачі змінюють ту саму структуру без захисту. На одноядерному чипі перемикання відбувається в передбачуваних точках; на двоядерному — справді одночасно.

Черги: основний спосіб обміну

Черга — потокобезпечний буфер фіксованого розміру. Це **основний** інструмент передачі даних між задачами, і в більшості випадків правильна відповідь на питання «як передати дані».

```
QueueHandle_t cherga = xQueueCreate(10, sizeof(vymiryuvannya_t));

// відправник
vymiryuvannya_t v = { .temperatura = 23.5, .chas = esp_timer_get_time() };
xQueueSend(cherga, &v, pdMS_TO_TICKS(100));

// отримувач
vymiryuvannya_t v;
if (xQueueReceive(cherga, &v, portMAX_DELAY) == pdTRUE) {
    obrobyty(&v);
}
```

Чому чергу варто віддавати перевагу спільним змінним:

- **дані копіюються** — немає гонок за доступ;
- **отримувач блокується**, доки даних немає: не треба опитувати в циклі;
- **таймаут** дозволяє не зависати назавжди;
- **переповнення видиме**: `xQueueSend` повертає помилку, і це діагностика того, що споживач не встигає.

`portMAX_DELAY` означає «чекати скільки треба». Задача при цьому не споживає процесорного часу зовсім.

М'ютекси

Коли ресурс справді спільний — шина I²C, структура конфігурації, — доступ захищається м'ютексом:

```
SemaphoreHandle_t mutex = xSemaphoreCreateMutex();

if (xSemaphoreTake(mutex, pdMS_TO_TICKS(1000)) == pdTRUE) {
    // тільки одна задача тут одночасно
    xSemaphoreGive(mutex);
}
```

Завжди з таймаутом, а не `portMAX_DELAY`: взаємне блокування з таймаутом стає видимою помилкою, а без нього — тихим зависанням.

М'ютекс не можна брати з обробника переривання. Для ISR є окремі функції з суфіксом `FromISR`.

Семафори й групи подій

Двійковий семафор — сигнал «сталось». Типове застосування: ISR сигналізує, задача чекає.

Лічильний семафор — облік обмеженого ресурсу.

Група подій — набір прапорців, на комбінацію яких можна чекати. Зручно для «дочекатися, поки є і Wi-Fi, і час із SNTP»:

```
EventGroupHandle_t podiyi = xEventGroupCreate();
#define WIFI_OK    BIT0
#define TIME_OK    BIT1

xEventGroupWaitBits(podiyi, WIFI_OK | TIME_OK,
                    pdFALSE, pdTRUE, portMAX_DELAY);
```

Переривання: головне правило

НЕЗВОРОТНЕ

ISR має бути коротким. Прочитати, покласти в чергу, вийти. Усе інше — у задачі.

Що не можна робити в ISR: викликати `printf` і `ESP_LOGx`, виділяти пам'ять, брати м'ютекси, викликати блокувальні функції, чекати.

Довгий ISR блокує переривання й закінчується `Interrupt wdt timeout` — панікою, яку важко пов'язати з причиною (розділ 26).

УВАГА

Один виняток із заборони на лог, і він потрібен саме тоді, коли все інше не працює. ESP-IDF має набір `ESP_DRAM_LOGE`, `ESP_DRAM_LOGW` і далі за рівнями. Заголовок `esp_log.h` каже про них дослівно: «на відміну від звичайних макросів логування, цей можна вживати, коли переривання вимкнені або всередині ISR».

Ціна названа там само: рядки лягають у DRAM, а її мало, — тож використовувати «лише коли без цього ніяк». Тег теж мусить бути в DRAM: `ESP_DRAM_LOGE(DRAM_STR("mij_teg"), "...")`.

Це інструмент для відлагодження, а не для роботи. Але коли ISR поводить себе незрозуміло, а покласти в чергу нема чого, він єдиний.

Правильний зразок:

```
static void IRAM_ATTR gpio_isr(void *arg) {
    uint32_t pin = (uint32_t)arg;
    BaseType_t vyshche = pdFALSE;
    xQueueSendFromISR(cherga_podiy, &pin, &vyshche);
    if (vyshche) portYIELD_FROM_ISR();
}
```

IRAM_ATTR обов'язковий, якщо переривання може спрацювати під час операції з флешем (розділ 03). Функції FromISR — єдині дозволені. portYIELD_FROM_ISR перемикає на розбуджену задачу одразу після виходу.

Антидребезг кнопки

Найчастіший приклад, де правило про короткий ISR перевіряється на практиці. Механічний контакт при натисканні дає десятки перемикачів за мілісекунди, і спокуса поставити затримку прямо в обробнику дуже велика.

Робити цього не можна: затримка в ISR — це і є той довгий ISR, від якого приходить Interrupt wdt timeout. Правильний спосіб — порівняння часу без жодного очікування; готовий зразок і решта роботи з GPIO — розділ 33.

Програмні таймери

Коли треба виконати щось періодично, не заводячи окрему задачу:

```
TimerHandle_t t = xTimerCreate("perevirk", pdMS_TO_TICKS(5000),
                                pdTRUE, NULL, callback);
xTimerStart(t, 0);
```

Усі програмні таймери виконуються в **одній** службовій задачі. Довгий обробник таймера затримує всі інші таймери — тому в них теж має бути коротка робота.

Типові помилки

Задача без затримки. Task WDT.

Високий пріоритет + цикл без блокування. Система стоїть.

Замалий стек. Падіння в іншому місці, без зв'язку з причиною (розділ 30).

Спільна змінна без захисту. На двох ядрах ламається одразу.

Довгий ISR. Interrupt WDT.

Черга без перевірки результату. Дані тихо губляться, коли споживач не встигає.

М'ютекс із portMAX_DELAY. Взаємне блокування перетворюється на тихе зависання.

Що з цього треба запам'ятати

Задача не завершується і мусить віддавати керування.

Високий пріоритет означає «швидко відреагувати й заснути».

Черга — типова правильна відповідь на «як передати дані між задачами».

ISR: прочитати, покласти в чергу, вийти. Ніякого логування й пам'яті.

Двоядерність перетворює теоретичні помилки синхронізації на реальні.

32. Надійність і обробка помилок

Різниця між прототипом і виробом — не у функціях, а в поведінці при відмові. Прототип працює, коли все добре. Виріб працює, коли щось пішло не так, і продовжує працювати після того, як воно повторилося сто разів.

Цей розділ про те, як цього досягти на платформі, де немає ні операційної системи, яка перезапустить процес, ні людини, яка натисне кнопку.

esp_err_t: домовленість, на якій усе тримається

Майже кожна функція ESP-IDF повертає `esp_err_t` — код помилки, де `ESP_OK` дорівнює нулю.

```
esp_err_t err = i2c_master_probe(bus, ADDR, 100);
if (err != ESP_OK) {
    ESP_LOGE(TAG, "датчик 0x%02x не відповідає: %s",
              ADDR, esp_err_to_name(err));
    return err;
}
```

`esp_err_to_name` перетворює число на читабельне ім'я. Без нього в лозі буде `0x105`, і його доведеться шукати по заголовках (розділ 25).

ESP_ERROR_CHECK: інструмент, який часто застосовують не там

```
ESP_ERROR_CHECK(nvs_flash_init());
```

Макрос перевіряє результат і, якщо це не `ESP_OK`, **викликає паніку й перезавантажує чип**, надруквавши файл і рядок.

НЕЗВОРОТНЕ

`ESP_ERROR_CHECK` — це `assert`, а не обробка помилок. У прикладах він скрізь, бо приклад має бути коротким; у виробі, що поїхав у поле, кожен такий виклик — потенційна нескінченна перезавантаження.

Класичний випадок: `ESP_ERROR_CHECK` навколо під'єднання до Wi-Fi. Точка доступу вимкнена — пристрій перезавантажується. Знову не бачить — знову перезавантажується. Пристрій, який мав пережити відсутність мережі, стає цеглинкою на весь час її відсутності.

Правило. `ESP_ERROR_CHECK` доречний там, де помилка означає, що далі працювати неможливо і немає сенсу: ініціалізація NVS, створення базових об'єктів при старті. Усе, що може не спрацювати **під час роботи** — мережа, датчик, файл, — обробляється явно.

Три стратегії реакції

Для кожної помилки, яку ви обробляєте, є три чесні варіанти. Обрати треба свідомо, а не за замовчуванням.

Повторити. Мережа, шина, тимчасово недоступний ресурс. Обов'язково з обмеженням кількості спроб і зростаючою паузою — інакше пристрій захлинається спробами:

```
esp_err_t sprobuvaty(int max_sprob) {
    int pauza = 100;
    for (int i = 0; i < max_sprob; i++) {
        esp_err_t err = operatsiya();
        if (err == ESP_OK) return ESP_OK;
        ESP_LOGW(TAG, "спроба %d/%d: %s", i + 1, max_sprob,
                  esp_err_to_name(err));
        vTaskDelay(pdMS_TO_TICKS(pauza));
        pauza = pauza * 2 > 5000 ? 5000 : pauza * 2;
    }
    return ESP_FAIL;
}
```

Деградувати. Працювати далі з меншими можливостями. Немає мережі — складати вимірювання в буфер і віддати потім. Немає одного з трьох датчиків — працювати на двох і повідомити про це.

Це найцінніша й найрідше реалізована стратегія. Саме вона відрізняє виріб від прототипу.

Перейти у безпечний стан. Коли продовжувати небезпечно: вимкнути нагрівач, зупинити двигун, знеструмити виконавчий механізм.

НЕЗВОРОТНЕ

Безпечний стан має бути станом за замовчуванням апаратно, а не лише програмно.

Реле, що вмикається високим рівнем на GPIO, при зависанні або скиданні чипа лишається у визначеному стані — вимкненому — тільки якщо на лінії є підтягувальний резистор до землі (розділ 05). Без нього під час скидання пін «висить», і виконавчий механізм може ввімкнутися сам.

Питання, яке варто ставити до кожного виходу: **що станеться, якщо чип зникне просто зараз?** Якщо відповідь незручна — це виправляється резистором, а не кодом.

Watchdog як інструмент

Watchdog — таймер, який перезавантажує чип, якщо його не «годувати» (розділ 26). Природна реакція розробника — вимкнути його, бо він заважає. Правильна — використати.

Task WDT стежить за тим, що задачі віддають керування. Задачу можна підписати на нього явно:

```
esp_task_wdt_add(NULL);           // підписати поточну задачу
while (1) {
    esp_task_wdt_reset();         // «я живий»
    robota();
    vTaskDelay(pdMS_TO_TICKS(100));
}
```

Тепер зависання цієї задачі — навіть якщо решта системи жива — призведе до перезавантаження. Для пристрою без обслуговування це саме те, що потрібно: краще перезавантаження, ніж тихо мертвий вузол.

УВАГА

Логіка «годування» має відображати **корисну роботу**, а не факт виконання циклу.

Задача, що годує watchdog у циклі, який давно нічого не робить, — це watchdog, вимкнений складним способом. Годувати треба після того, як цикл справді щось зробив: прочитав датчик, обробив пакет, оновив стан.

Watchdog і OTA-відкат — одна система: непідтверджена прошивка повертається на попередню лише тоді, коли пристрій перезавантажиться, а зависла прошивка сама цього не зробить (розділ 19).

Поведінка при зникненні живлення

Живлення зникає без попередження, у довільний момент. Три наслідки, про які варто подумати наперед.

Незавершений запис у флеш. NVS до цього стійкий за задумом; файлова система — залежить від типу (розділ 18). Останній записаний файл може бути втрачений — це проєктне обмеження, а не помилка.

Виконавчі механізми. Див. вище про безпечний стан.

Втрачений стан у RAM. Усе, що не в NVS чи RTC RAM, зникає. Пристрій має вміти стартувати з нуля й відновитися сам, без ручного втручання.

Практичний прийом для лічильників: не записувати кожну зміну у флеш (це зношує його), а зберігати з запасом наперед. Записали «дійшли до 1000», працюєте до 1000 у RAM, потім записали «до 2000». Після аварійного скидання лічильник відновиться з невеликим перескоком уперед, але ніколи не піде назад — а це саме та властивість, яка потрібна.

Реальний час без RTOS-фанатизму

«240 МГц» звучить так, ніби встигнеться все. Що насправді визначає час реакції:

Пріоритети. Задача з нижчим пріоритетом не виконується, поки готова вища (розділ 31).

Радіостек. Wi-Fi і Bluetooth забирають час, іноді помітними шматками. На одноплатних чипах це відчутно сильніше.

Операції з флешем. Запис у NVS зупиняє виконання коду з флешу (розділ 03).

Переривання мають пріоритет над усіма задачами.

Практичні орієнтири, які варто перевіряти вимірюванням, а не приймати на віру:

Потрібна точність	Чим робити
секунди	звичайна задача з <code>vTaskDelay</code>
десятки мілісекунд	задача з підвищеним пріоритетом
одиночки мілісекунд	апаратний таймер + переривання
мікросекунди	апаратна периферія: RMT, MCPWM, PCNT
гарантований жорсткий час	окремий мікроконтролер (розділ 57)

УВАГА

Останній рядок — не поразка, а нормальне інженерне рішення. ESP32 чудовий у зв'язку й посередній у гарантованих таймінгах. Задача, де потрібне й те, й те, природно ділиться між двома чипами: STM32 тримає таймінги, ESP32 стоїть збоку і забезпечує зв'язок (розділ 01).

Там, де таймінги критичні, найкраще рішення — **не робити їх у коді взагалі**. RMT формує імпульси WS2812 апаратно; MCPWM тримає мертвий час; PCNT рахує імпульси. Периферія не залежить від того, чим зайнятий процесор (розділ 33).

Мінімальний набір для виробу

Те, що варто мати в кожній прошивці, яка їде в поле:

- логування причини скидання першим рядком `app_main` (розділ 26);
- Task WDT, підписаний на головні робочі задачі;
- обмежені повтори зі зростаючою паузою для всього мережевого;
- деградація замість зупинки, де це можливо;
- безпечний стан виконавчих механізмів, забезпечений апаратно;
- періодичне логування мінімуму вільної пам'яті — виявляє витік (розділ 30);
- `coredump` у флеші (розділ 26);
- OTA з відкатом, якщо пристрій має оновлюватися (розділ 19).

Це перелік, який відрізняє пристрій, що працює місяцями без уваги, від пристрою, до якого треба їздити.

Що з цього треба запам'ятати

`ESP_ERROR_CHECK` — це `assert`. У виробі він доречний лише там, де помилка робить подальшу роботу безглуздою.

Три чесні стратегії: повторити з обмеженням, деградувати, перейти в безпечний стан. Деградація — найцінніша й найрідше реалізована.

Безпечний стан забезпечується резистором, а не кодом. Питання до кожного виходу: що станеться, якщо чип зникне зараз?

Watchdog — інструмент, а не перешкода; годувати його треба після корисної роботи, а не в порожньому циклі.

Критичні таймінги віддають апаратній периферії або окремому чипу.

33. Периферія з коду: GPIO, таймери, PWM, ADC

Практична робота з блоками, описаними оглядово в розділі [04](#). Головна ідея розділу: **більшість того, що виглядає як задача для коду, робиться апаратно** — і саме тому працює надійно незалежно від завантаження системи.

GPIO

```
gpio_config_t cfg = {
    .pin_bit_mask = (1ULL << GPIO_NUM_2) | (1ULL << GPIO_NUM_4),
    .mode = GPIO_MODE_OUTPUT,
    .pull_up_en = GPIO_PULLUP_DISABLE,
    .pull_down_en = GPIO_PULLDOWN_DISABLE,
    .intr_type = GPIO_INTR_DISABLE,
};
gpio_config(&cfg);
gpio_set_level(GPIO_NUM_2, 1);
```

`pin_bit_mask` — бітова маска, тому кілька пінів налаштовуються однією дією. `1ULL` обов'язково: на пінах вище 31 звичайний `1` переповниться.

Вхід із підтягуванням:

```
gpio_config_t in = {
    .pin_bit_mask = (1ULL << GPIO_NUM_5),
    .mode = GPIO_MODE_INPUT,
    .pull_up_en = GPIO_PULLUP_ENABLE,
    .intr_type = GPIO_INTR_NEGEDGE,
};
```

Перед вибором піна — картка [K9](#): strapping, тільки-вхідні, зайняті флешем (розділ [07](#)).

Переривання і антидребезг

Обробник має бути коротким: покласти в чергу й вийти (розділ [31](#)).

```
static void IRAM_ATTR isr(void *arg) {
    uint32_t pin = (uint32_t)arg;
    xQueueSendFromISR(cherga, &pin, NULL);
}

gpio_install_isr_service(0);
gpio_isr_handler_add(GPIO_NUM_5, isr, (void *)GPIO_NUM_5);
```

Механічна кнопка дає десятки перемикачів за мілісекунди. Антидребезг робиться **не затримкою в ISR** (це прямий шлях до Interrupt wdt timeout), а порівнянням часу:

```
static int64_t ostannya;

static void IRAM_ATTR isr(void *arg) {
    int64_t teper = esp_timer_get_time();    // мікросекунди
    if (teper - ostannya < 50000) return;    // 50 мс — ігнорувати
    ostannya = teper;
    xQueueSendFromISR(cherga, &teper, NULL);
}
```

Таймери

`esp_timer` — програмні таймери з мікросекундною роздільною здатністю. Для більшості періодичних задач цього досить:

```
static void callback(void *arg) { /* коротко */ }

esp_timer_create_args_t args = { .callback = callback, .name = "опыт" };
esp_timer_handle_t t;
esp_timer_create(&args, &t);
esp_timer_start_periodic(t, 1000000); // раз на секунду
```

Обробники всіх `esp_timer` виконуються в одній задачі — довгий обробник затримує решту.

`esp_timer_get_time()` повертає мікросекунди від старту у 64-бітному числі. Це основний спосіб міряти час: переповнення не станеться за час життя пристрою.

Апаратні таймери потрібні там, де важлива точність незалежно від завантаження системи, — переривання формується апаратно.

LEDC: PWM для світлодіодів і серво

```
ledc_timer_config_t tcfg = {
    .speed_mode = LEDC_LOW_SPEED_MODE,
    .duty_resolution = LEDC_TIMER_13_BIT,
    .timer_num = LEDC_TIMER_0,
    .freq_hz = 5000,
};
ledc_timer_config(&tcfg);

ledc_channel_config_t ccfg = {
    .gpio_num = GPIO_NUM_2,
    .speed_mode = LEDC_LOW_SPEED_MODE,
    .channel = LEDC_CHANNEL_0,
    .timer_sel = LEDC_TIMER_0,
    .duty = 4096,
};
ledc_channel_config(&ccfg);
```

Частота і розрядність пов'язані: що вища частота, то менше розрядів доступно. 5 кГц із 13 розрядами — робоче поєднання для світлодіодів.

УВАГА

Яскравість світлодіода **не лінійна** щодо коефіцієнта заповнення. Око сприймає яскравість логарифмічно: перехід від 10 % до 20 % видно, від 80 % до 90 % — майже ні.

Плавне згасання, зроблене лінійно, виглядає як різкий стрибок наприкінці. Лікується таблицею або квадратичною залежністю.

Серво керується імпульсами 50 Гц: приблизно 1 мс — один край, 2 мс — інший, 1.5 мс — середина. Період при 50 Гц — 20 мс, тому значення *duty* рахується як $2^{\text{розрядність}} \times \text{тривалість} / 20 \text{ мс}$. Для 16-розрядної роздільності це приблизно 3277 (1 мс), 4915 (1.5 мс) і 6554 (2 мс).

ЖИВЛЕННЯ

Серво живиться **окремо**, не від піна 3V3 плати. Навіть невелике серво в момент рушання бере сотні міліампер, і бортовий стабілізатор цього не витримує: пристрій перезавантажується по brownout саме тоді, коли механізм починає рух (розділ 06).

Спільна земля обов'язкова (розділ 48).

MCPWM: коли LEDC замало

classic **S3** MCPWM зроблений для силової електроніки й уміє те, чого LEDC не вміє:

- **мертвий час** між верхнім і нижнім плечем моста — без нього обидва ключі на мить відкриті одночасно, і це наскрізний струм;
- **апаратне аварійне вимкнення** за зовнішнім сигналом — швидше за будь-яку реакцію коду;
- **синхронізація каналів**.

Для керування двигуном через мостовий драйвер це не зручність, а захист силового каскаду (розділ 48).

RMT: точні імпульси апаратно

RMT задумувався для інфрачервоних пультів, а виявився універсальним формувачем імпульсних послідовностей із наносекундною точністю.

Головне застосування — **адресні світлодіоди WS2812**. Їхній протокол кодує біти тривалістю імпульсів у сотні наносекунд. Робити це в коді означає заборонити переривання на весь час передачі — і все одно отримати збої, коли втрутяться радіо.

RMT формує послідовність апаратно: процесор віддає дані й вільний.

```
led_strip_handle_t strip;
led_strip_config_t scfg = { .strip_gpio_num = 18, .max_leds = 30 };
led_strip_rmt_config_t rcfg = { .resolution_hz = 10 * 1000 * 1000 };
led_strip_new_rmt_device(&scfg, &rcfg, &strip);

led_strip_set_pixel(strip, 0, 255, 0, 0);
led_strip_refresh(strip);
```

Номер піна в прикладі довільний — беріть свій за картою K9. На платах розробки з бортовим адресним світлодіодом він у кожної свій: **C3** на C3-DevKitM це GPIO8, **S3** на S3-DevKitC — GPIO48. **classic** На classic GPIO8 брати не можна взагалі: там флеш (розділ 07).

RMT уміє й приймати — вимірювати тривалість вхідних імпульсів. Це правильний спосіб читати ІЧ-пульти й датчики з імпульсним виходом.

PCNT: рахувати без переривань

Апаратний лічильник імпульсів. Енкодер, витратомір, лічильник обертів — усе це не потребує переривання на кожен імпульс: процесор читає накопичене, коли йому зручно.

Перевага критична на високих частотах: десять тисяч імпульсів на секунду через переривання з'їдять помітну частину ядра; PCNT не з'їсть нічого.

PCNT уміє й апаратний фільтр коротких сплесків — антидребезг без коду.

ADC

```
adc_oneshot_unit_handle_t adc;
adc_oneshot_unit_init_cfg_t ucfg = { .unit_id = ADC_UNIT_1 };
adc_oneshot_new_unit(&ucfg, &adc);

adc_oneshot_chan_cfg_t ccfg = {
    .bitwidth = ADC_BITWIDTH_DEFAULT,
    .atten = ADC_ATTEN_DB_12,
};
adc_oneshot_config_channel(adc, ADC_CHANNEL_6, &ccfg);

int raw;
adc_oneshot_read(adc, ADC_CHANNEL_6, &raw);
```

УВАГА

classic **S2** **S3** ADC2 не працює при увімкненому Wi-Fi (розділ 07).

Симптом: датчик читається правильно, доки не викликано `esp_wifi_start`. Вимірювання переносяться на ADC1.

Затухання (attenuation) задає діапазон вхідної напруги. Без нього ADC міряє лише невелику частину діапазону; з максимальним затуханням доступний

майже весь до 3.3 В. Пам’ятайте: вхід не толерантний до перевищення — понад живлення подавати не можна.

Точність. ADC ESP32 нелінійний, і сирі відліки не переводяться в вольти простим множенням. Штатний шлях — калібрування:

```
adc_cali_handle_t cali;
adc_cali_curve_fitting_config_t cfg = {
    .unit_id = ADC_UNIT_1,
    .atten = ADC_ATTEN_DB_12,
    .bitwidth = ADC_BITWIDTH_DEFAULT,
};
adc_cali_create_scheme_curve_fitting(&cfg, &cali);

int mv;
adc_cali_raw_to_voltage(cali, raw, &mv);
```

Калібрувальні коефіцієнти зашиті в eFuse кожного чипа на заводі — ця функція їх використовує.

Боротьба з шумом, у порядку дієвості:

- 1. **Усереднення** 16–64 відліків. Найдешевше і найдієвіше.
- 2. **Конденсатор** 100 нФ від входу до землі.
- 3. **Тихе живлення** аналогової частини (розділ 53).
- 4. **Коротші проводи** до джерела сигналу.
- 5. **Зовнішній ADC** по SPI — коли потрібна справжня точність.

ЖИВЛЕННЯ

Вимірювання напруги акумулятора через дільник — класична задача, у якій дільник **сам розряджає акумулятор**. Два резистори по 100 кОм — це 200 кОм між плюсом і землею: при 3.6 В вони постійно беруть 18 мкА. Для пристрою, що споживає уві сні одиниці мікроампер, дільник стає головним джерелом розряду — більшим за сам чип.

Лікується вмиканням дільника транзистором лише на час вимірювання (розділ 53).

DAC

Справжній аналоговий вихід, 8 розрядів, два канали. Піни **різні** за сімействами:

	Канал 1	Канал 2
classic	GPI025	GPI026
S2	GPI017	GPI018

Більше ніде в лінійці DAC немає (розділ 04). Де потрібен на інших чипах — зовнішній ЦАП по I²C або згладжений PWM.

УВАГА

GPIO25 і GPIO26 на S2 **не існують узагалі**: у нього немає пінів 22–25. Тобто помилитися тут не «майже те саме», а неробочий код і ESP_ERR_INVALID_ARG при налаштуванні.

Що з цього треба запам'ятати

Антидребезг — порівнянням часу, ніколи не затримкою в ISR.

Яскравість світлодіода нелінійна щодо коефіцієнта заповнення.

Серво живиться окремо; спільна земля обов'язкова.

WS2812 керуються через RMT апаратно — у коді це робити не варто.

PCNT рахує імпульси без переривань і має апаратний антидребезг.

classic **S2** **S3** ADC2 не працює при Wi-Fi; ADC потребує калібрування й усереднення.

Дільник для вимірювання акумулятора розряджає акумулятор.

Частина ІХ. Дротові інтерфейси

UART, I²C, SPI, 1-Wire, CAN.

34. UART, RS-485, Modbus

UART — найстаріший і найнадійніший спосіб з'єднати два пристрої. Два дрони, жодного протоколу поверх, працює завжди. Саме тому це основний канал між ESP32 і «дорослим» контролером (розділ 57), і саме тому консоль зроблена на ньому.

Апаратні порти

classic ESP32 classic має три контролери UART, S3 — три, C3 — два (розділ 04).

UART0 зайнятий консоллю. Через нього йде boot-лог і прошивка (розділ 16). Використати його під щось інше можна, але тоді ви втрачаєте і лог, і зручну прошивку — тобто саме те, чим діагностують проблеми.

Правило: чіпати UART0 лише тоді, коли пінів справді не лишилося.

Решта портів вільні, і завдяки матриці GPIO їх можна вивести майже на будь-які піни (розділ 04):

```
uart_config_t cfg = {
    .baud_rate = 115200,
    .data_bits = UART_DATA_8_BITS,
    .parity = UART_PARITY_DISABLE,
    .stop_bits = UART_STOP_BITS_1,
    .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
    .source_clk = UART_SCLK_DEFAULT,
};
uart_driver_install(UART_NUM_1, 2048, 0, 0, NULL, 0);
uart_param_config(UART_NUM_1, &cfg);
uart_set_pin(UART_NUM_1, GPIO_NUM_17, GPIO_NUM_16,
    UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);
```

Читання з таймаутом:

```
uint8_t buf[128];
int n = uart_read_bytes(UART_NUM_1, buf, sizeof(buf), pdMS_TO_TICKS(100));
if (n > 0) obrobty(buf, n);
```

УВАГА

Розмір буфера драйвера має значення. Дані приходять, поки ваша задача зайнята чимось іншим; якщо буфер переповниться, вони губляться мовчки.

Для потоку на 115200 бод це близько 11 КБ на секунду. Буфер на 256 байтів означає, що задача мусить читати частіше ніж кожні 22 мілісекунди — а вона цього не гарантує. 2 КБ і більше — розумний старт.

Що йде не так

Не та швидкість. Найчастіше. У моніторі — сміття зі стабільною структурою (розділ 25).

Переплутані TX і RX. Друга за частотою. З'єднання завжди перехресне: TX одного до RX другого.

Немає спільної землі. Обмін не працює або йде з випадковими помилками (розділ 05).

Різні логічні рівні. Пристрій на 5 В подає 5 В на вхід ESP32 (розділ 47).

Порт зайнятий консоллю — випадково взяли UART0.

RS-485: той самий UART на сотні метрів

Звичайний UART працює на десятки сантиметрів. RS-485 передає той самий сигнал диференціально — двома дротами з протилежними рівнями — і це дає **сотні метрів** та стійкість до завад. Стандарт промислової автоматики.

Потрібен трансивер: MAX485, SP3485 або аналог. MAX485 живиться від 5 В; для 3.3 В беруть версію на 3.3 В, інакше потрібне узгодження рівнів.

Напівдуплекс. Лінія одна, тому в кожен момент говорить хтось один. Напрямоком керує окремий пін DE/RE:

```
gpio_set_level(PIN_DE, 1);           // передавання
uart_write_bytes(UART_NUM_1, data, len);
uart_wait_tx_done(UART_NUM_1, portMAX_DELAY); // ДОЧЕКАТИСЯ
gpio_set_level(PIN_DE, 0);           // приймання
```

НЕЗВОРОТНЕ

uart_wait_tx_done **обов'язковий**. uart_write_bytes лише кладе дані в буфер і повертається одразу — фізична передача ще триває. Перемкнути напрямок відразу після нього означає обрізати власну посилку посередині.

Це найчастіша помилка при роботі з RS-485, і виглядає вона як «інколи губляться відповіді».

Термінатори. На обох кінцях лінії — резистор 120 Ом між лініями А і В. На коротких лініях працює і без них; на довгих без термінаторів з'являються відбиття й помилки, які виглядають як випадкові збої.

Багато модулів мають термінатор на платі, іноді припаяний намертво. Три пристрої з термінаторами на одній лінії — це втричі менший опір, ніж треба, і сигнал просідає.

Полярність. Лінії А і В маркуються по-різному в різних виробників. Якщо обмін не йде — поміняти місцями. Це безпечно і розв'язує половину випадків.

Modbus RTU, оглядово

Протокол поверх RS-485, стандарт промислового обладнання: лічильники, частотні перетворювачі, датчики, контролери.

Модель проста: один ведучий (master), кілька ведених (slave), у кожного свій адресний номер від 1 до 247. Ведучий питає — ведений відповідає. Сам ведений не говорить ніколи.

Дані — набір регістрів, доступ до яких дає жменя функцій: прочитати регістри зберігання, прочитати вхідні, записати один, записати кілька.

ESP-IDF має штатний компонент `esp-modbus` для обох ролей: ESP32 може бути й ведучим (опитувати обладнання), і веденим (виглядати як стандартний пристрій для чужої системи).

Друга роль часто найцінніша: ваш виріб стає доступним для будь-якої SCADA чи ПЛК без жодної домовленості про формат.

УВАГА

При налагодженні Modbus логічний аналізатор економить години (розділ 28). Видно одразу: чи вийшла посилка, чи відповів ведений, чи збігається контрольна сума, чи не обрізано кінець через ранній перехід на приймання.

ESP32 як допоміжний контролер

Найпоширеніша роль UART у цій книзі: основний контролер робить свою роботу, ESP32 стоїть збоку і забезпечує зв'язок (розділ 01).

Кілька практичних порад щодо власного протоколу на такій лінії:

Текстовий формат простіший, ніж здається. Рядки виду `GET TEMP\n` і `TEMP 23.5\n` налагоджуються звичайним монітором порту, без жодного інструменту. Для обміну раз на секунду цього досить із запасом.

Двійковий формат — коли треба часто або багато. Тоді обов'язково: байт початку, довжина, контрольна сума. Без них перше ж втрачене байтове зміщення розсинхронізує обмін назавжди.

Таймаут і відновлення. Приймач має вміти викинути недобудований пакет і почати спочатку. Протокол без цього працює до першої завади.

Обидві сторони мають переживати відсутність іншої. Ні ESP32, ні основний контролер не повинні зависати, чекаючи відповіді, якої не буде (розділ 32).

Що з цього треба запам'ятати

UART0 — це консоль; чіпати в останню чергу.

TX до RX перехресно, спільна земля обов'язкова.

Буфер драйвера робити з запасом: переповнення губить дані мовчки.

RS-485: `uart_wait_tx_done` перед перемиканням напрямку, інакше посилка обрізається.

Термінатори 120 Ом на кінцях лінії, і лише на кінцях.

Свій протокол — краще текстовий; двійковий — тільки з довжиною і контрольною сумою.

35. I²C

I²C — дві лінії, багато пристроїв, невисока швидкість. Найзручніша шина для датчиків і найчастіше джерело питання «чому нічого не знаходиться».

Скільки саме пристроїв: адресація 7-бітна, отже 128 адрес, з яких зарезервовано 16 (0x00–0x07 і 0x78–0x7F) — лишається 112. На практиці до цієї межі не доходять ніколи: раніше впирається або збіг адрес у двох однакових датчиків, або ємність шини (обидва питання нижче).

Майже всі проблеми з I²C — не програмні. Вони у фізиці шини, і саме тому їх не видно з коду.

Як воно влаштоване

Дві лінії: SDA (дані) і SCL (тактування). Обидві **open-drain** (розділ 05): пристрої вміють лише притискати лінію до землі, а вгору її тягнуть **підтягувальні резистори**.

НЕЗВОРОТНЕ

Без підтягувальних резисторів I²C не працює взагалі. Не «працює гірше» — не працює.

Типовий номінал: **4.7 кОм** від SDA до 3.3 В і від SCL до 3.3 В. Один комплект на всю шину, не на кожен пристрій.

Це не єдине правильне число, а середина робочого діапазону. ESP-IDF рекомендує **2–5 кОм**, називаючи припустимим 1–10 кОм, і формулює залежність прямо: **що вища частота, то менший резистор** — але не менше 1 кОм, бо далі струм через відкритий вихід стає з великим.

Практично: 4.7 кОм годиться для 100 кГц і коротких дротів. На 400 кГц або на десятках сантиметрів фронт не встигає піднятися, і саме тут допомагає перехід на 2.2 кОм.

Багато модулів датчиків мають власні резистори на платі. Це зручно, поки модуль один. Три модулі зі своїми резисторами по 4.7 кОм дають сумарно близько 1.6 кОм.

Всупереч поширеній думці, **фронт від цього не гіршає, а кращає**: стала часу лінії дорівнює $R_p \times C_b$, тож утричі менший опір утричі скорочує наростання. Те, що не проходило на 400 кГц при 4.7 кОм, при 1.6 кОм проходить.

Струм теж у нормі: при 3.3 В і 1.6 кОм це 2.1 мА, а базове припущення шини — 3 мА. Мінімальний допустимий опір при 3.3 В — близько 1.1 кОм, тобто 1.6 кОм лишається з запасом.

Тобто три модулі самі по собі шини не ламають. Проблема починається далі: **шість** модулів дають 780 Ом, і це вже нижче за мінімум — ведені не зможуть притиснути лінію до потрібного низького рівня. Порядок дій той самий, але причина інша: рахувати паралельний опір, а не знімати резистори навмання.

Перевірка мультиметром за десять секунд: у спокої на обох лініях має бути 3.3 В. Нуль або щось проміжне — розбиратися треба тут, а не в коді (розділ 28).

Швидкість, довжина, ємність

Стандартні швидкості — 100 кГц і 400 кГц. Реальна межа задається **ємністю шини**: що довші проводи й більше пристроїв, то повільніше піднімається лінія після відпускання.

Практичні орієнтири:

- до **20 см** на макетці — 400 кГц зазвичай працює;
- **30–50 см** — знижувати до 100 кГц;
- **понад метр** — I²C не призначений для цього. Беріть RS-485 (розділ 34) або зменшуйте підтягувальні резистори до 2.2 кОм, розуміючи, що це підвищить споживання.

Симптом зовеликої ємності: обмін іде, але з випадковими помилками, які частішають від довших проводів. На аналізаторі видно завалені фронти (розділ 28).

Сканер: перше, що треба зробити

Перш ніж писати код датчика, треба переконатися, що він узагалі на шині і за якою адресою. Сканер перебирає всі адреси й друкує ті, що відповіли:

```
for (uint8_t addr = 1; addr < 0x7F; addr++) {
    if (i2c_master_probe(bus, addr, 50) == ESP_OK) {
        ESP_LOGI(TAG, "знайдено пристрій: 0x%02x", addr);
    }
}
```

Це замінює логічний аналізатор для питання «чи є пристрій і де він». П'ять хвилин роботи, які економлять години.

УВАГА

Адреса в datasheet буває вказана зі зсувом. Частина виробників пише 7-бітну адресу (0x76), частина — 8-бітну з бітом читання-запису (0xEC). Це та сама адреса, записана по-різному.

ESP-IDF очікує 7-бітну. Якщо пристрій не відповідає за адресою з datasheet — спробуйте поділити її навпіл. Сканер знімає це питання взагалі: він показує те, що є насправді.

Конфлікти адрес

Адреса зашита у пристрій виробником. Два однакові датчики на одній шині — конфлікт, і жоден не працює нормально.

Варіанти розв’язання:

Перемичка на модулі. Більшість датчиків мають один-два піни вибору адреси. BME280 — 0x76 або 0x77, залежно від рівня на піні SDO.

Друга шина. `classic` У classic два контролери I²C, і другу шину можна вивести на інші піни.

Мультиплексор. TCA9548A дає вісім окремих шин за однією адресою. Правильне рішення, коли треба вісім однакових датчиків.

Робота з коду

```
i2c_master_bus_config_t bus_cfg = {
    .i2c_port = I2C_NUM_0,
    .sda_io_num = GPIO_NUM_21,
    .scl_io_num = GPIO_NUM_22,
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .glitch_ignore_cnt = 7,
    .flags.enable_internal_pullup = true,
};
i2c_master_bus_handle_t bus;
i2c_new_master_bus(&bus_cfg, &bus);

i2c_device_config_t dev_cfg = {
    .dev_addr_length = I2C_ADDR_BIT_LEN_7,
    .device_address = 0x76,
    .scl_speed_hz = 100000,
};
i2c_master_dev_handle_t dev;
i2c_master_bus_add_device(bus, &dev_cfg, &dev);

uint8_t reg = 0xD0, id;
i2c_master_transmit_receive(dev, &reg, 1, &id, 1, pdMS_TO_TICKS(100));
```

УВАГА

enable_internal_pullup вмикає **вбудовані** підтягувальні резистори чипа. Вони мають опір у десятки кілоомів — цього замало для I²C на будь-якій відчутній довжині.

Для одного датчика на коротких проводах може спрацювати. Для чогось серйознішого зовнішні 4.7 кОм обов'язкові. Вбудоване підтягування — не заміна, а рятувальний круг.

Clock stretching

Ведений має право притримати лінію SCL, коли не встигає підготувати дані. Ведучий мусить це витримати.

ESP32 це підтримує, але з обмеженням таймаутом. Повільний ведений (наприклад, датчик, що довго міряє) може вийти за межу, і обмін зірветься.

Симптом: пристрій відповідає через раз, помилки таймауту при регулярному опитуванні. На аналізаторі видно розтягнутий SCL (розділ 28).

Лікування: знизити частоту шини, збільшити таймаут у налаштуваннях, або не опитувати датчик частіше, ніж він здатен міряти.

УВАГА

Усе вище — про ESP32 у ролі **ведучого**, який терпить розтягування від чужого веденого. Зворотний бік окремих і про нього рідко попереджають: **сам ESP32 у ролі веденого розтягувати SCL не вміє**. Контролер цього не підтримує.

Практично це означає, що ваш пристрій, який прикидається I²C-датчиком для чужої системи, мусить встигати відповідати завжди. Ведучий, що опитує швидше, ніж ваш обробник готує дані, отримає не затримку, а зіпсовані дані — і жодного сигналу про це.

Якщо роль веденого потрібна, а встигати не гарантовано — синхронізацію доводиться робити рівнем вище: окремою лінією «дані готові» на GPIO або перевіркою цілісності в самому протоколі.

Порядок пошуку несправності

Строго за порядком, від найчастішого:

1. **Мультиметр: 3.3 В на обох лініях у спокої**. Немає — підтягування.
2. **Прозвонка спільної землі**.
3. **Живлення датчика** — чи є на ньому напруга і яка саме.
4. **Сканер** — чи відповідає хоч щось.
5. **Скоротити проводи**, знизити швидкість до 100 кГц.
6. **Від'єднати решту пристроїв** — лишити один.
7. **Аналізатор** — АСК чи НАСК (розділ 28).

Дев'ять із десяти випадків закриваються на кроках 1–4.

Що з цього треба запам'ятати

Без зовнішніх підтягувальних резисторів 4.7 кОм I²C не працює. Вбудовані — не заміна.

Один комплект резисторів на шину; кілька модулів зі своїми дають завеликий струм.

У спокої на обох лініях 3.3 В — перевірка мультиметром за десять секунд.

Адреса в datasheet буває 8-бітною; сканер знімає це питання.

I²C — це десятки сантиметрів. Метри — це RS-485.

36. SPI

SPI — швидка шина для того, де I²C замало: дисплеї, картки пам'яті, радіомодулі, зовнішні АЦП. Десятки мегагерц замість сотень кілогерц, ціною більшої кількості пінів.

Чотири лінії

Лінія	Інші назви	Що робить
SCK	SCLK, CLK	тактування, задає ведучий
MOSI	SDI, DIN, SDA	від ведучого до веденого
MISO	SDO, DOUT	від веденого до ведучого
CS	SS, NSS, CE	вибір пристрою, активний нуль

Різнобій у назвах — постійне джерело помилок при підключенні незнайомого модуля (розділ 44). Орієнтуватися треба на функцію, а не на напис: DIN модуля — це вхід модуля, тобто до нього йде MOSI контролера.

CS — на кожен пристрій свій. Решта три лінії спільні. Пристрій слухає шину лише тоді, коли його CS притиснутий до нуля; решту часу він відпускає MISO і не заважає іншим.

Звідси головна арифметика SPI: 4 + n пінів на n пристроїв.

Режими: CPOL і CPHA

Це те, на чому спотикаються всі, і причина класичного симптому «пристрій повертає нулі або сміття».

Два біти визначають, у якому стані лінія тактування в спокої і по якому фронту читаються дані:

Режим	CPOL	CPHA	Спокій	Читання	Фронт
0	0	0	низький	по першому фронту	наростання ↑
1	0	1	низький	по другому	спадання ↓
2	1	0	високий	по першому	спадання ↓
3	1	1	високий	по другому	наростання ↑

Режим **0** — найпоширеніший, і саме він стоїть за замовчуванням.

УВАГА

Останній стовпець варто прочитати уважно, бо саме тут роблять помилку.

СРНА каже, по **котрому за ліком** фронту читаються дані — першому чи другому, — а не в який бік цей фронт іде. Напрямок виходить із поєднання: у режимі 0 перший фронт наростаючий, у режимі 2 той самий «перший» уже спадний, бо тактування в спокої високе.

Практичний наслідок, який економить час: **режими 0 і 3 читають по одному й тому самому фронту** — наростаючому. Різниця між ними лише в рівні тактування в спокої, і багато пристроїв її не помічають. Тому той самий дисплей у різних бібліотеках буває описаний і як «режим 0», і як «режим 3», причому працює в обох.

Так само парою поводяться режими 1 і 2: обидва читають по спадному фронту.

Отже перебирати варто не всі чотири, а спершу одну пару: 0 і 3. Якщо дані безглузді в обох — пробувати 1 і 2.

УВАГА

Симптом неправильного режиму: обмін начебто йде, тактування є, але дані безглузді — нулі, одиниці або зсунуті на біт значення.

З боку коду це виглядає як несправний модуль. На логічному аналізаторі видно одразу: сигнали правильні, а дані зчитуються не по тому фронту (розділ 28).

Правильний режим завжди зазначений у datasheet модуля. Якщо datasheet немає — перебрати займає хвилину, і починати треба з пари 0 і 3.

Швидкість

SPI витримує десятки мегагерц, але не завжди.

На рідних пінах контролера межа — 80 МГц. **Через матрицю GPIO** (розділ 04) — 40 МГц: рівно вдвічі, і це та ціна, яку платять за свободу вибору пінів.

Але ціна ця сплачується не завжди. **На 40 МГц і нижче різниці немає взагалі** — матриця поводить себе так само, як рідні піни. Тобто для microSD, дисплеїв і датчиків, тобто для майже всього в цій книзі, питання «чи мої піни рідні» практичного значення не має.

Ще одна деталь, яку легко не помітити: **достатньо однієї лінії поза рідними пінами, щоб через матрицю пішли всі**. Проміжного стану немає, і «майже рідний» набір пінів дає ту саму межу 40 МГц, що й повністю довільний.

Довгі проводи й макетка знижують межу різко. На макетці з Dupont-дротами розраховувати більш ніж на кілька мегагерц не варто.

Практичний підхід: почати з 1 МГц, переконатися, що працює, потім піднімати. Помилка, що з'являється при підвищенні швидкості, — це фізика, а не код.

Робота з коду

```
spi_bus_config_t bus = {
    .mosi_io_num = GPIO_NUM_23,
    .miso_io_num = GPIO_NUM_19,
    .sclk_io_num = GPIO_NUM_18,
    .quadwp_io_num = -1,
    .quadhd_io_num = -1,
    .max_transfer_sz = 4096,
};
spi_bus_initialize(SPI2_HOST, &bus, SPI_DMA_CH_AUTO);

spi_device_interface_config_t dev = {
    .clock_speed_hz = 1 * 1000 * 1000,
    .mode = 0,
    .spics_io_num = GPIO_NUM_5,
    .queue_size = 7,
};
spi_device_handle_t handle;
spi_bus_add_device(SPI2_HOST, &dev, &handle);

uint8_t tx[2] = { 0x8F, 0x00 }, rx[2] = { 0 };
spi_transaction_t t = { .length = 16, .tx_buffer = tx, .rx_buffer = rx };
spi_device_transmit(handle, &t);
```

`.length` задається **в бітах**, не в байтах. Це найчастіша помилка при першому знайомстві з драйвером.

Драйвер сам керує CS, якщо вказано `spics_io_num` — окремо смикати його не треба.

DMA: коли даних багато

Для великих передач — кадр дисплея, блок з картки — DMA передає дані без участі процесора.

Дві вимоги, які легко порушити:

Буфер має бути придатним для DMA. Звичайний `malloc` може віддати непридатну пам'ять (розділ 30):

```
uint8_t *buf = heap_caps_malloc(4096, MALLOC_CAP_DMA);
```

`max_transfer_sz` у конфігурації шини має вміщати найбільшу передачу. За замовчуванням він невеликий, і спроба надіслати кадр дисплея мовчки обрізається.

Скільки контролерів вільно

Частина контролерів SPI зайнята флешем і PSRAM (розділ 07). Реально вільних:

classic два (SPI2_HOST, SPI3_HOST), **C3** один.

Кілька пристроїв вішаються на одну шину — саме для цього існує CS. Обмеження лише в тому, що вони не можуть говорити одночасно.

УВАГА

Пристрій, який погано відпускає MISO. Коректний ведений при неактивному CS переводить MISO у високоімпедансний стан. Дешеві модулі іноді цього не роблять і заважають решті шини.

Симптом: два пристрої окремо працюють, разом — ні. Перевірка: від'єднати один і спробувати другий.

Лікується резистором на MISO або окремою шиною для проблемного пристрою.

SPI чи I²C

	I ² C	SPI
Пінів	2 на всіх	4 + 1 на пристрій
Швидкість	сотні кГц	десятки МГц
Підтягування	обов'язкове	не потрібне
Адресація	адреса в пристрої	окремий пін
Конфлікт адрес	буває	неможливий
Довжина	десятки см	ще менше

I²C — коли пристроїв багато, дані маленькі, пінів мало: датчики температури, тиску, годинники реального часу, невеликі OLED-дисплеї.

SPI — коли даних багато: кольорові дисплеї, картки пам'яті, радіомодулі, зовнішні АЦП з високою частотою вибірки.

Багато модулів підтримують обидва інтерфейси — вибір задається перемичкою. Для дисплея це часто різниця між плавним оновленням і помітним перемальовуванням.

Порядок пошуку несправності

1. Живлення й спільна земля.
2. MOSI/MISO не переплутані — звірити за функцією, не за назвою.

3. **cs під'єднаний і керується** — на аналізаторі видно, чи опускається.
4. **Режим** — перебрати чотири.
5. **Швидкість** — знизити до 1 МГц.
6. **Один пристрій на шині** — від'єднати решту.

Що з цього треба запам'ятати

CS на кожен пристрій свій; решта ліній спільні.

Режим SPI — причина класичного «повертає нулі»; правильний зазначений у datasheet, перебір чотирьох займає хвилину.

`.length` у транзакції задається в бітах.

Через матрицю GPIO межа 40 МГц замість 80; на 40 МГц і нижче різниці немає жодної.

Буфер для DMA виділяється через `heap_caps_malloc` із `MALLOC_CAP_DMA`.

Назви ліній у модулів різняться — орієнтуватися на функцію.

37. 1-Wire

1-Wire — протокол, у якому дані й (за бажанням) живлення йдуть **одним дротом**. Практично весь його світ для нас — це датчики температури DS18B20, і саме тому розділ короткий.

Чим він цікавий

Один дріт плюс земля. Мінімум проводів на великі відстані.

Багато пристроїв на одній лінії. У кожного унікальний 64-бітний адресний код, зашитий на заводі. Двох однакових не буває.

Довгі лінії. Десятки метрів — нормально, на відміну від I²C (розділ 35).

Лінія працює за принципом open-drain (розділ 05) і потребує **підтягувального резистора 4.7 кОм** до живлення. Без нього не працює.

DS18B20

Найпоширеніший датчик температури в саморобній техніці. Діапазон приблизно від -55 до +125 °C, роздільність налаштовується від 9 до 12 розрядів.

Три виводи: GND, DQ (дані), VDD (живлення).

Продається у двох виглядах: чип у корпусі TO-92 і герметичний зонд у металевій гільзі на кабелі. Другий — те, що ставлять у ґрунт, у воду, на вулицю.

ЗАКУПІВЛЯ

Ринок наповнений підробками DS18B20. Ознаки: датчик працює, але роздільність нижча за заявлену; значення стрибають; не працює паразитне живлення; кілька датчиків на лінії конфліктують.

Практична перевірка при отриманні: прочитати роздільність і порівняти показання кількох датчиків у тих самих умовах — розбіжність понад півградуса між сусідніми датчиками в одній склянці води підозріла.

Для відповідальних вимірювань варто брати в постачальника компонентів, а не на маркетплейсі.

Час вимірювання

Головна практична особливість: перетворення при 12 розрядах займає **близько 750 мілісекунд**. Це багато.

УВАГА

Більшість бібліотек для Arduino реалізують читання просто: запустити перетворення, `delay(750)`, прочитати результат. Задача при цьому стоїть три чверті секунди (розділ 12).

Правильно — розділити на дві дії: запустити перетворення, зайнятися іншим, повернутися через секунду по результат. Якщо ви читаєте температуру раз на хвилину, це нічого не ускладнює, а звільняє задачу.

Альтернатива — знизити роздільність. При 9 розрядах перетворення займає близько 94 мс, а точність 0.5 °C для більшості задач достатня.

Кілька датчиків на одній лінії

Основна перевага 1-Wire. Порядок роботи:

Пошук. Спеціальна процедура перебору знаходить адреси всіх пристроїв на лінії. Виконується один раз при старті.

Зіставлення. Адреси треба **записати** й прив'язати до фізичного розташування: який датчик у теплиці, який на вулиці. Порядок, у якому вони знаходяться при пошуку, залежить від адрес, а не від порядку підключення, і при заміні датчика зміниться.

Це та річ, яку роблять на етапі монтажу й записують у паспорт виробу (розділ 56). Інакше через рік невідомо, який із чотирьох датчиків де.

Одночасний запуск. Команда перетворення без адреси запускає всі датчики разом — потім результати читаються по черзі. Для десяти датчиків це різниця між 750 мс і сімома з половиною секундами.

Паразитне живлення

Датчик може житися **з лінії даних**, беручи енергію в паузах між обміном. Тоді потрібно всього два драти: `DQ` і `GND`.

Виглядає привабливо і працює нестабільно, особливо на довгих лініях і з кількома датчиками. Під час перетворення датчик споживає помітно більше, і лінія має його прогодувати через той самий резистор.

УВАГА

Практична порада: не використовуйте паразитне живлення без потреби. Третій дріт коштує дешевше, ніж пошук причини, чому один із п'яти датчиків іноді повертає -127°C .

Значення -127°C — це не температура, а код помилки: датчик не відповів. Найчастіші причини — відсутнє підтягування, погана пайка, паразитне живлення на межі, задовга лінія.

Довгі лінії

Десятки метрів — реально, але з умовами:

- **Підтягувальний резистор** може знадобитися менший: 2.2 кОм замість 4.7 кОм.
- **Топологія** має значення. Лінійна (датчики вздовж одного кабелю) працює краще за зіркову (кожен своїм кабелем від контролера).
- **Кабель** — краще звита пара, DQ із землею в одній парі.
- **Живлення окреме**, не паразитне. Див. вище.

Робота з коду

В ESP-IDF 1-Wire реалізують через компонент реєстру (шукати за `onewire`), який використовує RMT (розділ 33) для формування точних таймінгів. Це правильний підхід: протокол вимагає імпульсів із мікросекундною точністю, і робити їх програмно означає залежати від того, чим зайнята система.

В Arduino це бібліотеки `OneWire` і `DallasTemperature` — найпоширеніша пара, повно прикладів.

Що з цього треба запам'ятати

Підтягувальний резистор 4.7 кОм обов'язковий.

Перетворення при 12 розрядах — близько 750 мс. Не блокувати задачу на цей час; за потреби знизити роздільність.

Адреси датчиків записати й прив'язати до розташування на етапі монтажу.

-127 °C — це код помилки, а не температура.

Паразитне живлення економить дріт і додає нестабільності. Третій дріт дешевший.

38. CAN (TWAI)

CAN — шина, придумана для автомобілів і прижилася скрізь, де треба надійно і на відстань: промислові контролери, техніка, приводи, акумуляторні батареї з BMS.

В ESP32 контролер CAN називається **TWAI** (Two-Wire Automotive Interface). Назва інша через ліцензування торгової марки — протокол той самий, сумісність повна.

Це головний спосіб з'єднати ESP32 з «дорослою» системою (розділ 57).

Чим CAN відрізняється від інших шин

Немає ведучого. Будь-який вузол може почати передачу коли завгодно.

Немає адрес отримувача. Пакет має **ідентифікатор** — не адресу пристрою, а тип повідомлення. Кожен вузол сам вирішує, які ідентифікатори йому цікаві.

Апаратний арбітраж. Коли двоє починають одночасно, перемагає той, у кого менший ідентифікатор — а другий автоматично відкладає передачу і повторює. Це відбувається на рівні заліза, без втрати даних і без колізій у звичному сенсі.

Апаратне підтвердження й повтор. Контролер сам перевіряє контрольну суму, підтверджує прийом і повторює передачу при помилці.

УВАГА

Наслідок для проектування: у CAN ви думаєте не «хто кому надсилає», а «яка інформація існує в системі». Вузол публікує температуру з ідентифікатором 0x180; хто хоче — слухає. Додати ще одного слухача не означає нічого змінювати в передавачі.

Менший ідентифікатор = вищий пріоритет. Тому аварійні повідомлення отримують малі номери, а телеметрія — великі.

Фізика: трансивер обов'язковий

ESP32 має контролер, але **не має трансивера**. Потрібна зовнішня мікросхема: SN65HVD230, TJA1050, MCP2551 або аналог.

НЕЗВОРОТНЕ

Трансивер має відповідати логіці **3.3 В**. SN65HVD230 живиться від 3.3 В і підходить прямо. TJA1050 і MCP2551 розраховані на 5 В: їхній вихід RX може подавати 5 В на пін ESP32 і спалити його (розділ 05).

Готові модулі на 5-вольтових трансиверах продаються масово й підписуються «для ESP32». Перевіряти треба самому: або трансивер на 3.3 В, або конвертер рівнів на лінії RX.

Дві лінії: CANH і CANL, диференціальна пара — звідси стійкість до завад і довжина.

Термінатори 120 Ом на **обох** кінцях лінії, і тільки на кінцях. Це не опція: без них сигнал відбивається і шина не працює. З трьома термінаторами — так само не працює.

Багато модулів мають термінатор на платі, часто припаяний намертво. Перед складанням шини з кількох вузлів варто перевірити, скільки їх насправді: між CANH і CANL знеструмленої шини має бути близько 60 Ом (два по 120 паралельно).

Швидкість і довжина

Швидкість і довжина обернено пов'язані. Орієнтири:

Швидкість	Довжина
1 Мбіт/с	до 40 м
500 кбіт/с	до 100 м
250 кбіт/с	до 250 м
125 кбіт/с	до 500 м

Швидкість має бути однаковою в усіх вузлів. Один вузол із неправильною швидкістю не просто не чує — він псує обмін решті, бо не підтверджує пакети й генерує помилки.

Робота з коду

```
twai_general_config_t g = TWAI_GENERAL_CONFIG_DEFAULT(
    GPIO_NUM_21, GPIO_NUM_22, TWAI_MODE_NORMAL);
twai_timing_config_t t = TWAI_TIMING_CONFIG_500KBITS();
twai_filter_config_t f = TWAI_FILTER_CONFIG_ACCEPT_ALL();

twai_driver_install(&g, &t, &f);
twai_start();

twai_message_t msg = {
    .identifier = 0x180,
    .data_length_code = 4,
    .data = { 0x01, 0x02, 0x03, 0x04 },
};
twai_transmit(&msg, pdMS_TO_TICKS(100));

twai_message_t in;
if (twai_receive(&in, pdMS_TO_TICKS(1000)) == ESP_OK) {
    ESP_LOGI(TAG, "id 0x%03lx, %d байт", in.identifier, in.data_length_code);
}
```

Пакет несе до 8 байтів. Це жорстке обмеження класичного CAN. Більше — розбивати на кілька пакетів або використовувати протокол поверх (наприклад, ISO-TP).

НЕЗВОРОТНЕ

CAN FD жодне з сімейств у цій книзі не розуміє — ні classic, ні S2, ні S3, ні C3, ні C6, ні H2. І поводить воно гірше, ніж «просто не працює»: контролер сприймає FD-кадр як **помилку**.

Наслідок для розвідки чужої шини серйозний. Сучасний автомобіль або промислова установка цілком може ходити на CAN FD; ваш вузол не просто нічого не прочитає — він почне генерувати кадри помилок і псувати обмін решті, рівно як вузол із неправильною швидкістю.

Тому `LISTEN_ONLY` (нижче) — не порада для акуратних, а спосіб не зіпсувати чужу шину, поки не з'ясовано, що вона класична. Якщо на ній FD — потрібен інший контролер, зовнішній (наприклад, MCP2518FD по SPI) або не ESP32 узагалі.

Це та властивість, яка з'являється в новіших чипах Espressif; перед проектуванням під FD звіряйте `soc_caps.h` свого чипа на `SOC_TWAI_SUPPORT_FD`.

Фільтр дозволяє контролеру приймати лише потрібні ідентифікатори, не турбуючи процесор рештою. На завантаженій шині це суттєво.

УВАГА

Режим `TWAI_MODE_LISTEN_ONLY` — найкорисніший для розвідки. Вузол слухає шину й **нічого не передає**, навіть підтверджень.

Це правильний спосіб під'єднатися до чужої працюючої системи: ви бачите весь обмін і гарантовано нічого не порушуєте. Для тріажу незнайомого обладнання (розділ 23) — саме те.

Помилки і стан шини

CAN сам стежить за помилками й має лічильники. При накопиченні помилок контролер спершу переходить у стан «пасивних помилок», далі — вимикається з шини (bus-off).

```
twai_status_info_t st;
twai_get_status_info(&st);
ESP_LOGI(TAG, "стан %d, помилок TX %lu, RX %lu",
          st.state, st.tx_error_counter, st.rx_error_counter);
```

Зростання лічильника помилок — це діагностика фізики: термінатори, швидкість, проводка. Стан bus-off потребує явного відновлення (twai_initiate_recovery), і в надійному виробі це треба обробляти (розділ 32).

Найчастіша причина зростання помилок при першому запуску: **на шині один вузол**. CAN потребує щонайменше двох — передані пакети нема кому підтверджувати, і передавач вважає це помилкою.

Для перевірки одного вузла є TWAI_MODE_NO_ACK — режим самотестування.

Обмін між двома ESP32

Мінімальна робоча схема: два ESP32, два трансивери, дві лінії між ними, два термінатори по 120 Ом. Спільна земля бажана — на коротких лініях без неї часто працює, але це не привід її не робити.

Це найпростіший спосіб перевірити всю схему перед під'єднанням до чогось справжнього.

Обмін із «дорослим» контролером

Практична послідовність при під'єднанні до чужої системи:

1. **Дізнатися швидкість**. Без неї нічого не вийде. Якщо документації немає — перебрати стандартні значення в режимі LISTEN_ONLY, доки не з'являться коректні пакети.
2. **Слухати в LISTEN_ONLY** і записати, які ідентифікатори ходять і як часто.
3. **Зіставити з документацією** на протокол, якщо вона є.
4. **Лише потім** починати передавати.

НЕЗВОРОТНЕ

Передача в чужу працюючу шину — дія з наслідками. Пакет із чужим ідентифікатором може бути сприйнятий як команда керування.

Правило: **спершу тільки слухати**, і починати передавати лише тоді, коли зрозуміло, що саме ви надсилаєте і хто це отримає. У техніці, яка рухається або гріє, це не формальність.

Що з цього треба запам'ятати

TWAI — це CAN; назва інша, протокол той самий.

Трансивер обов'язковий і має бути на 3.3 В, інакше він спалить пін.

Термінатори 120 Ом на обох кінцях; на знеструмленій шині між лініями має бути близько 60 Ом.

Швидкість однакова в усіх вузлів; один неправильний вузол псує обмін усім.

Ідентифікатор — це тип повідомлення, а не адреса; менший номер означає вищий пріоритет.

`LISTEN_ONLY` — правильний спосіб знайомитися з чужою шиною.

Один вузол на шині дає помилки: підтверджувати нема кому.

Частина X. Бездротовий зв'язок

Wi-Fi, мережеві протоколи, BLE, ESP-NOW, LoRa.

39. Wi-Fi

Wi-Fi — головна причина, чому беруть ESP32 (розділ 01). Він працює «з коробки», і саме тому легко не помітити місця, де він підводить: у живленні, в антені й у поведінці при втраті зв'язку.

Три режими

STA (station). Пристрій під'єднується до наявної точки доступу. Основний режим.

AP (access point). Пристрій сам роздає мережу. Використовується для початкового налаштування і для роботи там, де мережі немає.

APSTA. Обидва одночасно. Зручно: пристрій під'єднаний до роутера і водночас доступний напряму для налаштування.

УВАГА

У режимі APSTA обидві ролі ділять **один** радіомодуль і мусять бути на **одному каналі**. Коли пристрій під'єднується до точки доступу на каналі 6, його власна точка теж переїжджає на канал 6 — навіть якщо ви задали інший.

Наслідок: клієнт, під'єднаний до вашої точки, втратить зв'язок у момент, коли пристрій перепід'єднається до роутера на іншому каналі. Це не помилка, а фізика одного радіо.

Що обмежує ESP32

НЕЗВОРОТНЕ

ESP32 не бачить мережі 5 ГГц. Уся лінійка, крім ESP32-C5, працює тільки в діапазоні 2.4 ГГц (розділ 02).

Це найчастіша причина «мережі немає в списку». Сучасні роутери часто мають однакове ім'я для обох діапазонів, і телефон під'єднується до 5 ГГц, а ESP32 не бачить нічого. Виглядає як несправність.

Перевірка: подивитися на телефоні, у якому діапазоні працює мережа, або тимчасово розділити імена в налаштуваннях роутера.

Інші обмеження, що трапляються:

Канали 12 і 13 доступні не за всіх налаштувань регіону. Якщо роутер працює на 13-му, ESP32 із неправильно заданою країною його не побачить.

WPA3 підтримується в новіших версіях ESP-IDF; старіші — ні. Роутер, переведений у режим «тільки WPA3», відрізає такі пристрої.

Прихований SSID потребує явного налаштування — сканування його не знайде.

Тільки один канал одночасно. Звідси обмеження APSTA вище і співіснування з ESP-NOW (розділ 42).

Під'єднання

```
wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
esp_wifi_init(&cfg);

wifi_config_t sta = {
    .sta = {
        .ssid = "merezha",
        .password = "parol",
        .threshold.authmode = WIFI_AUTH_WPA2_PSK,
    },
};
esp_wifi_set_mode(WIFI_MODE_STA);
esp_wifi_set_config(WIFI_IF_STA, &sta);
esp_wifi_start();
esp_wifi_connect();
```

Робота йде **через події**: під'єднання асинхронне, і код мусить реагувати на `WIFI_EVENT_STA_DISCONNECTED`, `IP_EVENT_STA_GOT_IP` та інші.

УВАГА

Помилка новачка — вважати, що після `esp_wifi_connect()` мережа вже є. Її ще немає: під'єднання займає від сотень мілісекунд до кількох секунд, а IP-адреса приходить окремою подією ще пізніше.

Правильний спосіб дочекатися — група подій FreeRTOS (розділ 31), яку встановлює обробник `IP_EVENT_STA_GOT_IP`. Саме наявність IP, а не факт під'єднання, означає, що можна працювати з мережею.

Перепід'єднання: те, що відрізняє виріб від прототипу

Зв'язок обірветься. Роутер перезавантажиться, живлення блимне, пристрій опиниться на межі покриття. Виріб має пережити це без втручання.

НЕЗВОРОТНЕ

Дві помилки, що перетворюють пристрій на цеглинку.

ESP_ERROR_CHECK навколо під'єднання (розділ 32). Точка доступу вимкнена — пристрій перезавантажується, знову не бачить — перезавантажується знову.

Перепід'єднання без паузи. Обробник `STA_DISCONNECTED`, що негайно викликає `esp_wifi_connect()`, створює нескінченний цикл спроб. Пристрій гріється, з'їдає батарею і не робить нічого корисного.

Правильна схема — повтори зі зростаючою паузою і збереженням працездатності:

```
static int pauza = 1000; // мс, початкова

static void on_disconnect(void) {
    ESP_LOGW(TAG, "зв'язок втрачено, спроба через %d мс", pauza);
    esp_timer_start_once(timer_reconnect, (uint64_t)pauza * 1000);
    if (pauza < 30000) pauza *= 2;
}

static void on_got_ip(void) {
    pauza = 1000; // ← без цього рядка схема не працює
}
```

Скидання паузи при успішному під'єднанні — не косметика. Без нього пристрій, який один раз пережив довгу відсутність мережі, назавжди лишається з тридцятисекундною паузою: наступний обрив на секунду коштуватиме півхвилини недоступності, і причини цьому не буде видно ніде.

І головне: **пристрій має працювати без мережі**. Скласти вимірювання в буфер, продовжувати керувати, віддавати дані потім (розділ 32).

Креденшели

Зашивати SSID і пароль у код — нормально для прототипу і погано для виробу: змінити мережу можна лише перепрошиванням, а пароль лежить у прошивці відкритим текстом і дістається з дампа за п'ять хвилин (розділ 24).

Правильно — зберігати в NVS (розділ 18), а вводити одним із способів provisioning:

Власна точка доступу з веб-формою. Пристрій, що не знайшов збереженої мережі, піднімає AP; людина під'єднується телефоном, відкриває сторінку, вводить дані. Найзрозуміліший для користувача спосіб.

SoftAP або BLE provisioning — штатні механізми ESP-IDF із застосунками для телефона.

SmartConfig / ESP-Touch — передача креденшелів широкомовними пакетами. Працює не з усіма роутерами й телефонами; підходить як запасний варіант, не як основний.

УВАГА

Обов'язково передбачте **скидання налаштувань**: довге утримання кнопки стирає збережену мережу й піднімає точку доступу. Без цього пристрій, переїхавши в інше місце, стає непридатним — і це найчастіша причина повернень виробів.

RSSI і реальна дальність

```
wifi_ap_record_t ap;
esp_wifi_sta_get_ap_info(&ap);
ESP_LOGI(TAG, "RSSI %d дБм, канал %d", ap.rssi, ap.primary);
```

RSSI	Що це означає
від -50 дБм	відмінно
-50...-65	добре
-65...-75	робоче
-75...-85	межа: обриви, повільно, ОТА може не пройти
нижче -85	практично непрацездатно

УВАГА

Різниця між «зв'язок є» і «зв'язок працює» проявляється саме на межі. Пристрій із RSSI -82 успішно пінгується і навіть віддає невеликі пакети — але ОТА-оновлення на кілька сотень кілобайтів не проходить жодного разу (розділ 19).

Тому RSSI варто логувати завжди: у полі це перше, що пояснює дивну поведінку.

Реальна дальність у приміщенні — десятки метрів, і кожна стіна з'їдає помітну частину. Залізобетон і фольгована ізоляція вбивають сигнал майже повністю.

Анени

PCB-антена — доріжка на платі модуля (WROOM-1). Дешево, компактно, достатньо для більшості задач.

Зовнішня антена через роз'єм IPEX — модулі з літерою U (WROOM-1U). Потрібна, коли пристрій у металевому корпусі або треба дальність.

НЕЗВОРОТНЕ

Метал убиває радіо. Пристрій у металевому боксі не має зв'язку, хоч би що ви робили з кодом. Варіанти: пластиковий корпус, вікно в металі, або зовнішня антена, винесена назовні.

Це те, що виявляється після складання виробу, і тоді вже дорого. Питання корпусу й антени вирішується на етапі проектування (розділ 54).

Правила розміщення РСВ-антени, які варто дотримувати:

- зона антени на власній платі має бути **вільна від міді** з усіх боків і знизу;
- антена має виступати за край плати або стояти на краю;
- не класти під антену дроти, акумулятор, дисплей;
- відстань до металу — щонайменше сантиметр, краще більше.

Споживання

Wi-Fi — головний споживач у пристрої (розділ 06). Що з цим роблять:

Modem sleep — радіо вимикається між маячками, з'єднання зберігається. Вмикається за замовчуванням і майже безкоштовне.

Рідше передавати. Найдієвіше. Раз на п'ять хвилин замість раз на секунду — це не оптимізація коду, а зміна постановки задачі.

Не під'єднуватися взагалі. Для датчика на батарейці ESP-NOW (розділ 42) виграє на порядок: передача без під'єднання займає мілісекунди замість секунд.

Знизити потужність передавача `esp_wifi_set_max_tx_power` — коли точка доступу поруч, повна потужність не потрібна.

Що з цього треба запам'ятати

ESP32 не бачить 5 ГГц — найчастіша причина «мережі немає в списку».

Нааявність IP, а не факт під'єднання, означає готовність працювати.

Перепід'єднання — зі зростаючою паузою; `ESP_ERROR_CHECK` навколо під'єднання перетворює виріб на цеглинку.

Пристрій має працювати без мережі.

Скидання налаштувань кнопкою — обов'язкове.

RSSI логувати завжди: на межі ОТА не проходить, коли пінги ще ходять.

Метал убиває радіо; це вирішується на етапі корпусу, не коду.

40. Мережеві протоколи

Wi-Fi дає канал; далі треба вирішити, що по ньому передавати і як. Розділ про рівень вище фізичного: сокети, HTTP, WebSocket, MQTT, TLS.

TCP чи UDP

TCP гарантує доставку й порядок. Ціна — встановлення з'єднання, підтвердження, повтори; при поганому зв'язку затримки ростуть.

UDP нічого не гарантує. Пакет або дійшов, або ні, і дізнатися про це ви не можете. Зате він швидкий, не потребує з'єднання і дозволяє широкомовну розсилку.

Практичне правило: **TCP для команд і налаштувань, UDP для потоку вимірювань**. Втрачений відлік температури нічого не змінює; втрачена команда «вимкнути» — змінює.

HTTP-сервер на пристрої

Найзручніший інтерфейс для людини: жодного застосунку, працює з будь-якого телефона.

```
httpd_config_t cfg = HTTPD_DEFAULT_CONFIG();
httpd_handle_t server = NULL;
httpd_start(&server, &cfg);

httpd_uri_t uri = {
    .uri = "/api/stan",
    .method = HTTP_GET,
    .handler = stan_handler,
};
httpd_register_uri_handler(server, &uri);
```

УВАГА

Обробник виконується в задачі веб-сервера з **обмеженим стеком**. Великі буфери там — прямий шлях до переповнення стека, яке проявиться пізніше й деінде (розділ 30).

Розмір стека сервера задається в HTTPD_DEFAULT_CONFIG і його часто доводиться збільшувати. Особливо якщо в обробнику формується JSON або використовується TLS.

Статичні файли — HTML, CSS, скрипти — кладуть у розділ файлової системи (розділ 18) або вбудовують у прошивку як ресурси. Друге простіше для невеликих сторінок і не потребує окремого розділу.

WebSocket

HTTP влаштований як «запит — відповідь»: сервер не може сам щось надіслати. Для сторінки, яка має оновлюватися наживо, це означає опитування щосекунди — марний трафік і затримки.

WebSocket дає двонаправлений канал поверх того самого з'єднання. Для графіка вимірювань у реальному часі це правильний інструмент.

Обмеження практичне: кожен клієнт — це відкрите з'єднання й пам'ять під нього. На ESP32 кілька одночасних клієнтів — межа, і поводитися з нею треба свідомо.

mDNS: щоб не шукати IP

Пристрій отримує адресу від роутера, і вона може змінитися. Змушувати людину шукати IP — поганий інтерфейс.

mDNS дає постійне ім'я:

```
mdns_init();
mdns_hostname_set("teplytsia");
mdns_instance_name_set("Датчики теплиці");
```

Далі пристрій доступний як `teplytsia.local` — з телефона, з ноутбука, з браузера.

УВАГА

mDNS працює не скрізь. Android історично підтримував його неповно; корпоративні мережі часто ріжуть широкомовний трафік; гостьові мережі ізолюють клієнтів одне від одного.

Тому: mDNS — це зручність, а не єдиний спосіб дістатися пристрою. Показувати IP-адресу теж треба — хоча б у логу або на дисплеї.

SNTP: точний час

Годинник у чипі неточний і скидається при вимиканні живлення (розділ 03). Якщо є мережа, час беруть з інтернету:

```
esp_sntp_setoperatingmode(ESP_SNTP_OPMODE_POLL);
esp_sntp_setservername(0, "pool.ntp.org");
esp_sntp_init();
setenv("TZ", "EET-2EEST,M3.5.0/3,M10.5.0/4", 1);
tzset();
```

Рядок `TZ` вище — правило переходу на літній час для України; воно працює автономно, без оновлень.

Час приходить **не миттєво**: перша синхронізація займає секунди. Код, що ставить мітки часу, має вміти працювати, доки часу ще немає, — інакше в логах з'являються записи з 1970 року.

Немає мережі й потрібен точний час — зовнішня мікросхема RTC із батареєю (розділ 60).

MQTT

Протокол, придуманий саме для таких пристроїв: легкий, тримає одне з’єднання, працює через погані канали.

Модель — «публікація і підписка». Пристрої не знають одне про одного: є **брокер** посередині, є **топіки** — ієрархічні імена. Хто хоче — публікує, хто хоче — підписується.

```
esp_mqtt_client_config_t cfg = {
    .broker.address.uri = "mqtt://192.168.1.10",
    .credentials.username = "datchyk",
    .session.keepalive = 60,
};
esp_mqtt_client_handle_t client = esp_mqtt_client_init(&cfg);
esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, handler, NULL);
esp_mqtt_client_start(client);

esp_mqtt_client_publish(client, "teplytsia/temperatura", "23.5", 0, 1, 0);
```

Структура топіків — це проєктне рішення, і міняти її потім дорого. Робоча схема: <об’єкт>/<вузол>/<величина>, наприклад `teplytsia/datchyk1/temperatura`. Керування — окремою гілкою: `teplytsia/datchyk1/cmd/rele`.

QoS:

Рівень	Гарантія	Коли
0	доставка не гарантована	телеметрія, часті вимірювання
1	доставлено щонайменше раз, можливі дублі	команди
2	рівно раз, найдорожчий	рідко потрібен

Обробник команд має бути **ідемпотентним**: при QoS 1 те саме повідомлення може прийти двічі, і «увімкнути реле» двічі має означати те саме, що один раз.

УВАГА
Дві можливості MQTT, які роблять систему живою і які часто пропускають.

Retained-повідомлення. Брокер зберігає останнє значення в топіку і віддає його новому підписнику одразу. Без цього інтерфейс після перезавантаження показує порожнечу, доки датчик не надішле наступне значення — а це може бути через п’ять хвилин.

Last Will and Testament. Пристрій наперед каже брокеру: «якщо я зникну, опублікуй у такий-то топик слово `offline`». Тепер система дізнається про мертвий вузол сама, без опитувань. Для будь-якого виробу, що працює без нагляду, це обов’язкове.

TLS: мінімум

Незашифрований обмін означає, що будь-хто в мережі бачить дані й може підмінити команди. Для пристрою в полі це не теорія.

Мінімум, що реально працює:

Перевірка сертифіката сервера. Пристрій має знати, з ким говорить. Сертифікат вбудовується в прошивку як ресурс.

Зашивати сертифікат центру сертифікації, а не сервера. Сертифікат сервера протермінується через рік, і всі пристрої одночасно втратять зв’язок. Сертифікат CA живе значно довше (розділ 19).

Не вимикати перевірку. Спокуса велика — воно одразу починає працювати. Але тоді TLS дає лише шифрування без автентифікації, тобто захищає від підслуховування і не захищає від підміни. Це половина захисту, яка створює відчуття повного.

ЖИВЛЕННЯ

TLS коштує ресурсів: кілька кілобайтів RAM на з’єднання, помітний час на рукостискання, і сплеск споживання. На C3 з його 400 КБ (розділ 02) два одночасні TLS-з’єднання вже відчутні.

Апаратні прискорювачі криптографії в чипі роблять це прийнятним (розділ 04), але не безкоштовним. Для пристрою на батарейці рукостискання при кожній передачі — головна стаття витрат; краще тримати одне з’єднання постійно.

Що обрати

Задача	Чим
Налаштування людиною	HTTP-сервер на пристрої
Графік наживо	WebSocket
Телеметрія в систему	MQTT
Обмін між своїми вузлами	ESP-NOW (розділ 42) або UDP
Команди керування	MQTT з QoS 1 або TCP

Задача**Чим**

Доступ без знання IP

mDNS плюс показ IP

Що з цього треба запам'ятати

TCP для команд, UDP для потоку вимірювань.

Стек задач веб-сервера часто треба збільшувати.

mDNS — зручність, а не єдиний шлях; IP показувати теж.

Retained і Last Will у MQTT роблять систему живою; обробник команд має бути ідемпотентним.

Зашивати сертифікат CA, а не сервера. Не вимикати перевірку.

Код має працювати, доки часу ще немає.

41. Bluetooth і BLE

Bluetooth на ESP32 — тема, де найлегше витратити час даремно, бо відповідь на «як зробити» залежить від чипа сильніше, ніж деінде.

Найважливіше рішення вже ухвалене за вас

НЕЗВОРОТНЕ

Bluetooth Classic є лише в ESP32 classic. S3, C3, C6, H2 — **тільки BLE**. ESP32-S2 не має Bluetooth узагалі (розділ [02](#)).

Це відсутність апаратного блоку, а не нереалізована функція. Ніяка бібліотека і ніяка версія IDF цього не змінить.

Практичний наслідок величезний: профіль **SPP** — послідовний порт по Bluetooth, на якому тримається безліч старих проєктів і на який розраховані прості термінальні застосунки для телефона, — існує **тільки на classic**.

Проєкт на SPP, що переїжджає на S3, доведеться переписувати на BLE. Це не порт, а переробка: інша модель обміну.

Classic проти BLE

	Bluetooth Classic	BLE
Де є	тільки classic	уся лінійка, крім S2
Модель	потік байтів	набір іменованих значень
Швидкість	сотні кбіт/с	десятки кбіт/с
Споживання	висока	дуже низька
Спарювання	так, з PIN	не обов'язкове
Пам'ять у прошивці	багато	менше
Термінал на телефоні	простий (SPP)	потрібен BLE-застосунок

Classic зручний там, де треба «просто послідовний порт по повітрю»: налагодження, обмін із застарілим обладнанням, термінал.

BLE — коли треба батарейка, багато пристроїв або сучасні телефони. iOS із Bluetooth Classic для довільних пристроїв практично не працює — це окрема причина, чому SPP не варто закладати в новий проєкт.

Модель BLE: GATT

BLE не передає потік. Він публікує **структуру даних**, і клієнт читає або пише її частини.

Сервіс — логічна група. Наприклад, «датчики середовища».

Характеристика — окреме значення всередині сервісу: температура, вологість. У кожної є права: читати, писати, сповіщати.

Notify — сервер сам надсилає значення клієнту при зміні. Це те, що замінює потік: клієнт підписується один раз і отримує оновлення.

UUID — ідентифікатор сервісу чи характеристики. Стандартні профілі мають короткі 16-бітні номери; власні — 128-бітні, які генеруються один раз.

УВАГА

Спокуса відтворити SPP через BLE — зробити характеристику «дані» і ганяти через неї байти — виникає в усіх, хто прийшов із Classic. Так роблять, і воно працює (це називають BLE UART або Nordic UART Service).

Але правильніше описати дані як дані: окрема характеристика на температуру, окрема на стан реле. Тоді пристрій самоописовий — будь-який універсальний BLE-застосунок покаже осмислені значення без вашого клієнта. Для виробу, який обслуговуватимуть інші люди, це велика різниця.

Bluedroid і NimBLE

В ESP-IDF два стеки BLE, і вибір між ними — не смак.

Bluedroid — повний стек, підтримує і Classic, і BLE. Займає значно більше пам'яті.

NimBLE — тільки BLE, компактніший, займає в рази менше RAM і флешу.

УВАГА

Для BLE-проєкту беріть NimBLE. Різниця в пам'яті вимірюється десятками кілобайтів — на C3 з його 400 KB (розділ [02](#)) це різниця між «вміщається» і «ні».

Bluedroid потрібен лише тоді, коли треба Classic, тобто тільки на classic-чипі.

Перемикається в menuconfig: Component config → Bluetooth → Host.

Скільки це коштує пам'яті

Bluetooth — найважчий компонент після Wi-Fi. Прошивка з BLE помітно більша, а RAM з'їдається стеком постійно, а не лише під час обміну.

Практичні наслідки:

Wi-Fi і Bluetooth одночасно працюють, але ділять одне радіо і конкурують за час. Продуктивність обох падає, споживання росте. Для провізювання (розділ 39) це нормально; для постійної одночасної роботи — привід подумати.

На С3 з 400 КБ BLE плюс Wi-Fi плюс TLS плюс власна логіка — це вже тісно.

Вимикати, коли не треба. Пристрій, у якому BLE потрібен лише для початкового налаштування, має його вимикати після — це звільняє пам'ять і знижує споживання.

Споживання

BLE спроектований для батарейок, і його головний параметр — **інтервал реклами** (advertising interval).

Пристрій, що рекламує себе кожні 100 мс, знаходиться миттєво і їсть відчутно. Той, що рекламує раз на секунду, знаходиться за секунду і їсть у десять разів менше.

Для датчика на батарейці різниця в місяцях роботи. Розумний підхід: частий інтервал перші хвилини після старту (щоб зручно було під'єднатися), далі рідкий.

Інтервал з'єднання визначає затримку обміну після під'єднання. Тут теж компроміс між швидкістю реакції і споживанням, і телефон має право його змінити.

Безпека

BLE без спарювання доступний **будь-кому поруч**. Для датчика температури це прийнятно; для керування чимось — ні.

Мінімум для пристрою, що приймає команди:

- **спарювання з підтвердженням** — щоб під'єднатися міг не кожен;
- **характеристики керування тільки для запису після автентифікації**
- **не публікувати в рекламі нічого зайвого** — ім'я пристрою видно всім, хто сканує, включно з сусідами.

НЕЗВОРОТНЕ

Найпоширеніша реальна вразливість — характеристика запису без жодного захисту, доступна на відстані десятків метрів. Будь-хто зі смартфоном і безкоштовним застосунком може під'єднатися і надіслати команду.

Питання, яке варто поставити до кожної характеристики із правом запису: **що станеться, якщо туди напише сторонній?** (розділ 50)

Обмін із телефоном

Найпоширеніший сценарій: пристрій — сервер, телефон — клієнт.

Для розробки й налагодження є універсальні BLE-застосунки: вони сканують пристрої, показують сервіси, характеристики й дозволяють читати й писати вручну. Це найшвидший спосіб перевірити, що ваш GATT правильний, до написання власного застосунку.

Що варто врахувати:

Розмір пакета обмежений — за замовчуванням ATT MTU дорівнює 23 байтам, з яких 3 службові, тобто на корисні дані лишається **20 байтів**. Довгі рядки доведеться або ділити, або узгоджувати більший MTU — і друге телефон має право не підтримати.

Телефон керує параметрами з'єднання. Ваші налаштування інтервалів — побажання, а не команда.

iOS суворіший щодо структури GATT і кешує її. Змінили структуру сервісів — телефон може продовжувати бачити стару, доки не забути пристрій у налаштуваннях. Це джерело дуже заплутаних годин налагодження.

Що з цього треба запам'ятати

Classic — тільки на classic-чипі; SPP на S3 і C3 не існує в принципі.

Для BLE-проєкту брати NimBLE: різниця в пам'яті вирішальна на C3.

BLE публікує структуру даних, а не потік; описуйте значення окремими характеристиками.

Інтервал реклами — головний важіль споживання.

Характеристика запису без захисту доступна будь-кому поруч.

iOS кешує структуру GATT: після зміни сервісів пристрій треба забути в налаштуваннях телефона.

42. ESP-NOW

ESP-NOW — власний протокол Espressif для прямого обміну між пристроями на ESP32 і ESP8266. Без роутера, без точки доступу, без IP-адрес.

Для автономних датчиків це часто найкраще технічне рішення в усій книзі, і причина одна: **передача без під'єднання**.

Чому це важливо

Звичайний Wi-Fi перед першою передачею мусить під'єднатися до точки доступу: сканування, автентифікація, асоціація, отримання IP. Це секунди — від однієї до десяти, а на межі покриття може не відбутися взагалі (розділ 39).

ESP-NOW не робить нічого з цього. Пакет іде **одразу**, за мілісекунди.

ЖИВЛЕННЯ

Для датчика на батарейці, який прокидається, міряє й засинає (розділ 06), різниця виглядає так:

Wi-Fi: прокинувся → 3 секунди на під'єднання → передав → заснув.
Активна фаза — три з половиною секунди при сотні міліампер.

ESP-NOW: прокинувся → 10 мілісекунд на передачу → заснув. Активна фаза — частки секунди.

Це різниця у **два порядки** в споживанні на цикл, тобто різниця між місяцем і роками роботи від тих самих батарейок.

Як воно влаштоване

Обмін іде за **MAC-адресами**. Кожен пристрій має унікальну MAC від заводу (розділ 20), і вона ж є адресою в ESP-NOW.

```
esp_now_init();

esp_now_peer_info_t peer = {
    .channel = 1,
    .encrypt = false,
};
memcpy(peer.peer_addr, mac_pryimacha, 6);
esp_now_add_peer(&peer);

esp_now_send(mac_pryimacha, (uint8_t *)&dani, sizeof(dani));
```

Прийом — через зареєстрований обробник:

```
static void on_recv(const esp_now_recv_info_t *info,
                  const uint8_t *data, int len) {
    // виконується в контексті Wi-Fi — коротко, без важкої роботи
    xQueueSend(cherga, data, 0);
}
esp_now_register_recv_cb(on_recv);
```

УВАГА

Обробник прийому виконується в контексті **задачі** Wi-Fi, а не в перериванні. Звідси дві речі одразу.

Правило поведінки те саме, що для ISR (розділ 31): скопіювати дані, покласти в чергу, вийти. Важка робота там блокує радіостек.

А от функції — **не ті самі**: тут потрібен звичайний `xQueueSend`, а не `xQueueSendFromISR`. Варіант `FromISR` у задачі не спрацює як треба, і помилка ця тиха. Нульовий таймаут у `xQueueSend` теж не випадковий: чекати на місце в черзі всередині радіостека не можна.

І окремо: `esp_now_recv_info_t`, на який указує `info`, живе **лише поки триває виклик**. Знадобився MAC відправника — копіювати його тут, а не зберігати показчик.

Обмеження, які визначають проєкт

Розмір пакета — до 250 байтів. Це жорстко. Більше — ділити самому.

Немає гарантії доставки. Є підтвердження на рівні кадру (`send_cb` повідомляє, чи пакет прийнято сусідом), але немає повторів і немає контролю порядку. Це UDP-подібна модель.

Треба знати MAC отримувача. Або зашити, або передати при налаштуванні, або використати широкомовну адресу.

Один канал. Усі учасники мають бути на одному радіоканалі.

Кількість реє-ів обмежена жорстко: 20 усього, з них не більше 6 захищених. Друге число визначає проєкт значно сильніше за перше: конструкція «багато датчиків → один приймач» із шифруванням упирається в шість датчиків на приймач, а не в двадцять. Обходиться це або broadcast-обміном із власним шифруванням у полі корисних даних, або другим приймачем.

Broadcast

Широкомовна адреса `FF:FF:FF:FF:FF:FF` дозволяє передавати всім, хто слухає, не знаючи адрес.

Зручно для виявлення пристроїв і для розсилки команд одразу всім. Обмеження: broadcast **не шифрується** — це властивість протоколу, а не налаштування.

Практична схема: broadcast для початкового знайомства, далі — адресний обмін із шифруванням.

Шифрування

ESP-NOW підтримує шифрування з ключами PMK і LMK.

```
esp_now_set_pmk((uint8_t *) "pmk1234567890123"); // рівно 16 байтів
peer.encrypt = true;
memcpy(peer.lmk, "lmk1234567890123", 16);
```

НЕЗВОРОТНЕ

Без шифрування ESP-NOW — це відкритий радіоефір. Будь-хто з ESP32 поруч може слухати ваш обмін і, знаючи MAC, надсилати пакети від чужого імені.

Для датчика температури це може бути прийнятним. Для будь-чого, що керує — ні. Питання те саме, що в розділі [41](#): що станеться, якщо туди напише сторонній? (розділ [50](#))

І окремо: ключі, зашиті в код, дістаються з дампа прошивки за п'ять хвилин (розділ [24](#)). Місце ключів — NVS, записаний окремо на кожен пристрій (розділ [21](#)).

Співіснування з Wi-Fi

Найважча практична частина. ESP-NOW і Wi-Fi ділять одне радіо і **один канал**.

Коли пристрій під'єднаний до точки доступу, він працює на її каналі. Щоб ESP-NOW працював, партнери мусять бути **на тому самому каналі** — а він визначається роутером і може змінитися.

Схеми, що працюють:

Тільки ESP-NOW. Найнадійніше. Усі вузли на фіксованому каналі, Wi-Fi не використовується. Для замкненої мережі датчиків — оптимально.

Шлюз із двома ролями. Датчики працюють тільки по ESP-NOW; один вузол (шлюз) під'єднаний до Wi-Fi і приймає ESP-NOW. Шлюз мусить тримати канал ESP-NOW рівним каналу точки доступу — і, якщо роутер змінить канал, повідомити датчики або перейти сам.

Динамічне перемикання. Пристрій вимикає Wi-Fi, передає по ESP-NOW, вмикає назад. Працює, але з'їдає час і ускладнює логіку.

УВАГА

Найчастіша проблема ESP-NOW у реальних установках: **усе працювало на столі, а на об'єкті перестало**. Причина майже завжди — роутер змінив

канал (автоматичний вибір каналу увімкнений за замовчуванням у більшості роутерів), і шлюз переїхав, а датчики лишилися.

Лікування: зафіксувати канал у налаштуваннях роутера або передбачити процедуру повторного узгодження каналу.

Міст «багато датчиків → один приймач»

Найпоширеніша й найвдаліша конструкція на ESP-NOW.

Десять датчиків прокидаються за розкладом, надсилають по 20 байтів на відомий MAC і засинають. Приймач постійно живиться, слухає, накопичує і віддає далі — у Wi-Fi, MQTT (розділ 40) чи на дисплей.

Чому це працює добре:

- датчики не витрачають енергію на під'єднання;
- вихід з ладу приймача не заважає датчикам працювати (вони просто передають у порожнечу);
- додати датчик означає прописати його MAC у приймачі (пам'ятаючи про межу реєстрів вище);
- немає роутера — немає залежності від його налаштувань.

Що варто передбачити:

Лічильник у пакеті. Приймач бачить пропуски й може оцінити якість зв'язку.

Мітка часу від приймача, а не від датчика: у датчика немає точного часу (розділ 40).

Буфер у датчику. Не дійшло — спробувати ще раз наступного разу, надіславши два вимірювання.

Де ESP-NOW не виграє

Багато даних. 250 байтів на пакет і відсутність контролю потоку роблять його непридатним для потокової передачі.

Велика відстань. Дальність та сама, що у Wi-Fi. Потрібні кілометри — це LoRa (розділ 43).

Обмін із чимось, крім ESP. Протокол власний: телефон, комп'ютер чи чужий контролер його не розуміють.

Потрібна гарантована доставка. Доведеться будувати підтвердження самому.

Що з цього треба запам'ятати

Головна перевага — передача без під'єднання: мілісекунди замість секунд, два порядки економії на циклі.

250 байтів на пакет, немає гарантії доставки.

Усі учасники на одному каналі; співіснування з Wi-Fi — головна складність.

Роутер, що сам змінив канал, — найчастіша причина «працювало на столі, не працює на об'єкті».

Без шифрування це відкритий ефір; ключі зберігати в NVS, не в коді.

Конструкція «багато датчиків → один приймач» — найвдаліше застосування.

43. LoRa та інші радіо

Коли потрібні кілометри замість десятків метрів, Wi-Fi і ESP-NOW не допоможуть. LoRa розв'язує саме цю задачу — ціною швидкості.

Розділ оглядовий: що це, як під'єднати, чого чекати, і де межа.

Що таке LoRa

Спосіб модуляції, розрахований на дальність і стійкість, а не на швидкість. Сигнал «розмазується» по смузі частот так, що приймач витягує його з-під рівня шуму.

Практичний результат:

	Wi-Fi / ESP-NOW	LoRa
Дальність	десятки метрів	одиниці — десятки км
Швидкість	Мбіт/с	сотні біт/с — одиниці кбіт/с
Розмір пакета	кілобайти	десятки — сотні байтів
Час передачі пакета	мілісекунди	до секунд
Споживання при передачі	сотні мА	десятки мА

УВАГА

Головне непорозуміння з LoRa — очікування, що це «Wi-Fi на кілометри». Це радіо для **коротких повідомлень зрідка**: рівень бака, стан датчика, координата, тривога.

Передати фотографію по LoRa технічно можливо і практично безглуздо: це години ефіру. Якщо в задачі є слово «потік» — LoRa не підходить.

Модулі

SX1276 / RFM95 — класика, широко доступні, море бібліотек. RFM95 — це модуль на чипі SX1276, тому в бібліотеках вони взаємозамінні.

SX1262 — новіше покоління Semtech: те саме LoRa, менше споживання. Готові плати останніх років ідуть переважно на ньому.

RA-01, RA-02 — дешеві модулі на базі SX127x. Працюють, але якість пайки й екранування нерівна.

УВАГА

RFM69 — це не LoRa. RFM69HCW і RFM69W схожі на RFM95 назвою, ціною й виглядом, продаються в тих самих крамницях поруч на полиці — а модуляція в них інша: FSK, GFSK, MSK, GMSK і OOK. Слова «LoRa» в документації RFM69 немає жодного разу.

З RFM95 чи SX1262 він не зв'яжеться ніколи. Симптом при цьому буде рівно той самий, що при різних SF або несправній антені, — тиша. І бібліотека не підкаже: RadioLib підтримує обидва сімейства, тож код збереться й запуститься.

Ознака для крамниці: у LoRa-модулів HopeRF номер починається з RFM9 (RFM95, RFM96, RFM98). RFM69 — сусіднє сімейство для вузькосмугового FSK, і воно теж корисне, але для інших задач і без дальності LoRa.

Готові плати (LilyGO T-Beam, Heltec WiFi LoRa 32) містять ESP32, LoRa-модуль, роз'єм антени й часто дисплей та тримач акумулятора. Для початку це найшвидший шлях: нічого не паяти, розводка вже правильна.

Під'єднання — по **SPI** (розділ 36) плюс кілька додаткових ліній: скидання і D100 для сигналу про завершення прийому чи передачі.

Антенa: єдина річ, яку не можна пропустити

НЕЗВОРОТНЕ

Ніколи не вмикати LoRa-модуль без антени. Передавач без узгодженого навантаження відбиває потужність назад у вихідний каскад і **вигорає**.

Це не «може зіпсуватися» — це типовий спосіб убити модуль за секунди. Модулі приходять без антени, і спокуса «швидко перевірити» дуже велика.

Правило: антена або узгоджене навантаження **приєднані завжди**, ще до подачі живлення.

Антенa має відповідати діапазону. Модулі продаються на різні частоти (433, 868, 915 МГц), і антенa на 433 МГц із модулем на 868 МГц дає дальність у десятки метрів замість кілометрів — при формально робочій схемі.

Розміщення визначає результат більше за все інше: вище, вертикально, подалі від металу й від самої плати. Різниця між антеною, покладеною поруч на стіл, і піднятою на два метри — десятки разів за дальністю.

Дозволені діапазони й потужність

НЕЗВОРОТНЕ

Радіопередавач — це регульоване обладнання. Дозволені діапазони, максимальна потужність і допустимий коефіцієнт заповнення ефіру (duty cycle) визначаються нормами і різняться між країнами.

Модулі продаються на частоти, дозволені не скрізь. Модуль на 915 МГц — це діапазон, дозволений в інших регіонах, і його використання поза ними може бути порушенням.

Конкретні значення в цьому розділі не наводяться навмисно (P5, P13): норми змінюються, а застаріла цифра в друкованій книзі гірша за її відсутність. Актуальні дані — у датованому вкладиші радіочастотних норм, із зазначенням джерела й дати перевірки.

Перед розгортанням системи звіряйтеся з чинними нормами, а не з налаштуваннями за замовчуванням у бібліотеці.

Окремо про **duty cycle**: у частині діапазонів обмежено не лише потужність, а й частку часу, яку передавач може займати ефір. Це прямо впливає на проект: пристрій, що передає раз на 10 секунд, може виявитися поза нормою, і це виявиться після розгортання.

Point-to-point: те, що варто робити

Найпростіша й найнадійніша конструкція: два модулі, свій формат пакета, жодної інфраструктури.

Це та схема, яку варто брати за замовчуванням. Вона робить рівно те, що треба, і не залежить ні від чого зовнішнього.

Що доведеться зробити самому — того, що LoRa не дає:

Адресація. У пакеті має бути, кому він і від кого. Інакше два ваші пристрої чутимуть одне одного і сусідську систему.

Підтвердження й повтори. LoRa нічого не гарантує. Для важливих повідомлень — підтвердження від приймача і повтор при його відсутності.

Шифрування. Радіо відкрите; хто завгодно з таким самим модулем може слухати й передавати. Для будь-чого, що керує, шифрування обов'язкове (розділи 42, 50).

Захист від повторів. Записаний і відтворений пакет спрацює знову, якщо в ньому немає лічильника чи мітки часу.

Параметри й компроміс

Три налаштування визначають усе, і вони пов'язані між собою:

Spreading Factor (SF) — робочий діапазон 7...12. Більший SF означає більшу дальність і чутливість, але **різко довший** час передачі й вищу витрату енергії на пакет.

Апаратно існує ще й SF6, і бібліотеки його приймають, — але це окремий режим: він працює лише з неявним заголовком (implicit header), вимагає власних налаштувань детектування і не сумісний із рештою на рівні «просто поставили менше число». Для звичайного обміну його не беруть, тому діапазон і названо від сімки.

Bandwidth — ширша смуга означає швидше і менш далеко.

Coding Rate — надлишковість коду: більше стійкості, більше часу.

УВАГА

Обидва боки мусять мати **однакові** параметри — інакше вони не почують одне одного взагалі. Це найчастіша причина «нічого не працює» при першому запуску, і діагностується вона важко, бо симптом той самий, що при несправній антені.

Порядок дій при запуску: покласти модулі поруч на стіл (з антенами!), виставити мінімальний SF, переконатися, що обмін іде, і лише потім розносити й підвищувати SF.

Практичний підхід: почати з середніх значень, виміряти реальну дальність на об'єкті, підвищувати SF лише за потреби — кожен крок коштує часу ефіру й енергії.

LoRaWAN, оглядово

LoRaWAN — мережева надбудова над LoRa: шлюзи, сервер, ключі, автентифікація, роумінг.

Коли має сенс: пристрої розкидані по місту й є доступ до мережі (своєї або спільної); потрібна централізована робота з ключами й пристроями.

Коли не має: два-три вузли на своєму об'єкті. Тоді point-to-point простіший, надійніший і не залежить від чужої інфраструктури.

802.15.4: Thread, Zigbee, Matter

C6 **H2** Окрема гілка лінійки — чипи з апаратною підтримкою 802.15.4 (розділ 02). Це база для Zigbee, Thread і Matter.

Zigbee — сталий стандарт автоматики будівель, велика екосистема готових пристроїв.

Thread — сучасніша мережа з IPv6, самовідновлювана комірчаста топологія.

Matter — рівень сумісності поверх Thread і Wi-Fi, задуманий, щоб пристрої різних виробників працювали разом.

Спільна властивість комірчастих мереж: вузли ретранслюють одне одного, і покриття будується з кількох пристроїв замість одного потужного передавача. Для будівлі це часто краще за LoRa: менші відстані, але вищі швидкості й готова екосистема.

Обмеження — потрібен S6 або H2 і координатор мережі.

Що обрати

Задача	Радіо
Датчики в межах будівлі, є Wi-Fi	Wi-Fi або ESP-NOW
Датчики на батареях, свій парк	ESP-NOW (розділ 42)
Кілька кілометрів, короткі повідомлення	LoRa point-to-point
Місто, багато пристроїв, є шлюзи	LoRaWAN
Автоматика будівлі, готові пристрої	Zigbee / Thread C6 H2
Потік даних на відстань	жодне з цього; кабель або інша технологія

Що з цього треба запам'ятати

LoRa — це кілометри для коротких повідомлень зрідка, а не Wi-Fi на відстань.

Ніколи не вмикати передавач без антени: це типовий спосіб убити модуль.

Антенa має відповідати діапазону модуля; розміщення важить більше за все інше.

Параметри мають збігатися в обох боків, інакше не буде чути нічого.

Адресація, підтвердження, шифрування і захист від повторів — ваша робота, LoRa їх не дає.

Норми й потужності — у датованому вкладиші, і звіряти їх треба перед розгортанням.

Частина XI. Периферія

Як підключити те, чого ви раніше не бачили, і що саме варто підключати.

44. Як підключити незнайомий модуль

Універсальна процедура, яка працює з будь-яким модулем — датчиком, дисплеєм, драйвером, радіо. Порядок кроків важливіший за знання конкретного пристрою: він захищає від помилок, що коштують заліза.

Це один із найкорисніших розділів книги, бо застосовується щоразу.

Крок 1. Дізнатися, що це

На платі модуля є маркування головної мікросхеми — саме воно, а не назва в оголошенні продавця. BME280, SSD1306, MAX485, A4988.

Пошук за цим позначенням дає datasheet мікросхеми. Пошук за назвою модуля з оголошення дає сторінку магазину.

Datasheet мікросхеми — джерело істини про електрику й протокол. **Схема модуля** (якщо є) — про те, що навколо неї: стабілізатор, підтягування, конвертер рівнів.

Крок 2. Живлення: чого хоче модуль

Перше, що шукається в datasheet, — **допустима напруга живлення**.

Тут є пастка, яка коштує плат частіше за все інше. Багато модулів мають на платі власний стабілізатор і підписані VCC 5 В. Сама мікросхема при цьому працює від 3.3 В.

Три можливі випадки:

Що на платі	Живлення	Логічні рівні на виводах
Немає стабілізатора	3.3 В	3.3 В — під'єднувати прямо
Є стабілізатор 5→3.3	5 В	може бути 3.3 або 5 — перевіряти
Є стабілізатор і конвертер рівнів	5 В	5 В — потрібен конвертер

НЕЗВОРОТНЕ

Живлення і логічні рівні — різні питання. Модуль може житися від 5 В і при цьому мати виводи на 3.3 В, або житися від 3.3 В і мати виводи, що не сприймають 3.3 В як одиницю.

Не можна вважати, що коли модуль живиться від 3.3 В, то з ним усе гаразд. І навпаки: 5-вольтове живлення не завжди означає 5-вольтові сигнали.

Перевіряти треба саме **рівні на сигнальних виводах**, і саме в datasheet, а не за розташуванням написів (розділ 05).

Класичний приклад — HC-SR04: живиться від 5 В і видає **5 В на виводі ЕСНО**. Пряме під'єднання до GPIO вбиває пін. Потрібен дільник або конвертер (розділ 47).

Крок 3. Який інтерфейс

Визначається за кількістю і назвами виводів:

Виводи	Інтерфейс	Розділ
VCC, GND, SDA, SCL	I ² C	35
VCC, GND, SCK, MOSI/SDI, MISO/SDO, CS	SPI	36
VCC, GND, TX, RX	UART	34
VCC, GND, DQ/DATA	1-Wire	37
VCC, GND, один сигнал	GPIO, PWM або власний протокол	33
VCC, GND, A0/OUT аналоговий	ADC	33

Назви виводів SPI різняться в різних виробників (розділ 36). Орієнтуватися треба на функцію: DIN модуля — це його вхід, туди йде MOSI контролера.

Крок 4. Адреса або протокол

Для I²C — адреса з datasheet, і одразу сканер (розділ 35): він покаже реальну адресу, зніме питання про 7 чи 8 біт і підтвердить, що модуль узагалі живий.

Для SPI — режим (CPOL/CPHA) з datasheet. Не знайшли — перебрати чотири (розділ 36).

Для UART — швидкість. Не знайшли — перебрати стандартні.

Крок 5. Мінімальний тест окремо

УВАГА

Найважливіший крок у всьому розділі. Незнайомий модуль перевіряється на голій платі з трьома-чотирма дротами і найкоротшою можливою програмою — а не всередині проєкту з двадцятьма з'єднаннями.

Причина проста: коли не працює мінімальний тест, ви знаєте, що справа в модулі, підключенні або конфігурації. Коли не працює всередині проєкту

— підозрюваних двадцять, і серед них конфлікт пінів, брак пам'яті, конкуренція задач і ще десяток речей.

Це та економія, що вимірюється днями.

Мінімальний тест для I²C — сканер. Для SPI — прочитати регістр ідентифікації, який є майже в кожній мікросхемі (в BME280 це 0x00, що має повернути 0x60). Для UART — просто вивести все, що приходить.

Мета — **отримати від модуля хоч якусь осмислену відповідь**. Далі вже можна працювати.

Для цього кроку дуже зручний MicroPython (розділ 14): смикнути шини з консолі — три рядки замість написання, збирання й прошивання.

Крок 6. Бібліотека

Спершу шукати готову — під ESP-IDF у реєстрі компонентів, під Arduino в менеджері бібліотек (розділи 11, 12).

Що перевірити перед тим, як брати:

Дата останнього оновлення. Бібліотека, не оновлювана роками, могла не пережити перехід Arduino core 2.x → 3.x (розділ 12).

Чи є блокувальні затримки. `delay(750)` у середині — нормально для AVR і руйнівні тут (розділ 37).

Чи перевіряються помилки. Бібліотека, що мовчки ігнорує NACK на шині, дасть вам сміття замість даних і жодної підказки.

Що робити, коли бібліотеки немає

Не катастрофа. Порядок:

Знайти бібліотеку для іншої платформи. Код для Arduino AVR, STM32 чи Raspberry Pi показує послідовність команд — а це і є найцінніше. Переписати обмін по шині простіше, ніж розібратися з протоколом з нуля.

Прочитати datasheet у розділі опису регістрів. Для більшості датчиків обмін зводиться до: записати конфігурацію в один регістр, почекаати, прочитати результат із інших.

Почати з ідентифікації. Майже кожна мікросхема має регістр із відомим значенням. Прочитали правильне — обмін працює, далі справа техніки.

Мінімальний обмін вручну:

```
// написати в регістр конфігурації
uint8_t cfg[2] = { REG_CTRL, 0x27 };
i2c_master_transmit(dev, cfg, 2, pdMS_TO_TICKS(100));

// прочитати результат
uint8_t reg = REG_DATA, buf[3];
i2c_master_transmit_receive(dev, &reg, 1, buf, 3, pdMS_TO_TICKS(100));
```

Перетворення сирих даних — те, де datasheet незамінний. У багатьох датчиків є калібрувальні коефіцієнти, зашиті в самій мікросхемі, які треба прочитати й застосувати за формулою з документації. Без цього значення виглядають правдоподібно і є неправильними.

Порядок пошуку, коли не працює

1. **Живлення на модулі** — зміряти прямо на його виводах, а не на платі.
2. **Спільна земля** — прозвонити.
3. **Рівні** — чи не подано 5 В на GPIO; чи сприймає модуль 3.3 В.
4. **Підтягування** для I²C — 3.3 В на лініях у спокої (розділ 35).
5. **Пін не той** — strapping, тільки-вхідний, зайнятий флешем (картка K9).
6. **Сканер / регістр ідентифікації** — чи відповідає взагалі.
7. **Аналізатор** — що реально йде по лінії (розділ 28).

Дев'ять із десяти випадків закриваються на перших чотирьох кроках.

Що з цього треба запам'ятати

Маркування мікросхеми, а не назва в оголошенні.

Живлення і логічні рівні — різні питання; перевіряти обидва.

Мінімальний тест окремо від проєкту — крок, що економить дні.

Регістр ідентифікації — найшвидший спосіб довести, що обмін працює.

Бібліотеки немає — datasheet і код для іншої платформи розв'язують це.

Калібрувальні коефіцієнти з datasheet: без них значення виглядають правдоподібно і є неправильними.

45. Сенсори

Огляд датчиків, які реально використовуються, з чесною оцінкою кожного. Процедура підключення незнайомого — розділ 44 тут про те, що саме варто брати і чого від нього чекати.

Температура і вологість

DHT11 і DHT22. Дешеві, поширені, у кожному наборі для початківців.

ЗАКУПІВЛЯ

Чесна оцінка: **DHT22 гірший за BME280 і SHT3x майже за всіма параметрами**, а коштує не набагато менше.

Власний протокол на одному дроті з жорсткими таймінгами замість нормальної шини; опитування не частіше ніж раз на дві секунди; нестабільна точність вологості; чутливість до якості живлення. DHT11 — ще гірший: роздільність вологості цілими відсотками.

Вони згадуються тут тому, що ви їх зустрінете — у наборах, у чужих проєктах, у статтях. Для нового проєкту вибір інший.

BME280 — тиск, температура, вологість, I²C або SPI. Стандартний вибір для середовища.

BMP280 — те саме без вологості, дешевший.

SHT3x, SHT4x — точна вологість, коли саме вона головна.

DS18B20 — тільки температура, але на довгому дроті, у герметичному зонді, кілька на одній лінії (розділ 37). Незамінний для вимірювання в ґрунті, воді, на вулиці.

УВАГА

Датчик температури міряє **власну температуру**, а не температуру повітря навколо. Розміщений біля стабілізатора, всередині щільного корпусу або поруч із самим ESP32, він показує тепло від них.

Типова помилка: датчик у боксі з платою показує на 5–10 градусів більше за реальну. Лікується виносом датчика за межі корпусу — і це проєктне рішення, а не налаштування (розділ 54).

Тиск і освітленість

BMP280 / BME280 — атмосферний тиск. Роздільність достатня, щоб бачити зміну висоти на кілька метрів.

ВН1750 — освітленість у люксах по $I^{\circ}C$. Простий, лінійний, дає осмислені одиниці.

Фоторезистор — дешево і приблизно: нелінійний, розкид між екземплярами великий, одиниць немає. Годиться для «світло/темно», не для вимірювання.

Відстань і рух

HC-SR04 — ультразвуковий далекомір, 2–400 см.

НЕЗВОРОТНЕ

HC-SR04 живиться від 5 В і видає **5 В на виводі ЕСНО**. Пряме під'єднання до GPIO вбиває пін.

Потрібен дільник (10 кОм + 20 кОм) або конвертер рівнів (розділ 47).

Це, ймовірно, найчастіша конкретна причина спалених пінів серед початківців, бо модуль є в кожному наборі, а застереження — не в кожному підручнику.

Вимірювання — тривалість імпульсу, і для нього правильний інструмент PCNT або RMT (розділ 33), а не цикл із опитуванням.

Обмеження, яке вирішує, чи годиться датчик узагалі. Специфікація вимагає ціль **площею від 0.5 м²** — це квадрат приблизно 70×70 см — і поверхню якомога рівнішу. Менша ціль дає то показ, то нуль, і жодної ознаки несправності при цьому немає.

Практично це викреслює три задачі, у яких HC-SR04 беруть найчастіше: рівень води у вузькому баку, присутність людини в проході, відстань до руки чи дверцят. У всіх трьох ціль менша за 0.5 м².

Решта обмежень ультразвуку: похилі поверхні відбивають убік, м'які поглинають, показ залежить від температури повітря, конус вимірювання широкий.

VL53L0X / VL53L1X — лазерний далекомір, $I^{\circ}C$. Точніший, вузький промінь, працює на дзеркальних і м'яких поверхнях. Дорожчий; погано працює під прямим сонцем.

PIR (HC-SR501) — датчик руху. Дає простий цифровий сигнал. Особливості: потребує 30–60 секунд на прогрів після подачі живлення, реагує на рух теплого тіла (не на присутність), і має два підстроювальні резистори, які треба налаштувати руками.

Інерціальні, оглядово

MPU6050 — акселерометр і гіроскоп, $I^{\circ}C$. Дуже дешевий, дуже поширений, застарілий. Значний дрейф гіроскопа, потребує фільтрації.

BNO055 — має вбудований процесор, що сам зводить дані й видає готову орієнтацію. Дорожчий, але знімає найскладнішу частину роботи.

УВАГА

Робота з інерціальними датчиками — це не читання значень, а обробка: калібрування, злиття даних (sensor fusion), фільтрація дрейфу. Сирі показання гіроскопа безкорисні через хвилину інтегрування.

Це окрема інженерна тема, значно більша за підключення. Якщо орієнтація потрібна як частина задачі, а не як предмет дослідження, — беріть датчик, що робить це сам.

Аналогові вимірювання

Будь-який резистивний датчик читається через **дільник напруги** (розділ 05) на вхід ADC. Про обмеження ADC — розділ 33: нелінійність, потреба калібрування,

classic **S2** **S3** недоступність ADC2 при Wi-Fi.

Вимірювання напруги акумулятора — класика, у якій дільник сам розряджає акумулятор більше, ніж це робить сплячий чип. Числа й розв'язання — у розділі 33, схематехніка вимкнення дільника — у розділі 53.

Струм і напруга

INA219, INA226 — вимірювання струму й напруги по I^2C , із вбудованим шунтом і перерахунком у міліампери. Правильний вибір, коли треба міряти споживання чи заряд.

ACS712 — датчик струму на ефекті Холла, аналоговий вихід. Дає гальванічну розв'язку — може міряти струм у колі, не з'єднаному з вашою землею. Шумний, потребує усереднення, дрейфує від температури.

Струмовий шунт — резистор малого опору й вимірювання падіння на ньому. Найточніше і найдешевше, потребує підсилювача для малих струмів.

GPS/GNSS

Модулі (NEO-6M, NEO-8M і подібні) під'єднуються по UART і **самі надсилають** потік рядків NMEA — текстових речень із координатами, часом, кількістю супутників.

```
$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,...
$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,...
```

Розбір — це розбір тексту. Готові бібліотеки є, але написати самому нескладно.

Практичні особливості:

Холодний старт займає хвилини. Модуль без резервного живлення щоразу шукає супутники з нуля. Батарейка на модулі зберігає альманах і скорочує старт до секунд.

У приміщенні не працює. Потрібне небо. Це не несправність.

Точний час — безкоштовний бонус. GPS дає час високої точності без мережі, і для автономного логера це часто цінніше за координати (розділ 60).

НЕЗВОРОТНЕ

Пристрій, що визначає й передає своє розташування, — це пристрій, що розповідає про своє розташування. Разом із передавачем це створює наслідки, які варто продумати до розгортання, а не після: хто отримує дані, як вони захищені, що станеться, якщо їх перехоплять.

Технічний бік цього — розділ 50 рішення про доцільність — ваше.

Загальні правила

Датчик міряє себе. Розміщення визначає результат більше за модель.

Усереднення майже завжди потрібне. 16–64 відліки для аналогових.

Перевіряти на осмисленість. Значення поза фізично можливим діапазоном означає збій зв'язку, а не екстремальні умови. -127°C від DS18B20 — це код помилки (розділ 37).

Калібрувати за відомим. Порівняти з повіреним приладом або з очевидною точкою: танення льоду — це 0°C .

Датчик виходить з ладу мовчки. Він продовжує віддавати значення — просто неправильні. Виріб має розрізняти «датчик каже 25» і «датчик завис на 25»: слідкувати за тим, що значення взагалі змінюються (розділ 32).

Що з цього треба запам'ятати

DHT22 гірший за BME280 майже в усьому; для нового проекту вибір інший.

HC-SR04 видає 5 В на ЕСН0 — дільник обов'язковий.

Датчик температури міряє власну температуру; розміщення — проектне рішення.

Дільник для вимірювання акумулятора сам його розряджає (розділ 33).

Значення поза фізичним діапазоном — це збій зв'язку, а не показання.

Датчик виходить з ладу мовчки: слідкувати, чи значення змінюються.

46. Дисплеї і введення

Дисплей перетворює пристрій із «чорної коробки, яку треба опитувати» на щось, що показує свій стан саме там, де стоїть. Для польового приладу це часто важливіше за мережу.

Вибір типу

Тип	Розмір	Інтерфейс	Кольори	Особливості
OLED SSD1306	0.96–1.3"	I ² C або SPI	моно	контраст, малий, дешевий
OLED SH1106	1.3"	I ² C	моно	схожий на SSD1306, зсув на 2 пікселі
TFT ST7789	1.3–2.4"	SPI	65 тис. ¹	яскравий, швидкий
TFT ILI9341	2.4–3.2"	SPI	65 тис. ¹	великий, класика
E-paper	1.5–7.5"	SPI	моно / 3	тримає зображення без живлення

¹ Це формат пікселя RGB565, а не стея контролера: ILI9341 уміє 262 тис. кольорів, і формат перемикається командою COLMOD (3Ah). RGB565 стоїть у таблиці тому, що його дають типово TFT_eSPI і LovyanGFX: удвічі менше байтів на піксель — удвічі швидший кадр. Для більшості задач книги це правильний вибір, але це **вибір**, а не межа заліза.

OLED SSD1306 — типовий вибір для службової інформації. Дешевий, читається під будь-яким кутом, працює по двох дротах I²C.

УВАГА

SH1106 продається під виглядом SSD1306 і майже сумісний — але має внутрішню пам'ять ширшу за екран, через що зображення зсувається на два пікселі й з'являється смужка збоку.

Симптом «купив SSD1306, зображення зсунуте» означає, що це SH1106. Більшість бібліотек мають окремий режим — треба лише його ввімкнути.

E-paper заслуговує окремої згадки для автономних пристроїв: він споживає енергію **тільки під час оновлення**, а далі тримає зображення без живлення взагалі. Для датчика на батарейці, який показує значення раз на годину, це ідеально.

Ціна — оновлення займає секунди й помітно блимає, а часткове оновлення підтримують не всі моделі.

Живлення і рівні

ЖИВЛЕННЯ

Кольоровий TFT із підсвіткою споживає десятки міліампер постійно — більше, ніж сам ESP32 у режимі очікування. Для пристрою на батарейці це головна стаття витрат, і керування підсвіткою через транзистор або PWM — не оптимізація, а необхідність (розділ 06).

Логічні рівні перевіряти обов’язково (розділ 44): частина модулів дисплеїв розрахована на 5 В, частина на 3.3 В, і зустрічаються обидві.

Бібліотеки

U8g2 — монохромні дисплеї. Підтримує майже все, що існує, має багато шрифтів, у тому числі з кирилицею. Працює і в Arduino, і в ESP-IDF.

TFT_eSPI — кольорові TFT, дуже швидка. Особливість, що дивує: конфігурація (модель дисплея, піни) задається **правкою файлу всередині самої бібліотеки**, а не в проєкті. Це незручно і ламається при оновленні бібліотеки; варто одразу зафіксувати конфігурацію в проєкті.

LovyanGFX — сучасніша альтернатива TFT_eSPI, конфігурується нормально, з коду.

LVGL — не драйвер, а повноцінна графічна бібліотека: кнопки, списки, графіки, анімація, обробка дотиків.

УВАГА

LVGL дає красиві інтерфейси і коштує ресурсів. Потрібен кадровий буфер: для 320×240 у 16 бітах це 150 КБ (320 × 240 × 2 байти) — на С3 з його 400 КБ SRAM (розділ 02) це порівнянно з усією купою, що лишається після Wi-Fi, тобто повний буфер туди просто не поміщається.

Практично LVGL — це S3 із PSRAM або дисплей невеликого розміру. На класичному ESP32 без PSRAM він працює з частковими буферами й обмеженнями.

Для показу трьох чисел і графіка LVGL надлишковий: U8g2 або пряме малювання зроблять те саме без витрат.

Кирилиця

Стандартні шрифти бібліотек часто містять лише латиницю. Українські літери перетворюються на порожні прямокутники.

Що з цим робити:

U8g2 має вбудовані шрифти з кирилицею — треба обрати правильний (у назві є *cyrillic*).

Для **TFT** шрифт із кирилицею зазвичай генерують окремо і вбудовують у прошивку.

Кодування. Рядки в коді — UTF-8; бібліотека має вміти його читати. Частина старих бібліотек очікує однобайтове кодування, і тоді українські літери, що займають два байти, ламають вивід.

Перевірка проста: вивести рядок Гг Єє Іі Ії і подивитися, що з'явилося.

Кнопки

Механічний контакт дає дребезг — десятки перемикань за мілісекунди. Анти-дребезг робиться порівнянням часу, ніколи не затримкою в обробнику переривання (розділ 33).

Три речі, які варто передбачити в будь-якому виробі з кнопками:

Довге натискання як окрема подія. Це найзручніший спосіб дати доступ до службових функцій, не додаючи кнопок.

Скидання налаштувань довгим утриманням (розділ 39). Без нього пристрій, що переїхав в іншу мережу, стає непридатним.

Підтягування. Кнопка на землю плюс `pull_up_en`. classic Пам'ятати про піни 34–39, у яких вбудованого підтягування немає (розділ 07).

Енкодери

Обертовий енкодер дає два сигнали зі зсувом фаз — за їхнім порядком визначається напрямок обертання.

Правильний спосіб читання — **PCNT** (розділ 33): апаратний лічильник із вбудованим фільтром дребезгу. Читання енкодера в коді через переривання працює, але з'їдає час і збивається при швидкому обертанні.

Більшість енкодерів має ще й кнопку на валу — зручно для «обрати значення і підтвердити».

Матричні клавіатури

Клавіатура 4×4 — вісім пінів на шістнадцять кнопок: рядки й стовпці опитуються по черзі.

Вісім пінів — це багато (розділ 07). Альтернативи: розширювач портів по I²C (PCF8574), аналогова клавіатура з резистивним дільником на один пін ADC, або сенсорні кнопки на вбудованому Touch classic S3.

УВАГА

Дешеві плівкові матричні клавіатури мають ненадійні контакти, і дребезг у них значно гірший за кнопковий. Якщо введення критичне, варто розглянути окремі кнопки або енкодер.

Скільки часу займає малювання

Оновлення дисплея — повільна операція. Кольоровий TFT 320×240 по SPI на 40 МГц оновлюється десятки мілісекунд; по I²C монохромний — теж помітно.

Це означає, що **малювання не можна робити в задачі з жорсткими таймінгами**. Правильна схема: окрема задача малювання нижчого пріоритету, яка бере дані з черги (розділ 31).

І окремо: **не перемальовувати весь екран, коли змінилося одне число**. Часткове оновлення на порядок швидше.

Що з цього треба запам'ятати

Зсунуте зображення на «SSD1306» означає, що це SH1106.

Е-рарег тримає зображення без живлення — для автономних пристроїв це вирішальна властивість.

Підсвітка TFT споживає більше, ніж сам ESP32 в очікуванні.

LVGL потребує кадрового буфера: практично це S3 із PSRAM.

Кирилицю перевіряти рядком `Гг Єє Іі Її` до того, як писати інтерфейс.

Скидання налаштувань довгим натисканням — обов'язкове.

Малювання — окрема задача низького пріоритету; оновлювати частково.

47. Ключі, реле, узгодження рівнів

Пін ESP32 віддає до 40 мА на максимальній силі драйвера — і це **типове значення, а не гранично допустиме** за замовчуванням драйвер стоїть на середній силі, тобто менше, а при кількох активних пінах домену віддача просідає приблизно до 29 мА (розділ 05). Усе, що споживає більше, вмикається через щось. Цей розділ — про це «щось» і про безпечне з’єднання світів із різними напругами.

Що чим комутувати

Навантаження	Чим	Чому
Світлодіод, до 10 мА	напрямую через резистор	вистачає піна
Стрічка, двигун, до кількох А, спільна земля	MOSFET	дешево, тихо, швидко
Мережеве живлення, змінний струм	реле або симістор	потрібна ізоляція
Мала потужність, треба ізоляція	оптопара	розв’язка без механіки
Часте перемикання, ШІМ	MOSFET	реле не витримає ресурсу

MOSFET як ключ

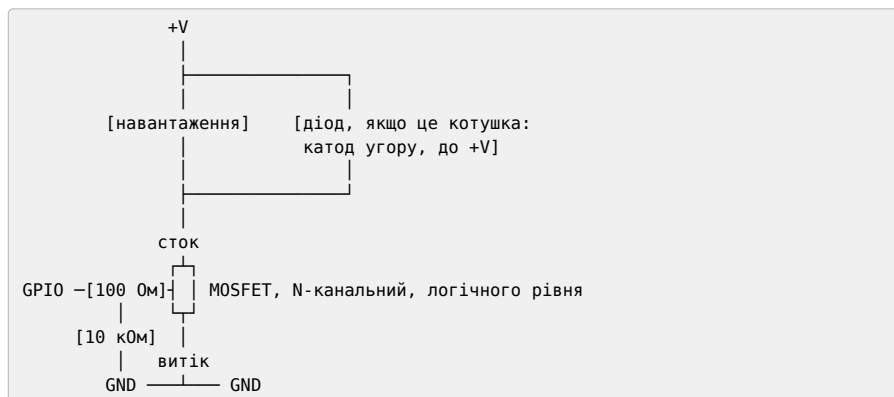
НЕЗВОРОТНЕ

Потрібен **N-канальний MOSFET логічного рівня** (logic level). Ключове слово — «логічного рівня»: такий повністю відкривається від 3.3 В.

Звичайний MOSFET розрахований на 10 В на затворі. Від 3.3 В він відкривається **частково**, має підвищений опір, гріється і згорає — не одразу, а через хвилини або години роботи.

Симптом: схема працює, потім транзистор стає гарячим, потім перестає вимикатися (пробій). Причина не в схемі, а в тому, який саме транзистор поставили.

Базова схема low-side (навантаження зверху, ключ знизу):



Читається це так: навантаження висить між +V і стоком, ключ замикає його на землю. Діод, якщо навантаження індуктивне, стоїть **паралельно самому навантаженню**, катодом до +V — тобто в нормальній роботі закритий, а викид при вимиканні пропускає по колу навантаження, минаючи транзистор.

Два резистори, які виглядають зайвими і не є ними:

100–220 Ом послідовно з затвором обмежує кидок струму в момент перемикання. Затвор MOSFET — це ємність, і без резистора вона заряджається струмом, що перевищує можливості піна.

10 кОм із затвора на землю тримає ключ закритим, коли GPIO ще не налаштований.

НЕЗВОРОТНЕ

Другий резистор — це не дрібниця, а безпека. Під час завантаження чипа (розділ 16) GPIO перебуває у високоімпедансному стані: він не тримає ні нуля, ні одиниці. Затвор без підтягування «висить» і може відкрити ключ.

Для світлодіода це блимання при старті. Для нагрівача, двигуна чи насоса — увімкнення на секунду при кожному перезавантаженні, включно з тими, що відбуваються в boot loop (розділ 32).

Питання, яке ставиться до кожного виходу: що станеться, якщо чип зникне або перезавантажиться просто зараз?

High-side комутація (ключ зверху, навантаження на землю) складніша: потрібен P-канальний транзистор або драйвер із підвищувальним живленням. Якщо є вибір — робіть low-side.

Захисний діод

НЕЗВОРОТНЕ

Будь-яке індуктивне навантаження — реле, двигун, електромагнітний клапан — потребує захисного діода.

Котушка при вимиканні струму створює викид напруги зворотної полярності в сотні вольтів. Він пробиває транзистор — іноді одразу, іноді після сотні спрацювань.

Діод (1N4007 або швидкий на кшталт 1N4148 для малих струмів) ставиться **паралельно котушці, у зворотному напрямку**: катодом (смуга) до плюса живлення. У нормальній роботі він закритий і не заважає.

Готові релейні модулі зазвичай мають діод на платі. Саморобна схема з реле чи клапаном — ні.

Реле

Механічне реле — контакти, які фізично замикаються. Комутує що завгодно, включно з мережевим живленням, дає повну ізоляцію.

Ціна: обмежений ресурс (десятки-сотні тисяч спрацювань), клацання, повільність (одиниці мілісекунд), і котушка, яка споживає десятки міліампер постійно, поки реле ввімкнене.

Готові релейні модулі — найзручніший варіант. На платі вже є транзистор, діод і часто оптопара.

Дві речі, які треба перевірити перед підключенням:

Живлення котушки. Більшість модулів розраховані на **5 В**. Від 3.3 В реле може не спрацювати або спрацьовувати нестабільно — класичний симптом «іноді вмикається».

Логіка керування. Багато модулів мають **інверсну** логіку: реле вмикається логічним **нулем** на вході. Це означає, що під час завантаження, поки пін висить, реле може бути ввімкнене. Перевіряти обов'язково.

ЖИВЛЕННЯ

Живити котушку реле від піна 3V3 плати не можна: десятки міліампер на котушку в момент спрацювання просаджують живлення й дають brownout (розділ 06). Реле живиться від того самого джерела, що й плата, або від окремого — зі спільною землею.

Твердотільне реле (SSR) — без механіки, безшумне, ресурс необмежений, швидке. Дорожче; для змінного струму вмикається в момент переходу через нуль, що добре; гріється під навантаженням і потребує радіатора.

НЕЗВОРОТНЕ

Робота з мережевим живленням 230 В — окрема тема з окремими правилами безпеки, які виходять за межі цієї книги. Тут ідеться лише про те, як подати сигнал керування з мікроконтролера.

Монтаж силової частини, ізоляція, запобіжники, розділення низьковольтної й високовольтної частин на платі — це те, чого не можна робити «на макетці, щоб перевірити». Якщо ви не впевнені — не вмикайте в розетку.

Оптопара

Оптопара (РС817 та подібні) передає сигнал світлом: світлодіод з одного боку, фототранзистор з іншого. Електричного з'єднання між сторонами немає.

Навіщо це потрібно:

Різні землі. Пристрої, чиї землі не можна з'єднувати — наприклад, через різницю потенціалів або з міркувань безпеки.

Захист. Пробій на боці навантаження не доходить до мікроконтролера.

Завади. Довга лінія до датчика в промисловому середовищі.

Обмеження: швидкість. Дешеві оптопари працюють до десятків кілогерц — для сигналу керування досить, для швидкої шини ні.

Узгодження логічних рівнів

Найчастіша задача — з'єднати ESP32 (3.3 В) із пристроєм на 5 В (розділ 05).

Напрямок 3.3 → 5 В. Часто працює без нічого: більшість 5-вольтових входів сприймає 3.3 В як одиницю. Не завжди — звернутися з datasheet.

Напрямок 5 → 3.3 В. Потрібне зниження **обов'язково**.

Дільник напруги — два резистори (розділ 05). Найдешевше. Обмеження: тільки однонаправлені й повільні сигнали. Для SPI на кількох мегагерцях дільник завалює фронти; для I²C він не працює в принципі, бо шина двонаправлена.

Модуль конвертера рівнів на польових транзисторах — правильне рішення. Двонаправлений, працює з I²C, коштує копійки. Чотири-вісім каналів на платі.

Схема підключення: сторона LV до 3.3 В, сторона HV до 5 В, землі з'єднані, сигнали через відповідні канали.

ЗАКУПІВЛЯ

Пара модулів конвертера рівнів у шухляді — те, що варто мати завжди. Вони дешеві, а потреба виникає раптово: купили датчик, а він на 5 В.

Альтернатива «спробую напряму, може обійдеться» коштує піна, а іноді плати.

Спільна земля

НЕЗВОРОТНЕ

Усі з'єднані пристрої мусять мати спільну землю — крім тих випадків, коли між ними стоїть гальванічна розв'язка (оптопара, ізолюваний трансивер).

Це найчастіша помилка при живленні від окремого джерела: плюс подали, землю з'єднати забули. Симптом — пристрої живляться, обміну немає, або обмін із випадковими помилками (розділ 29).

Зворотна помилка теж буває: з'єднати землі там, де стоїть розв'язка саме для того, щоб їх не з'єднувати. Тоді ізоляція перестає існувати, а схема виглядає працездатною.

Що з цього треба запам'ятати

MOSFET має бути логічного рівня, інакше він гріється і згорає.

Резистор 10 кОм із затвора на землю не дає навантаженню ввімкнутися під час завантаження чипа.

Будь-яка котушка — з діодом.

Релейні модулі часто на 5 В і часто з інверсною логікою; перевіряти обидва.

Дільник не годиться для I²C і для швидкого SPI; конвертер рівнів годиться.

Спільна земля — крім тих місць, де стоїть розв'язка.

48. Двигуни й сервоприводи

Двигун — це індуктивність, великий струм і завади. Три причини, через які керування двигуном ламає те, що поруч, частіше за будь-яку іншу периферію.

Загальні правила, які стосуються всіх типів

ЖИВЛЕННЯ

Двигун живиться від окремого джерела. Не від піна 5V плати, не від USB. Пусковий струм двигуна в кілька разів перевищує робочий, і навіть маленький двигун у момент рушання просаджує живлення настільки, що ESP32 перезавантажується по brownout (розділ 06).

Земля спільна між джерелом двигуна і платою — інакше сигнали керування не мають точки відліку (розділ 47).

Конденсатор великої ємності поруч із драйвером двигуна — обов'язково. Він гасить кидки, що інакше йдуть у все живлення.

Це три речі, які пояснюють переважну більшість «пристрій перезавантажується, коли вмикається двигун».

Колекторні двигуни постійного струму

Найпростіші: подали напругу — крутиться, поміняли полярність — крутиться назад. Швидкість задається ШІМ.

Драйвер потрібен завжди — напругу з піна двигун не вмикається.

DRV8833 — сучасний, компактний, два канали, вбудований захист від перегріву й перевантаження. Падіння напруги мале. Робочий вибір за замовчуванням для малих двигунів.

TB6612FNG — схожий, трохи потужніший, теж хороший вибір.

L298N — той, що продається всюди.

ЗАКУПІВЛЯ

L298N застарів на десятиліття, але лишається найпоширенішим. Він зроблений на біполярних транзисторах і втрачає на собі приблизно два вольти. Практично це означає: подали 12 В — двигун отримав 10, а різниця пішла в нагрів масивного радіатора, який стоїть на платі саме тому.

Для живлення від акумулятора це прямі втрати ємності. Для 5-вольтового живлення — часто взагалі непрацездатно: після падіння лишається 3 В.

L298N згадується тут тому, що ви його зустрінете в наборах, старих проектах і на кожному маркетплейсі. Для нового проекту беріть DRV8833 або TB6612: дешевше, менше, ефективніше.

Керування — два піни на канал: напрямок і ШІМ, або два ШІМ. Використовується LEDC (розділ 33) або, для серйозних задач, **MCSPWM** (розділ 04): він дає мертвий час між плечима моста й апаратне аварійне вимкнення. Для силового каскаду це захист від наскрізного струму, а не зручність.

УВАГА

Двигун постійного струму на низькій швидкості може не рушити: моменту не вистачає, щоб подолати тертя спокою. Практичний прийом — короткий імпульс повної потужності на старті, далі перехід на робочий коефіцієнт заповнення.

Крокові двигуни

Крокують на фіксований кут за імпульс. Знають своє положення без датчика — поки не пропустили крок.

A4988 і DRV8825 — драйвери для біполярних двигунів (NEMA17 і подібних). Керування мінімальне: пін STEP (імпульс = крок) і пін DIR (напрямок).

НЕЗВОРОТНЕ

Обмеження струму на драйвері треба виставити до першого запуску.

На платі є підстроювальний резистор; його положення задає струм через обмотки.

Не виставили — двигун і драйвер перегріваються за хвилини. Виставили завелике — те саме. Виставили замале — двигун пропускає кроки.

Процедура: виміряти напругу на контрольній точці драйвера відносно землі й підлаштувати відповідно до формули з datasheet конкретного драйвера й струму вашого двигуна.

І окремо: **не від'єднувати двигун від драйвера при увімкненому живленні** — це типовий спосіб убити драйвер.

Мікрокрок — драйвер ділить крок на частини (1/2, 1/4, ..., 1/32), що дає плавніший рух і менше шуму. Задається перемичками на платі.

28BYJ-48 з ULN2003 — дешевий уніполярний двигун із редуктором у комплекті, часто в наборах. Слабкий і повільний, але для шторки, заслінки чи стрілки індикатора цілком достатній. Керування — чотири піни послідовністю.

Генерація кроків. Точні імпульси з рівними інтервалами — це задача для апаратури, а не для циклу з затримками. Правильні інструменти: RMT (розділ 33), апаратний таймер або LEDC у режимі імпульсів. Цикл із `vTaskDelay` дає рвані кроки, щойно система зайнята чимось іншим.

Сервоприводи

Серво приймає імпульси **50 Гц**: тривалість близько 1 мс — один крайній кут, близько 2 мс — інший, 1.5 мс — середина.

ЖИВЛЕННЯ

Серво живиться окремо. Це не рекомендація.

Навіть маленьке SG90 в момент рушання бере сотні міліампер, а під навантаженням — більше. Бортовий стабілізатор плати цього не витримує: пристрій перезавантажується саме тоді, коли механізм починає рух (розділ 06).

Схема: серво живиться від окремого джерела 5 В, сигнальний провід іде на GPIO, **земля спільна**. Три дроти серво — це живлення, земля і сигнал; сигнальний зазвичай помаранчевий або жовтий.

Керування через LEDC (розділ 33). Практичні особливості:

Реальні межі відрізняються. Заявлені 1–2 мс на конкретному екземплярі можуть бути 0.6–2.4 мс. Крайні положення варто визначити дослідно й обмежити в коді, інакше серво впирається в упор і гріється.

Дрижання в спокої. Серво постійно підправляє положення. Лікується вимиканням сигналу після досягнення потрібного кута — більшість серво при цьому лишається на місці.

Плавність. Різка зміна кута — це ривок і кидок струму. Плавний рух робиться поступовою зміною положення невеликими кроками.

Безколекторні (BLDC) і ESC

Потребують окремого контролера (ESC), який приймає той самий сигнал, що й серво. З боку ESP32 керування виглядає так само.

НЕЗВОРОТНЕ

BLDC-двигун із гвинтом — це небезпечний механізм. Він набирає обертів миттєво, і помилка в коді або перезавантаження чипа під час роботи означає неконтрольований рух.

Мінімальні правила: випробування **без гвинта** ESC із процедурою арму, що не дає запуститися випадково; апаратний вимикач живлення силової частини в межах руки; безпечний стан за замовчуванням (розділ 32).

Зворотний зв'язок

Двигун без датчика не знає свого положення. Кроковий знає, поки не пропустить крок — а пропуск нічим не виявляється.

Для точного позиціювання потрібен енкодер (розділ 46) або кінцевий вимикач. Мінімум для механізму з обмеженим ходом — **кінцевий вимикач у крайніх положеннях**: він рятує механіку, коли логіка помилилася.

Завади

Колекторні двигуни іскрять на щітках і створюють завади в широкому спектрі. Наслідки: збої І²С, помилки на шинах, а іноді перезавантаження.

Що допомагає:

- **конденсатори 100 нФ** на виводах двигуна і між кожним виводом і корпусом;
- **фізичне розділення**: силові дроти окремо від сигнальних, не в одному джгуті;
- **скручені пари** для силових проводів;
- **окреме джерело** й розв'язка по живленню.

Що з цього треба запам'ятати

Двигун і серво живляться окремо; земля спільна; конденсатор поруч із драйвером.

L298N втрачає близько двох вольтів на собі — для нового проєкту беріть DRV8833 або TB6612.

Обмеження струму на кроковому драйвері виставляється **до** першого запуску.

Не від'єднувати кроковий двигун при увімкненому живленні.

Кроки генерувати апаратно, а не циклом із затримками.

Реальні межі серво визначати дослідно й обмежувати в коді.

Кінцевий вимикач рятує механіку, коли логіка помилилася.

49. Карти пам'яті, камери, звук

Три теми, об'єднані спільною рисою: усі три впираються в пам'ять і в пропускну здатність, і саме тому на різних чипах поводяться по-різному.

microSD

Два способи під'єднання, і різниця між ними велика.

SPI — чотири лінії (розділ 36). Працює на будь-якому чипі, на будь-яких пінах, повільно (одиниці мегабіт).

SDMMC — рідний інтерфейс карток, одна або чотири лінії даних. Помітно швидший, але вимагає конкретних пінів і є не на всіх чипах.

Для конфігурації, логу раз на хвилину чи невеликих файлів SPI достатньо з запасом. Для запису відео чи звуку — потрібен SDMMC.

ЖИВЛЕННЯ

Картка споживає помітний струм при записі — десятки, іноді сотні міліампер сплесками. На платі з бортовим стабілізатором це може дати brownout саме в момент запису (розділ 06).

Конденсатор поруч із роз'ємом картки — обов'язковий, а не бажаний.

Надійний запис

НЕЗВОРОТНЕ

Файлова система FAT на картці не переживає зникнення живлення посеред запису. Це не дефект реалізації, а властивість FAT: пошкоджується не лише останній файл, а таблиця розміщення — тобто картка цілком.

Пристрій, що пише в картку і живиться від нестабільного джерела, рано чи пізно втратить усі дані, а не останній запис.

Що з цим роблять:

Закривати файл після кожного запису. Повільніше, але дані потрапляють на носій, а не лишаються в буфері.

Писати короткими сесіями: відкрив, дописав рядок, закрив.

Резервне живлення — конденсатор великої ємності або невеликий акумулятор, якого вистачає, щоб коректно завершити запис при зникненні основного (розділ 53).

Дублювати критичне в NVS: останній стан у NVS, історія в картку.

Не тримати файл відкритим годинами. Це найпоширеніша помилка логера.

Практичні дрібниці

Картки великого обсягу форматуються в exFAT, який підтримується не завжди. Для сумісності — FAT32, і обсяг до 32 ГБ.

Дешеві картки з маркетплейсів часто мають менший реальний обсяг, ніж заявлений, і виходять з ладу від інтенсивного запису швидко. Для логера, що пише постійно, варто брати картки, розраховані на запис (промислові або відеонаглядові).

ESP32-CAM

Найдешевший спосіб отримати мережеву камеру, і найнезручніша плата в книзі (розділ 08).

Що з нею не так:

- **немає USB-роз'єму** — потрібен окремий USB-UART-перехідник;
- GPIO0 замикається на землю **перемичкою вручну** для входу в download mode (картка K4);
- камера займає більшість пінів, вільних лишається кілька;
- бортовий стабілізатор слабкий.

ЖИВЛЕННЯ

ESP32-CAM дуже чутлива до живлення. Класичний симптом — камера ініціалізується, а при першому кадрі плата перезавантажується, або в лозі з'являється Camera probe failed.

Причина майже завжди — живлення: USB-UART-перехідник не тягне пікового струму. Живити треба від окремого джерела на 5 В через пін 5V, а не від перехідника.

OV2640 — типовий модуль камери. Роздільність до 2 Мп, апаратне стиснення в JPEG.

MJPEG-потік — найпростіший спосіб віддати відео: послідовність JPEG-кадрів через HTTP. Браузер показує це без плагінів.

Межі FPS. Визначаються не процесором, а пам'яттю й каналом. На класичному ESP32 з PSRAM реалістично: висока роздільність — одиниці кадрів на секунду; низька — десятки.

Без PSRAM камера обмежена дуже маленькими кадрами: буфер просто не може тримати (розділ 03).

S3 На S3 ситуація краща: більше пам'яті, швидший інтерфейс, є апаратні інструкції для обробки. Але фізика та сама — Wi-Fi лишається вузьким місцем.

УВАГА

Очікування «зроблю відеоспостереження на ESP32-CAM» варто одразу привести до реальності: це камера для періодичних знімків і короткого перегляду, а не для цілодобового потоку в високій якості. Для останнього беруть готові IP-камери.

Де ESP32-CAM справді хороша: зробити знімок за подією (спрацював PIR), надіслати, заснути. Тут вона незамінна за ціною.

Звук: I²S

I²S — цифровий інтерфейс звуку (розділ 04). Три-чотири лінії, працює з мікрофонами й підсилювачами напругу, без аналогових каскадів.

INMP441 — цифровий MEMS-мікрофон по I²S. Віддає готові відліки, не потребує підсилювача й аналогової обв'язки. Для запису звуку, вимірювання рівня шуму чи виявлення звукових подій — правильний вибір.

MAX98357A — цифровий підсилювач класу D по I²S. Приймає потік і керує динаміком напругу.

ЖИВЛЕННЯ

Підсилювач під навантаженням споживає відчутно, і сплески збігаються з гучними звуками. Живлення окреме або з великим конденсатором — інакше голосний звук дає просадку і перезавантаження.

Що реалістично на ESP32: відтворення заздалегідь підготовлених звуків, простих мелодій, голосових повідомлень із флешу чи картки; запис звуку; вимірювання рівня; виявлення простих звукових подій.

Що нереалістично: якісна обробка звуку в реальному часі, розпізнавання мовлення на пристрої (для складних моделей бракує ресурсів навіть на S3), багатоканальне зведення.

Аналоговий шлях — мікрофон із підсилювачем на вхід ADC — теж працює, але дає гірший результат: ADC шумний і нелінійний (розділ 33). Цифровий мікрофон краще майже завжди.

Спільне: усе це впирається в пам'ять

Три теми розділу об'єднує одне: буфери.

Кадр камери, блок для запису на картку, буфер звуку — усе це десятки й сотні кілобайтів, і виділяти їх треба один раз при старті, а не в циклі (розділ 30).

PSRAM вирішує. Для камери вона обов'язкова; для звуку й картки бажана. Чипи без PSRAM — C3, C6, H2 (розділ 02) — для цих задач підходять погано або не підходять зовсім.

DMA обов'язковий. Передавати такі обсяги через процесор означає з'їсти ядро повністю. Буфери для DMA виділяються через `heap_caps_malloc` із `MALLOC_CAP_DMA` (розділ 36).

Що з цього треба запам'ятати

FAT на картці не переживає зникнення живлення посеред запису — псується не файл, а картка цілком.

Не тримати файл відкритим годинами; закривати після кожного запису.

ESP32-CAM потребує окремого живлення 5 В; `Camera probe failed` — це майже завжди живлення.

Камера без PSRAM практично непрацездатна.

ESP32-CAM — для знімків за подією, а не для цілодобового потоку.

Цифровий мікрофон по I²S краще за аналоговий через ADC майже завжди.

Буфери виділяти один раз при старті; для DMA — з правильними властивостями.

Частина XII. Безпека

Кілька дешевих рішень, кожне з яких закриває реальний сценарій, і два незворотні, які варто розуміти до, а не після.

50. Безпека пристрою

Розділ про те, як не зробити пристрій, який працює проти власника. Він не про криптографію — про кілька рішень, кожне з яких коштує небагато й закриває реальні сценарії.

Головне питання, яке варто ставити до кожної функції пристрою: **що станеться, якщо це зробить не той, хто має право?**

Модель загроз: із чого починати

Перш ніж вирішувати, як захищати, треба вирішити **від чого**. Це не формальність — від відповіді залежить усе інше.

Хто має доступ до ефіру. Wi-Fi, BLE і LoRa — це радіо. Сусід, людина на вулиці, будь-хто з таким самим модулем. Для BLE і ESP-NOW достатньо бути поруч (розділи 41, 42).

Хто має доступ до мережі. Пристрій у спільній мережі доступний усім, хто в ній.

Хто має фізичний доступ. Це визначає найбільше. Пристрій, який можна взяти в руки, віддає весь свій вміст (розділ 24) — питання лише часу.

Що станеться, якщо пристрій захоплять. Чи компрометує це решту системи? Чи є в ньому ключі, спільні для всіх пристроїв?

Останнє питання найважливіше і найчастіше пропущене.

Креденшели

НЕЗВОРОТНЕ

Паролі, ключі й токени, зашиті в код, дістаються з дампа прошивки за п'ять хвилин. Не «складно» — за п'ять хвилин командою strings (розділ 24).

Це не теоретичний ризик: саме так знаходять паролі до Wi-Fi, ключі API і паролі адміністратора в серійних виробках.

Місце credenшелів — **NVS** (розділ 18), записаний окремо на кожен пристрій при виробництві (розділ 21).

НЕЗВОРОТНЕ

Один ключ на всі пристрої означає, що компрометація одного компрометує всі. Хтось узяв один датчик, витяг ключ — і тепер може видавати себе за будь-який пристрій у системі.

Унікальний ключ на екземпляр коштує одного кроку на виробництві й перетворює катастрофу на локальний інцидент.

Дефолтні паролі — не варіант. Пристрій, що піднімає точку доступу з паролем 12345678 або веб-інтерфейс без пароля, доступний усім у радіусі дії. Мінімум: пароль, унікальний для екземпляра, надрукований на наліпці (і змінюваний).

Шифрування NVS

NVS за замовчуванням лежить у флеші **відкритим текстом**. Той, хто зняв дамп, читає збережені паролі так само легко, як зашиті в код.

ESP-IDF підтримує шифрування NVS: ключі зберігаються в окремому розділі, а сам розділ захищається Flash Encryption. Вмикається в `menuconfig`.

Важливо розуміти межу: шифрування NVS має сенс **разом із** Flash Encryption. Без неї ключ шифрування NVS лежить у тому самому флеші.

TLS: мінімум, що працює

Для пристрою, що спілкується з сервером (розділи 19, 40):

Перевіряти сертифікат сервера. Без перевірки TLS дає шифрування без автентифікації — захист від підслуховування без захисту від підміни. Це половина захисту, яка створює відчуття повного.

Зашивати сертифікат СА, а не сервера. Сертифікат сервера протермінується, і всі пристрої одночасно втратять зв'язок. Сертифікат центру сертифікації живе значно довше.

Не вимикати перевірку «тимчасово». Тимчасове доїжджає до замовника.

Слідкувати за терміном. Навіть СА колись протермінується. Пристрій, що має жити роками, потребує механізму оновлення довірених сертифікатів — через OTA.

Оновлення

OTA — це канал, яким у пристрій потрапляє код. Незахищений OTA означає, що будь-хто може залити свою прошивку.

Мінімум: HTTPS із перевіркою сервера (розділ 19). Наступний рівень: перевірка підпису самого образу — тоді навіть скомпрометований сервер не залиє чуже.

Flash Encryption і Secure Boot

НЕЗВОРОТНЕ

Це найпотужніші механізми платформи і **єдині незворотні** (розділ 20).

Flash Encryption шифрує вміст флешу ключем, що не покидає чип. Дамп, знятий після ввімкнення, марний: він зашифрований ключем, якого ви не маєте.

Secure Boot змушує чип перевіряти підпис прошивки перед запуском. Чип перестає приймати непідписані образи назавжди.

Обидва реалізовані через **eFuse** — біти, які пропалюються фізично й не скидаються ніколи, жодною командою, жодним програматором.

Наслідки, які треба усвідомити до, а не після:

- дамп, знятий **до** ввімкнення, ще можна залити назад; знятий після — ні;
- помилка в підготовці ключів означає пристрій, який неможливо пере-прошити;
- налагодження ускладнюються: JTAG може бути вимкнений;
- у release-режимі зворотного шляху немає взагалі.

Пробувати це можна **тільки на платі, яку не шкода**. Ніколи — на робочому пристрої, ніколи — «щоб подивитися, що буде».

Коли це справді потрібно: серійний виріб, що потрапляє до недоброзичливця; захист інтелектуальної власності в прошивці; відповідальність за те, що на пристрої не з'явиться чужий код.

Коли не потрібно: прототип, одиничний пристрій, навчальний проект, пристрій під вашим контролем.

Проміжний варіант — увімкнути в **development-режимі**, який лишає можливість перепрошивки. Це дозволяє перевірити, що все працює, до незворотного кроку.

Припущення про фізичний доступ

Це найважливіший розділ для реального проектування.

Без Flash Encryption фізичний доступ = повний доступ. Дамп флешу читається за хвилини й містить усе: прошивку, налаштування, ключі, збережені дані (розділ 24).

Практичні висновки:

Не покладайтесь на секретність прошивки. Логіка роботи, формати пакетів, адреси серверів — усе це видиме.

Пристрій має бути обмежений у правах. Датчик, що передає телеметрію, не потребує ключа, яким можна керувати всією системою. Захоплений датчик має давати можливість підробити свої показання — і не більше.

Сервер не довіряє пристрою. Перевірка прав, обмеження частоти, контроль осмисленості значень — на стороні сервера, а не пристрою.

Передбачте відкликання. Можливість заборонити конкретний пристрій — за MAC, за ідентифікатором, за ключем.

Що робити з чужим пристроєм

Дзеркальний бік розділу 24. Якщо в дампі знайшлися паролі й ключі:

Це знахідка, а не здобич. Повідомити власника — правильна дія.

Дамп — документ із чутливими даними. Не класти в загальнодоступне місце, не надсилати «щоб подивилися».

Зміна конфігурації чужого пристрою може мати наслідки за межами самого пристрою: він може бути частиною системи, про яку ви не знаєте (розділ 24).

Мінімальний перелік для виробу

Те, що варто зробити в будь-якому пристрої, який виходить за межі столу:

- креншелли в NVS, унікальні на екземпляр — не в коді;
- жодних дефолтних паролів; пароль на веб-інтерфейс і точку доступу;
- HTTPS із перевіркою CA для всього, що йде назовні;
- OTA тільки через захищений канал;
- BLE і ESP-NOW із шифруванням, якщо через них щось керується;
- права пристрою обмежені його роллю;
- перевірка на стороні сервера, а не пристрою;
- можливість відкликати конкретний пристрій;
- Flash Encryption і Secure Boot — свідомо, для серійного виробу, після випробування на платі, яку не шкода.

Що з цього треба запам'ятати

Ключі в коді дістаються за п'ять хвилин; місце їм у NVS, унікальними на екземпляр.

Один ключ на всі пристрої перетворює локальний інцидент на катастрофу.

TLS без перевірки сертифіката — половина захисту з відчуттям повного.

Зашивати CA, а не сертифікат сервера.

Flash Encryption і Secure Boot незворотні: пробувати тільки на платі, яку не шкода.

Без Flash Encryption фізичний доступ означає повний доступ. Проектувати треба з цього припущення.

Сервер не довіряє пристрою.

Частина XIII. Збирання і супровід

Паяння, монтаж, живлення, корпус, польовий ремонт і передача виробу іншій людині.

51. Паяння і монтаж

Паяння — навичка, яка ставиться за один вечір і працює все життя. Погана пайка дає збої, які виглядають як завгодно: плаваючі помилки, «іноді працює», несправність, що зникає від дотику.

Розділ для того, хто не паяв. Інструмент — розділ 10.

Що таке хороша пайка

Припій має **змочити** обидві поверхні — вивід і контактну площадку — і розтектися по них, утворивши плавну галтель.

Хороша: блискуча, увігнута, плавно переходить від виводу до площадки. Форма схожа на маленький вулкан.

Холодна: матова, зерниста, кулькою на виводі. Припій не змочив поверхню, а просто застиг на ній. Електричний контакт може бути — поганий, нестабільний, залежний від температури й вібрації.

Холодна пайка — головне джерело збоїв «іноді працює». Причини: замало температури, забруднена поверхня, немає флюсу, рух під час застигання.

Порядок дій

1. **Очистити.** Окислена поверхня не змочується. Спирт і серветка; для сильного окислення — механічно.
2. **Флюс.** Не «для професіоналів», а обов'язковий крок: він знімає окисел і дає припою розтектися.
3. **Прогріти обидві поверхні одночасно.** Жало торкається і виводу, і площадки. Це головне: припій тече на **гаряче**.
4. **Подати припій у місце контакту**, а не на жало. Припій, розплавлений на жалі, втрачає флюс іще до контакту.
5. **Прибрати припій, потім жало.**
6. **Не рухати** до застигання — секунду-дві.
7. **Оглянути.** Під лупою, якщо є.
8. **Змити флюс** спиртом.

Уся операція займає 2–3 секунди на з'єднання. Довше — перегрів.

УВАГА

Найпоширеніша помилка початківця — **паяти жалом, а не припоєм**: набрати припій на жало й «намазати» на з'єднання. Виходить кулька, що лежить зверху, — класична холодна пайка.

Правильно: жало гріє, припій подається окремо, у точку контакту.

Температура

Свинцевий припій — приблизно 300–320 °C. Безсвинцевий — на 30–40 градусів більше.

Занизька: припій не тече, з'єднання холодне. Зависока: вигорає флюс, окислюється жало, **відриваються контактні площадки**.

НЕЗВОРОТНЕ

Контактна площадка тримається на склотекстоліті клейовим шаром, який руйнується від тривалого нагріву. Відірвана площадка означає обірвану доріжку — це вже ремонт із перемичкою (розділ 55).

Правило: **три секунди на з'єднання**. Не вийшло — прибрати жало, дати охолонути, додати флюсу, спробувати знову. Гріти довше — не «доробити», а зіпсувати.

Свинець і безпека

Свинцевий припій паяється легше і надійніше.

Правила прості: **не їсти під час роботи, мити руки після**, працювати з провітрюванням. Небезпечний не сам свинець при паянні, а потрапляння в організм із рук.

Дим від флюсу дратує дихальні шляхи. Витяжка або хоча б відкрите вікно; не нахилятися над місцем пайки.

Гребінки й дроти

Гребінки (pin headers). Спершу припаяти **один крайній вивід**, перевірити, що гребінка стоїть рівно й притиснута до плати, поправити (розігрівши цей один вивід), і лише потім паяти решту.

Гребінка, припаяна криво по всій довжині, знімається важко й псує плату.

Дроти. Залудити кінець перед пайкою. Термоусадка надівається **до** пайки — після вже пізно, і це помилка, яку роблять усі один раз.

Багатожильний дріт не залужувати повністю до самої ізоляції: жорстка ділянка на межі ламається від вигину.

ТНТ-компоненти

Наскрізний монтаж: вивід у отвір, пайка з іншого боку.

Вивід має виступати на 1–2 мм. Довші — відкусити **після** пайки, не до: короткий вивід важко тримати.

Полярність перевіряти **до** пайки. Електролітичний конденсатор (смуга — мінус), діод (смуга — катод), світлодіод (довга ніжка — плюс), мікросхема (крапка або виїмка — перший вивід).

УВАГА

Випаяти щось із наскрізного монтажу значно важче, ніж припаяти: припій треба видалити з отвору опліткою або відсмоктувачем, інакше компонент не витягнеться, а спроба витягнути силою відриває площадку.

Тому полярність перевіряється двічі до пайки, а не один раз після.

Типові дефекти

Дефект	Вигляд	Причина
Холодна пайка	матова кулька	замало температури, немає флюсу
Перемичка	припій між сусідніми	забагато припою
Відірвана площадка	мідь відстала	перегрів
Непропай	вивід не змочений	не прогріли обидві поверхні
Тріщина	волосяна лінія	рухали до застигання
Залишки флюсу	липкий наліт	не змили

Залишки флюсу — не косметика: активний флюс із часом роз’їдає доріжки й дає струми витоку між сусідніми з’єднаннями. Змивати треба завжди.

Перемичка між пінами

Найчастіший дефект після холодної пайки. Виправляється **опліткою**: покласти на перемичку, притиснути жалом, оплітка вбирає припій.

Спроба «розтягнути» перемичку жалом зазвичай розмазує її далі.

Перемичку між сусідніми пінами не завжди видно неозброєним оком — під лупою обов’язково, а на плутаній платі ще й прозвонити (розділ 28).

Робота з платами й модулями

Не паяти під живленням. Різниця потенціалів між паяльником і платою пробиває входи (розділ 05). Живлення від’єднується завжди.

Антистатика. Торкнутися заземленого металу перед роботою.

Тримати плату за краї, не за виводи.

Модуль ESP32 демонтується тільки феном: у нього багато виводів по периметру, і жалом його зняти неможливо, не зруйнувавши плату (розділ 55).

Скільки тренуватися

Достатньо одного вечора на непотрібній платі. Тридцять з'єднань — і рука починає розуміти, скільки це «три секунди» і як виглядає момент, коли припій розтікся.

Тренуватися на робочому проекті — найдорожчий спосіб учитися.

Що з цього треба запам'ятати

Флюс обов'язковий; жало гріє обидві поверхні; припій подається в точку контакту, а не на жало.

Три секунди. Не вийшло — охолодити й спробувати ще, а не гріти довше.

Термоусадка надівається до пайки.

Полярність перевіряти двічі до пайки: випаяти важче, ніж припаяти.

Флюс змивати завжди — залишки роз'їдають доріжки.

Не паяти під живленням.

Один вечір на непотрібній платі економить зіпсовані робочі.

52. Від макетки до постійного монтажу

Пристрій, зібраний на макетній платі, працює на столі й перестає працювати, щойно його кудись понесли. Це не випадковість, а властивість макетки.

Межі макетної плати

Макетка — це десятки пружинних контактів, кожен із яких має опір і ємність, і кожен може відійти.

Що з цього виходить:

Контакти окислюються й слабшають. Схема, що працювала вчора, сьогодні поводить інакше.

Опір контактів помітний. Для сигналів байдуже, для живлення двигуна чи стрічки — ні: на контактах падає напруга й вони гріються.

Ємність між рядами псує швидкі сигнали. SPI на кількох мегагерцях на макетці працює гірше, ніж на дротах.

Вібрація й переміщення виривають дроти. Достатньо перенести плату зі столу на стіл.

УВАГА

Правило, яке варто прийняти одразу: **макетка — для перевірки ідеї, не для роботи**. Пристрій, що має пропрацювати довше за один сеанс за столом, збирається інакше.

Найпідступніше в макетці те, що вона працює «майже завжди», і збої виглядають як помилки в коді. Години, витрачені на пошук бага, який насправді був відійшлим дротом, — типова історія.

Окремо: **DuPont-роз'єми не тримаються під вібрацією**. Вони тримаються тертям, і достатньо струсу, щоб контакт зник. Для чогось, що рухається або їде, вони непридатні.

Перфбورد: робочий стандарт одиничних виробів

Перфбورد (макетна плата під пайку) — плата з отворами й контактними площадками. Компоненти паяються, з'єднання робляться дротом або припоем.

Для одиничного виробу це правильний вибір: надійно, дешево, не потребує виготовлення, робиться за вечір.

Практичні поради:

Сплануйте розташування до пайки. Накидати на папері або в редакторі, де що стоїть. Перепаювати перфборд дуже незручно.

Гребінки під модулі, не пайка модулів навпростець. ESP32 ставиться на гребінку — тоді його можна зняти, замінити, перепрошити окремо.

Живлення — товстішим дротом, окремими лініями від входу, а не ланцюжком через усю плату.

Земля — суцільною лінією або кількома, з'єднаними в одній точці.

Конденсатори по живленню біля кожного споживача (розділ 05).

Механічне кріплення роз'ємів. Роз'єм, що тримається лише на пайці, відірветься разом із площадками при першому натягу кабелю.

УВАГА

Найважливіше при переході з макетки на перфборд: не змінювати схему. Перенести один в один те, що працювало.

Спокуса «заодно покращу» призводить до того, що новий пристрій не працює, і незрозуміло, справа в монтажі чи у змінах. Спершу перенести й переконатися, що працює. Потім міняти.

Дроти й джгути

Скручені пари для сигналів, що йдуть поруч із силовими.

Силові окремо від сигнальних — не в одному джгуті (розділ 48).

Запас довжини на кожному дроті: короткий дріт натягується й ламає пайку.

Strain relief — механічне закріплення кабелю біля місця пайки: стяжка, крапля термоклею, петля. Без нього натяг передається на пайку (розділ 54).

Власна плата: коли й навіщо

Перехід до виготовленої плати має сенс, коли:

- потрібно більше кількох екземплярів;
- пристрій має бути малим;
- є швидкі сигнали або чутлива аналогова частина;
- потрібна надійність, що не залежить від акуратності монтажника;
- виріб має виглядати як виріб.

KiCad — вільний і повноцінний пакет для розробки плат. Наскрізний шлях виглядає так:

Схема. Малюється з бібліотечних компонентів. Модуль ESP32 у бібліотеках є.

Призначення посадкових місць. Кожному компоненту — фізичний відповідник.

Розводка. Розташувати компоненти, провести доріжки. Найдовший етап.

Перевірка правил (DRC). Ловить те, чого не видно оком: занадто вузькі доріжки, замалі зазори, непід'єднані ланцюги.

Експорт файлів для виробництва (Gerber) і замовлення.

НЕЗВОРОТНЕ

П'ять правил розводки, кожне з яких коштує окремого виготовлення плати, якщо його порушити.

Зона антени вільна від міді. Під PCB-антеною модуля не має бути ні доріжок, ні полігону землі, ні з іншого боку плати. Антена, покладена на мідь, не працює (розділ 39).

Конденсатори якомога ближче до виводів живлення мікросхем.

Земля суцільним полігоном, не тонкими доріжками.

Силові доріжки ширші. Ширина рахується за струмом; тонка доріжка гріється й падає на ній напруга.

Шість контактів для прошивки: TX, RX, **пін входу в бутлоадер**, EN, 3V3, GND. Коштують нічого на етапі розводки і вирішують, чи можна буде перепрошити виріб через рік (розділ 08).

Третій із них залежить від сімейства: **classic** **S2** **S3** це GPIO0, **C3** **C6** **H2** — GPIO9 (розділ 07). Помилитися тут дорого й непомітно: площадка буде, до бутлоадера вона не заведе, а з'ясується це на готовій партії.

І шосте, не менш важливе: **перевірити посадкові місця перед замовленням.** Роздрукувати плату в масштабі 1:1 і прикласти реальні компоненти. Помилка в розмірі роз'єму виявляється при отриманні партії, і тоді вже пізно.

Скільки це коштує часу

Чесно: перша власна плата — це кілька вечорів на освоєння KiCad плюс тиждень-два очікування виготовлення. Друга — значно швидше.

Для одного пристрою перфборд майже завжди швидший і дешевший. Плата починає вигравати від трьох-п'яти екземплярів або там, де перфборд не дає потрібної якості.

Що з цього треба запам'ятати

Макетка — для перевірки ідеї; збої на ній виглядають як помилки в коді.

Dupont не тримається під вібрацією.

Перфборд — робочий стандарт одиничного виробу.

При переході з макетки нічого не змінювати в схемі: спершу перенести й переконатися.

Модуль ставити на гребінку, не паяти навпростець.

На власній платі: вільна зона під антеною, суцільна земля, шість контактів для прошивки.

Роздрукувати плату 1:1 і прикласти компоненти перед замовленням.

53. Автономне живлення

Пристрій на батарейці — це не пристрій без дроту, а окрема інженерна задача: бюджет енергії (розділ 06), безпека, зарядка, захист і вимірювання залишку.

Li-ion 18650 і аналоги

Найпоширеніший вибір: доступні, ємні, дешеві.

Напруга: 4.2 В повністю заряджений, **3.6 В номінальна**, 2.5 В — межа розряду за паспортом. Ємність ходових елементів — від 2500 до 3500 мА·год.

Два уточнення, які варто знати, бо вони розходяться з тим, що переказують у мережі.

Номінальна — 3.6 В, а не 3.7. Так її подають усі виробники: Samsung INR18650-25R і -30Q, LG INR18650-MJ1 і HG2, Murata VTC6. «3.7 В» — округлення, яке прижилося в обігу. Різниця мала, але якщо ви рахуєте запас енергії за номіналом, беріть 3.6.

Схема зупиняється раніше за елемент. Паспортна межа розряду — 2.5 В, а в частини елементів 2.0 В. Практично ж перетворювач припиняє роботу близько 3.0 В, і саме цю цифру ви побачите у своєму пристрої. Різниця між 3.0 і 2.5 В — невикористаний залишок; він невеликий за енергією (крива внизу дуже крута) і платиться за нього ресурсом елемента, тож вибирати його до кінця не варто.

НЕЗВОРОТНЕ

Літієві акумулятори небезпечні при неправильному поводженні. Не теоретично: перезаряд, глибокий розряд, коротке замикання й механічне пошкодження призводять до займання, яке неможливо погасити водою.

Правила, що не обговорюються:

Захист обов'язковий. Або вбудований у сам елемент (protected), або окрема плата BMS. Незахищений елемент напряму до навантаження — це питання часу.

Не заряджати нижче 0 °C. Заряджання замерзлого літію руйнує його зсередини, і це не видно ззовні. Паспорти більшості елементів нормують заряджання від 0 до +45 °C — саме так у Panasonic NCR18650B і LG MJ1. Окремі елементи дозволяють нижче (LG HG2 — від -5 °C), але 0 °C лишається правильним правилом для схеми, яка не знає, який елемент у неї вставлять.

Не залишати заряджання без нагляду в приміщенні з людьми, доки схема не перевірена.

Механічне пошкодження = утилізація. Зім'ятий, пробитий чи роздутий елемент не використовується. Ніколи.

Не паяти напряму до елемента без досвіду: перегрів руйнує внутрішню структуру. Або точкове зварювання, або елементи з привареними виводами, або тримач.

Заряджання: TP4056

Найпоширеніший модуль зарядки. Дешевий, працює, доступний скрізь.

ЗАКУПІВЛЯ

TP4056 продається у **двох варіантах**: із захистом (додаткова мікросхема DW01 і два транзистори на платі) і без.

Брати треба з захистом. Різниця в ціні незначна, різниця в наслідках велика: варіант без захисту не має ні захисту від глибокого розряду, ні від короткого замикання.

Відрізнити просто: на платі із захистом біля роз'єму акумулятора стоять дві додаткові мікросхеми в маленьких корпусах.

Струм заряджання задається резистором на платі, типово 1 А. Для малих елементів це забагато — резистор доводиться міняти.

Одночасна робота й заряджання. Проста схема «акумулятор і навантаження паралельно на виході TP4056» працює, але заряджання при цьому не завершується коректно: модуль не бачить, коли елемент заповнився, бо струм іде і в навантаження.

Для пристрою, що має працювати під час заряджання, потрібна схема з розділенням шляхів (power path) або окремий контролер.

BMS для збірок

Кілька елементів послідовно — потрібна плата BMS, яка не лише захищає, а й **балансує**: вирівнює напругу на елементах.

Без балансування один елемент розряджається глибше за інші, деградує швидше, і збірка виходить з ладу передчасно.

Для однієї банки достатньо захисту.

Перетворення напруги

Акумулятор дає від 4.2 В до межі, на якій зупиниться ваш перетворювач — практично близько 3.0 В; ESP32 потребує стабільних 3.3 В. Три варіанти.

LDO (лінійний). Простий, тихий, не створює завад. Працює лише поки вхід вищий за вихід плюс падіння на самому LDO. Для звичайного LDO це означає, що при 3.5 В на акумуляторі виходу вже немає — тобто половина ємності недоступна.

LDO з малим падінням (low dropout, від 100–200 мВ) виправляє це частково.

Buck (понижувальний імпульсний). Ефективність 85–95 %, працює, доки вхід вищий за вихід. Для 4.2 → 3.3 В підходить, але при розряді до 3.3 В теж закінчується.

Buck-boost — працює і коли вхід вищий, і коли нижчий за вихід. Це єдиний спосіб використати ємність акумулятора **повністю**, до 3.0 В. Дорожчий і складніший.

ЖИВЛЕННЯ

Вибір перетворювача прямо визначає, скільки ємності ви реально використовуєте.

Звичайний LDO на 3.3 В із падінням 0.5 В відсікає все нижче 3.8 В — це приблизно **половина** ємності 18650. Buck-boost дає майже все.

Для пристрою, що має жити місяцями, це різниця вдвічі — більша за будь-яку оптимізацію коду.

Тихе живлення для аналогових вимірювань

Імпульсні перетворювачі створюють високочастотні пульсації, які потрапляють у вимірювання ADC (розділ 33).

Що робити, коли потрібна точність:

LDO після buck для аналогової частини: buck робить основну роботу ефективно, LDO чистить те, що йде на аналог.

Окремий фільтр: дросель або резистор плюс конденсатор на живленні аналогової частини.

Вимірювати, коли радіо мовчить. Найбільші завади дає передавач.

Захист від переполюсовки

Помилка при під'єднанні акумулятора трапляється, і без захисту вона означає спалену плату.

Діод послідовно — найпростіше, ціною падіння 0.3–0.7 В. Для автономного пристрою це помітна втрата.

P-канальний MOSFET — падіння мізерне, захист повний. Правильне рішення для акумуляторного живлення.

Роз'єм, який фізично не вставляється навпаки, — найдешевший і найнадійніший захист. Варто закладати на етапі проектування.

Вимірювання заряду

Найпростіше — виміряти напругу через дільник на ADC (розділ 33).

НЕЗВОРОТНЕ

Дільник розряджає акумулятор — і на порядок сильніше за сплячий чип (числа в розділі 33). Схемотехнічних розв'язань два.

Вмикати дільник транзистором лише на час вимірювання. Правильний варіант: у сні через дільник не тече нічого.

Брати резистори значно більшого номіналу — з розумінням, що вхід ADC має скінченний вхідний опір, і з мегаомним дільником показання попливуть, доки не поставити буфер.

Напруга погано відображає залишок. Крива розряду літію в середній частині майже пласка: від 80 % до 20 % напруга змінюється мало. Плюс вона просідає під навантаженням і відновлюється в спокої.

Що з цим робити:

- **міряти в спокої**, коли радіо вимкнене;
- **усереднювати** кілька відліків;
- **не показувати відсотки з точністю до одиниць** — це самообман; чотири градації (повний, більше половини, менше, критично) чесніші;
- для точного обліку — **окрема мікросхема-паливомір**, і тут важливо розрізняти два різні класи, які легко сплутати за назвою.

Лічильник кулонів у прямому сенсі (LTC2941, LTC2942, MAX17260 і подібні) вимірює струм на шунті й **інтегрує його в часі**. Дає чесний облік заряду, що вийшов і зайшов, ціною зайвого резистора в силовому колі й власного споживання.

Моделльний паливомір (MAX17048, MAX17043 і решта сімейства ModelGauge) шунта **не має взагалі**: він міряє ту саму напругу, тільки зіставляє її з моделлю розряду елемента, враховуючи історію і навантаження. Тобто це не «замість напруги», а «розумніше за напругу». Схема простіша й споживання менше (одиниці мікроампер), а точність залежить від того, наскільки ваш елемент схожий на модель.

Для датчика на батарейці модельний варіант зазвичай доречніший: він не додає нічого в силове коло. Там, де треба знати саме витрачений заряд, — лічильник кулонів.

Бюджет і його перевірка

Розрахунок — розділ 06. Тут головне доповнення: **розрахунок треба перевірити вимірюванням.**

Причини розбіжності майже завжди ті самі:

- час під'єднання до Wi-Fi більший за очікуваний (розділ 39);
- плата розробки споживає уві сні на порядки більше за чип: USB-міст, стабілізатор і світлодіод не сплять (розділ 06);
- дільник вимірювання напруги;
- підтягувальні резистори, що постійно течуть;
- периферія, яка не вимикається перед сном;
- холод: на морозі ємність падає відчутно.

УВАГА

Плата розробки не годиться для вимірювання реального споживання уві сні. 20 мА замість 20 мкА — це світлодіод і міст, а не чип.

Для справжньої автономності потрібен власний монтаж без цієї обв'язки або плата, спроектована під низьке споживання.

Що з цього треба запам'ятати

Захист акумулятора обов'язковий; TP4056 брати з захистом.

Не заряджати нижче нуля; пошкоджений елемент утилізувати.

Вибір перетворювача визначає, чи використаєте ви половину ємності чи майже всю.

Дільник для вимірювання напруги стає головним споживачем уві сні.

Напруга погано показує залишок; чотири градації чесніші за відсотки.

Плата розробки не годиться для вимірювання сну.

Бюджет енергії перевіряється вимірюванням, а не розрахунком.

54. Корпус і захист середовища

Корпус — не оздоблення. Це те, що визначає, чи переживе пристрій зиму, вібрацію, конденсат і людину, яка потягне за кабель.

І це те, що вирішується до складання, бо після виявляється, що антена не працює, а датчик міряє власне тепло.

Ступінь захисту IP

Дві цифри: перша — від твердих частинок, друга — від води.

Позначка	Що означає	Де застосовне
IP20	пальці й великі предмети	приміщення
IP54	пил частково, бризки	під навісом
IP65	пил повністю, струмені	вулиця
IP67	занурення на 30 хв	вулиця, тимчасове затоплення

Для вулиці мінімум — **IP65**. Готові бокси є в широкому асортименті й коштують небагато.

УВАГА

Клас захисту має корпус у зібраному стані. Просвердлений отвір під кабель, невставлена заглушка, перекручена кришка або затиснута прокладка — і IP65 перетворюється на IP20.

Кожен отвір має бути закритий сертифікованим вводом, а не силіконом навколо дроту.

Кабельні вводи

Гермоввід (сальник) — стандартна деталь, що затискає кабель і герметизує отвір. Розмір підбирається під діаметр кабелю: занадто товстий кабель не затиснеться, занадто тонкий не ущільниться.

НЕЗВОРОТНЕ

Гермоввід має охоплювати **кабель**, а не окремі дроти. Кілька тонких дротів у сальнику не ущільнюються, і вода йде **всередині** ізоляції по капілярах.

І окремо: кабель має входити **знизу** або мати петлю нижче вводу. Вода стікає по кабелю і збирається саме там, де він входить у корпус.

Strain relief

Кабель, що тримається лише на пайці, відірветься разом із площадками при першому натягу.

Механічне закріплення всередині корпусу: стяжка до стійки, притискна планка, петля з фіксацією. Правило: **натяг має прийматися корпусом, а не пайкою**.

Це п'ять хвилин при складанні й найчастіша причина ремонту польових пристроїв (розділ 55).

Антенa й метал

НЕЗВОРОТНЕ

Металевий корпус — це екран. Пристрій усередині металу не має зв'язку, і жодні налаштування цього не змінять (розділ 39).

Варіанти: пластиковий корпус; металевий із діелектричним вікном; зовнішня антена, винесена назовні через герметичний ввід.

Виявляється це після складання, і тоді вже дорого. Питання вирішується на етапі вибору корпусу.

Навіть у пластиковому корпусі важливо: не класти під антену акумулятор, дисплей, дроти чи металеві кріплення. Сантиметр вільного простору навколо — мінімум.

Конденсат

Найпідступніша проблема герметичних корпусів, і вона неочевидна.

Корпус закрили теплою дня — всередині лишилося вологе повітря. Вночі температура падає, волога конденсується на платі. Вдень випаровується, вночі знову. Через місяці — окислення, струми витоків, корозія.

УВАГА

Герметичний корпус не захищає від конденсату — він його створює.

Вода не заходить ззовні, а та, що всередині, не виходить.

Що з цим роблять:

Пакетик силікагелю всередині. Дешево, працює, потребує заміни раз на рік-два.

Мембранний клапан (Gore-Tex і аналоги) — пропускає пар, не пропускає воду. Правильне рішення для серйозного виробу.

Conformal coating — захисне покриття плати лаком. Захищає доріжки навіть при конденсаті.

Скласти в сухому приміщенні, а не на вулиці у вологий день.

Conformal coating наноситься пензликом або аерозолем на очищену плату. Роз'єми, кнопки й антену закривають малярним скотчем. Перед покриттям плату треба **відмити від флюсу** (розділ 51) — інакше лак запечатає забруднення разом із дошкою.

Вібрація

Транспорт, механізми, вітер. Що страждає першим:

Dupont-роз'єми. Не тримаються. Для будь-чого рухомого — не використовувати (розділ 52).

Гвинтові клеми послаблюються. Для вібрації — клеми з пружинним притиском або пайка.

Важкі компоненти — електролітичні конденсатори, реле, роз'єми — розхитують власну пайку. Механічне кріплення: клей, стяжка, притиск.

Плата в корпусі має бути закріплена на стійках, а не лежати вільно.

Температура

Промисловий діапазон більшості модулів — від -40 до $+85$ °C. Але це діапазон **модуля**, а не чипа й не пристрою, і плутанина коштує дорого в обидва боки.

Чип витримує більше: $-40...+125$ °C. Стелю модуля знижує не кремній ESP32, а флеш-кристал під тим самим екраном. Тому «ESP32 витримує 125» — правда про чип і неправда про те, що ви припаяєте на плату.

Угору запас теж є: у ряду WROOM поруч зі звичайними -N4 і -N8 стоять версії -N4 і -N8 на $+105$ °C. Якщо виріб гріється, це дешевший хід, ніж перероблення корпусу.

І головне, чого не видно з самої цифри: **85 °C міряються одразу за межами модуля** — так це означає його datasheet. Тобто не надворі, а всередині вашого корпусу, поруч зі стабілізатором і власним нагрівом плати. Діапазон модуля не є діапазоном пристрою, і різницю задає корпус, а не Espressif.

Що обмежує раніше:

Акумулятор. Літій не заряджається нижче 0 °C і втрачає ємність на морозі (розділ 53).

Дисплеї. Рідкокристалічні на морозі гальмують і темніють; e-paper має вузький діапазон.

Конденсатори. Електролітичні на морозі втрачають ємність, на спеці швидко старіють.

Пластик корпусу на сонці деформується; чорний корпус на сонці нагрівається значно вище температури повітря.

живлення

Перегрів у замкненому корпусі — часта причина «на столі працює, у корпусі ні». ESP32 з Wi-Fi гріється, стабілізатор гріється, і в замкненому об'ємі це нікуди не дівається.

Наслідки: тротлінг, збої, перезавантаження, скорочення терміну служби.

Що допомагає: розміщення нагрітих елементів ближче до стінок; металева стінка як радіатор; збільшення об'єму; зниження частоти ядра; зменшення потужності передавача.

Монтаж на об'єкті

Не на сонці. Прямі промені дають десятки градусів перегріву.

Датчик — за межами корпусу. Датчик температури всередині боксу міряє бокс (розділ 45).

Доступ для обслуговування. Пристрій, який треба перепрошити, має бути досяжним. Роз'єм для прошивки — там, де до нього можна дістатися, не розбираючи все.

Підпис. Що це, чиє, коли встановлено, як зв'язатися. Пристрій без підпису через рік — загадка для всіх, включно з тим, хто його ставив (розділ 56).

Що з цього треба запам'ятати

IP має корпус у зібраному стані; кожен отвір — сертифікованим вводом.

Кабель входить знизу або з петлею; натяг приймає корпус, а не пайка.

Метал убиває радіо — це вирішується вибором корпусу, не кодом.

Герметичний корпус створює конденсат; силікагель, мембрана або покриття плати.

Плату відмити від флюсу перед conformal coating.

Датчик температури — за межами корпусу.

Пристрій підписати.

55. Польова діагностика й ремонт

Пристрій не працює, і він не на столі. Часу мало, інструменту небагато, запасних частин може не бути взагалі.

Розділ про те, як за наявним визначити, що саме зламалося, і що з цим робити на місці.

Порядок, який економить найбільше часу

Не розбирати одразу. Спершу зібрати відомості:

- індикатори живлення горять?
- чи гріється щось?
- чи є запах — перегрітий пластик, згорілий компонент?
- чи змінилися умови: живлення, кабель, погода, хтось щось робив?

Потім живлення. До будь-якого втручання в код чи прошивку.

Потім зв'язок і лог. Якщо є доступ до UART — картка K6.

Потім вузли.

Стисла версія симптомів — картка K8, розгорнута — розділ 29.

Що і де міряти мультиметром

Це основний інструмент у полі (розділ 10). Порядок точок:

Вхідна напруга на роз'ємі живлення. Є взагалі? Відповідає очікуваній?

Після захисту. Запобіжник, діод, ключ. Напруга до і після: різниця означає обрив або пробитий елемент.

Вихід стабілізатора — 3V3. Має бути близько 3.3 В. Помітно менше — або стабілізатор мертвий, або перевантажений замиканням далі.

Під навантаженням. Найважливіше і найчастіше пропущене. Виміряти, коли пристрій працює і радіо ввімкнене. Просадка нижче 3.0 В — причина знайдена (розділ 06).

Прозвонка землі між усіма вузлами.

Прозвонка на замикання 3V3–GND і 5V–GND при вимкненому живленні. Дзвенить — далі шукати замикання, а не вмикати.

УВАГА

Просадка під навантаженням — найчастіша знахідка польової діагностики. Пристрій, що місяцями працював, починає збоїти, коли акумулятор постарів, кабель окислився або контакт послабшав.

Виміряти дві напруги — на холостому ходу і під навантаженням — займає хвилину і закриває більшу частину випадків.

Типові апаратні дефекти

За частотою в польових умовах:

Роз'єм живлення або USB. Механічно розхитаний, відірваний із площадками, окислений. Симптом: працює, якщо притиснути кабель; переривчасте живлення.

Холодна пайка. Проявляється з часом і від температурних циклів. Симптом: працює після постукування, або навпаки перестає (розділ 51).

Стабілізатор. Перегрів під навантаженням, пробій. Симптом: працює кілька хвилин, потім перезавантаження; корпус гарячий.

Кабель. Обрив усередині ізоляції, окислені контакти. Найдешевша перевірка — замінити на завідомо справний.

Корозія від конденсату. Зеленуватий наліт, темні доріжки. Причина — корпус без силікагелю чи мембрани (розділ 54).

Антенa. Обірваний коаксіальний кабель, відкручений роз'єм, зламана PCB-антена. Симптом: зв'язок є впритул і немає на відстані.

Флеш. Рідко, але буває — від зношення при частому записі. Симптом: не проходить verify-flash, помилки читання (розділ 17).

Спалений пін. Один GPIO не працює, решта нормально. Причина — 5 В або статика (розділ 05). Лікується перепризначенням піна в коді, якщо є вільний.

Від симптому до вузла

Що бачимо	Найімовірніший вузол
Немає живлення взагалі	кабель, роз'єм, запобіжник
ЗВЗ занижена	стабілізатор або замикання далі
Гарячий стабілізатор	перевантаження або пробій
Перезавантаження під навантаженням	живлення, кабель, конденсатор
Немає зв'язку, живлення норм	антена, роз'єм, налаштування

Що бачимо	Найімовірніший вузол
Один датчик мовчить	його живлення, дріт, сам датчик
Усі датчики мовчать	шина, підтягування, спільна земля
Не прошивається	кабель, міст, GP100 (картка K4)
Пахне горілим	не вмикати; шукати візуально

Що можна полагодити на місці

Замінити кабель. Найчастіший і найшвидший ремонт.

Підтягнути клеми, перевиткнути роз'єми. Окислені контакти — почистити.

Перепаяти холодне з'єднання. Якщо є паяльник.

Jumper-ремонт. Обірвана доріжка чи відірвана площадка обходиться дротом від найближчої точки того самого ланцюга. Дріт фіксується клеєм або термоусадкою, щоб не працював на злам.

Перепризначити спалений пін у коді на вільний і перепрошити.

Замінити модуль чи плату цілком. Найшвидше, коли є запасна і коли пристрій зроблений так, що це можливо (розділ 08).

УВАГА

Найдорожча помилка польового ремонту — **лагодити те, що не зламане**. Пристрій розібрали, щось перепаяли, зібрали — не працює, і тепер незрозуміло, чи це початкова несправність, чи додана.

Правило: **одна зміна за раз, з перевіркою після кожної**. І фотографія до розбирання (розділ 22) — щоб було куди повернутися.

Заміна модуля

Модуль ESP32 має виводи по периметру й **не знімається жалом**. Потрібен фен: рівномірний прогрів по периметру, поки припій не розплавиться по всій площі, потім зняти пінцетом без зусилля.

Спроба піддіти жалом або силою відриває площадки й перетворює ремонт на брут.

Тому: у польових умовах модуль зазвичай не міняють. Міняють плату цілком — і саме тому виріб варто проектувати з платою на роз'ємі чи гребінці (розділ 52).

Що взяти в поле

Комплект — картка K12 і розділ 10. Для ремонту критично:

Мультиметр — без нього діагностика зводиться до здогадок.

Запасний перевірений кабель.

Запасна плата з залитою робочою прошивкою. Найшвидший ремонт — заміна.

Паяльник — якщо є куди ввімкнути. Портативний на акумуляторі варто мати.

Дроти, термоусадка, стяжки, ізоляційна стрічка.

Роздруковані картки K1–K15. Заламіновані, у кишені.

Ноутбук з офлайн-комплектом (розділ 15, додаток F): тулчейн, образи прошивок разом із .elf, документація.

Коли зупинитися

Чесна порада: якщо після перевірки живлення, кабелю, роз'ємів і зв'язку причина не знайдена — **везти на стіл**, а не розбирати далі в полі.

У полі мало світла, немає лупи, немає приладів і немає запчастин. Ймовірність додати другу несправність до першої висока, і тоді ремонт подорожчає в рази.

Замінити пристрій цілком і розібратися потім — майже завжди дешевше.

НЕЗВОРОТНЕ

Перед тим як щось міняти в пристрої, який ще подає ознаки життя, — зняти дамп флешу, якщо є доступ (картка K2). У NVS може лежати конфігурація, якої немає більше ніде, і після заміни плати вона знадобиться.

Що з цього треба запам'ятати

Спершу зібрати відомості, потім живлення, потім лог, потім вузли.

Просадка під навантаженням — найчастіша знахідка; міряти обов'язково під навантаженням.

Одна зміна за раз; фото до розбирання.

Модуль знімається тільки феном; у полі міняють плату, а не модуль.

Не знайшли за чотири кроки — везти на стіл, а не розбирати далі.

Дамп перед заміною, якщо пристрій ще живий.

56. Паспорт виробу і передача

Пристрій, який розуміє лише той, хто його зробив, — це не виріб, а особиста справа. Цей розділ про документ, що робить його виробом.

Питання, на яке відповідає паспорт: **чи зможе інша людина через рік полагити це без вас?**

Мінімальний паспорт

Одна-дві сторінки. Не «повна документація», а те, без чого неможливо працювати.

1. Що це і що робить.

Три речення. Призначення, вхід, вихід.

2. Живлення.

Напруга, полярність, споживання типове й пікове, тип роз'єму. Що станеться при переполюсовці.

3. Пінаут і з'єднання.

Що куди підключено — по сигналах, не по кольорах (розділ 22):

```
GPI04 → датчик DS18B20, лінія DATA (підтяжка 4.7 кОм до 3V3)
GPI021 → дисплей SSD1306, SDA
GPI022 → дисплей SSD1306, SCL
GPI025 → реле насоса, ключ на MOSFET (низький = вимкнено)
```

4. Порядок прошивки.

Конкретні команди з адресами, чип, як увійти в download mode на цьому виробі, де взяти образ:

```
esptool --port /dev/ttyUSB0 write-flash 0x0 nasos-v1.4.bin
```

5. Налаштування.

Що зберігається в NVS, як задається, як скинути на заводські.

6. Індикація.

Що означає кожен стан світлодіода чи дисплея.

7. Дії при збої.

Найкорисніший розділ: симптом → що робити.

8. Контакти й дата.

Хто зробив, коли, як зв'язатися.

Схема, за якою зможе зібрати інший

Для одиничного виробу не обов'язково креслити схему в редакторі. Достатньо:

Фото зібраного пристрою з обох боків, чіткі, з читабельним маркуванням.

Таблиця з'єднань — та сама, що в пункті 3 паспорта.

Перелік компонентів із конкретними позначеннями: не «датчик температури», а DS18B20 у герметичному зонді, 3 м кабель.

Особливості, які неочевидні: «резистор 4.7 кОм між DATA і 3V3 обов'язковий», «реле вмикається низьким рівнем», «GPIO12 не використовувати».

УВАГА

Останній пункт — найцінніший і той, що першим забувається. Усі неочевидні рішення, ухвалені під час розробки, існують лише у вашій голові. Через рік вони не існуватимуть і там.

Записувати треба **саме те, що здалося дивним і зажадало пояснення**: чому цей пін, чому цей номінал, чому не зробили простіше. Це те, що рятує наступного, включно з вами.

Версіонування прошивок

Версія має бути видима з самого пристрою. Не «десь у git», а в boot-лозі, у веб-інтерфейсі, на дисплеї:

```
ESP_LOGI(TAG, "Насос-контролер v1.4, зібрано %s %s", __DATE__, __TIME__);
```

Це перше, що питають при діагностиці, і єдиний надійний спосіб дізнатися, що саме залито в конкретний пристрій.

УВАГА

`__DATE__` і `__TIME__` мають ціну, про яку варто знати: вони роблять кожне збирання побайтово відмінним від попереднього. Тобто зібрати «точно такий самий» образ повторно стає неможливо **за визначенням**, а не через тулчейн (розділ 24).

Це не привід їх прибирати: відповідь на «яка це збірка» коштує дорожче за побайтову відтворюваність, і саме тому книга наполягає зберігати сам файл образу, а не сподіватися перезібрати його (розділ 21). Але якщо відтворюваність збирання для вас є вимогою — мітку часу підставляють ззовні, зі змінної збирання, а не з `__DATE__`.

Правило нумерації — будь-яке, аби послідовне. Практичний мінімум: збільшувати число при кожній прошивці, що поїхала кудись за межі столу.

Що зберігати разом із версією

НЕЗВОРОТНЕ

Для кожної версії, що поїхала до замовника чи в поле, зберігається:

- **сам образ** `.bin` — той, що залито;
- **`.elf` того самого збирання** — без нього `backtrace` із поля нерозшифрований (розділ 26);
- **`sdkconfig` проєкту**;
- **версія ESP-IDF і версії компонентів**;
- **дата й перелік змін**.

Перезібрати «такий самий» образ пізніше майже ніколи не виходить точно: змінився тулчейн, змінилася бібліотека. Адреси зсунуться, і `.elf` перестане відповідати тому, що в полі (розділ 21).

Це кілька мегабайтів, які вирішують, чи буде збій із поля розібраний за десять хвилин, чи не буде розібраний узагалі.

Changelog

Короткий файл, рядок на версію, людською мовою:

v1.4	2026-08-26	Додано контроль сухого ходу насоса. Виправлено: втрата з'єднання не зупиняла насос.
v1.3	2026-07-14	OTA через веб-інтерфейс.
v1.2	2026-06-02	Скидання налаштувань довгим натисканням.

Пишеться **для того, хто читатиме через рік** — з описом наслідку, а не внутрішньої зміни. «Виправлено обробку події `disconnect`» нічого не каже; «втрата з'єднання не зупиняла насос» каже все.

Інструкція оператору

Окремий документ. **Одна сторінка**, для людини, яка нічого не знає про ESP32 і не має знати.

Що в ній:

- як увімкнути й вимкнути;
- що означає кожен індикатор;
- що робити, коли не працює — три-чотири прості дії;
- **чого не робити** (не мити струменем, не закривати вентиляцію, не від'єднувати живлення під час оновлення);
- кому дзвонити.

Без термінів, без назв мікросхем, без командного рядка. Якщо оператору потрібен командний рядок — інструкція не зроблена.

Роздрукувати й покласти в корпус або приклеїти зсередини кришки.

Маркування самого пристрою

На корпусі, стійко до вологи й сонця:

- **що це** — коротка назва;
- **номер екземпляра**
- **версія прошивки** (олівцем або наліпкою, що змінюється);
- **дата встановлення**
- **контакт**.

УВАГА

Пристрій без маркування через рік — загадка для всіх, включно з тим, хто його ставив. Скільки таких стоїть, які в них версії, який із них проблемний — питання без відповідей.

Наліпка коштує нічого і клеїться на бік, який видно в зібраному стані. Наліпка всередині корпусу не існує (розділ 21).

Передача іншій людині

Що передається разом із пристроєм:

- паспорт;
- інструкція оператору;
- образ прошивки і .elf;
- вихідні тексти, якщо домовлено;
- перелік компонентів;
- фотографії;
- запасні частини, якщо є.

І одна річ, яку варто зробити явно: **розповісти, чого ви не зробили**. Відомі обмеження, невирішені проблеми, місця, де пристрій поводить дивно. Це не визнання слабкості, а економія часу наступного — і захист від того, що обмеження виявяться в найгірший момент.

Що з цього треба запам'ятати

Паспорт — одна-дві сторінки; питання, на яке він відповідає: чи зможе інший полатити це без вас.

З'єднання записувати по сигналах, а не по кольорах.

Неочевидні рішення записувати обов'язково — через рік їх не буде і у вашій голові.

Версія має бути видима з пристрою, у boot-лозі.

.elf зберігати разом із кожним образом, що поїхав.

Changelog писати наслідками, а не внутрішніми змінами.

Інструкція оператору — одна сторінка без термінів.

Наліпка на боці, який видно в зібраному стані.

Частина XIV. Від задачі до пристрою

Інженерний процес: постановка, прототип, доведення до надійного вузла.

57. Постановка і архітектура

Найдорожчі помилки в проєкті робляться до першого рядка коду. Обраний не той чип, не врахований бюджет живлення, не продумана поведінка при відмові — і це виявляється тоді, коли пристрій уже зібраний.

Розділ про питання, які варто поставити на самому початку.

Питання, з яких усе починається

Що на вході? Скільки датчиків, які, як часто опитуються, яка потрібна точність.

Що на виході? Що вмикається, з якою потужністю, як часто, що станеться при помилковому спрацюванні.

Які затримки допустимі? Реакція за мілісекунди чи за хвилини — це різні пристрої (розділ 32).

Який канал зв'язку? Wi-Fi, ESP-NOW, LoRa, дріт — визначається відстанню, енергією й тим, що вже є на об'єкті (розділи 39–43).

Звідки живлення? Мережа, акумулятор, сонце. Це найжорсткіше обмеження: воно визначає й чип, і протокол, і режим роботи (розділ 06).

Яке середовище? Приміщення, вулиця, вібрація, температура, вологість (розділ 54).

Що має статися при відмові? Кожної частини окремо: зник зв'язок, завис контролер, зникло живлення, збрехав датчик.

УВАГА

Останнє питання найважливіше і найчастіше пропущене — а відповідь на нього визначає схемотехніку, а не код.

Насос, що вмикається високим рівнем на GPIO, при зависанні чипа лишається ввімкненим. Нагрівач — теж. Питання «що станеться, якщо чип зникне просто зараз» має ставитися до кожного виходу **на етапі проєктування**, бо виправляється воно резистором на платі, а не програмою (розділ 32, 47).

Вибір чипа й плати

Після відповідей вище вибір стає механічним (розділ 02):

Обмеження	Наслідок
Потрібен Bluetooth Classic	тільки classic
Потрібна PSRAM, камера, дисплей із буфером	classic, S2 або S3

Обмеження	Наслідок
Батарейка, без Wi-Fi, роками	H2
Zigbee, Thread, Matter	C6 або H2
Дешево, просто, 400 КБ вистачає	C3
Новий проєкт без особливих умов	S3

Потім — плата чи модуль (розділ 08), і одразу питання: скільки пінів потрібно і скільки лишиться (розділ 07). Це часто виявляється вужчим місцем, ніж пам'ять чи швидкість.

Блок-схема

Малюється до схемотехніки, на папері. Прямокутники — вузли, стрілки — що куди йде.

Що вона одразу показує:

- **скільки інтерфейсів** насправді потрібно і чи вистачить пінів;
- **де межі відповідальності** — що робить ESP32, а що ні;
- **де вузькі місця** — один канал зв'язку на все, одне джерело живлення на всіх;
- **що станеться, якщо вузол зникне** — стрілка, яка обірветься.

П'ятнадцять хвилин на папері економлять переробку плати.

ESP32 як companion-контролер

Архітектурне рішення, яке варто розглядати за замовчуванням, а не як запасне (розділ 01).

Ідея. Основний контролер робить свою роботу — керує механізмом, тримає таймінги, забезпечує безпеку. ESP32 стоїть **збоку** і відповідає лише за зв'язок: віддає телеметрію, приймає команди, оновлює себе.

Обмін — по UART (розділ 34) або CAN (розділ 38).

Чому це часто правильно:

- проблеми з радіо не впливають на основну логіку: зв'язок зник — механізм працює далі;
- основний контролер обирається під задачу — там, де потрібні гарантовані таймінги, це не ESP32 (розділ 32);
- відповідальність розділена, кожен частину можна тестувати окремо;
- ESP32 можна перепрошити або замінити, не чіпаючи основну систему;
- безпека: захоплений модуль зв'язку не дає прямого керування механізмом (розділ 50).

Розділення відповідальності — головне проектне рішення в цій схемі:

Основний контролер	ESP32
керування механізмом	зв'язок і протоколи
таймінги й реальний час	буферизація телеметрії
аварійні блокування	веб-інтерфейс, ОТА
робота при втраті зв'язку	розбір команд

НЕЗВОРОТНЕ

Ключове правило цієї схеми: **аварійні блокування живуть в основному контролері, а не в ESP32**. Зупинка за граничним значенням, кінцевий вимикач, захист від перегріву — усе, що рятує механіку й людей, не повинно залежати від вузла, який може перезавантажитися через оновлення чи проблему з мережею.

ESP32 може **запросити** зупинку. Виконати її має той, хто керує.

Протокол між контролерами

Практичні поради (розділ 34):

Текстовий формат для нечастого обміну: налагоджується монітором порту, без інструментів.

Двійковий — коли часто або багато, і обов'язково з байтом початку, довжиною і контрольною сумою.

Обидві сторони переживають зникнення іншої. Ні ESP32, ні основний контролер не зависають, чекаючи відповіді, якої не буде.

Періодичний сигнал життя в обидва боки. Зникнення сигналу — подія, на яку є реакція.

План тестів і критерії приймання

Записуються до складання, у вигляді перевірок із однозначною відповіддю:

- усі датчики читаються, значення в очікуваних межах;
- при зникненні зв'язку пристрій продовжує працювати і буферизує дані;
- при відновленні зв'язку накопичене передається;
- при зникненні живлення виконавчі механізми переходять у безпечний стан;
- пристрій витримує 24 години безперервної роботи без витоку пам'яті (розділ 30);
- ОТА проходить і відкат працює (розділ 19);

- споживання відповідає розрахунку (розділ 53).

УВАГА

Критерій «працює» — не критерій. Він завжди виконується на столі в день складання.

Критерій має бути перевіркою, яку можна провалити: «24 години без перевантажень», «мінімум вільної пам'яті не зменшився», «10 циклів втрати й відновлення зв'язку без втрати даних».

Список, складений до розробки, ще й формує саму розробку: побачивши пункт про втрату зв'язку, ви напишете код інакше.

Що врахувати наперед

Розбивку флешу з ОТА — навіть якщо ОТА поки не потрібен. Змінити її дистанційно неможливо (розділ 18).

Шість контактів для прошивки на платі (розділ 52).

Доступ до пристрою після монтажу (розділ 54).

Унікальні ключі на екземпляр — інакше переробляти виробництво (розділ 50).

Місце для наліпки з версією і номером (розділ 56).

Кожен пункт коштує нічого на етапі проектування й дорогого — після.

Що з цього треба запам'ятати

Найдорожчі помилки робляться до першого рядка коду.

Питання «що станеться, якщо чип зникне зараз» ставиться до кожного виходу на етапі проектування.

ESP32 як companion-контролер — рішення за замовчуванням, а не запасне.

Аварійні блокування живуть у тому, хто керує, а не в модулі зв'язку.

Критерій приймання — це перевірка, яку можна провалити.

Розбивка з ОТА, контакти для прошивки й унікальні ключі закладаються наперед.

58. Прототип → тест → надійний вузол

Між «працює на столі» і «працює в полі» лежить робота, яку легко недооцінити. Цей розділ — про неї.

Три стани пристрою

Прототип доводить, що задача розв’язується. Живе на макетці, працює, поки на нього дивляться. Мета — знання, а не надійність.

Тестовий зразок працює безперервно й дає дані про те, як він поводить. Зібраний постійним монтажем, з логуванням.

Виріб працює без нагляду, переживає відмови й може бути переданий іншому.

Помилка — намагатися зробити виріб одразу. Тоді ви одночасно розбираєтеся з задачею, з надійністю і з монтажем, і незрозуміло, що саме не працює.

Мінімальний прототип

Правило одне: **мінімальний означає мінімальний**.

Один датчик, а не всі. Дріт замість корпусу. Живлення від USB. Найпростіший спосіб вивести результат.

Мета — відповісти на найризикованіше питання проєкту. Зазвичай це щось одне: чи вистачить дальності, чи читається цей датчик, чи вкладемося в бюджет живлення.

УВАГА

Перевіряти треба **найризикованіше першим**, а не найпростіше.

Типова помилка — почати з веб-інтерфейсу, бо він зрозумілий, і за тиждень з’ясувати, що на потрібній відстані зв’язку немає взагалі. Тиждень витрачено на частину, яку доведеться викинути разом із рештою.

Питання «що тут найімовірніше не спрацює» ставиться на початку, і саме воно перевіряється першим (розділ 44).

Логування як інструмент доведення

Логування — не для того, щоб ловити помилки, а для того, щоб **знати, що відбувалося** (розділ 25).

Що варто логувати від самого початку:

Причину скидання першим рядком `app_main`. Пристрій сам розповідає про свою попередню смерть (розділ 26).

Переходи станів із часовими мітками. Не «тут», а «ОЧІКУВАННЯ → РОБОТА, тиск 4.2».

Мінімум вільної пам'яті періодично. Це єдиний спосіб побачити повільний витік, який інакше проявиться через тижні (розділ 30).

Помилки з кодом і назвою через `esp_err_to_name`.

RSSI при кожному сеансі зв'язку (розділ 39).

Лог пишеться у **файл**, а не читається з екрана: `tee` при налагодженні, кільцевий буфер чи картка в полі.

Тестова обв'язка

Кілька прийомів, що прискорюють доведення:

Прискорений час. Пристрій, що має щось робити раз на годину, під час випробувань робить це раз на хвилину. Добова поведінка перевіряється за двадцять хвилин.

Штучні відмови. Не чекати, поки зв'язок обірветься сам — вимкнути роутер. Не чекати розряду — під'єднати регульоване джерело й знизити напругу. Не чекати, поки датчик зламається — від'єднати його.

Друга плата як імітатор. Замість реального обладнання, якого ще немає або яке не можна чіпати.

Довгий прогін — доба мінімум, із логом у файл. Більшість проблем із пам'яттю й накопиченням стану проявляються саме тут.

НЕЗВОРОТНЕ

Помилка, яку не вміють відтворювати, не полагоджена.

Збій, що зник після зміни, але не відтворювався стабільно до неї, — просто зараз не видно. Він повернеться в полі, у найгірший момент, і там його вже не спіймати.

Перш ніж закрити питання, треба вміти викликати збій за бажанням. Це займає час і воно того варте (розділ 29).

Зміцнення: від тестового зразка до виробу

Перелік того, що додається на цьому етапі. Кожен пункт — розділ книги.

Обробка помилок. Замінити `ESP_ERROR_CHECK` на явну обробку скрізь, де помилка можлива в роботі. Три стратегії: повторити, деградувати, безпечний стан (розділ 32).

Повтори зі зростаючою паузою для всього мережевого (розділ 39).

Watchdog на головні робочі задачі, з годуванням після корисної роботи (розділ 32).

Безпечний стан апаратно — резистори на затворах ключів, щоб навантаження не вмикалося під час завантаження (розділ 47).

ОТА з відкатом, якщо пристрій має оновлюватися. Підтвердження справності — після реальної умови, не на початку (розділ 19).

Coredump у флеші (розділ 26).

Постійний монтаж замість макетки (розділ 52).

Корпус із урахуванням антени, конденсату й натягу кабелю (розділ 54).

Безпека: креденшели в NVS, унікальні ключі, HTTPS із перевіркою (розділ 50).

Приймальні випробування

Проводяться за списком, складеним до розробки (розділ 57), із записаним результатом кожного пункту.

Типовий мінімум:

Перевірка	Критерій
Безперервна робота	24 год без перезавантажень
Пам'ять	мінімум вільної не зменшується
Втрата зв'язку	10 циклів без втрати даних
Зникнення живлення	безпечний стан, коректний старт
ОТА	оновлення і відкат проходять
Споживання	відповідає розрахунку $\pm 20\%$
Температура	працює в заявленому діапазоні

УВАГА

Випробування зникнення живлення варто робити **грубо**: вимикати живлення в довільні моменти, багато разів, зокрема під час запису у флеш і під час ОТА.

Це найдешевший спосіб знайти проблеми, які інакше з'являться в полі через півроку — і виглядатимуть як містика (розділи [18](#), [49](#)).

Документування результату

Останній крок, який робить пристрій виробом: паспорт (розділ [56](#)).

Робиться **одразу після приймання**, поки все свіже. Через місяць неочевидні рішення забудуться, а саме вони найцінніші.

Разом із паспортом зберігається версія прошивки, `.elf` того самого збирання, `sdkconfig` і перелік версій компонентів (розділ [21](#)).

Скільки це триває

Чесна пропорція для типового проєкту: прототип — це приблизно чверть роботи. Решта — доведення, монтаж, корпус, випробування й документація.

Пристрій, що «майже готовий» після тижня, зазвичай готовий на чверть. Це не песимізм, а планування: знаючи пропорцію, ви не обіцяєте неможливого.

Що з цього треба запам'ятати

Не робити виріб одразу: прототип, тестовий зразок, виріб — три різні речі.

Перевіряти найризикованіше першим, а не найпростіше.

Логувати причину скидання, переходи станів і мінімум вільної пам'яті з самого початку.

Штучні відмови замість очікування природних.

Помилка, яку не вміють відтворювати, не полагоджена.

Живлення вимикати грубо й багато разів, зокрема під час запису.

Прототип — чверть роботи.

Частина XV. Проєкти

П'ять наскрізних прикладів: постановка, схема, ВОР, повний код, збирання, тест, розвиток.

59. Моніторинг датчиків з веб-інтерфейсом

Пристрій міряє температуру, вологість і тиск, тримає історію й показує її у браузері. Доступний за іменем, без пошуку IP-адреси.

Це базовий проєкт: на ньому зустрічаються Wi-Fi, I²C, веб-сервер, mDNS, зберігання стану й обробка помилок.

Постановка

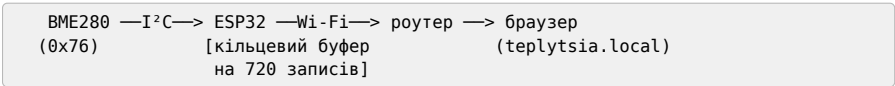
Вхід: BME280 по I²C — температура, вологість, тиск.

Вихід: веб-сторінка з поточними значеннями і графіком за останні години; JSON для машинного зчитування.

Живлення: мережа через USB. Автономність не потрібна.

Поведінка при відмові: немає мережі — вимірювання продовжуються й накопичуються; датчик мовчить — пристрій живий і повідомляє про це.

Блок-схема



Складові

Позиція	Кількість	Примітка
ESP32-S3-DevKitC-1 або classic DevKitC	1	будь-який із Wi-Fi
BME280, модуль I ² C	1	звірити адресу: 0x76 або 0x77
Резистори 4.7 кОм	2	підтягування I ² C, якщо немає на модулі
Дроти Dupont	4	
Корпус, живлення	—	розділ 54

Ціни й наявність — у вкладиші ринку (P5).

Піни за платами

Проект розрахований на дві плати, і **піни в них різні**. Спочатку оберіть рядок, далі читайте схему з ним.

Сигнал	classic DevKitC	S3-DevKitC-1
SDA	GPI021	GPI08
SCL	GPI022	GPI09

НЕЗВОРОТНЕ

S3 **GPI022 на S3 не існує**. У S3 немає пінів 22–25 узагалі — це видно прямо в ESP-IDF, де маска дійсних пінів їх вирізає. Схема з GPI022, перенесена з classic, не запрацює: `i2c_new_master_bus` поверне `ESP_ERR_INVALID_ARG`, і шина лишиться німою.

Драйвер при цьому **називає причину в консолі** — типово, без жодного налаштування:

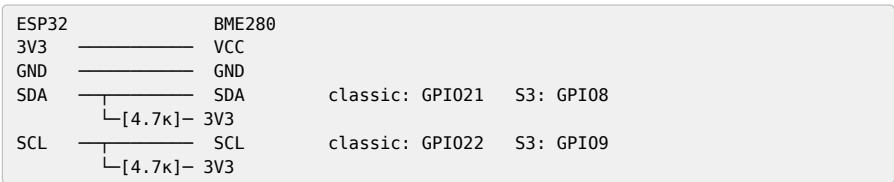
```
E (315) i2c.master: i2c_new_master_bus(1049): invalid SDA/SCL pin number
```

Тобто ловиться це за секунду, **якщо дивитися в лог**. Мовчазним воно стає лише тоді, коли код повернення не перевіряють, а лог гортають повз: без `ESP_ERROR_CHECK` програма спокійно йде далі до першої транзакції з дескриптором, якого немає.

Це загальне правило для всіх проєктів цієї частини: **ВОМ із двома сімействами означає дві розпіновки**, і жодну з них не можна отримати з іншої заміною одного числа (додаток A).

Схема

Нижче — варіант для classic. Для S3 підставте піни з таблиці вище.



Підтягування обов'язкове (розділ 35). Багато модулів BME280 мають власні резистори — тоді зовнішні не ставити.

Код

Проект ESP-IDF. Конфігурація через `menuconfig` або `sdkconfig.defaults`.

Читання датчика

Піни винесені в одне місце нагорі — так їх видно й так вони не розповзаються по коду:

```
// Піни за платою. Одне місце на весь проєкт.
#if CONFIG_IDF_TARGET_ESP32S3
# define PIN_SDA  GPIO_NUM_8
# define PIN_SCL  GPIO_NUM_9
#else               // ESP32 classic
# define PIN_SDA  GPIO_NUM_21
# define PIN_SCL  GPIO_NUM_22
#endif
```

CONFIG_IDF_TARGET_* виставляє сама збірка після idf.py set-target, тож перемика-
ння плати не потребує правок у коді (розділ 11).

```
#include "driver/i2c_master.h"
#include "esp_log.h"

#define BME_ADDR  0x76
#define REG_ID    0xD0
#define REG_RESET 0xE0
#define REG_CTRL_H 0xF2
#define REG_CTRL_M 0xF4
#define REG_CONFIG 0xF5
#define REG_DATA   0xF7
#define REG_CALIB1 0x88
#define REG_CALIB2 0xE1

static const char *TAG = "BME";
static i2c_master_dev_handle_t bme;

// калібрувальні коефіцієнти живуть у самому датчику
static uint16_t T1; static int16_t T2, T3;
static uint16_t P1; static int16_t P2, P3, P4, P5, P6, P7, P8, P9;
static uint8_t  H1, H3; static int16_t H2, H4, H5; static int8_t H6;
static int32_t  t_fine;

static esp_err_t bme_read(uint8_t reg, uint8_t *buf, size_t len) {
    return i2c_master_transmit_receive(bme, &reg, 1, buf, len,
                                       pdMS_TO_TICKS(100));
}

static esp_err_t bme_write(uint8_t reg, uint8_t val) {
    uint8_t b[2] = { reg, val };
    return i2c_master_transmit(bme, b, 2, pdMS_TO_TICKS(100));
}

esp_err_t bme_init(i2c_master_bus_handle_t bus) {
    i2c_device_config_t cfg = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address = BME_ADDR,
        .scl_speed_hz = 100000,
    };
    ESP_RETURN_ON_ERROR(i2c_master_bus_add_device(bus, &cfg, &bme),
```

```

TAG, "не додано пристрій на шину");

// регістр ідентифікації: доводить, що обмін працює (розділ 44)
uint8_t id = 0;
ESP_RETURN_ON_ERROR(bme_read(REG_ID, &id, 1), TAG, "немає відповіді");
if (id != 0x60) {
    ESP_LOGE(TAG, "чужий пристрій: id 0x%02x, очікували 0x60", id);
    return ESP_ERR_NOT_FOUND;
}

uint8_t c[26];
ESP_RETURN_ON_ERROR(bme_read(REG_CALIB1, c, 26), TAG, "калібрування");
T1 = c[0] | (c[1] << 8); T2 = c[2] | (c[3] << 8);
T3 = c[4] | (c[5] << 8); P1 = c[6] | (c[7] << 8);
P2 = c[8] | (c[9] << 8); P3 = c[10] | (c[11] << 8);
P4 = c[12] | (c[13] << 8); P5 = c[14] | (c[15] << 8);
P6 = c[16] | (c[17] << 8); P7 = c[18] | (c[19] << 8);
P8 = c[20] | (c[21] << 8); P9 = c[22] | (c[23] << 8);
H1 = c[25];

uint8_t h[7];
ESP_RETURN_ON_ERROR(bme_read(REG_CALIB2, h, 7), TAG, "калібрування H");
H2 = h[0] | (h[1] << 8);
H3 = h[2];
// старший байт H4 і H5 знаковий — розширення знака обов'язкове
H4 = ((int16_t)(int8_t)h[3] * 16) | (h[4] & 0x0F);
H5 = ((int16_t)(int8_t)h[5] * 16) | (h[4] >> 4);
H6 = (int8_t)h[6];

bme_write(REG_CTRL_H, 0x01); // osrs_h = x1
bme_write(REG_CONFIG, 0xA8); // t_sb 1000 мс, фільтр x4
bme_write(REG_CTRL_M, 0x27); // osrs_t x1, osrs_p x1, режим normal
ESP_LOGI(TAG, "BME280 знайдено і налаштовано");
return ESP_OK;
}

```

УВАГА

Регістр ідентифікації читається **першим** і перевіряється. Це доводить, що обмін по шині працює, ще до того, як з'являться перше значення (розділ 44).

Без цієї перевірки несправна шина дає нулі, які виглядають як правдоподібні дані.

Порядок трьох записів не довільний: за datasheet зміна `ctrl_hum` набуває чинності **лише після запису в `ctrl_meas`**, тому `ctrl_meas` завжди останній. Записаний першим, він лишив би вологість невимірюною — і датчик віддавав би нулі, схожі на дані.

УВАГА

Два зсуви в розборі калібрування виглядають однаково і такими не є. H4 і H5 — **знакові** 16-бітові величини, і старший байт кожної береться зі знаком:

$(\text{int16_t})(\text{int8_t})h[3] * 16$, а не $h[3] \ll 4$. Різниця виникає лише тоді, коли в старшому байті виставлено сьомий біт, тобто на частині екземплярів датчика — і виявляється як стабільно неправильна вологість при правильних температурі й тиску.

Це те місце, де варто звернутися з референсною реалізацією виробника, а не з чужим прикладом: у прикладах в інтернеті ця помилка трапляється частіше, ніж правильний варіант.

Перетворення сирих відліків — за формулами з datasheet. Без калібрувальних коефіцієнтів значення виглядають правдоподібно і є неправильними:

```
esp_err_t bme_measure(float *temp, float *hum, float *pres) {
    uint8_t d[8];
    ESP_RETURN_ON_ERROR(bme_read(REG_DATA, d, 8), TAG, "читання");

    int32_t adc_p = (((uint32_t)d[0] << 12) | ((uint32_t)d[1] << 4) | (d[2] >> 4));
    int32_t adc_t = (((uint32_t)d[3] << 12) | ((uint32_t)d[4] << 4) | (d[5] >> 4));
    int32_t adc_h = (((uint32_t)d[6] << 8) | (uint32_t)d[7]);

    int32_t v1 = (((adc_t >> 3) - ((int32_t)T1 << 1))) * ((int32_t)T2) >> 11;
    int32_t v2 = (((((adc_t >> 4) - ((int32_t)T1)) *
        ((adc_t >> 4) - ((int32_t)T1))) >> 12) * ((int32_t)T3)) >> 14;
    t_fine = v1 + v2;
    *temp = ((t_fine * 5 + 128) >> 8) / 100.0f;

    int64_t p1 = ((int64_t)t_fine) - 128000;
    int64_t p2 = p1 * p1 * (int64_t)P6 + ((p1 * (int64_t)P5) << 17)
        + (((int64_t)P4) << 35);
    p1 = ((p1 * p1 * (int64_t)P3) >> 8) + ((p1 * (int64_t)P2) << 12);
    p1 = (((((int64_t)1) << 47) + p1) * ((int64_t)P1)) >> 33;
    if (p1 == 0) return ESP_ERR_INVALID_STATE;
    int64_t p = 1048576 - adc_p;
    p = (((p << 31) - p2) * 3125) / p1;
    p1 = (((int64_t)P9) * (p >> 13) * (p >> 13)) >> 25;
    p2 = (((int64_t)P8) * p) >> 19;
    *pres = (((p + p1 + p2) >> 8) + (((int64_t)P7) << 4)) / 25600.0f;

    int32_t h = t_fine - 76800;
    h = (((((adc_h << 14) - (((int32_t)H4) << 20) - (((int32_t)H5) * h)) +
        16384) >> 15) * ((((((h * ((int32_t)H6)) >> 10) *
        ((h * ((int32_t)H3)) >> 11) + 32768)) >> 10) + 2097152) *
        ((int32_t)H2) + 8192) >> 14));
    h -= (((((h >> 15) * (h >> 15)) >> 7) * ((int32_t)H1)) >> 4);
    if (h < 0) h = 0;
    if (h > 419430400) h = 419430400;
    *hum = (h >> 12) / 1024.0f;
    return ESP_OK;
}
```

Кільцевий буфер історії

```
#define ISTORIYA 720 // 12 годин при вимірюванні раз на хвилину

typedef struct {
    int64_t chas; // мікросекунди від старту
    float temp, hum, pres;
    bool valid;
} zapys_t;

static zapys_t istoriya[ISTORIYA];
static size_t idx = 0, kilkist = 0;
static SemaphoreHandle_t mutex;

static void dodaty(float t, float h, float p, bool ok) {
    xSemaphoreTake(mutex, portMAX_DELAY);
    istoriya[idx] = (zapys_t){ esp_timer_get_time(), t, h, p, ok };
    idx = (idx + 1) % ISTORIYA;
    if (kilkist < ISTORIYA) kilkist++;
    xSemaphoreGive(mutex);
}
```

Буфер виділяється **статично**, один раз. Ніякого malloc у циклі (розділ 30).

Задача вимірювання

```
static void task_vymir(void *arg) {
    int pomylok_pospil = 0;
    while (1) {
        float t, h, p;
        esp_err_t err = bme_measure(&t, &h, &p);
        if (err == ESP_OK) {
            pomylok_pospil = 0;
            dodaty(t, h, p, true);
            ESP_LOGI(TAG, "%.2f °C, %.1f %, %.1f rPa", t, h, p);
        } else {
            pomylok_pospil++;
            dodaty(0, 0, 0, false);
            ESP_LOGW(TAG, "датчик не відповідає (%d поспіль): %s",
                pomylok_pospil, esp_err_to_name(err));
            // деградуємо, а не перезавантажуємось (розділ 32)
        }
        ESP_LOGD(TAG, "вільно RAM: %lu, мінімум: %lu",
            esp_get_free_heap_size(), esp_get_minimum_free_heap_size());
        vTaskDelay(pdMS_TO_TICKS(60000));
    }
}
```

Датчик, що замовк, не зупиняє пристрій: записується позначка про збій, і робота триває. Веб-інтерфейс покаже, що дані застаріли.

Веб-сервер

```
static esp_err_t json_handler(httpd_req_t *req) {
    char *buf = malloc(16384);
    if (!buf) return httpd_resp_send_500(req);
```

```

xSemaphoreTake(mutex, portMAX_DELAY);
int n = snprintf(buf, 16384, "{\\"zapysiv\\":%u,\\"dani\\":[", kilkist);
size_t start = (kilkist == ISTORIYA) ? idx : 0;
bool pershyy = true; // ← не «i == 0», див. нижче
for (size_t i = 0; i < kilkist && n < 16000; i++) {
    zapys_t *z = &istoriya[(start + i) % ISTORIYA];
    if (!z->valid) continue;
    n += snprintf(buf + n, 16384 - n,
        "%s{\\"t\\":%lld,\\"temp\\":%.2f,\\"hum\\":%.1f,\\"pres\\":%.1f}",
        pershyy ? "" : ",",
        z->chas / 1000000, z->temp, z->hum, z->pres);
    pershyy = false;
}
xSemaphoreGive(mutex);
snprintf(buf + n, 16384 - n, "]}");

httpd_resp_set_type(req, "application/json");
esp_err_t r = httpd_resp_sendstr(req, buf);
free(buf);
return r;
}

```

УВАГА

Окремий прапорець `pershyy` замість перевірки `i == 0` — не педантизм. Записи зі збоєм пропускаються через `continue`, тож індекс циклу `i` і номер **виведеного** елемента розходяться. Варіант `i ? ", " : ""` при першому ж збійному запису на початку історії поставить кому перед першим елементом, і JSON стане несинтаксичним: `"dani":[, {...}]`.

Ламається це рівно тоді, коли датчик відмовив, — тобто саме тоді, коли на графік дивляться. Це типова форма помилки в цій книзі: код правильний для щасливого шляху й невірний для того, заради якого писався.

УВАГА

Буфер тут виділяється з купи й одразу звільняється — це **не** цикл виділень, а разова операція на запит. Різниця з правилом розділу 30 у тому, що частота низька й розмір фіксований.

Уважніше треба з іншим: обробник виконується в задачі веб-сервера з обмеженим стеком. Тому 16 КБ беруться з купи, а не оголошуються як локальний масив — інакше стек переповниться (розділ 30).

Головна функція

```

void app_main(void) {
    ESP_LOGI(TAG, "старт, причина скидання: %d", esp_reset_reason());

    esp_err_t err = nvs_flash_init();
    if (err == ESP_ERR_NVS_NO_FREE_PAGES || err == ESP_ERR_NVS_NEW_VERSION_FOUND) {

```

```

    ESP_ERROR_CHECK(nvs_flash_erase());
    err = nvs_flash_init();
}
ESP_ERROR_CHECK(err);

mutex = xSemaphoreCreateMutex();

i2c_master_bus_config_t bus_cfg = {
    .i2c_port = I2C_NUM_0,
    .sda_io_num = PIN_SDA,    // classic 21 / S3 8 — див. таблицю пінів
    .scl_io_num = PIN_SCL,    // classic 22 / S3 9
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .glitch_ignore_cnt = 7,
};
i2c_master_bus_handle_t bus;
ESP_ERROR_CHECK(i2c_new_master_bus(&bus_cfg, &bus));

if (bme_init(bus) != ESP_OK) {
    ESP_LOGE(TAG, "датчик не знайдено — працюємо без нього");
    // не ESP_ERROR_CHECK: веб-інтерфейс має піднятися й показати проблему
}

wifi_start();                // під'єднання з повторами, розділ 39
mdns_init();
mdns_hostname_set("teplytsia");
mdns_service_add(NULL, "_http", "_tcp", 80, NULL, 0);

web_start();
xTaskCreate(task_vymir, "vymir", 4096, NULL, 5, NULL);
}

```

НЕЗВОРОТНЕ

ESP_ERROR_CHECK тут стоїть лише навколо NVS і створення шини — того, без чого пристрій не має сенсу.

Навколо ініціалізації датчика його **немає** свідомо: несправний датчик не повинен перетворювати пристрій на цеглину. Веб-інтерфейс має піднятися й показати, що датчик мовчить (розділ 32).

Збирання і перевірка

```

idf.py set-target esp32s3
idf.py menuconfig          # Wi-Fi, розбивка флешу з OTA (розділ 18)
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor

```

Порядок перевірки:

1. У лозі — BME280 знайдено і налаштовано. Немає — сканер I²C (розділ 35).
2. Перше вимірювання з осмисленими значеннями.
3. `teplytsia.local` відкривається у браузері.

4. Від'єднати датчик на ходу: пристрій лишається живим, у лозі попередження, веб показує застарілі дані.
5. Вимкнути роутер: вимірювання тривають, після відновлення веб знову доступний.
6. Доба безперервної роботи: мінімум вільної пам'яті не зменшується (розділ 58).

Розвиток

- **MQTT** замість або разом із веб-інтерфейсом (розділ 40);
- **другий датчик** — DS18B20 на вулиці (розділ 37);
- **ОТА** — розбивку вже закладено (розділ 19);
- **e-rareg** для показу на місці (розділ 46);
- **автономність**: перехід на deep sleep і ESP-NOW перетворює це на проєкт 60.

60. Автономний логер

Пристрій прокидається за розкладом, міряє, записує на картку й засинає. Живе від акумулятора місяцями.

Головна тема проєкту — **бюджет енергії** (розділ 06) і надійність запису при зникненні живлення (розділ 49).

Постановка

Вхід: BME280 (I²C) і DS18B20 (1-Wire) для виносного вимірювання.

Вихід: файл CSV на microSD, один рядок на вимірювання.

Живлення: 18650, ціль — не менше трьох місяців при вимірюванні раз на 15 хвилин.

Час: RTC DS3231 із власною батареєю — мережі немає, SNTP недоступний (розділ 40).

Поведінка при відмові: картка недоступна — вимірювання накопичуються в RTC RAM; датчик мовчить — записується позначка; акумулятор розряджений — пристрій засинає назавжди, не псуючи картку.

Складові

Позиція	Кількість	Примітка
ESP32 classic або C3	1	не плата розробки для фінальної версії
BME280	1	I ² C
DS18B20 у зонді	1	1-Wire, резистор 4.7 кОм
DS3231 модуль RTC	1	I ² C, з батареєю CR2032
Модуль microSD	1	SPI
18650 з захистом	1	розділ 53
TP4056 з захистом	1	розділ 53
Перетворювач buck-boost на 3.3 В	1	ключове рішення, див. нижче
Резистори 4.7 кОм	3	I ² C і 1-Wire
MOSFET малий + резистори	1	ключ дільника вимірювання напруги

Піни за платами

Проекту потрібно вісім пінів, і на двох сімействах вони **різні повністю**, а не одним номером.

Сигнал	classic	C3
ADC дільника	GPI034	GPI03
Ключ дільника (вихід)	GPI013	GPI02 ⚠
I ² C SDA / SCL	GPI021 / GPI022	GPI04 / GPI05
1-Wire	GPI04	GPI01
SPI SCK / MOSI / MISO	GPI018 / GPI023 / GPI019	GPI06 / GPI07 / GPI010
SPI cs microSD	GPI05	GPI00

classic Четвірка 18/23/19/5 на classic — це рідні піни SPI3 (VSPI) один в один. **C3** На C3 рідними лишилися тільки SCK і MOSI: рідний MISO там GPI02, а він уже пішов під ключ дільника. Для microSD це не коштує нічого — межа матриці 40 МГц (розділ 36).

classic Один пін у цій таблиці варто назвати чесно: **GPI05 на classic — теж strapping-пін**, як і GPI02 на C3. Позначки ⚠ він не отримав, і це послідовно: CS — вихід, у спокої високий, а strapping GPI05 має внутрішнє підтягування вгору й задає таймінги SDIO-веденого, яких у цьому проєкті немає. Ризику тут немає, але правило книги — називати strapping-піни там, де їх беруть, — діє на обидві колонки.

НЕЗВОРОТНЕ

C3 Жодного з пінів GPI022, GPI023, GPI034 на C3 не існує. У C3 усього 22 піни, GPI00–GPI021, і класична розпіновка з розділу 07 переноситься на нього не заміною чисел, а перерозподілом усіх трьох шин.

Ще гірше те, що з цих 22 пінів вільні далеко не всі: GPI012–GPI017 зайняті флешем, GPI018 і GPI019 — USB-Serial-JTAG, GPI020 і GPI021 — консоль UART0, а GPI02, GPI08, GPI09 — strapping.

І окремо GPI011 — це живлення флешу. Його майданчик у матриці називається VDD_SPI, і за замовчуванням він не GPIO взагалі, а вивід, з якого живиться сама флеш-пам'ять. Перевести його в GPIO можна лише пропаленням eFuse VDD_SPI_A5_GPIO — **незворотним**, як усі eFuse (картка K11). На модулі з внутрішнім флешем це означає забрати в флешу живлення, тобто перетворити модуль на цеглину.

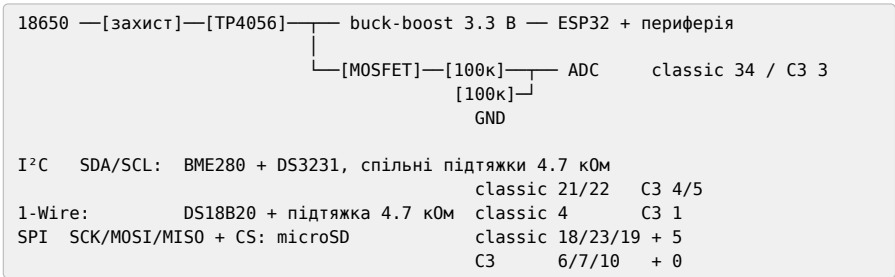
Тобто відняти треба тринадцять пінів, а не дванадцять. Лишається рівно вісім безумовно вільних: 0, 1, 3, 4, 5, 6, 7, 10.

Проекту треба **дев'ять**. Тому ключ дільника доводиться вішати на strapping-пін GPIO2 — це припустимо лише тому, що він працює тут **виключно як вихід** і лише після старту (розділ 07), а зовнішньої обв'язки на ньому немає.

Тобто **цей проєкт вичерпує C3 повністю й трохи більше**. Додати ще один датчик буде нікуди. Це саме той випадок, коли вибір чипа робиться на етапі схеми, а не після: на classic такої тісноти немає, і саме тому в BOM він стоїть першим.

Схема

Нижче — варіант для classic. Для C3 підставте піни з таблиці вище.



ЖИВЛЕННЯ

Buck-boost, а не LDO — головне рішення проєкту. Звичайний LDO перестає давати 3.3 В, коли акумулятор просів до 3.8 В, і відсікає приблизно половину ємності. Buck-boost працює до 3.0 В.

Різниця — вдвічі за часом роботи, і вона більша за будь-яку оптимізацію коду (розділ 53).

НЕЗВОРОТНЕ

Дільник вимірювання напруги вмикається транзистором. Постійно під'єднаний дільник із двох резисторів по 100 кОм бере на порядок більше, ніж чип уві сні (числа — розділ 33), тобто стає головним споживачем усього пристрою.

Без ключа розрахунок на три місяці перетворюється на три тижні (розділ 53).

Піни в коді

Уся розпіновка — в одному місці нагорі, а не розсіяна по викликах. Це не охайність заради охайності: саме розсіяні числа й роблять перенесення на інший чип неможливим без пропусків.

```
// Піни й канал ADC за платою (таблиця вище).
#ifdef CONFIG_IDF_TARGET_ESP32C3
# define PIN_SDA      GPIO_NUM_4
# define PIN_SCL      GPIO_NUM_5
# define PIN_I2C_WIRE GPIO_NUM_1
# define PIN_SCK      GPIO_NUM_6
# define PIN_MOSI     GPIO_NUM_7
# define PIN_MISO     GPIO_NUM_10
# define PIN_CS_SD     GPIO_NUM_0
# define PIN_DILNYK_EN GPIO_NUM_2 // ▲ strapping: лише вихід
# define ADC_CHANNEL   ADC_CHANNEL_3 // GPIO3
#else
# define PIN_SDA      GPIO_NUM_21
# define PIN_SCL      GPIO_NUM_22
# define PIN_I2C_WIRE GPIO_NUM_4
# define PIN_SCK      GPIO_NUM_18
# define PIN_MOSI     GPIO_NUM_23
# define PIN_MISO     GPIO_NUM_19
# define PIN_CS_SD     GPIO_NUM_5
# define PIN_DILNYK_EN GPIO_NUM_13
# define ADC_CHANNEL   ADC_CHANNEL_6 // GPIO34 = ADC1_6
#endif
```

УВАГА

СЗ Рядок PIN_DILNYK_EN на СЗ — свідомий компроміс і єдине місце, де довелося взяти strapping-пін. Вільних більше немає, а GPIO2 тут працює **лише як вихід** і лише після старту, тож на завантаження не впливає (розділ 07). Зовнішньої обв'язки на ньому бути не повинно — затвор MOSFET підключається напряму.

Якщо ця умова незручна, висновок той самий, що вище: беріть classic.

Стан, що переживає сон

Deep sleep — це перезавантаження: RAM втрачається, app_main починається спочатку (розділ 06). Переживає сон лише RTC RAM.

```
RTC_DATA_ATTR static uint32_t nomer_cyklu = 0;
RTC_DATA_ATTR static uint32_t pomylok_karty = 0;
RTC_DATA_ATTR static zapys_t bufer[BUFER_ROZMIR]; // накопичення при збої
RTC_DATA_ATTR static uint8_t u_buferi = 0;
```

RTC RAM невелика — одиниці кілобайтів. Буфер на 20–30 записів у неї вміщається; більше — ні.

Головний цикл

```
void app_main(void) {
    nomer_cyklu++;
    ESP_LOGI(TAG, "цикл %lu, причина пробудження: %d",
              nomer_cyklu, esp_sleep_get_wakeup_cause());

    float napruga = zmiryaty_akumulyator(); // з ключем, див. нижче
    if (napruga < 3.2f) {
        ESP_LOGE(TAG, "акумулятор %.2f В – засинаємо назавжди", napruga);
        esp_deep_sleep_start(); // без таймера: не прокинеться
    }

    zapys_t z = { .chas = rtc_chas(), .napruga = napruga };
    z.ok_bme = (bme_measure(&z.temp, &z.hum, &z.pres) == ESP_OK);
    z.ok_ds = (ds18b20_read(&z.temp_zovni) == ESP_OK);

    if (!zapysaty_na_kartku(&z)) {
        pomylok_karty++;
        if (u_buferi < BUFER_ROZMIR) bufer[u_buferi++] = z;
        ESP_LOGW(TAG, "картка недоступна, у буфері %u", u_buferi);
    } else if (u_buferi > 0) {
        skynyty_bufer(); // дописати накопичене
    }

    zasnuty(15 * 60);
}
```

НЕЗВОРОТНЕ

Перевірка напруги стоїть **першою**, до будь-якої роботи з картою.

Причина: запис у картку при просілому живленні — найнадійніший спосіб пошкодити файлову систему FAT так, що втрачається не останній файл, а картка цілком (розділ 49).

Розряджений акумулятор має призводити до чистого засинання, а не до спроби записати ще один рядок.

Вимірювання напруги через ключ

```
static float zmiyaty_akumulyator(void) {
    gpio_set_level(PIN_DILNYK_EN, 1);
    vTaskDelay(pdMS_TO_TICKS(10)); // дати зарядитися ємності

    int sum = 0;
    for (int i = 0; i < 32; i++) { // усереднення (розділ 33)
        int raw;
        adc_oneshot_read(adc, ADC_CHANNEL, &raw);
        sum += raw;
    }
    gpio_set_level(PIN_DILNYK_EN, 0); // вимкнути одразу

    int mv;
    adc_cal_i_raw_to_voltage(cali, sum / 32, &mv);
    return mv * 2.0f / 1000.0f; // дільник 1:2
}
```

Надійний запис на картку

```
static bool zapysaty_na_kartku(const zapys_t *z) {
    if (sd_mount() != ESP_OK) return false;

    FILE *f = fopen(MOUNT "/dani.csv", "a");
    if (!f) { sd_unmount(); return false; }

    fprintf(f, "%lld,%.2f,%.1f,%.1f,%.2f,%.2f,%d,%d\n",
            z->chas, z->temp, z->hum, z->pres,
            z->temp_zovni, z->napruga, z->ok_bme, z->ok_ds);

    fflush(f);
    fsync(fileno(f)); // змусити записати на носій, а не в буфер
    fclose(f);
    sd_unmount(); // розмонтувати одразу
    return true;
}
```

НЕЗВОРОТНЕ

Чотири дії наприкінці — `fflush`, `fsync`, `fclose`, `sd_unmount` — виглядають надмірними і не є ними.

Файл, залишений відкритим, означає, що дані лежать у буфері в RAM. Deep sleep стирає RAM. Зникнення живлення — тим більше.

Правило логера: **відкрив, дописав рядок, закрив, розмонтував**. Ніколи не тримати файл відкритим між циклами (розділ 49).

Засинання

```
static void zasnuty(uint32_t sekund) {
    // вимкнути все, що споживає
    sd_unmount();
    gpio_set_level(PIN_DILNYK_EN, 0);
    gpio_set_level(PIN_ZHYVLENNYA_PERYFERIYI, 0); // ключ живлення датчиків

    esp_sleep_enable_timer_wakeup((uint64_t)sekund * 1000000ULL);
    ESP_LOGI(TAG, "засинаємо на %lu c", sekund);
    esp_deep_sleep_start();
}
```

ЖИВЛЕННЯ
Живлення периферії вмикається ключем. Модуль microSD, BME280 і RTC споживають уві сні — разом це можуть бути мілі-, а не мікроампери. Один MOSFET, що знімає живлення з усієї периферії на час сну, часто економить більше, ніж усі налаштування сну разом. Виняток — RTC DS3231: він має власну батарейку й лишається живим сам.

Бюджет енергії

Розрахунок для циклу раз на 15 хвилин:

Фаза	Час	Струм	Заряд
Пробудження й ініціалізація	300 мс	40 мА	12 мА·с
Вимірювання (DS18B20 — 750 мс)	900 мс	40 мА	36 мА·с
Запис на картку	400 мс	80 мА	32 мА·с
Сон	899 с	30 мкА	27 мА·с
Разом за цикл	900 с		≈107 мА·с

За добу: $96 \text{ циклів} \times 107 \text{ мА}\cdot\text{с} \approx 10\,272 \text{ мА}\cdot\text{с} \approx \mathbf{2.85 \text{ мА}\cdot\text{год}}$.
З 18650 на 2500 мА·год, беручи 70 % на старіння і холод: $1750 / 2.85 \approx 614 \text{ дб}$.

УВАГА
Розрахунок дає **понад рік**, і саме тому його треба перевірити вимірюванням (розділ 58).
Практика зазвичай дає менше, і причини завжди ті самі: струм сну вищий за очікуваний (плата розробки!), холод з’їдає ємність, картка споживає більше при записі, ніж у datasheet.

Ціль «три місяці» в постановці — свідомо занижена. Розрахунок на рік із запасом утричі означає, що три місяці ви отримаєте навіть при неприємних сюрпризах.

Плата розробки для фінальної версії не годиться: USB-міст, стабілізатор і світлодіод споживають уві сні на порядки більше за чип. 20 мА замість 30 мкА перетворює рік на добу (розділ 06).

Перевірка

1. Кілька циклів на столі з логом: значення осмислені, рядки в CSV з'являються.
2. **Виміряти струм сну** USB-тестером або, краще, шунтом і осцилографом (розділ 06). Це головна перевірка проєкту.
3. Вийняти картку на ходу — пристрій продовжує, накопичує в буфер.
4. Вставити назад — накопичене дописується.
5. Знизити напругу живлення лабораторним джерелом до 3.1 В — пристрій має заснути назавжди, не пошкодивши картку.
6. Вимикати живлення грубо, багато разів, зокрема під час запису (розділ 58). Картка має лишатися читабельною.
7. Тиждень безперервної роботи з реальним акумулятором; порівняти реальне падіння напруги з розрахунком.

Розвиток

- **ESP-NOW** замість картки: передавати на приймач замість запису (розділ 42, проєкт 61) — прибирає картку й додає надійності;
- **e-rarerg** для показу останнього значення без витрат енергії (розділ 46);
- **сонячна панель** із контролером заряду;
- **ULP-співпроцесор** для пробудження за подією, а не за розкладом (розділ 03).

61. Радіоканал між двома платами

Надійний обмін між двома пристроями без роутера, без інтернету й без інфраструктури. ESP-NOW із шифруванням, підтвердженням і контролем зв'язку.

Це основа для будь-якої системи «датчик — приймач» і найкорисніший проєкт для автономних вузлів (розділ 42).

Постановка

Задача: передавач раз на хвилину надсилає вимірювання, приймач отримує, показує і віддає далі. Обидва мають знати, чи живий партнер.

Живлення: передавач від акумулятора (deep sleep між передачами), приймач від мережі.

Захист: шифрування; чужий пристрій не може ні прочитати, ні підробити пакет.

Поведінка при відмові: передавач не отримав підтвердження — повторює й накопичує; приймач не бачить пакетів — повідомляє про втрату вузла.

Чому ESP-NOW, а не Wi-Fi

Для передавача на батарейці різниця вирішальна (розділ 42):

	Wi-Fi	ESP-NOW
Час до передачі	1–10 с	10 мс
Заряд на цикл	500 мА·с	5 мА·с
Потрібен роутер	так	ні

Два порядки економії — це різниця між місяцем і роками.

Формат пакета

Спільний заголовок для обох боків. Це те, що варто продумати один раз: змінити формат після розгортання означає перепрошити всі вузли.

```

#define PROTO_VERSIYA 1
#define TYP_VYMR 1
#define TYP_PIDTVERDZH 2

typedef struct __attribute__((packed)) {
    uint8_t versiya; // версія протоколу – перше поле завжди
    uint8_t typ; // тип повідомлення
    uint8_t vuzol; // номер вузла
    uint8_t rezerv;
    uint32_t nomer; // лічильник пакетів: виявляє пропуски
    int32_t temp_x100; // ціле замість float – менше й переносніше
    uint16_t hum_x10;
    uint16_t napruga_mv;
} paket_t;

_Static_assert(sizeof(paket_t) <= 250, "ESP-NOW: максимум 250 байтів");

```

УВАГА

Три рішення в цій структурі, кожне з причиною.

versiya першим полем. Коли формат зміниться, старий приймач зможе розпізнати незнайому версію і не розібрати сміття як дані.

__attribute__((packed)) прибирає вирівнювання, яке компілятор додав би сам. Без нього структура на двох різних чипах може мати різний розмір.

Цілі замість float. Менший розмір, немає питань про представлення чисел, і на чипах без FPU дешевше (розділ 02).

Передавач

```

static const uint8_t MAC_PRYIMACHA[6] = { 0x24, 0x6F, 0x28, 0x11, 0x22, 0x33 };

RTC_DATA_ATTR static uint32_t nomer = 0;
RTC_DATA_ATTR static paket_t bufer[10];
RTC_DATA_ATTR static uint8_t u_buferi = 0;

static volatile bool dostavleno = false;

static void on_sent(const esp_now_send_info_t *info,
                    esp_now_send_status_t status) {
    dostavleno = (status == ESP_NOW_SEND_SUCCESS);
}

static bool nadislaty(const paket_t *p) {
    dostavleno = false;
    if (esp_now_send(MAC_PRYIMACHA, (const uint8_t *)p, sizeof(*p)) != ESP_OK)
        return false;

    // чекати підтвердження рівня кадру – це не гарантія доставки,
    // але воно відрізняє «сусід почув» від «нікого немає»
    for (int i = 0; i < 50 && !dostavleno; i++)
        vTaskDelay(pdMS_TO_TICKS(2));
}

```

```

    return dostavleno;
}

void app_main(void) {
    radio_init(); // Wi-Fi у режимі STA, канал фіксований
    espnow_init_with_key();

    paket_t p = {
        .versiya = PROTO_VERSIYA,
        .typ = TYP_VYMIR,
        .vuzol = NOMER_VUZLA,
        .nomer = ++nomer,
    };
    zmiryaty(&p);

    if (!nadislaty(&p)) {
        if (u_buferi < 10) bufer[u_buferi++] = p;
        ESP_LOGW(TAG, "не доставлено, у буфері %u", u_buferi);
    } else {
        // дійшло – спробувати віддати накопичене
        while (u_buferi > 0 && nadislaty(&bufer[u_buferi - 1]))
            u_buferi--;
    }

    esp_sleep_enable_timer_wakeup(60ULL * 1000000);
    esp_deep_sleep_start();
}

```

УВАГА

esp_now_send повертається до фактичної передачі. Статус приходить у зворотний виклик on_sent, і саме його треба дочекатися перед засинанням — інакше чип засне посеред передачі.

Це та сама помилка, що з uart_wait_tx_done у RS-485 (розділ 34), і проявляється так само: «інколи не доходить».

Сигнатура зворотного виклику змінилася: до ESP-IDF 5.4 першим аргументом був const uint8_t *mac_addr, тепер — const esp_now_send_info_t *. Приклади з інтернету старшого віку не зберуться, і повідомлення компілятора вкаже на тип, а не на причину.

Шифрування

```
static void espnow_init_with_key(void) {
    ESP_ERROR_CHECK(esp_now_init());
    ESP_ERROR_CHECK(esp_now_register_send_cb(on_sent));

    uint8_t pmk[16], lmk[16];
    nvs_read_key("pmk", pmk);           // з NVS, не з коду
    nvs_read_key("lmk", lmk);
    ESP_ERROR_CHECK(esp_now_set_pmk(pmk));

    esp_now_peer_info_t peer = { .channel = KANAL, .encrypt = true };
    memcpy(peer.peer_addr, MAC_PRYIMACHA, 6);
    memcpy(peer.lmk, lmk, 16);
    ESP_ERROR_CHECK(esp_now_add_peer(&peer));
}
```

НЕЗВОРОТНЕ

Ключі читаються з **NVS**, а не зашиті в код. Зашитий ключ дістається з дампа прошивки за п'ять хвилин (розділи 24, 50).

І ключі мають бути **різні на кожен пару вузлів**, а не один на всю систему: інакше захоплення одного датчика компрометує всю мережу.

Записуються при виробництві разом із номером вузла (розділ 21).

Приймач

```
typedef struct {
    uint32_t ostanniy_nomer;
    int64_t ostanniy_chas;
    uint32_t vtracheno;
    bool zhyvyy;
} stan_vuzla_t;

static stan_vuzla_t vuzly[MAX_VUZLIV];
static QueueHandle_t cherga;

static void on_recv(const esp_now_recv_info_t *info,
                    const uint8_t *data, int len) {
    // виконується в контексті задачі Wi-Fi: скопіювати й вийти (розділ 42)
    if (len != sizeof(paket_t)) return;
    paket_t p;
    memcpy(&p, data, sizeof(p));
    xQueueSend(cherga, &p, 0);    // не FromISR: це задача, а не переривання
}

static void task_obrobka(void *arg) {
    paket_t p;
    while (xQueueReceive(cherga, &p, portMAX_DELAY) == pdTRUE) {
        if (p.versiya != PROTO_VERSIYA) {
            ESP_LOGW(TAG, "чужа версія протоколу: %u", p.versiya);
            continue;
        }
    }
}
```

```

    if (p.vuzol >= MAX_VUZZIV) continue;

    stan_vuzla_t *v = &vuzly[p.vuzol];
    if (v->ostanniy_nomer && p.nomer > v->ostanniy_nomer + 1)
        v->vtracheno += p.nomer - v->ostanniy_nomer - 1;

    v->ostanniy_nomer = p.nomer;
    v->ostanniy_chas = esp_timer_get_time();
    v->zhyvvy = true;

    ESP_LOGI(TAG, "вузол %u: %.2f °C, %.1f %, %.2f B "
                "(пакет %lu, втрачено %lu)",
             p.vuzol, p.temp_x100 / 100.0f, p.hum_x10 / 10.0f,
             p.napruga_mv / 1000.0f, p.nomer, v->vtracheno);

    vidaty_dali(&p); // MQTT, веб, дисплей
}
}

```

Контроль зв'язку

Приймач має сам помічати, що вузол зник, — не чекати, поки хтось спитає:

```

static void task_kontrol(void *arg) {
    while (1) {
        int64_t teper = esp_timer_get_time();
        for (int i = 0; i < MAX_VUZZIV; i++) {
            stan_vuzla_t *v = &vuzly[i];
            if (!v->zhyvvy) continue;
            // три пропущені інтервали — вважаємо вузол мертвим
            if (teper - v->ostanniy_chas > 3 * 60 * 1000000LL) {
                v->zhyvvy = false;
                ESP_LOGE(TAG, "вузол %d не виходить на зв'язок", i);
                povidomyty_pro_vtratu(i);
            }
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

```

Лічильник `vtracheno` — безкоштовна оцінка якості зв'язку. Він одразу показує, чи проблема в конкретному вузлі, чи в загальних умовах.

Канал: головна складність розгортання

НЕЗВОРОТНЕ

Усі учасники мають бути **на одному каналі**. Якщо приймач також під'єднаний до Wi-Fi, його канал визначає **роутер** — і більшість роутерів обирають канал автоматично й змінюють його самі.

Це найчастіша причина «працювало на столі, на об'єкті перестало» (розділ 42).

Три робочі варіанти:

Фіксований канал у роутері. Найпростіше, якщо роутер ваш.

Приймач без Wi-Fi. Тільки ESP-NOW на фіксованому каналі, дані далі йдуть по дроту.

Два чипи в приймачі. Один слухає ESP-NOW на фіксованому каналі, другий тримає Wi-Fi; між ними UART (розділ 34). Найнадійніше і найдорожче.

Перевірка

1. Дві плати поруч: пакети доходять, лічильник росте без пропусків.
2. Рознести на робочу відстань, тиждень роботи: подивитися *vtacheno*.
3. Вимкнути передавач: приймач має за три хвилини повідомити про втрату.
4. Увімкнути назад: накопичене в буфері має дійти.
5. Спробувати надіслати пакет із третього пристрою без ключа: приймач має його не прийняти.
6. Виміряти струм передавача за цикл — має бути частки міліампер-секунди (розділ 60).

Розвиток

- **Кілька передавачів на один приймач** — структура вже готова (масив *vuzly*);
- **Двонаправлений обмін:** приймач надсилає команди у відповідь на пакет, поки передавач не заснув;
- **Заміна на LoRa** (розділ 43), коли потрібні кілометри: формат пакета й логіка лишаються, змінюється транспорт;
- **Ретрансляція** через проміжний вузол для збільшення покриття.

62. Керування виконавчими механізмами

Пристрій вмикає й вимикає щось реальне: насос, клапан, нагрівач, двигун. Тема проекту — **безпечна поведінка**, а не функціональність.

Різниця з попередніми проектами принципова: помилка тут має фізичні наслідки. Насос, що не вимкнувся, заливає приміщення; нагрівач — підпалює.

Постановка

Задача: керувати насосом за розкладом і за датчиком рівня, з можливістю ручного втручання по мережі.

Виходи: реле насоса, індикація стану.

Входи: датчик рівня (поплавковий вимикач), кнопка «стоп», кнопка ручного пуску.

Безпека:

- насос вимкнений при зникненні живлення, при зависанні, при перезавантаженні;
- жорсткий ліміт часу безперервної роботи;
- захист від сухого ходу;
- аварійна зупинка апаратна, не програмна;
- втрата зв'язку не змінює стан механізму.

Складові й піни

Позиція	Кількість	Примітка
ESP32 classic DevKitC	1	схема нижче — для classic
Модуль реле з оптопарою	1	живлення котушки 5 В, розділ 47
Аварійний вимикач	1	механічний, у силовому колі
Поплавковий вимикач	1	1. зовнішня підтяжка 10 кОм
Кнопки «пуск» і «стоп»	2	
Резистори 220 Ом, 10 кОм	по 1	

Сигнал	classic	S3	C3
Вихід на реле	GPIO25	GPIO4	GPIO4
Поплавковий вимикач	GPIO34	GPIO5	GPIO5
Кнопка «стоп»	GPIO26	GPIO6	GPIO6
Кнопка «пуск»	GPIO27	GPIO7	GPIO7

УВАГА

Схема й код нижче написані під classic, бо GPIO34 там **тільки вхідний** — і це принципово саме для цього проєкту: пін, який фізично не вміє бути виходом, не може випадково подати сигнал на реле через помилку в коді.

На S3 і C3 тільки-вхідних пінів немає взагалі (розділ 07), тож цю властивість там не отримати. Замість неї доведеться покладатися на дисципліну коду — а в проєкті, де помилка заливає приміщення, це гірший захист.

Тому вибір classic тут не інерція, а рішення. Числа для інших сімейств наведені, щоб перенесення було свідомим, а не «замінити 34 на щось».

Безпечний стан: апаратно, до коду

НЕЗВОРОТНЕ

Питання, з якого починається проєкт: що станеться, якщо чип зникне просто зараз?

Відповідь має бути «насос вимкнеться», і забезпечується вона схемотехнікою, а не програмою (розділи 32, 47).

Три речі, які треба зробити на платі:

Резистор 10 кОм, що утримує керувальний вхід у стані «вимкнено».

Під час завантаження GPIO перебуває у високоімпедансному стані — без резистора вхід висить, і реле може ввімкнутися саме в момент старту. При boot loop (розділ 20) це означає блимання насосом раз на секунду. Куди саме йде цей резистор — на землю чи на 3V3 — залежить від того, яким рівнем вмикається ваш модуль; таблиця в наступному підрозділі.

Реле з нормально розімкненими контактами. Знеструмлене реле = вимкнений насос. Ніколи навпаки.

Апаратний аварійний вимикач у розрив живлення насоса, не в логіку.

Кнопка, що подає сигнал на GPIO, не працює, коли чип завис. Кнопка, що розриває коло, працює завжди.

Перевірка цих трьох речей — перший пункт випробувань, і робиться вона до написання логіки: подати живлення, поспостерігати за реле під час завантаження; висмикнути живлення під час роботи насоса.

Схема

Силове коло — усе послідовно, і аварійний вимикач у ньому фізично:

+12 В — [насос] — [реле: контакти N0] — [аварійний вимикач] — GND

Коло керування реле — окреме, від 5 В, а не від 3V3:

```

5 В — VCC модуля реле
GND — GND модуля (спільна з ESP32)

GPIO25 — [220 Ом] — IN модуля
      |
      + [10 кОм] ← напрямок підтяжки залежить від модуля, див. нижче
      |
      GND або 3V3
  
```

Входи:

```

GPIO34 — поплавковий вимикач — GND (+ зовнішня підтяжка 10 кОм до 3V3!)
GPIO26 — кнопка «стоп» — GND (внутрішня підтяжка)
GPIO27 — кнопка «пуск» — GND
  
```

НЕЗВОРОТНЕ

Два питання до релейного модуля, які треба закрити до монтажу (розділ 47).

Від чого живиться котушка. Більшість готових модулів розраховані на 5 В. Від 3.3 В реле або не спрацює, або спрацює через раз — класичне «іноді вмикається». Тому VCC модуля йде на 5 В, а не на 3V3.

Чи приймає вхід 3.3 В. Наявність оптопари цього **не гарантує**, хоч так часто пишуть. Світлодіод оптопари живиться від того самого VCC через резистор, підібраний під 5 В, і при керуванні від 3.3 В струм крізь нього падає — на частині модулів нижче порога спрацювання. Симптом той самий «іноді вмикається», лише причина інша.

Перевірити треба до монтажу, і це п'ять хвилин: подати 3.3 В на вхід і переконатися, що реле клацає надійно, а не на межі. Надійніше — зміряти струм через вхід і звірити з порогом оптопари в її datasheet. Якщо струму бракує — транзисторний ключ між GPIO і входом модуля (розділ 47) або модуль, у якого вхідне коло розраховане на 3.3 В.

Яким рівнем вмикається. Багато модулів **інверсні**: реле вмикається логічним нулем. Від цього залежить напрямок підтяжки, і помилитися тут означає отримати рівно те, від чого весь цей розділ:

Модуль вмикається	Підтяжка 10 кОм	Стан під час завантаження
високим рівнем	на GND	реле вимкнене
низьким рівнем (інверсний)	на 3V3	реле вимкнене

Правило одне і формулюється не через напрямок, а через результат: **резистор утримує вхід у стані «вимкнено», поки GPIO ще не налаштований**. Перевіряється це не за схемою, а дослідом: подати живлення десять разів і подивитися на реле (пункт 1 випробувань нижче).

УВАГА

classic GPIO34 — тільки вхід і без вбудованого підтягування (розділ 07).

Поплавковому вимикачу потрібен зовнішній резистор 10 кОм до 3.3 В, інакше вхід бовтається й насос вмикається від наводок.

Це саме той випадок, коли «датчик іноді спрацьовує сам» — не датчик, а відсутній резистор.

Автомат станів

Логіка керування будується як явний автомат, а не набором прапорців. Стан завжди один, переходи явні, і кожен перехід логується.

```
typedef enum {
    STAN_STOP,           // зупинено, чекає команди
    STAN_ROBOTA,         // насос працює
    STAN_AVARIYA,        // помилка, потрібне ручне скидання
    STAN_BLOKUVANNYA,    // пауза після роботи
} stan_t;

static stan_t stan = STAN_STOP;
static int64_t stan_vid;

static void perejty(stan_t novyy, const char *prychyna) {
    if (novyy == stan) return;
    ESP_LOGI(TAG, "%s -> %s: %s", nazva(stan), nazva(novyy), prychyna);
    stan = novyy;
    stan_vid = esp_timer_get_time();
    nasos_keruvaty(novyy == STAN_ROBOTA);
    onovyty_indykaciyu();
}
```

Логувати перехід із **причиною** — те, що робить лог придатним для розбору через місяць (розділ 25).

Захисти

```
#define MAX_ROBOTY_S    600    // 10 хвилин безперервно
#define PAUZA_PISLYA_S  300    // 5 хвилин відпочинку
#define ZV_YAZOK_TAIMAUT 120  // втрата зв'язку

static void task_keruvannya(void *arg) {
    esp_task_wdt_add(NULL);
    while (1) {
        esp_task_wdt_reset();
        int64_t u_stani = (esp_timer_get_time() - stan_vid) / 1000000;

        // 1. Аварійна кнопка — найвищий пріоритет
        if (!gpio_get_level(PIN_STOP))
            perejty(STAN_AVARIYA, "натиснуто STOP");

        // 2. Сухий хід: працюємо, а рівня немає
        if (stan == STAN_ROBOTA && !riven_je() && u_stani > 10)
            perejty(STAN_AVARIYA, "сухий хід: немає рівня");

        // 3. Ліміт часу безперервної роботи
        if (stan == STAN_ROBOTA && u_stani > MAX_ROBOTY_S)
            perejty(STAN_BLOKUVANNYA, "перевищено ліміт часу");

        // 4. Кінець паузи
        if (stan == STAN_BLOKUVANNYA && u_stani > PAUZA_PISLYA_S)
            perejty(STAN_STOP, "пауза завершена");

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

НЕЗВОРОТНЕ

Ліміт часу безперервної роботи — найважливіший захист у проєкті.

Він рятує від помилок **керування**: збрехлого датчика, помилки в логіці, забутої ручної команди, зависання задачі. Насос, що працює десять хвилин замість двох, — незручність; насос, що працює добу, — аварія.

Ліміт ставиться завжди, навіть коли «за логікою такого не буває». Саме те, чого не буває за логікою, і трапляється.

НЕЗВОРОТНЕ

Чого програмний ліміт не рятує — і це треба знати точно.

Ліміт часу знімає сигнал із котушки. Далі все залежить від того, чи розмикається реле, коли котушка знеструмлена.

Відмова	Ліміт рятує?
датчик бреше, логіка помилилася, команда забута	так
задача зависла, а watchdog перезавантажив плату	так, якщо після скидання пін у безпечному стані
контакти реле зварилися	ні
пробитий ключ у силовому колі	ні
обрив у колі керування, реле лишилося замкненим	ні

Зварені контакти — не «залипання», яке можна відпустити. Це фізично з'єднаний метал: зняття струму з котушки не роз'єднує його, і пружина не долає зварювання. Реле в такому стані замкнене назавжди.

Саме тому в силовому колі цього проекту стоїть **аварійний вимикач**, і саме тому він механічний і послідовний (схема вище). Він — єдиний захист від двох нижніх рядків таблиці, і жоден рядок коду його не замінює.

Якщо наслідок відмови серйозніший за калюжу — потрібне не програмне рішення, а схемне: друге реле в розрив, контактор із примусово зв'язаними контактами й контролем стану, або незалежне зняття живлення. Це вже царина норм на безпеку машин, і вибрати там за довідником загального призначення не можна.

НЕЗВОРОТНЕ

Стан AVARІYA не скидається сам. Вихід із нього — лише ручне втручання: кнопка або команда з мережі, і бажано після того, як людина подивилася, що сталося.

Автоматичне скидання аварії перетворює захист на затримку: пристрій циклічно вмикає насос, ловить сухий хід, чекає, вмикає знову.

Втрата зв'язку

НЕЗВОРОТНЕ

Втрата зв'язку не змінює стан механізму — це проектне рішення, і воно має бути свідомим.

Два можливі варіанти, і обирати треба за наслідками:

Продовжити за локальною логікою. Правильно для насоса, що працює за розкладом і датчиком: мережа потрібна для нагляду, а не для роботи.

Зупинитися. Правильно для механізму, яким керує лише оператор: без зв'язку немає нагляду, тому рух припиняється.

Найгірший варіант — **не думати про це**: тоді поведінка визначається випадковим місцем у коді, де мережевий виклик заблокувався або ESP_ERROR_CHECK викликав перезавантаження (розділ 32).

У цьому проекті обрано перший варіант, і він записується в паспорт (розділ 56).

Команди з мережі

```
static esp_err_t cmd_handler(httpd_req_t *req) {
    char buf[64];
    int n = httpd_req_recv(req, buf, sizeof(buf) - 1);
    if (n <= 0) return ESP_FAIL;
    buf[n] = 0;

    if (strcmp(buf, "pusk") == 0) {
        if (stan == STAN_AVARIYA)
            httpd_resp_sendstr(req, "avariya: potriben skydannya");
        else if (!riven_je())
            httpd_resp_sendstr(req, "nemaye rivnya");
        else {
            perejty(STAN_ROBOTA, "команда з мережі");
            httpd_resp_sendstr(req, "ok");
        }
    } else if (strcmp(buf, "stop") == 0) {
        perejty(STAN_STOP, "команда з мережі");
        httpd_resp_sendstr(req, "ok");
    } else if (strcmp(buf, "skydannya") == 0) {
        if (stan == STAN_AVARIYA) perejty(STAN_STOP, "ручне скидання");
        httpd_resp_sendstr(req, "ok");
    }
    return ESP_OK;
}
```

УВАГА

Команда **stop** виконується завжди й без умов. Це правило для будь-якого керування: зупинка не має перевірок, не має підтверджень і не залежить від поточного стану.

Усе інше може бути відхилене; зупинка — ніколи.

Обробник ідемпотентний: «стоп» двічі означає те саме, що один раз (розділ 40).

Індикація

Стан має бути видимим на місці, без мережі:

Світлодіод	Стан
не горить	STOP
горить рівно	РОБОТА
блимає повільно	БЛОКУВАННЯ (пауза)
блимає швидко	АВАРІЯ

Людина біля пристрою мусить розуміти, що відбувається, не відкриваючи бра-узер (розділ 56).

Перевірка

Порядок обов'язковий, і перші три пункти — до підключення насоса:

1. **Реле при завантаженні.** Подати живлення десять разів, спостерігати за реле. Жодного клацання під час старту.
2. **Зникнення живлення.** Висмикнути живлення при ввімкненому реле — реле має відпустити.
3. **Аварійний вимикач.** Розриває коло насоса незалежно від стану чипа.
4. **Ліміт часу:** запустити, дочекатися десяти хвилин — має перейти в блокування.
5. **Сухий хід:** запустити й від'єднати датчик рівня — аварія за десять секунд.
6. **Аварія не скидається сама:** почекати п'ять хвилин, стан лишається.
7. **Втрата зв'язку:** вимкнути роутер під час роботи — поведінка відповідає записаній у паспорті.
8. **Зависання:** викликати штучне зависання задачі — watchdog має перезавантажити чип, реле — відпустити.

НЕЗВОРОТНЕ

Пункти 1–3 виконуються **з від'єднанням насосом**. Перевіряти захисти на працюючому механізмі — це перевіряти їх наслідками.

Розвиток

- **Кілька механізмів** із взаємними блокуваннями;
- **Логування подій** у NVS: історія пусків і аварій переживає перезавантаження (розділ 18);
- **Companion-схема** (розділ 57): критичну логіку — на окремий контролер, ESP32 лишити зв'язок;
- **Датчик струму** (розділ 45) для контролю, що насос справді крутиться, а не просто отримав живлення.

63. Протокольний міст

Пристрій, що з’єднує дві сторони, які не розмовляють одна з одною: обладнання з послідовним портом і мережу, RS-485 і Wi-Fi, CAN і MQTT.

Це найпоширеніша реальна роль ESP32 у чужій системі (розділ 57), і проєкт зібраний як каркас, що налаштовується під конкретний випадок.

Постановка

Задача: прозоро передавати дані між послідовним інтерфейсом і мережею, у обидва боки, з логуванням і можливістю налаштувати параметри без перепрошивки.

Варіанти, які покриває один каркас:

Міст	Один бік	Другий бік
Термінал по мережі	UART	TCP-сервер
Modbus TCP → RTU	RS-485	TCP-сервер
Телеметрія	UART або CAN	MQTT
Шлюз шини	CAN	Wi-Fi + MQTT

Поведінка при відмові: зникла мережа — дані з послідовного боку буферизуються; зник послідовний бік — мережевий клієнт отримує повідомлення, а не тишу.

Архітектура

Два незалежні напрямки, кожен зі своєю задачею і чергою. Це ключове рішення: жодна сторона не має блокувати іншу.

UART/RS-485/CAN

|

[task_rx_serial] → cherga_do_merezhi → [task_tx_net]

[task_tx_serial] ← cherga_do_serial ← [task_rx_net]

мережа

|

[task_tx_net]

[task_rx_net]

```
typedef struct {
    uint16_t dovzhyna;
    uint8_t dani[512];
} blok_t;

static QueueHandle_t do_merezhi; // від послідовного боку в мережу
static QueueHandle_t do_serial;  // з мережі в послідовний бік
```

УВАГА

Черги фіксованого розміру — це і буфер, і **захист від перевантаження**. Коли одна сторона швидша за іншу, черга заповнюється, і `xQueueSend` повертає помилку — тобто ви **дізнаєтеся** про перевантаження замість того, щоб мовчки з'їсти пам'ять.

Міст без обмеження буфера при відсутній мережі з'їдає всю купу за хвилини й падає (розділ 30).

Послідовний бік

```
static void task_rx_serial(void *arg) {
    blok_t b;
    while (1) {
        int n = uart_read_bytes(UART_PORT, b.dani, sizeof(b.dani),
                                pdMS_TO_TICKS(20));

        if (n <= 0) continue;
        b.dovzhyna = n;
        if (xQueueSend(do_merezhi, &b, 0) != pdTRUE) {
            vtracheno_do_merezhi += n;
            ESP_LOGW(TAG, "черга в мережу повна, втрачено %d Б", n);
        }
    }
}
```

Читання з коротким таймаутом, а не порція фіксованого розміру: дані з послідовного порту приходять довільними шматками, і чекати повного буфера означає додавати затримку.

Розмір буфера драйвера беруть із запасом: на 115200 бод це близько 11 КБ на секунду, і 256 байтів переповнюються за 22 мілісекунди (розділ 34).

RS-485: керування напрямком

```
static void nadislaty_serial(const blok_t *b) {
    #if REZHYM_RS485
        gpio_set_level(PIN_DE, 1);
    #endif
    uart_write_bytes(UART_PORT, b->dani, b->dovzhyna);
    #if REZHYM_RS485
        uart_wait_tx_done(UART_PORT, portMAX_DELAY); // 0Б0В'ЯЗК0В0
        gpio_set_level(PIN_DE, 0);
    #endif
}
```

НЕЗВОРОТНЕ

`uart_wait_tx_done` перед перемиканням напрямку — найчастіша помилка роботи з RS-485 (розділ 34).

uart_write_bytes лише кладе дані в буфер і повертається одразу. Перемкнути напрямок відразу після нього означає обрізати власну посилку посередині — і виглядає це як «інколи губляться відповіді», що збиває з пантелику надовго.

Мережевий бік: TCP-сервер

```
static void task_tcp(void *arg) {
    int listen_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_addr.s_addr = htonl(INADDR_ANY),
        .sin_port = htons(PORT),
    };
    bind(listen_sock, (struct sockaddr *)&addr, sizeof(addr));
    listen(listen_sock, 1);

    while (1) {
        struct sockaddr_in klient;
        socklen_t len = sizeof(klient);
        int sock = accept(listen_sock, (struct sockaddr *)&klient, &len);
        if (sock < 0) continue;

        ESP_LOGI(TAG, "клієнт під'єднався");
        // очистити чергу: клієнт не має отримати те, що накопичилося
        xQueueReset(do_merezhi);

        obsluhovuvaty(sock);          // до розриву з'єднання
        close(sock);
        ESP_LOGI(TAG, "клієнт від'єднався");
    }
}
```

УВАГА

xQueueReset при під'єднанні нового клієнта — важлива дрібниця. Без неї клієнт одразу отримує все, що накопичилося за час, поки ніхто не слухав: для терміналу це сотні рядків старих даних.

Виняток — міст, де дані не можна втрачати. Тоді, навпаки, накопичене треба віддати, і черга має бути більшою.

Один клієнт одночасно — свідоме обмеження. Кілька клієнтів на одному послідовному порту означають перемішані відповіді: обладнання не знає, кому воно відповідає. Для Modbus це прямий шлях до хаосу.

Налаштування без перепрошивки

Міст, у якого швидкість зашита в код, доводиться перепрошивати при кожній зміні обладнання. Параметри йдуть у NVS (розділ 18):

```
typedef struct {
    uint32_t baud;
    uint8_t data_bits, parity, stop_bits;
    uint8_t rezhym;           // UART, RS-485, CAN
    uint16_t tcp_port;
    char mqtt_uri[128];
    char mqtt_topic[64];
} nalashtuvannya_t;
```

Задаються через веб-форму (розділ 40), зберігаються в NVS, читаються при старті. Скидання на заводські — довгим натисканням кнопки (розділ 39).

Варіант CAN

```
static void task_rx_can(void *arg) {
    twai_message_t msg;
    while (1) {
        if (twai_receive(&msg, pdMS_TO_TICKS(100)) != ESP_OK) continue;

        blok_t b;
        b.dovzhyna = snprintf((char *)b.dani, sizeof(b.dani),
                               "%03lx:%d:", msg.identifier,
                               msg.data_length_code);
        for (int i = 0; i < msg.data_length_code; i++)
            b.dovzhyna += snprintf((char *)b.dani + b.dovzhyna,
                                   sizeof(b.dani) - b.dovzhyna,
                                   "%02x", msg.data[i]);
        b.dani[b.dovzhyna++] = '\n';
        xQueueSend(do_merezhi, &b, 0);
    }
}
```

Текстове представлення обрано свідомо: такий потік читається людиною в терміналі й розбирається будь-яким скриптом без домовленостей про двійковий формат.

НЕЗВОРОТНЕ

Міст на CAN за замовчуванням має працювати в режимі **LISTEN_ONLY** (розділ 38).

Передача в чужу працюючу шину — дія з наслідками: пакет із чужим ідентифікатором може бути сприйнятий як команда керування. У техніці, що рухається або гріє, це не формальність.

Передача вмикається окремим свідомим налаштуванням, і за замовчуванням вона вимкнена.

Діагностика

Міст, який мовчки не працює, — найгірший варіант. Мінімум лічильників, доступних через веб:

```
static struct {
    uint32_t serial_rx, serial_tx;
    uint32_t net_rx, net_tx;
    uint32_t vtracheno_do_merezhi, vtracheno_do_serial;
    uint32_t pomylok_serial;
    int64_t ostannya_aktivnist;
} statystyka;
```

Ці сім чисел відповідають на головне питання діагностики: **на якому боці зупинилося**. Дані йдуть із послідовного, але не йдуть у мережу — проблема в мережі. Не йдуть із послідовного — проблема на тому боці або в налаштуваннях порту.

Без цих лічильників те саме з'ясовується логічним аналізатором і годинами (розділ 28).

Перевірка

1. **Заглушка:** з'єднати TX і RX між собою. Усе, що надіслано з мережі, має повернутися.
2. **Реальне обладнання:** підключити, звірити швидкість і формат.
3. **Обрив мережі** під час обміну: лічильник втрат росте, пристрій живий, після відновлення все працює.
4. **Обрив послідовного боку:** мережевий клієнт отримує повідомлення про це.
5. **Перевантаження:** подати потік швидший, ніж може прийняти другий бік. Черга має переповнитися **з логом**, а не з'їсти пам'ять.
6. **Доба роботи** з реальним трафіком: мінімум вільної пам'яті не зменшується (розділ 58).

Розвиток

- **Modbus RTU ↔ TCP** зі штатним компонентом esp-modbus замість прозорого мосту (розділ 34);
- **Кілька послідовних портів** одночасно;
- **Фільтрація** на боці CAN — приймати лише потрібні ідентифікатори, не турбуючи процесор рештою;
- **Буферизація у флеш** для мостів, де втрата даних неприйнятна;
- **Companion-роль** (розділ 57): міст як постійна частина великої системи, з паспортом і версіонуванням (розділ 56).

Додатки

Кожен додаток — розгорнута версія відповідних карток з тих самих джерел (P10): картка — польова верхівка, додаток — повна таблиця.

Додаток А. Повні пінаут-таблиці

Розгорнута версія картки К9. Обмеження пояснені в розділі 07 тут — повні таблиці для звірки.

УВАГА
Pinout плати важливіший за pinout чипа. Виробник міг вивести не всі піни, підписати їх власними іменами, повісити світлодіод чи резистор. Таблиці нижче описують **чип** конкретну плату звіряйте з її схемою (розділ 08).

ESP32 classic (WROOM-32)

GPIO	Обмеження	ADC	Touch	Примітка
0	strapping	ADC2_1	T1	BOOT; низький = download mode
1	UART0 TX	—	—	консоль
2	strapping	ADC2_2	T2	часто вбудований світлодіод
3	UART0 RX	—	—	консоль
4	—	ADC2_0	T0	вільний
5	strapping	—	—	типовий SPI CS
6–11	флеш — не чіпати	—	—	 ніколи
12	strapping (MTDI)	ADC2_5	T5	 високий при старті = не стартує
13	JTAG TCK	ADC2_4	T4	вільний
14	JTAG TMS	ADC2_6	T6	вільний
15	strapping (MTDO)	ADC2_3	T3	вільний з обережністю
16, 17	—	—	—	вільні; зайняті на WROVER (PSRAM)
18, 19	—	—	—	типові SPI SCK, MISO
21, 22	—	—	—	типові I ² C SDA, SCL
23	—	—	—	типовий SPI MOSI
25	DAC1	ADC2_8	—	

GPIO	Обмеження	ADC	Touch	Примітка
26	DAC2	ADC2_9	—	
27	—	ADC2_7	T7	
32, 33	—	ADC1_4/5	T9/T8	ADC1 — працює при Wi-Fi
34–39	тільки вхід, без підтягування	ADC1	—	

Вільні без застережень: 4, 13, 14, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33.

ADC1 (працює завжди): 32, 33, 34, 35, 36, 37, 38, 39. **ADC2** (не працює при Wi-Fi): 0, 2, 4, 12, 13, 14, 15, 25, 26, 27.

ESP32-S3 (WROOM-1)

GPIO	Обмеження	Примітка
0	strapping	BOOT
3	strapping	
19, 20	native USB D–, D+	втрачаються при перевизначенні
26–32	флеш і PSRAM	не чіпати
33–37	флеш/PSRAM на Octal-модулях	N16R8 і подібні
45, 46	strapping	 46 мусить бути низьким або вільним, інакше download mode недосяжний
1–10	ADC1	працює при Wi-Fi
11–20	ADC2	

Тільки-вхідних пінів немає — усі повнофункціональні.

УВАГА

S3 Модулі N8 і N16R8 мають **різну** кількість доступних пінів: Octal PSRAM з'їдає GPIO 33–37. Перед розводкою плати треба точно знати, який модуль стоїть (розділ 07).

ESP32-C3 (MINI-1, SuperMini)

GPIO	Обмеження	Примітка
2	strapping	
8	strapping	має бути високим для download mode
9	strapping	низький при скиданні = download mode
12–17	флеш	не чіпати
18, 19	USB-Serial-JTAG	втрачається налагодження
0–4	ADC1	
5	ADC2	 разовий режим не підтримується, див. нижче

 Комбінація GPIO8=0 і GPIO9=0 недійсна.

НЕЗВОРОТНЕ

C3 ADC2 на C3 непридатний для звичайних вимірювань. Це не конфлікт із Wi-Fi, як на classic, S2 і S3, а апаратна вада самого чипа: разовий режим (oneshot) на ADC2 в ESP-IDF **вимкнено**, бо результати нестабільні. Драйвер має примусовий перемикач, щоб увімкнути його назад, і вмикати його немає підстав.

Практичний наслідок для розводки плати: на C3 аналоговий вхід — це GPIO0–GPIO4, і більше нічого. П'ятий пін як аналоговий закладати не можна.

Touch і DAC відсутні.

Піни за замовчуванням

Тут змішано дві різні речі, і плутати їх дорого.

Апаратні піни (IOMUX) — ті, на яких блок працює на повній швидкості без проходу через матрицю. Значення нижче звірені з ESP-IDF (soc/<чип>/include/soc/uart_pins.h і spi_pins.h).

Функція	classic	S3	C3
UART0 TX / RX	1 / 3	43 / 44	21 / 20
SPI2 MOSI / MISO / CLK / CS	13/12/14/15	11/13/12/10	7/2/6/10
SPI3 MOSI / MISO / CLK / CS	23/19/18/5	—	—

classic SPI2 і SPI3 у classic історично звуться **HSPI** і **VSPI** у прикладах і бібліотеках частіше трапляється саме VSPI (23/19/18/5).

Домовленість, а не апаратна прив'язка. Для I²C апаратних піни за замовчуванням у ESP-IDF **немає взагалі** — їх задають у проєкті. Числа нижче — те, що прийнято в прикладах і в Arduino, не більше:

	classic	S3	C3
I ² C SDA / SCL	21 / 22	8 / 9	8 / 9

УВАГА

C3 Зверніть увагу на C3: типові для Arduino SDA = GPIO8 і SCL = GPIO9 — це **обидва strapping-піни** (таблиця вище).

Практично це означає дві речі. Перше: зовнішні підтягувальні резистори I²C тягнуть обидві лінії вгору, а GPIO8 при старті **має** бути високим, і GPIO9 високий означає звичайний старт, — тобто типова обв'язка I²C не заважає завантаженню, а навпаки збігається з потрібними рівнями. Друге, і гірше: ведений, що притискає SDA до землі в момент скидання, дає GPIO8 = 0 — недійсну комбінацію.

Симптом — плата, яка стартує через раз і лише коли датчик від'єднаний. Якщо пінів вистачає, I²C на C3 варто перенести на будь-яку іншу пару (розділ 04): матриця GPIO це дозволяє, а швидкість I²C від маршруту через матрицю не страждає.

УВАГА

Завдяки матриці GPIO будь-який із цих сигналів переноситься на інший пін (розділ 04). Ціна перенесення SPI з апаратного піна — нижча гранична частота: маршрут через матрицю додає затримку.

Для I²C і UART це не має значення; для SPI на десятках мегагерц — має.

Скільки пінів насправді

classic 38-пінова плата: з 34 GPIO відкидаємо 6 пінів флешу, 2 піни консолі й 6 тільки-вхідних → близько **20 повноцінних**, із яких 5 — strapping.

Коли не вистачає: розширювач по I²C (PCF8574, MCP23017), зсувний регістр (74HC595/165), аналоговий мультіплексор (CD4051), чип із більшою кількістю пінів, або другий мікроконтролер (розділ 07).

Додаток В. Симптом → причина → фікс

Повний показчик. Картка K8 — польова верхівка з п’ятнадцяти найчастіших; розгорнутий розбір — розділ 29.

Три правила, що передують усьому

Спершу живлення, потім код. Більшість «плаваючих багів» — просадка напруги (картка K13).

Одна зміна за раз. Дві зміни — і невідомо, яка допомогла.

Мінімальний тест окремо. Незнайомий модуль — на голій платі з трьома дротоми (розділ 44).

Підключення і прошивка

Симптом	Причина	Дія	Розділ
Порт не з’являється	кабель без жил даних	інший кабель	09, K3
Порт не з’являється	немає драйвера мосту	CP2102/CH340/ CH9102 окремий	09
Порт не з’являється	відірваний USB-роз’єм	ремонт	55
Permission denied	права	група dialout, перезайти	09
Device or resource busy	порт зайнятий монітором	закрити монітор	09
Failed to connect	не в download mode	BOOT+EN вручну	K4
Failed to connect	зависока швидкість	--baud 115200	17
Failed to connect	обв’язка на strapping-піні	зняти	16
MD5 of file does not match	просадка живлення, кабель	коротший кабель, --baud нижче	06
Invalid head of packet	застосунок пише в UART	download mode вручну	17

Симптом	Причина	Дія	Розділ
Failed to start stub flasher	клон не приймає stub	--no-stub	17
This chip is X, not Y	не той --chip	прибрати --chip	17

Старт і завантаження

Симптом	Причина	Дія	Розділ
Прошилося, плата мовчить	адреса бутлоадера не та	0x1000 classic / 0x0 S3	16, K5
invalid magic byte	застосунку немає або не там	залити	18
Failed to verify partition table	немає таблиці розділів	повна прошивка	18
Плата не стартує взагалі	classic GPIO12 високий при старті	зняти обв'язку	07
Каша в моніторі	не та швидкість	ROM ESP32 завжди 115200	25
Лог читається на 74880	це ESP8266 , не ESP32	звірити маркування	23
Boot loop	паніка	перший дамп після живлення	26, K7
Boot loop	brownout	живлення	06, K13
Порожньо в моніторі	немає порту чи живлення	K3	09

Живлення

Симптом	Причина	Дія	Розділ
rst:0xf	просіло живлення	кабель, конденсатор, джерело 1 А	06, K13
Перезавантаження при Wi-Fi	джерело не тягне піків	джерело від 1 А, 470 мкФ	06
Перезавантаження при двигуні	немає окремого живлення	окреме джерело, спільна земля	48

Симптом	Причина	Дія	Розділ
Працює від USB, не від БЖ	немає спільної землі	з'єднати GND	05
Стабілізатор гарячий	перевантаження або слабкий клон	зовнішні 3.3 В	06
Сон 20 мА замість 20 мкА	плата розробки, не чип	власний монтаж	06, 53
Акумулятор сідає за тижні	дільник вимірювання напруги	ключ на дільник	53

Шини та інтерфейси

Симптом	Причина	Дія	Розділ
I ² C не знаходить пристрій	немає підтягування	4.7 кОм на SDA і SCL	35
I ² C не знаходить пристрій	немає спільної землі	прозвонити	35
I ² C не знаходить пристрій	не та адреса	сканер	35
I ² C з помилками	задовгі проводи, ємність	100 кГц, коротші дроти	35
I ² C таймаути через раз	clock stretching	знижити частоту	35
Кілька модулів I ² C — не працює	завелике сумарне підтягування	лишити один ком-плект	35
SPI повертає нулі	не той режим CPOL/CPHA	перебрати чотири	36
SPI: два разом не працюють	ведений не відпускає MISO	розділити шини	36
SPI: DMA не передає	буфер не для DMA	heap_caps_malloc(MALLOC_CAP_DMA)	36
UART: сміття	не та швидкість	перебрати	34
UART: нічого	переплутані TX/RX	перехресно	34
RS-485: губляться відповіді	немає uart_wait_tx_done	додати перед переми-канням	34

Симптом	Причина	Дія	Розділ
RS-485: помилки на довгій лінії	термінатори	120 Ом лише на кінцях	34
CAN: лічильник помилок росте	один вузол на шині	потрібні двоє	38
CAN: не працює	не та швидкість у вузла	однакова всюди	38
DS18B20: -127 °C	немає підтягування, погана лінія	4.7 кОм, окреме живлення	37

Периферія

Симптом	Причина	Дія	Розділ
ADC читає дурницю	classic ADC2 при Wi-Fi	перенести на ADC1	07, 33
ADC нелінійний	немає калібрування	adc_cal_*	33
ADC шумить	немає усереднення	16–64 відліки	33
GPIO дивно при старті	strapping-pін	інший пін	07
classic GPIO 6–11 не працюють	зайняті флешем	не використовувати	07
Кнопка на GPIO34 не працює	немає вбудованого підтягування	зовнішній резистор	07
Реле вмикається при старті	вхід ключа висить при завантаженні	10 кОм у бік «вимкнено»	47, 62
Реле не спрацьовує	модуль на 5 В	живити 5 В	47
Серво смикається, плата падає	немає окремого живлення	окреме джерело	48
Кроковий гріється	не виставлено обмеження струму	налаштувати до запуску	48
Дисплей зсунутий на 2 пікселі	це SH1106, не SSD1306	режим у бібліотеці	46
Кирилиця — прямокутники	шрифт без кирилиці	інший шрифт	46

Симптом	Причина	Дія	Розділ
Датчик гріється, значення плывуть	подано 5 В на 3.3 В	конвертер рівнів	44, K14
Датчик міряє не те	стоїть біля нагрітого	винести	45
Camera probe failed	живлення	окреме джерело 5 В	49

Мережа

Симптом	Причина	Дія	Розділ
Мережі немає в списку	5 ГГц	ESP32 бачить лише 2.4	39
Не під'єднується	пароль, канал 12–13, WPA3	звірити	39
Під'єднується й відвалюється	живлення або слабкий сигнал	RSSI, джерело	39
Пінги ходять, OTA не проходить	межа покриття	RSSI гірше –80	19, 39
Перезавантажується без мережі	ESP_ERROR_CHECK навколо connect	явна обробка	32, 39
Гріється, батарея сідає	перепід'єднання без паузи	зростаюча пауза	39
ESP-NOW: працювало на столі	роутер змінив канал	фіксувати канал	42
BLE: не вміщається	Bluedroid замість NimBLE	перемкнути	41
BLE: iOS бачить старі сервіси	кеш GATT	забути пристрій	41
SPP не працює на S3/C3	Classic є лише в classic	переписати на BLE	41
LoRa: нічого не чути	різні параметри в боків	однакові SF/BW/CR	43
LoRa: модуль згорів	увімкнули без антени	 незворотно	43

Пам'ять і стабільність

Симптом	Причина	Дія	Розділ
Падає через дні роботи	фрагментація купи	не виділяти в циклі	30
Падає в випадковому місці	переповнення стека задачі	збільшити стек	30
malloc = NULL, пам'ять є	немає суцільного блоку	largest_free_block	30
LoadProhibited, EXCVADDR 0	розіменування NULL	перевіряти malloc	26
Task WDT	цикл без затримки	vTaskDelay	31
Interrupt WDT	довгий ISR	коротко: у чергу і вийти	31
Система стоїть	висока пріоритетна задача без блокування	знижити або блокувати	31
Дані псуються випадково	спільна змінна без захисту	черга або м'ютекс	31

Корпус і поле

Симптом	Причина	Дія	Розділ
На столі працює, у корпусі ні	перегрів	вентиляція, менша потужність	54
Немає зв'язку в корпусі	метал екранує	пластик або зовнішня антена	39, 54
Через місяці корозія	конденсат	силікагель, мембрана, лак	54
Обрив після перевезення	вібрація, немає strain relief	закріпити кабель	54
Дроти вискочили	Dupont під вібрацією	постійний монтаж	52

Клас «іноді працює»

Майже ніколи не програмний. Порядок підозр **строго**:

живлення → пайка → земля → логічні рівні → таймінги → і лише потім код

НЕЗВОРОТНЕ

Збій, що зник після зміни, але не відтворювався стабільно до неї, — **не полагоджений**. Він повернеться в полі.

Перш ніж закрити питання, треба вміти викликати збій за бажанням (розділи [29](#), [58](#)).

Додаток С. Повні шпаргалки команд

Розгорнута версія картки K10.

Синтаксис esptool v5 (дефіси, без .py). Для v4 — підкреслення і суфікс .py. Перевірити своє: `esptool version`.

esptool

Розвідка

```
# сімейство, ревізію і MAC друкує шапка з'єднання — перед будь-якою
# командою; окремої команди для цього немає (розділ 17)
esptool --port /dev/ttyUSB0 flash-id      # виробник і обсяг флешу
esptool --port /dev/ttyUSB0 read-mac
esptool version
```

Читання

```
esptool --port PORT read-flash 0 ALL dump.bin      # повний дамп
esptool --port PORT read-flash 0 0x400000 dump.bin  # 4 МБ явно
esptool --port PORT read-flash 0x9000 0x6000 nvs.bin # лише NVS
esptool --port PORT read-flash 0x8000 0x1000 pt.bin # таблиця розділів
```

Розмір файлу має **точно** дорівнювати запитаному. Менший — обірваний дамп; повторити на `--baud 115200`.

Запис

```
esptool --port PORT --baud 460800 write-flash -z \
  0x1000 bootloader.bin 0x8000 partition-table.bin 0x10000 app.bin

esptool --port PORT write-flash 0x0 merged.bin      # зібраний образ
esptool --port PORT verify-flash 0x10000 app.bin    # звірити
```

Стирання

```
esptool --port PORT erase-flash      # 🚫 усе, спершу дамп
esptool --port PORT erase-region 0x9000 0x6000 # лише NVS
```

Складання образу

```
esptool --chip esp32 merge-bin -o vyrib.bin --flash-mode dio --flash-size 4MB \
  0x1000 bootloader.bin 0x8000 partition-table.bin 0x10000 app.bin
```

--chip тут **обов'язковий**: порту немає, автовизначенню нема звідки взятися. Без нього esptool одразу відповідає Specify the --chip argument. Значення має збіга-

тися з чипом, під який зібрано прошивку, — і з адресою бутлоадера з таблиці нижче.

Корисні прапорці

Прапорець	Навіщо
--baud 115200	коли обривається на високій швидкості
-z	стиснення при передачі. Уже ввімкнене — крім --no-stub
-u	вимкнути стиснення
--no-stub	коли клон не приймає допоміжну програму
--before no-reset	коли на платі немає DTR/RTS і скидання робиться рукою
--after no-reset	лишити чип у завантажувачі: кілька команд поспіль
--after watchdog-reset	S3 C3 застряг у download mode через native USB
--chip esp32s3	коли автовизначення заважає

Про -z варто знати точно: стиснення ввімкнене **за замовчуванням**, тож у звичайній команді цей прапорець нічого не змінює. Сенса він має рівно в одному випадку — разом із --no-stub, де стиснення типово вимкнене.

--before default-reset і --after hard-reset — теж значення за замовчуванням; писати їх, щоб «керувати скиданням», сенсу немає. Корисні саме інші значення, наведені в таблиці.

esepfuse

НЕЗВОРОТНЕ

Лише читання безпечне. burn-* пропалює біти **фізично й назавжди** (розділ 20).

```
esepfuse --port PORT summary      # безпечно: подивитися стан
```

idf.py

Проект

```
idf.py create-project imya
idf.py create-component imya
idf.py set-target esp32s3      # ▲ стирає sdkconfig
idf.py menuconfig             # пошук усередині — клавіша /
```

Збирання і прошивка

```
idf.py build
idf.py -p /dev/ttyUSB0 flash
idf.py -p /dev/ttyUSB0 monitor          # вихід Ctrl+]
idf.py -p /dev/ttyUSB0 flash monitor    # найчастіша команда
idf.py -p /dev/ttyUSB0 app-flash        # лише застосунок, швидше
idf.py fullclean                        # коли збирання поводитьсся дивно
idf.py merge-bin -o vyrib.bin           # один образ; адреси — з конфігурації
```

idf.py merge-bin кращий за esptool merge-bin завжди, коли проєкт під рукою: адресу бутлоадера, чип, режим і частоту флешу він бере з конфігурації, а не з набраного вручну рядка. Без -o результат — build/merged-binary.bin.

Аналіз

```
idf.py size          # скільки зайнято флешу і RAM
idf.py size-components # XTO CAME займає — найкорисніша
idf.py size-files
```

Діагностика

```
idf.py coredump-info    # розбір coredump із флешу
idf.py coredump-debug   # GDB на збереженому стані
idf.py openocd gdb      # покрокове налагодження (S3, C3)
idf.py monitor          # з розшифровкою backtrace на льоту
```

Компоненти

```
idf.py add-dependency "espressif/led_strip^3.0.3"
idf.py reconfigure
```

Розшифровка backtrace вручну

```
xtensa-esp32-elf-addr2line -pfiaC -e build/app.elf 0x400d1234 0x400d5678
xtensa-esp32s3-elf-addr2line -pfiaC -e build/app.elf 0x42001234
riscv32-esp-elf-addr2line -pfiaC -e build/app.elf 0x42001234
```

-і обов'язковий: без нього inline-кадри зникають.

Монітори портів

Програма	Вихід	Особливість
idf.py monitor	Ctrl+]	розшифровує backtrace; скидання Ctrl+T, Ctrl+R
picocom -b 115200 /dev/ttyUSB0	Ctrl+A, Ctrl+X	найпростіший
minicom -D /dev/ttyUSB0 -b 115200	Ctrl+A, X	
screen /dev/ttyUSB0 115200	Ctrl+A, K	є майже скрізь

Запис логу у файл:

```
picocom -b 115200 /dev/ttyUSB0 | tee log-2026-08-26.txt
```

Порти

```
ls /dev/ttyUSB* /dev/ttyACM*      # що є
ls -l /dev/serial/by-id/          # стабільні імена для скриптів
dmesg | tail -20                  # що ядро побачило
lsof /dev/ttyUSB0                 # хто тримає порт
sudo usermod -aG dialout $USER    # права; далі ПЕРЕЗАЙТИ в систему
```

/dev/ttyUSB* — зовнішній міст. /dev/ttyACM* — native USB **S3** **C3**.

PlatformIO

```
pio run                          # зібрати
pio run -e s3                     # конкретне середовище
pio run -t upload
pio device monitor
pio run -t clean
pio pkg update
```

Адреси у флеші

Що	classic, S2	S3, C3, C6, H2	P4, C5, H4
bootloader	0x1000	0x0	0x2000
таблиця розділів	0x8000	0x8000	0x8000
застосунок	0x10000	0x10000	0x10000
nvs (типово)	0x9000	0x9000	0x9000
зібраний merge-bin	0x0	0x0	0x0

Адресу бутлоадера задає ROM чипа (CONFIG_BOOTLOADER_OFFSET_IN_FLASH), і в ESP-IDF вона не налаштовується. Правила «що новіше, то ближче до нуля» немає: у P4, C5 і H4 перші два сектори віддані під ключі шифрування флешу, і бутлоадер зсунуто на 0x2000.

Розбір таблиці розділів із дампа

```
dd if=dump.bin of=pt.bin bs=1 skip=$((0x8000)) count=$((0x1000))
python $IDF_PATH/components/partition_table/gen_esp32part.py pt.bin
```


Розвідка чужої прошивки

```
strings -n 6 dump.bin | less
strings -n 6 dump.bin | grep -iE "v[0-9]+\.[0-9]+|20[0-9]{2}-"
strings -n 6 dump.bin | grep -iE "http|mqtt|ssid|pass"
```

Розділ 24.

Генерація NVS для серії

```
nvs_partition_gen.py generate config-0042.csv nvs-0042.bin 0x6000
esptool --port PORT write-flash 0x9000 nvs-0042.bin
```

Додаток D. Boot-повідомлення, скидання, паніки

Розгорнута версія карток K6 і K7. Пояснення — розділи 16 і 26.

Причини скидання: rst:

Числові коди з ROM-заголовка ESP-IDF (enum RESET_REASON), `classic`.

Код	Назва	Що сталося	Що робити
0x1	POWERON_RESET	подано живлення або EN	норма
0x3	SW_RESET	esp_restart() з коду	норма, якщо ваша
0x4	OWDT_RESET	застарілий watchdog	рідко
0x5	DEEPSLEEP_RESET	прокинувся з deep sleep	норма
0x6	SDIO_RESET	скидання модулем SLC	рідко
0x7	TG0WDT_SYS_RESET	watchdog таймера 0	розділ 32
0x8	TG1WDT_SYS_RESET	watchdog таймера 1	розділ 32
0x9	RTCWDT_SYS_RESET	RTC watchdog	розділ 32
0xa	INTRUSION_RESET	детектор втручання	рідко
0xb	TGWDT_CPU_RESET	watchdog скинув ядро	розділ 32
0xc	SW_CPU_RESET	програмне скидання ядра	типово після паніки
0xd	RTCWDT_CPU_RESET	RTC watchdog скинув ядро	розділ 32
0xe	EXT_CPU_RESET	APP CPU скинутий PRO CPU	норма
0xf	RTCWDT_BROWN_OUT_RESET	період живлення	 розділ 06
0x10	RTCWDT_RTC_RESET	RTC watchdog скинув усе	розділ 32

Три, що трапляються постійно: 0x1 (норма), 0xc (після паніки), 0xf (живлення).

ЖИВЛЕННЯ

`rst:0xf` — це **живлення**, не помилка в коді. Скільки б ви не читали код, причина в джерелі, кабелі або конденсаторах (картка K13).

З коду: `esp_reset_reason()`. Логувати першим рядком `app_main`.

Режим завантаження: boot:

Значення	Що це
<code>SPI_FAST_FLASH_BOOT</code>	звичайний старт із флешу
<code>DOWNLOAD_BOOT(UART0/UART1/...)</code>	download mode, GPIO0 низький
<code>DOWNLOAD(USB/UART0)</code>	те саме на чипах із власним USB
<code>SPI_FLASH_BOOT, SDIO_REI_FEO_V1_BOOT, ATE_BOOT</code>	обрано невідтримуваний режим

Останній рядок трапляється рідко, і саме тому спантеличує: плата стартувала кудись, чого читач не шукав у жодній інструкції. Причина в переважній більшості випадків одна — **другий strapping-пін високий, коли головний низький**: **classic** GPIO2 високий при низькому GPIO0. Тобто це не поломка, а комбінація, і лікується вона зняттям обв'язки з другого піна (розділ 07).

Саме число — це стани strapping-пінів

Найнедооціненіший рядок усього boot-логу. Число після `boot:` — не код режиму, а **бітова маска регістра GPIO_STRAP**: рівні, які чип зафіксував на strapping-пінах у момент відпускання скидання.

classic Для ESP32 classic біти такі:

Біт	Пін
0x01	GPIO5
0x02	GPIO15 (MTDO)
0x04	GPIO4
0x08	GPIO2
0x10	GPIO0
0x20	GPIO12 (MTDI)

Звідси читаються два найчастіші значення:

`boot:0x13 = 0x01 + 0x02 + 0x10` — GPIO5, GPIO15 і GPIO0 високі, решта низькі. Нормальний старт.

boot:0x3 = 0x01 + 0x02 — те саме, але **GPIO0 низький**. Звідси й download mode.

УВАГА

Це перетворює здогадки на вимірювання. Уся книга повторює, що зовнішня обв’язка на strapping-піні дає загадкові збої (розділи 07, 16) — а перевірити це можна прямо з логу, не беручи осцилографа.

Найцінніший біт — 0x20. Якщо він виставлений, GPIO12 при старті був високим, а отже флеш отримав 1.8 В замість 3.3 В. На більшості модулів це і є причина мовчазної плати (розділ 07).

Другий за цінністю — 0x04. GPIO4 не керує режимом завантаження й у переліку strapping-пінів не значиться, але його рівень чип фіксує теж, і в масці він видимий. Для діагностики це безкоштовний зайвий канал спостереження.

S3 **C3** На решті сімейств маска коротша — у ній лише два біти, і вони позначають ту саму пару, що вирішує режим:

Біт	classic	S3	C3
0x04	—	GPIO46	GPIO8
0x08	—	GPIO0	GPIO9

Читається так само: біт виставлений — пін був високим.

УВАГА

Далі значення не розшифровуються, і це свідоме рішення.

Спокусливо взяти цю таблицю бітів, скласти з нею правила strapping із розділу 07 і дістати «boot:0x4 означає ось це». Так робити не можна, і книга це вже пробувала: попередня редакція цього абзацу так і зробила й помилилася у двох випадках із трьох.

Причина в тому, що ROM класифікує **значення цілком**, а не пін за піном, і його власні макроси не збігаються з наївним складанням. У soc/boot_mode.h видно, що 0x4 потрапляє в ETS_IS_FLASH_BOOT, а не в завантаження по UART:

```
#define ETS_IS_FLASH_BOOT() (IS_1XXX(BOOT_MODE_GET()) ||  
IS_0100(BOOT_MODE_GET()))  
#define ETS_IS_JOINT_DOWNLOAD_BOOT() IS_00XX(BOOT_MODE_GET())
```

Тобто відповідність «біт — пін» із документації esptool і класифікація значень у ROM — це два різні рівні, і місток між ними лежить у technical reference manual, а не в тому, що доступне звідси.

Практично: **дивіться на рядок у дужках**, а не на число. Режим словами — SPI_FAST_FLASH_BOOT, DOWNLOAD_BOOT(...) — друкує сам ROM, і це розшифровка від того, хто має право її робити.

Друкує він її в **boot-лог по UART** (приклад одразу нижче), тобто видно її в моніторі: idf.py monitor, screen, picocom. У виводі esptool цього рядка немає — esptool скидає чип і виходить, лог після скидання читає вже монітор.

Типовий boot-лог

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
mode:DIO, clock div:2
load:0x3ffff0030,len:1344
entry 0x400805e4
I (29) boot: ESP-IDF v6.0.2 2nd stage bootloader
I (33) boot.esp32: SPI Flash Size : 4MB
I (52) boot: Partition Table:
I (56) boot: ## Label           Usage      Type ST Offset   Length
I (63) boot:  0 nvs             Wifi data   01 02 00009000 00006000
I (70) boot:  1 phy_init        RF data     01 01 0000f000 00001000
I (78) boot:  2 factory         factory app 00 00 00010000 00100000
I (xxx) cpu_start: Pro cpu up.
```

Звідси безкоштовно читається: **версія ESP-IDF, обсяг флешу очима бутлоадера і вся таблиця розділів з адресами** — готова відповідь на «що всередині чужого пристрою» (розділ 24).

Число в дужках — мілісекунди від старту. Стрибок показує, де прошивка задумалася.

Помилки бутлоадера

Рядки нижче — дослівні з ESP-IDF; %d, 0x%x і адреси підставляються.

Повідомлення	Причина	Розділ
image at 0x... has invalid magic byte (nothing flashed here?)	за адресою застосунку не образ	18
Factory app partition is not bootable	застосунку немає	K5
partition N invalid magic number 0x...	немає таблиці розділів	18
Failed to verify partition table	те саме	18
ota data partition invalid, falling back to factory	зіпсований otadata	19
Image hash failed - image is corrupt	образ пошкоджений	17

Повідомлення	Причина	Розділ
Detected size(...k) smaller than the size in the binary image header(...k). Probe failed.	конфігурація > реальний флеш	08
Detected size(...k) larger than ... Using the size in the binary image header.	конфігурація < реальний флеш; лише попередження	08

УВАГА
Розбіжність обсягу флешу дає **два різні рядки, і наслідки різні**.

Реальний флеш **менший** за налаштований — фатально: бутлоадер зупиняє пробу, бо частина розділів фізично не існує.

Реальний флеш **більший** — лише попередження: система працює, просто надлишок не використовується. Саме цей випадок трапляється з клонами, що продаються як 16 МБ, а стають 4 МБ у конфігурації.

Причини паніки

Причина	Що заборонено	Що шукати
LoadProhibited	читання з недійсної адреси	NULL або звільнений покажчик
StoreProhibited	запис за недійсною адресою	те саме, на запис
InstrFetchProhibited	перехід на недійсну адресу	зіпсований покажчик на функцію
IllegalInstruction	виконання не-коду	переповнення стека
LoadStoreAlignment	невирівняний доступ	32 біти з непарної адреси
IntegerDivideByZero	ділення на нуль	дільник із датчика без перевірки
Interrupt wdt timeout	переривання заблоковані задовго	довгий ISR, критична секція
Cache disabled but cached memory region accessed	доступ до флешу при вимкненому кеші	немає IRAM_ATTR

Читання дампа регістрів

Поле	Що означає
PC	адреса інструкції, на якій упало — де
EXCVADDR	адреса, за якою зверталися — куди
A1	вказівник стека
Backtrace	ланцюжок адреса: стек, читати знизу вгору

УВАГА

EXCVADDR — **найшвидша підказка**.

Близько нуля ($0x0-0x40$) → розіменування NULL зі зсувом поля структури. Це покриває більшість LoadProhibited на практиці.

Схожа на осмислену адресу, але доступ заборонено → покажчик на вже звільнену пам'ять.

Watchdog: розрізнення

Task WDT — задача не віддає керування:

```
E (5234) task_wdt: Task watchdog got triggered. The following tasks/users
did not reset the watchdog in time:
E (5234) task_wdt: - IDLE0 (CPU 0)
E (5234) task_wdt: Tasks currently running:
E (5234) task_wdt: CPU 0: my_task
```

Переліків тут **два, і вони різні**. Після першого рядка — ті, хто не встиг погодувати watchdog (IDLE0 — потерпілий). Після Tasks currently running: — те, що виконувалося в цю мить, і саме там винуватець: my_task.

Це діагностика, а не смерть системи.

Interrupt WDT — переривання заблоковані:

```
Guru Meditation Error: Core 0 panic'ed (Interrupt wdt timeout on CPU0)
```

Значно серйозніший. Причини: важкий код в ISR, довга критична секція, виклик забороненого в ISR (розділ 31).

Помилки стека і купи

Повідомлення	Причина
ERROR A stack overflow in task X has been detected.	замалий стек задачі

Повідомлення	Причина
CORRUPT HEAP: Bad tail at 0x... Expected 0x... got 0x...	запис за кінець блоку
CORRUPT HEAP: Bad head at 0x...	запис перед початком блоку
Guru Meditation ... IllegalInstruction	часто теж переповнення стека
assert failed: ...	порушено внутрішній інваріант
heap_caps_malloc failed	немає пам'яті або немає блоку потрібного розміру

УВАГА

Bad head і Bad tail — не однакові повідомлення. Купа тримає навколо кожного блоку контрольні слова-канарки, і зіпсована каже, з якого боку писали повз.

Bad tail — типове переповнення буфера: писали далі, ніж виділили. Шукати memcpy, sprintf, цикл із <= замість <.

Bad head — писали **до** початку блоку: від'ємний індекс, зсув покажчика назад, звільнення чужої адреси. Трапляється рідше й майже завжди означає помилку в арифметиці покажчиків.

Адреса в повідомленні — це адреса канарки, тобто край самого блоку. Її можна порівняти з тим, що повернув malloc.

Діагностика — розділ 30: uxTaskGetStackHighWaterMark, heap_caps_get_largest_free_block.

Порядок розбору збою

- 1. **rst**: у першому рядку. Живлення, watchdog чи паніка — три різні шляхи.
- 2. **Причина паніки і EXCVADDR**. Часто відповідь уже тут.
- 3. **Backtrace через .elf** того самого збирання. Знизу вгору.
- 4. **Відтворити**. Збій, який не відтворюється, не полагоджений.
- 5. Не відтворюється → coredump і логування переходів станів.

НЕЗВОРОТНЕ

Без .elf **того самого збирання** backtrace нерозшифровний. Перезібраний «такий самий» проект не підходить: адреси зсуваються від будь-якої зміни тулчейну чи бібліотеки.

.elf зберігається разом із кожним образом, що поїхав (розділ 21).

Boot loop

Дивитися **найперший** дамп після подачі живлення: відкрити монітор, **потім** подати живлення. У першому — причина, в решті — наслідки.

Швидке відсікання: залити свідомо справний мінімальний образ (hello_world). Працює — справа в прошивці; ні — у залізі чи живленні (розділ 20).

Додаток Е. Інтерфейс → пристрої

→ бібліотека

Таблиця для швидкого пошуку: що на чому працює і з чого починати. Процесура підключення незнайомого модуля — розділ 44.

I²C

Дві лінії, до кількох десятків пристроїв, десятки сантиметрів. Підтягування 4.7 кОм обов’язкове (розділ 35).

Пристрій	Адреса	Що дає	Бібліотека
BME280	0x76, 0x77	тиск, Т, вологість	реєстр IDF; Adafruit BME280
BMP280	0x76, 0x77	тиск, Т — без вологості	Adafruit BMP280
SHT3x / SHT4x	0x44, 0x45	точна вологість	Sensirion
BH1750	0x23, 0x5C	освітленість, люкс	BH1750
DS3231	0x68	RTC з батареєю	RTClib
MPU6050	0x68, 0x69	акселерометр, гіроскоп	MPU6050
BNO055	0x28, 0x29	готова орієнтація	Adafruit BNO055
INA219 / INA226	0x40+	струм і напруга	Adafruit INA219
VL53L0X / VL53L1X	0x29	лазерна відстань	VL53L0X
SSD1306 / SH1106	0x3C, 0x3D	OLED-дисплей	U8g2
PCF8574	0x20–0x27	розширювач портів	PCF8574
MCP23017	0x20–0x27	розширювач, 16 пінів	MCP23017
TCA9548A	0x70–0x77	мультиплексор шини	TCA9548A
AT24Cxx	0x50–0x57	EEPROM	—

УВАГА

DS3231 і MPU6050 мають однакову адресу 0x68. Разом на одній шині — конфлікт; розв’язується перемичкою на MPU6050 (адреса 0x69) або мультиплексором (розділ 35).

SPI

Швидко, чотири лінії плюс CS на кожен пристрій (розділ 36).

Пристрій	Режим	Що дає	Бібліотека
ST7789	0 або 3	TFT-дисплей	TFT_eSPI, LovyanGFX
ILI9341	0	TFT-дисплей	TFT_eSPI, LovyanGFX
microSD	0	картка пам'яті	штатний esp_vfs_fat
SX1276 / RFM95	0	LoRa	RadioLib, LoRa
SX1262	0	LoRa, новіший	RadioLib
NRF24L01	0	радіо 2.4 ГГц	RF24
MCP2515	0	зовнішній CAN	— (у ESP32 є свій, розділ 38)
MAX31855	0	термопара	Adafruit MAX31855
MAX6675	0	термопара, дешевший	Adafruit MAX6675
ADS1256, MCP3208	0/1	зовнішній точний ADC	—
W5500	0	дротовий Ethernet	Ethernet
E-paper (SSD16xx)	0	електронний папір	GxEPD2

Режим у таблиці — типовий; звернути з datasheet конкретного модуля.

Запис «0 або 3» не невизначеність: обидва режими читають по наростаючому фронту й відрізняються лише рівнем тактування в спокої, тож ST7789 працює в обох. Adafruit за замовчуванням ставить SPI_MODE0, частина інших бібліотек — третій (розділ 36).

UART

Два дрони, будь-яка відстань через RS-485 (розділ 34).

Пристрій	Швидкість	Що дає
GPS NEO-6M / NEO-8M	9600	координати, точний час (NMEA)
MAX485 / SP3485	будь-яка	RS-485, сотні метрів
PMS5003, SDS011	9600	пилові частинки
MH-Z19	9600	CO ₂
A6 / SIM800 / SIM7600	115200	стільниковий зв'язок

Пристрій	Швидкість	Що дає
Модулі відбитків пальців	57600	біометрія
Інший мікроконтролер	ваша	companion-схема (розділ 57)

1-Wire

Один дріт, десятки метрів, підтягування 4.7 кОм (розділ 37).

Пристрій	Що дає	Бібліотека
DS18B20	температура, кілька на лінії	OneWire + DallasTemperature
DS2431	EEPROM	—

CAN / TWAI

Промислова шина, потрібен трансивер на 3.3 В (розділ 38).

Пристрій	Що дає
SN65HVD230	трансивер 3.3 В — правильний вибір
TJA1050, MCP2551	трансивери 5 В —  потрібен конвертер на RX
BMS акумуляторних збірок	стан батареї
Частотні перетворювачі	керування приводом
Автомобільна електроніка	діагностика, телеметрія

I²S

Цифровий звук (розділ 49).

Пристрій	Що дає	Примітка
INMP441	цифровий мікрофон	без аналогової обв'язки
MAX98357A	підсилювач класу D	окреме живлення
PCM5102	ЦАП для звуку	

Прямі GPIO і PWM

Пристрій	Як	Розділ
WS2812 / SK6812	RMT, не програмно	33

Пристрій	Як	Розділ
Серво SG90, MG996R	LEDC 50 Гц, окреме живлення	48
HC-SR04	тригер + вимір ЕСНО (— 5 В!)	45
PIR HC-SR501	цифровий вхід	45
Енкодер	PCNT, не переривання	33
Реле, MOSFET	вихід + резистор на затворі	47
A4988 / DRV8825	STEP + DIR, кроки апаратно	48
DRV8833 / TB6612	PWM + напрямки	48
Кнопки	вхід із підтягуванням, антидребезг	33

Аналогові входи (ADC)

classic Лише ADC1 (GPIO 32–39) при Wi-Fi (розділ 07).

Джерело	Обв'язка
Дільник напруги акумулятора	2 резистори + ключ (розділ 53)
Фоторезистор, термістор	дільник із резистором
ACS712 (струм)	напряму, усереднення
Потенціометр	напряму
Датчик вологості ґрунту	дільник; ємнісні кращі за резистивні

Куди дивитися за бібліотекою

ESP-IDF: реєстр компонентів, `idf.py add-dependency`. Менший вибір, вища якість, фіксовані версії (розділ 11).

Arduino: менеджер бібліотек. Величезний вибір, дуже нерівна якість (розділ 12).

Приклади в самому ESP-IDF — каталог `examples/`. Найнедооціненіший ресурс: робочий приклад майже на кожен периферійний пристрій, точно під вашу версію.

Бібліотеки немає — розділ 44: `datasheet`, реєстр ідентифікації, код для іншої платформи.

УВАГА

Перед тим як брати бібліотеку: подивитися дату останнього оновлення (міг не пережити Arduino core 2.x → 3.x), пошукати всередині блокувальні `delay`, перевірити, чи взагалі обробляються помилки шини (розділ 12).

Додаток F. Офлайн-пак

Що завантажити й підготувати заздалегідь, поки є мережа. У полі інтернету немає, і саме там документація потрібна найбільше (розділ 15).

Перевірка готовності одна: **вимкнути мережу і спробувати зібрати та залити проєкт**. Що зламалося — того в паку бракує.

Тулчейн

- ☐ **ESP-IDF конкретної версії**, встановлений і **перевірений збиранням** хоча б одного проєкту. Не «завантажений інстальатор»: половина проблем встановлення вилазить саме при першому збиранні
- ☐ Тулчейни під усі цільові архітектури (Xtensa і RISC-V окремо)
- ☐ esptool — іде разом з IDF; перевірити версію (розділ 17)
- ☐ Arduino core, якщо використовується
- ☐ PlatformIO або pioarduino із **закешованими** платформами

Документація

- ☐ **ESP-IDF Programming Guide** — завантажується цілком, окремим архівом. Версія та сама, що у вашого IDF
- ☐ **Datasheet** кожного сімейства, з яким працюєте
- ☐ **Technical Reference Manual** на основне сімейство
- ☐ **Datasheet модуля** (WROOM-32, S3-WROOM-1, C3-MINI-1)
- ☐ **Схеми плат розробки**, які використовуєте
- ☐ **Пінаути** — картинками, не закладками браузера
- ☐ Datasheet кожного датчика й модуля вашого проєкту

УВАГА

Найчастіше забувають **схему конкретної плати**, а не datasheet чипа. Саме вона відповідає на питання «який пін куди виведений на цій платі», а без неї datasheet чипа мало допомагає (розділ 08).

Приклади й бібліотеки

- ☐ Каталог examples/ усередині ESP-IDF — **уже у вас**, це найнедооціненіший офлайн-ресурс
- ☐ Компоненти проєкту закешовані локально: проєкт, що тягне залежності при збиранні, у полі не збереться
- ☐ Бібліотеки Arduino, якщо використовуються
- ☐ Ваші власні напрацювання й шаблони проєктів

Драйвери

- ☐ **CP2102 / CP2102N** (Silicon Labs) — Windows
- ☐ **CH340 / CH341** (WCH) — Windows
- ☐ **CH9102** — **окремий** драйвер, не той, що для CH340
- ☐ **FT232** (FTDI) — Windows
- ☐ Утиліта заміни драйвера для USB-JTAG — Windows, для налагодження

Файлами, не посиланнями. Linux драйверів не потребує (розділ 09).

Прошивки й артефакти

- ☐ **Образи** всіх версій, що поїхали в поле
- ☐ **.elf того самого збирання** — без нього backtrace із поля нерозшифровний (розділи 21, 26)
- ☐ sdkconfig кожного проекту
- ☐ Зібрані merge-bin образи для швидкої заливки
- ☐ **Дампи флешу** пристроїв, з якими працюєте (картка K2)
- ☐ hello_world або blink, зібраний під кожен ваш чип — для відсікання «прошивки чи залізо» за дві хвилини (розділ 20)

Довідкове

- ☐ **Роздруковані й заламіновані картки K1–K15**
- ☐ Паспорти виробів (розділ 56)
- ☐ Журнали партій (розділ 21)
- ☐ Схеми з'єднань і фотографії ваших пристроїв
- ☐ Цей довідник у PDF

Інструменти

- ☐ **PulseView / sigrok** для логічного аналізатора (розділ 28)
- ☐ Термінальна програма: picocom, minicom, PuTTY
- ☐ mklittlefs / mkspiifs для роботи з файловими системами
- ☐ gen_esp32part.py — іде з IDF
- ☐ KiCad із бібліотеками, якщо розводите плати
- ☐ Редактор і засоби, до яких ви звикли

Де це тримати

ЗАКУПІВЛЯ

Не лише на робочому ноутбуку. Ноутбук ламається, лишається чужий.

Копія на флешці розгортається за годину. Зібраний наново пак — за день, і то якщо є мережа.

Флешку варто оновлювати після кожної зміни версії тулчейну чи випуску нової прошивки в поле.

Перевірка готовності

Раз на кілька місяців, на машині **без мережі**:

1. Створити новий проєкт із шаблону.
2. `idf.py set-target, build` — має зібратися без завантажень.
3. Залити на плату.
4. Відкрити монітор, побачити лог.
5. Відкрити Programming Guide локально.
6. Знайти пінаут потрібної плати.

Що не спрацювало — того в паку бракує. Це та перевірка, яку краще провалити вдома, ніж у полі.

Додаток G. Глосарій

Українська назва — канонічний англійський термін. Мовна політика — P8 і розділ 00: усталені в галузі терміни не перекладаються, бо шукати їх ви все одно будете англійською.

Не перекладаються

Терміни, які лишаються як є навіть в українському тексті.

Термін	Що це
brownout	скидання через просідання напруги живлення
watchdog	сторожовий таймер, що перезавантажує зависле
backtrace	ланцюжок викликів у момент збою
strapping	піни, стан яких при скиданні задає режим завантаження
bootloader	програма, що завантажує наступну програму
firmware	прошивка
datasheet	документація на компонент
pinout	розводка виводів
breadboard	макетна плата без пайки
pull-up / pull-down	підтягування вгору / вниз
open-drain	вихід, що вміє лише притискати лінію до землі
duty cycle	коефіцієнт заповнення: частка періоду в активному стані
deep sleep	глибокий сон із втратою вмісту RAM
coredump	знімок стану всіх задач у момент паніки
conformal coating	захисне лакове покриття плати
strain relief	механічне закріплення кабелю проти натягу
power path	схема одночасної роботи й заряджання
sensor fusion	злиття даних кількох датчиків
provisioning	початкове передавання пристрою креденшелів мережі

Українська ↔ англійська

Апаратне

Українською	English
кристал, мікросхема	die, SoC, chip
модуль	module
плата розробки	development board
пін, вивід	pin
контактна площадка	pad
гребінка	pin header
перемичка	jumper
доріжка	trace
шина	bus
земля, спільна земля	ground, common ground
живлення	power supply
стабілізатор	voltage regulator
понижувальний перетворювач	buck converter
підвищувальний перетворювач	boost converter
дільник напруги	voltage divider
конвертер рівнів	level shifter
розв'язувальний конденсатор	decoupling capacitor
захисний діод	flyback diode
ключ (транзисторний)	switch
оптопара	optocoupler
гальванічна розв'язка	galvanic isolation
термоусадка	heat shrink tubing
оплітка для випаювання	desoldering wick
припій	solder
холодна пайка	cold joint

Українською	English
перфбордин	perfboard, protoboard
корпус (виробу)	enclosure
гермоввід, сальник	cable gland

Програмне

Українською	English
прошивка	firmware
образ	image, binary
збирання	build
тулчейн	toolchain
компонент	component
задача	task
планувальник	scheduler
пріоритет	priority
черга	queue
семафор	semaphore
м'ютекс	mutex
група подій	event group
переривання	interrupt
обробник переривання	interrupt handler, ISR
критична секція	critical section
стек	stack
купа	heap
фрагментація	fragmentation
витік пам'яті	memory leak
переповнення стека	stack overflow
атомарна операція	atomic operation

Українською	English
гонка	race condition
взаємне блокування	deadlock
зворотний виклик	callback
таблиця розділів	partition table
розділ	partition
файлова система	filesystem
дамп	dump
прапорець	flag
автомат станів	state machine
ідемпотентність	idempotence
відтворюване збирання	reproducible build
відкат	rollback

Радіо і мережа

Українською	English
точка доступу	access point
станція, клієнт	station
канал	channel
рівень сигналу	RSSI, signal strength
антена	antenna
узгодження	matching
дальність	range
маячок	beacon
креденшели	credentials
брокер	broker
топік	topic
підписка	subscription

Українською	English
публікація	publish
сертифікат	certificate
центр сертифікації	certificate authority, CA
рукоштовування	handshake
широкомовна розсилка	broadcast
комірчаста мережа	mesh network
ретрансляція	relay

Вимірювання і живлення

Українською	English
напруга	voltage
струм	current
опір	resistance
потужність	power
ємність (конденсатора)	capacitance
ємність (акумулятора)	capacity
просадка	voltage drop, sag
пікове споживання	peak current
коефіцієнт заповнення	duty cycle
роздільність	resolution
точність	accuracy
калібрування	calibration
усереднення	averaging
шум	noise
завада	interference
шунт	shunt
навантаження	load

Українською	English
холостий хід	no load

Процес

Українською	English
постановка задачі	requirements
прототип	prototype
доведення	hardening
приймальні випробування	acceptance testing
критерій приймання	acceptance criterion
паспорт виробу	device documentation
журнал змін	changelog
версіонування	versioning
серійна прошивка	batch flashing
партія	batch
брак	defective unit
супровід	maintenance

Скорочення

Скорочення	Розшифровка
SoC	System on Chip
IDF	IoT Development Framework
RTOS	Real-Time Operating System
ISR	Interrupt Service Routine
DMA	Direct Memory Access
NVS	Non-Volatile Storage
OTA	Over-The-Air (оновлення)
ADC / DAC	Analog-to-Digital / Digital-to-Analog Converter
PWM	Pulse-Width Modulation

Скорочення	Розшифровка
GPIO	General-Purpose Input/Output
UART	Universal Asynchronous Receiver/Transmitter
SPI	Serial Peripheral Interface
I ² C	Inter-Integrated Circuit
I ² S	Inter-IC Sound
TWAI	Two-Wire Automotive Interface (CAN в ESP32)
JTAG	Joint Test Action Group (інтерфейс налагодження)
LDO	Low-Dropout regulator
BMS	Battery Management System
ESD	Electrostatic Discharge
PSRAM	Pseudo-Static RAM
IRAM / DRAM	Instruction / Data RAM
RTC	Real-Time Clock
ULP	Ultra-Low-Power (співпроцесор)
WDT	Watchdog Timer
TWDT	Task Watchdog Timer
MTU	Maximum Transmission Unit
SSR	Solid-State Relay
THT / SMD	Through-Hole Technology / Surface-Mount Device
DRC	Design Rule Check
BOM	Bill of Materials (перелік компонентів)

Мовні рішення довідника

Українська назва + англійський термін у дужках при першій згадці в розділі: «черги (queues)», «підтягування (pull-up)».

Назви команд, файлів, реєстрів і функцій не перекладаються і набираються моноширинним шрифтом: `idf.py`, `app_main`, `sdkconfig`.

Одиниці й позначення — за стандартом: В, А, Ом, Гц, Ф, °С. Шістнадцяткові числа — 0x і малі літери: 0x76, 0x3ff.

duty cycle — це «коефіцієнт заповнення», а не «шпаруватість». Ці дві величини **обернені** одна до одної: коефіцієнт заповнення — це t_i/T (частка періоду, коли сигнал активний, від 0 до 1), а шпаруватість — T/t_i (у скільки разів період довший за імпульс, від 1 і вище). Сплутати їх означає отримати обернене число, тому в довіднику вживається лише «коефіцієнт заповнення».

Додаток Н. Джерела й посилання

Де перевіряти те, що написано в цьому довіднику, і де шукати те, чого тут немає.

УВАГА

Довідник має дату ревізії, а кремній і тулчейн змінюються. Критичні числові значення — струми, межі, версії, адреси — звіряйте з першоджерелом на момент читання.

Що саме звірено при підготовці цієї ревізії, з якого джерела й коли — у файлі docs/fakty.md репозиторію. Це не формальність: у полі різниця між звіреним і згаданим з пам'яті вирішує, чи можна довіритися рядку.

Першоджерела Espressif

ESP-IDF Programming Guide — основна документація фреймворку.
docs.espressif.com/projects/esp-idf

Має версії. **Найчастіша причина «в документації написано, а не працює»** — **відкрита сторінка іншої версії**. Перевіряйте перемикач версії у лівому верхньому куті.

Datasheet сімейства — електричні параметри, межі, розводка виводів, таблиці strapping. Те, що відкривають перед проєктуванням плати. espressif.com/en/support/documents/technical-documents

Technical Reference Manual (TRM) — опис регістрів і периферії на низькому рівні. Товстий документ, який читають за покажчиком, коли треба зрозуміти, чому периферія поводить себе саме так.

Datasheet модуля (ESP32-WROOM-32, ESP32-S3-WROOM-1, ESP32-C3-MINI-1) — окремий документ від datasheet чипа. Саме він каже, які виводи доступні й скільки флешу та PSRAM у середині.

Hardware Design Guidelines — вимоги до розводки: зона антени, живлення, розв'язувальні конденсатори. Обов'язкове читання перед власною платою (розділ 52).

Репозиторії

github.com/espressif/esp-idf — сам фреймворк. Найцінніше в ньому, крім коду:

- **examples/** — робочий приклад майже на кожному периферію, точно під вашу версію. Найнедооціненіший ресурс у всій екосистемі;
- **SUPPORT_POLICY.md** — терміни підтримки версій;
- **Issues** — там відповідають розробники, і багато відповідей ніде більше не існує.

github.com/espressif/esptool — CHANGELOG.md пояснює, чому команди з інтернету не працюють: різниця синтаксису v4 і v5 (розділ 17).

github.com/espressif/arduino-esp32 — Arduino core, релізи, міграційні нотатки 2.x → 3.x (розділ 12).

components.espressif.com — реєстр компонентів для idf.py add-dependency (розділ 11).

Довідкове

sigrok.org — PulseView і підтримувані логічні аналізатори (розділ 28).

kicad.org — розробка друкованих плат (розділ 52).

Wokwi — симулятор ESP32 у браузері (розділ 14).

Документація на конкретні мікросхеми — сайти виробників: Bosch Sensortec (BME280), Sensirion (SHT3x), Semtech (SX127x/SX126x), Texas Instruments (драйвери, INA219), Maxim / Analog Devices (DS18B20, MAX485).

Як шукати відповідь

Порядок, що працює швидше за будь-який інший:

1. **examples/ у вашому ESP-IDF**. Локально, під вашу версію, точно робоче.
2. **Programming Guide вашої версії**. Не «latest», а вашої.
3. **Datasheet або TRM** — коли питання про залізо, а не про API.
4. **Issues репозиторію** — коли схоже на баг або на відому дивність.
5. **Форум Espressif** — коли питання про застосування.
6. **Пошук в інтернеті** — останнім, і з перевіркою дати й версії.

УВАГА

Пункт 6 останній не випадково. Величезна частина статей про ESP32 написана під ESP-IDF 4.x і Arduino core 2.x. Приклад, що не збирається, частіше означає **іншу версію**, ніж помилку.

Дивитися треба на дату статті й на те, під що вона написана (розділи 11, 12).

Що робити зі знайденою помилкою в довіднику

Довідник живе в репозиторії й перевидається ревізіями.

Знайшли помилку, застаріле значення або місце, де довідник підвів, — напишіть. Виправлення входить у наступну ревізію із зазначенням, що саме змінилося і чому.

Адреса репозиторію — на останній сторінці.

Ліцензування джерел

Технічні дані, наведені в довіднику, взяті з відкритої документації Espressif Systems і виробників компонентів. Вони належать своїм власникам і наводяться як фактичні відомості з посиланням на джерело.

Ліцензія самого довідника (CC BY-SA 4.0 на текст, MIT на код) на них не поширюється — розділ «Ліцензування» у репозиторії.

Вкладиші

*Швидкопсувне: ринок, компоненти,
радіочастотні норми. Мають власні дати
ревізій і перевидаються, не зачіпаючи нумерації
книги (P5).*

Вкладиш: ринок України

Ревізія вкладаша: 2026-08.

НЕЗВОРОТНЕ

Цей вкладаш **не містить цін і назв магазинів**, і це свідоме рішення (P5, P13).

Ціни змінюються швидше, ніж перевидається книга. Ціна, надрукована рік тому, гірша за її відсутність: людина побачить число і спланує бюджет за ним.

Вкладаш каже, **як шукати і на що дивитися**, а конкретні позиції з цінами збираються пошуком на момент закупівлі й дописуються сюди при перевиданні — із датою перевірки.

Де брати

Українські магазини електроніки. Найшвидше, є гарантія й повернення, можна подивитися перед покупкою. Дорожче за прямі замовлення.

Українські маркетплейси. Ширший вибір, продавці різної якості. Перевіряти відгуки саме про цю позицію, а не про продавця загалом.

Прямі замовлення з-за кордону. Найдешевше, найдовше. Для планової закупівлі підходить, для термінової — ні.

Місцеві радіоринки й майстерні. Витратне, роз'єми, дроти, іноді рідкісні компоненти. Немає гарантії.

Практика закупівлі

ЗАКУПІВЛЯ

Купувати з запасом і перевіряти одразу. Плати дешеві, і партія з десяти майже завжди містить одну проблемну. Знайти її на столі при отриманні дешевше, ніж у полі всередині виробу (розділ 08).

Три позиції, на яких не варто економити (розділ 10): мультиметр, паяльник із терморегулятором, USB-кабель.

Три позиції, де дешеве нормальне: макетка, Dupont-дроти, гребінки.

Не запасати датчики під майбутній проєкт — вони старіють на полиці, а вибір змінюється.

На що дивитися в оголошенні

Позиція	Перевіряти
Плата ESP32	кількість пінів (30 чи 38), який міст USB-UART
Модуль S3	N8 чи N16R8 — різна кількість вільних пінів
TP4056	з захистом чи без — на фото видно дві додаткові мікросхеми
Трансивер CAN	3.3 В (SN65HVD230) чи 5 В (TJA1050)
LoRa-модуль	діапазон частот — має відповідати нормам (вкладиш норм)
18650	protected чи ні; чесна ємність
Реле	напряга котушки і логіка керування
microSD	клас і призначення: для постійного запису потрібні окремі

Що перевірити після отримання

Хвилина на плату (розділ 08):

```
esptool --port /dev/ttyUSB0 flash-id # звірити шапку з написом
esptool --port /dev/ttyUSB0 flash-id # звірити обсяг
```

плюс boot-лог. Це ловить перемарковані модулі, меншу флеш і мертві плати.

Для датчиків — мінімальний тест окремо від проєкту (розділ 44).

Терміни й планування

Прямі замовлення — тижні. Це треба закладати в план проєкту, а не згадувати, коли компонент знадобився.

Практичний прийом: **критичні компоненти замовляти в перший день проєкту**, паралельно з роботою над рештою. Прототип на тому, що є, поки їде те, що потрібно.

Що дописати сюди при перевиданні

Для кожної позиції — **де, скільки, наявність і дата перевірки**:

- плати ESP32 основних моделей;
- датчики з переліку компонентного вкладиша;
- інструмент із розділу 10
- витратне: гребінки, дроти, термоусадка, припій, флюс;
- акумулятори й зарядні модулі.

Формат той самий, що в docs/fakty.md: значення, джерело, дата (додаток Н).

Вкладиш: компонентний довідник

Ревізія вкладиша: 2026-08.

Перевірені позиції й заміниники. Вкладиш перевидається окремо від книги й не зачіпає її нумерації (P5).

УВАГА

Тут немає цін і назв магазинів — вони у вкладиші ринку. Тут — **що саме брати і чому**: конкретні позначення, заміниники й ознаки підробок.

Плати

Позиція	Коли брати	На що дивитися
ESP32-S3-DevKitC-1	новий проект за замовчуванням	N8 чи N16R8 — різна кількість вільних пінів
ESP32-DevKitC V4	навчання, максимум прикладів	38 пінів, оригінал Espressif
DevKit V1 / DOIT	дешева заміна попередньої	буває 30- і 38-піновою — різні плати
ESP32-C3 SuperMini	простий дешевий вузол	400 КБ — стея, PSRAM немає
ESP32-CAM	камера за подією	немає USB , потрібен перехідник

Датчики: що брати і замість чого

Замість	Беріть	Чому
DHT11, DHT22	BME280 або SHT3x	власний протокол, нестабільна точність
Фоторезистор	BH1750	лінійний, у люксах, по I ² C
HC-SR04	VL53L0X, де можна	 5 В на ЕСНО; ціль від 0.5 м ² і рівна
MPU6050	BNO055, де потрібна орієнтація	сам зводить дані, менше роботи

Замість	Беріть	Чому
Резистивний датчик вологості ґрунту	ємнісний	резистивні кородують за місяці
ACS712	INA219 / INA226	цифровий, точніший, менше шуму

Лишаються добрим вибором: DS18B20 (довгий дріт, герметичний зонд, кілька на лінії), BMP280 (тиск без вологості, дешевше за BME280), PIR HC-SR501 (простий рух).

Драйвери й ключі

Позиція	Для чого	Замінники
DRV8833	двигун постійного струму	TB6612FNG
не L298N	—	застарів, втрачає 2 В на собі
A4988	кроковий	DRV8825 (більший струм, мікрокрок 1/32)
ULN2003	28BYJ-48	іде в комплекті
MOSFET логічного рівня	комутація навантаження	обов’язково logic level
Релейний модуль з оптопарою	мережеве живлення	перевіряти: 5 В і часто інверсна логіка
SSR	часте перемикання	потребує радіатора

Інтерфейси й зв’язок

Позиція	Для чого	Застереження
SN65HVD230	трансивер CAN	✓ 3.3 В — правильний вибір
TJA1050, MCP2551	трансивер CAN	⊖ 5 В, потрібен конвертер на RX
MAX485	RS-485	брати версію на 3.3 В
SX1262	LoRa	ефективніший за SX1276
SX1276 / RFM95	LoRa	більше бібліотек і прикладів

Позиція	Для чого	Застереження
Конвертер рівнів на польових	3.3 ↔ 5 В	двонаправлений, працює з I ² C

Живлення

Позиція	Для чого	Застереження
TP4056 з захистом	заряд Li-ion	 версія без захисту виглядає так само
Buck-boost 3.3 В	автономний пристрій	дає майже всю ємність, LDO — половину
LDO з малим падінням	тихе живлення для ADC	після buck
18650 protected	акумулятор	або окрема BMS
Тримач або елемент із привареними виводами	18650	не паяти напряму до елемента

Витратне: типові номінали

Номінал	Де використовується
220–330 Ом	резистор світлодіода
4.7 кОм	підтягування I ² C і 1-Wire
10 кОм	підтягування входів, затвор MOSFET на землю
100–220 Ом	послідовно з затвором MOSFET
120 Ом	термінатори RS-485 і CAN
100 нФ	біля кожної мікросхеми
470 мкФ	біля роз'єму живлення
1N4007	захисний діод на котушці

Ознаки підробок

Що	Як виявити
Перемаркований модуль	шапка esptool ≠ напис на кришці
Менша флеш	flash-id показує менше за заявлене

Що	Як виявити
Фейковий CP2102 / FT232	нестабільно, зникає після оновлення драйвера
Слабкий стабілізатор на клоні	гарячий, перезавантаження через хвилини
Підробка DS18B20	розбіжність понад 1.5 °C між датчиками в кімнатній воді
Картка microSD меншого обсягу	перевірити записом на повний обсяг

Поріг для DS18B20 не випадковий і не збігається з паспортною похибкою. Datasheet дає $\pm 0.5\text{ °C}$ **на один датчик** у діапазоні $-10\dots+85\text{ °C}$, тож два справні можуть відхилитися в різні боки й розійтися на цілий градус, лишаючись обидва в межах норми. Поріг 0.5 °C бракував би чесний товар.

Міряти треба у **кімнатній** воді, не в окропі: за межами $-10\dots+85\text{ °C}$ паспортна похибка зростає до $\pm 2\text{ °C}$, і там розбіжність не доводить нічого.

Перевірка нової партії — три команди й boot-лог, хвилина на плату (розділ 08).

Що тримати в запасі

Завжди: перевірені USB-кабелі, гребінки, Dupont усіх трьох видів, термоусадка, резистори 4.7 і 10 кОм, конденсатори 100 нФ і 470 мкФ, пара конвертерів рівнів.

Для ремонту: запасна плата з залитою робочою прошивкою, дроти, оплітка, флюс.

Не варто запасати: датчики під конкретний майбутній проєкт — вони старіють на полиці, а вибір змінюється.

Вкладиш: радіочастотні норми

Ревізія вкладки: 2026-08. Перевіряти перед кожним розгортанням.

НЕЗВОРОТНЕ

Цей вкладиш **не містить конкретних значень частот і потужностей** — і це свідоме рішення (P5, P13).

Норми змінюються, а застаріла цифра в друкованій книзі гірша за її відсутність: людина побачить число, повірить і не перевірить.

Вкладиш каже, **де і що саме перевіряти**, і збирається пошуком у першоджерелах, а не з пам'яті.

Що регулюється

Радіопередавач — обладнання, робота якого обмежена нормами. Регулюються щонайменше чотири речі, і кожна може зробити ваш пристрій незаконним:

Діапазон частот. Дозволені смути різняться між країнами. Модуль, куплений на маркетплейсі, може бути на частоту, дозволена в іншому регіоні.

Потужність передавача. Обмежується як випромінювана потужність (з урахуванням підсилення антени), а не як потужність на виході чипа.

Коефіцієнт заповнення ефіру (duty cycle). У частині діапазонів обмежено частку часу, яку передавач може займати. Це впливає на проект прямо: пристрій, що передає раз на десять секунд, може виявитися поза нормою (розділ 43).

Сертифікація готового виробу. Використання сертифікованого модуля знімає частину вимог, але не всі — особливо при власній антені.

Що це означає практично

Технологія	На що дивитися
Wi-Fi 2.4 ГГц	канали 12–13 доступні не в усіх регіонах; налаштування країни в коді
Wi-Fi 5 ГГц	тільки ESP32-C5; окремі обмеження на канали
BLE	зазвичай найменш проблемний
ESP-NOW	той самий діапазон, що Wi-Fi 2.4 ГГц
LoRa	найбільш чутливе: діапазон модуля має відповідати регіону
802.15.4	Zigbee, Thread — 2.4 ГГц

НЕЗВОРОТНЕ

LoRa-модулі продаються на різні частоти, і не всі дозволені в усіх регіонах. Модуль на діапазон, дозволений в іншій частині світу, — це не «працює гірше», а використання чужої смуги.

Перевіряти треба **до закупівлі партії**, а не після.

Налаштування країни в прошивці

ESP-IDF має налаштування регіону, що обмежує доступні канали й потужність. Значення за замовчуванням не обов'язково відповідає місцю, де стоятиме пристрій.

Для виробу, що йде в поле, це налаштування задається свідомо, а не лишається типовим (розділ 39).

Де перевіряти

Це те, що збирається пошуком на момент розгортання:

- **чинні національні норми** на використання радіочастотного спектра;
- **загальні дозволи** для малопотужних пристроїв малого радіуса дії (short range devices, SRD);
- **умови для конкретного діапазону**: потужність, duty cycle, каналний план;
- **вимоги до сертифікації** готового виробу;
- **документація на модуль**: на який регіон він розрахований.

Що записати в цей вкладиш при його перевиданні

Для кожного пункту — **значення, джерело і дата перевірки**, у форматі docs/fakty.md (додаток H):

- дозволені діапазони й каналні плани;
- максимальна випромінювана потужність по кожному діапазону;
- обмеження duty cycle, якщо є;
- вимоги до сертифікації;
- посилання на документ, яким це встановлено, і його редакція.

Окреме застереження

НЕЗВОРОТНЕ

Пристрій, що передає своє розташування, розповідає про своє розташування. Разом із передавачем це створює наслідки, які варто продумати **до** розгортання: хто отримує дані, як вони захищені, що станеться при переході (розділи 45, 50).

Технічний бік — розділ 50. Рішення про доцільність — не технічне.

ESP32: практичний довідник

Польовий довідник з розробки, прошивки, діагностики й ремонту

Ярослав Войтович

ревізія 2026-08-26

Знайшли помилку, застаріле значення або місце, де довідник вас підвів, — напишіть. Виправлення входять у наступну ревізію із зазначенням, що саме змінилося і чому.

github.com/ivoitovych/esp32-handbook-ua

CC BY-SA 4.0 · друкуйте і роздавайте вільно